

Crackz Documentation

AI Image Analysis

Version: 0.103.0

© 2026 Anaxiatech AB

Contents

01. Home
02. Installation
03. Quick Start
04. GUI Overview
05. AI Chat Assistant
06. Dashboard
07. Map Viewer
08. Image Tiling
09. Report Generation
10. CLI Reference
11. CLI Cookbook
12. Shell Completion
13. Project Configuration
14. Training Configuration
15. Data Splits & Import
16. Docker
17. Azure VM (GPU)
18. Azure Batch
19. VM vs Batch
20. Model Evaluation
21. Advanced Performance
22. Migration Guide
23. Architecture
24. Architecture Guidelines
25. Diagrams
26. Understand Anything
27. API Reference
28. Async Callbacks
29. MCP Integration
30. Python 3.15 Readiness
31. Tracing & Observability
32. Contributing
33. Release Flow
34. PR Review Workflow
35. Distribution
36. Customer Guide
37. Customer Access
38. License

Home

Crackz

Crackz Logo

AI-powered crack detection and analysis for harbor and bridge infrastructure.



The Crackz desktop app — project dashboard with model configuration and detection metrics.

Welcome

Crackz is a commercial-grade crack detection solution designed for infrastructure inspection. Reduce inspection costs by up to 70% while improving safety and consistency through automated AI-powered analysis.

> **New to Crackz?** Start with the Getting Started Guide to be up and running in minutes.
>
> **Ready to install?** Check out the Installation Guide for setup instructions.

Why Choose Crackz?

For Infrastructure Managers

- **Reduce costs** – Process thousands of images in hours instead of weeks
- **Improve safety** – Minimize physical inspections in hazardous locations
- **Early detection** – Identify structural issues before they become critical
- **Proven ROI** – Typical savings of $25,000-$75,000 annually per inspection program

For Technical Teams

- **Reproducible projects** – Self-contained folder structure for complete traceability
- **Fast experimentation** – Runtime overrides for rapid iteration
- **GUI + CLI** – Modern desktop app for visual workflows; powerful CLI for scripting and automation
- **Container friendly** – Docker images for all deployment scenarios
- **Extensible design** – Modular AI pipeline built on PyTorch Lightning

Model Capabilities

Crackz uses deep learning segmentation models to identify defects at the pixel level:

- **Binary segmentation**: Detect crack/no-crack with precise pixel masks (default for single-category projects)
- **Multi-class segmentation**: Identify and distinguish multiple defect types (cracks, corrosion, spalling) using the project annotation schema
- **Flexible backbones**: SegFormer-B2 by default; choose from SegFormer (B0–B5) or 20+ U-Net encoders (ResNet, EfficientNet, MobileNet, etc.)
- **Transfer learning**: Leverage pre-trained ImageNet weights for better accuracy with less data
- **Taxonomy-aware models**: Checkpoints record the annotation schema they were trained on; incompatible models are flagged automatically

Two Deployment Paths

1. Use Pre-Trained Models (Fastest) - Quick deployment with ready-made checkpoints - Standard crack detection scenarios - Consistent results across sites

2. Train Custom Models (Most Accurate) - Domain-specific imagery and unique defect types - Maximum accuracy for your infrastructure - Requires labeled training data (images + masks)

See the Getting Started Guide for detailed workflows.

Core Features

Category	Capabilities
Interfaces	GUI (`crackz-gui`), CLI (`crackz`), and Web Dashboard (Streamlit)
Data Management	Structured splits (raw/train/val/test) with automatic organization
AI Pipeline	Segmentation models with configurable encoders and metrics
Logging	Hybrid central + project logs for complete audit trails
Configuration	Pydantic-validated YAML configs with schema versioning
Deployment	Local, Docker (CPU/GPU), Azure Batch, standalone executables
Training	Runtime overrides for rapid experimentation

Quick Example

Using the desktop app:

1. Launch: `crackz-gui`
2. **File → New Project** — name your project and choose a folder
3. **Import Images** — drag in your harbor/bridge photos
4. **Run Detection** — click the Run button; results appear in real time
5. Browse flagged findings in the **Gallery** view

Or via the CLI (for scripting and automation):

```
crackz init --name demo
crackz import-images --project demo --src ./photos --type raw --size 512 512
crackz detect run --project demo
# Results written to demo/artifacts/detections/
```

Commercial Licensing

Crackz is distributed under a **commercial source-available license** for enterprise infrastructure management.

What's Included

- Perpetual license for internal use
- Source-available code for transparency and security audits
- Self-hosted deployment on your infrastructure
- Complete documentation and examples

Get Started

📧 **Contact**: theresia.sofia.snow@gmail.com

For pricing, proof-of-concept projects, and enterprise support options.

📖 **Learn More**:

- Getting Started – Quick start guide
- Installation – Setup instructions
- GUI Guide – Desktop workflow documentation

Documentation

For End Users

- Getting Started – Start here!
- Installation – Setup and configuration
- GUI Guide – Graphical interface documentation
- CLI Cookbook – Dedicated command-line workflows
- Dashboard Guide – Interactive web dashboard
- Configuration – Detailed settings reference

For Developers

- Architecture – System design and components
- Architecture Guidelines – Development patterns and principles

- API Reference – Programmatic access
- Contributing – Development guidelines

Advanced Topics

- Docker Deployment – Container usage
- Azure Batch – Large-scale processing
- Advanced Performance – Optimization tips

Need help? Start with the Getting Started Guide or contact us at theresia.sofia.snow@gmail.com

---

Installation

Installation

This page covers end-user (runtime) installation of Crackz. If you're developing the project itself, see the developer workflow in the Getting Started guide (`uv sync` etc.).

> Licensing: Crackz is distributed under a commercial (source-available) license. By installing you agree to the terms in the `LICENSE` file.

1. Requirements

- Python 3.11 – 3.13 (64-bit)
- OS: Linux, macOS (CPU), Windows 10/11
- **Postgres 16+** with the **pgvector extension (≥ 0.8)**; PostGIS recommended for geospatial queries — see Database setup for Docker, bare-metal, and hosted options and the `CRACKZ_DATABASE_URL` setting
- (Optional) NVIDIA GPU + matching CUDA drivers for accelerated training/detection
- Disk space: ~2–3 GB (Torch + model weights + cache)

2. Standard GUI Install (recommended default)

Standalone Executable (No Python Required)

Download the pre-built `crackz-gui` executable for your platform from GitHub Releases:

> **Availability:** the executables are built by the separate `pyinstaller-build.yml` workflow, which is run by hand, and it attaches them only to **draft or prerelease** releases. For a published release it skips the upload ("Skipping upload for immutable release"), so published releases, including v0.102.0, do not have them. If the asset list for your release has no `crackz-gui-*` archive, use the wheel install below.

Each build is an archive named `crackz-gui-<version>-<suffix>`; the executable (`crackz-gui` or `crackz-gui.exe`) is inside it.

Platform	Archive to download
Windows (CPU)	`crackz-gui-<version>-windows.zip`
Windows (CUDA 12.x — RTX 30/40, A100)	`crackz-gui-<version>-windows-cu128.zip`
Windows (CUDA 11.x — RTX 20, older)	`crackz-gui-<version>-windows-cu118.zip`
macOS	`crackz-gui-<version>-macos.tar.gz`
Linux (CPU)	`crackz-gui-<version>-linux.tar.gz`
Linux (CUDA 12.x — RTX 30/40, A100)	`crackz-gui-<version>-linux-cu128.tar.gz`
Linux (CUDA 11.x — RTX 20, older)	`crackz-gui-<version>-linux-cu118.tar.gz`

Check your CUDA version with `nvidia-smi` (look for the `CUDA Version` field) to pick the right CUDA variant. macOS uses Apple MPS automatically — no variant flag needed.

Linux / macOS:

```
tar -xzf crackz-gui-<version>-linux.tar.gz   # or the -macos / -linux-cu128 / -linux-cu118 archive
chmod +x crackz-gui
sudo install -m 755 crackz-gui /usr/local/bin/crackz-gui
crackz-gui
```

Windows: Extract the `.zip`, then double-click `crackz-gui.exe` to launch. Move it to a permanent folder and add that folder to `PATH` for command-line access.

> **Windows Defender / antivirus:** Unsigned executables may trigger SmartScreen. Click **More info → Run anyway** or add an exclusion. **Slow first start:** PyInstaller bundles are extracted to a temp directory on first launch (~30 s). Subsequent starts are fast.

Wheel Install (Python / uv)

Use this when you prefer a Python package install or need programmatic access.

Prerequisites: - GitHub account with repository access - GitHub CLI authenticated: `gh auth login` - Python 3.11+ and `uv`

Wheel Install: Private GitHub Releases

Use this when you need the Python package install instead of the standalone GUI binary.

Prerequisites: - GitHub account with repository access - GitHub CLI authenticated: `gh auth login` - Python 3.11+ and `uv`

```
gh auth login
gh release download --repo Anaxiatech-AB/crackz \
  --pattern 'crackz-*.whl' --pattern 'crackz_gui-*.whl'
uv pip install crackz-*.whl crackz_gui-*.whl
crackz-gui
```

The GUI wheel declares no dependency on the core package, because the core is published under three names (`crackz` , `crackz_cu118` , `crackz_cu128`) and one GUI wheel has to work with any of them. Install **one** core wheel next to it. For a CUDA build, install PyTorch for your CUDA version first (see GPU / CUDA Install; the CUDA wheels do not install PyTorch), download with `--pattern 'crackz_cu128-*.whl'` (or `crackz_cu118-*` , since the `crackz-*` pattern above does not match it), and install that wheel instead of `crackz-*.whl` .

> **Releases up to v0.102.0** still declare `crackz-core` in the GUI wheel and fail to resolve. Install the core wheel and `pyside6>=6.7,<=6.9.3` , then `uv pip install --no-deps crackz_gui-*.whl` .

Advanced Installation Methods

For developers or users who need more control over the installation:

Crackz is distributed as a **private commercial product**. Access is provided to licensed customers only through: - Private GitHub repository access - Private package index (if configured by your organization) - Standalone executables delivered directly to customers

Option A: Install from Private GitHub Repository

For customers with repository access:

If you've been granted access to the private Crackz repository, you can install from releases or directly from source.

Install from a specific release:

```
# Using pip with GitHub authentication
pip install git+https://github.com/OWNER/crackz.git@vVERSION

# Or download the wheel from the private repository's Releases page
# (Requires GitHub authentication - use personal access token)
pip install https://USERNAME:TOKEN@github.com/OWNER/crackz/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl
```

Install the latest version from main branch:

```
pip install git+https://github.com/OWNER/crackz.git
```

Using GitHub CLI (recommended):

```
# Login with GitHub CLI
gh auth login

# Download release asset
gh release download vVERSION --repo OWNER/crackz --pattern '*.whl'

# Install the downloaded wheel
pip install crackz-VERSION-py3-none-any.whl
```

Using Personal Access Token:

```
# Create a personal access token with 'repo' scope at:
# https://github.com/settings/tokens

# Set your token (keep it secure!)
export GITHUB_TOKEN="ghp_xxxxxxxxxxxx" # gitleaks:allow

# Install using token authentication
pip install git+https://${GITHUB_TOKEN}@github.com/OWNER/crackz.git@vVERSION
```

Security best practices: - Never commit tokens to version control - Use fine-grained personal access tokens with minimal scope (read-only repo access) - Store tokens securely (e.g., password manager, GitHub Secrets for CI/CD) - Rotate tokens regularly

Option B: Download Wheel from Private Repository

Manual download via GitHub web interface: 1. Navigate to the private repository: `https://github.com/OWNER/crackz` 2. Go to the Releases page 3. Download the wheel file for your desired version: `crackz-VERSION-py3-none-any.whl` 4. Install locally:

```
pip install crackz-VERSION-py3-none-any.whl
```

Using curl with authentication:

```
# Download wheel using GitHub token
curl -L \
  -H "Authorization: token ${GITHUB_TOKEN}" \
  -o crackz.whl \
  https://github.com/OWNER/crackz/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl
```

```
# Install the downloaded wheel
pip install crackz.whl
```

Option C: Private package index (if configured)

```
# If your organization has configured a private index
pip install --index-url https://pypi.company.com/simple/ crackz
```

For select customers with private index access:

If you've been provided access to a private package repository, you can install Crackz directly using pip:

One-time installation:

```
# Using the provided index URL and credentials
pip install crackz --index-url https://INDEX_URL/simple/

# Example with authentication (if required):
pip install crackz --index-url https://USERNAME:TOKEN@pypi.company.com/simple/
```

Configure pip permanently (recommended for easier updates):

Create or edit `~/.config/pip/pip.conf` (Linux/macOS) or `%APPDATA%\pip\pip.ini` (Windows):

```
[global]
index-url = https://USERNAME:TOKEN@pypi.company.com/simple/
```

Then install normally:

```
pip install crackz
pip install --upgrade crackz  # For updates
```

Using environment variables (for CI/CD or temporary use):

```
# Set credentials via environment variables
export PIP_INDEX_URL="https://pypi.company.com/simple/"
export PIP_EXTRA_INDEX_URL="https://pypi.org/simple"  # Fallback to PyPI for dependencies

pip install crackz
```

Supported private package index providers: - **Cloudsmith** - `https://dl.cloudsmith.io/TOKEN/org/repo/python/simple/` - **Artifactory** - `https://artifactory.company.com/artifactory/api/pypi/pypi-local/simple/` - **Gemfury** - `https://pypi.fury.io/TOKEN/USERNAME/` - **AWS CodeArtifact** - `https://aws:TOKEN@domain.d.codeartifact.region.amazonaws.com/pypi/repo/simple/` - **Azure Artifacts** - `https://pkgs.dev.azure.com/org/_packaging/feed/pypi/simple/` - **Self-hosted devpi** - `https://devpi.company.com/username/index/+simple/`

Security best practices: - Never commit credentials to version control - Use tokens instead of passwords when available - Store credentials in pip config file with restricted permissions:

```
chmod 600 ~/.config/pip/pip.conf  # Linux/macOS
```

- For CI/CD, use secret management (GitHub Secrets, etc.)

Troubleshooting private index installation: - **401 Unauthorized**: Check your credentials/token - **404 Not Found**: Verify the index URL and package name - **SSL errors**: Your organization may use custom CA certificates

```
pip install --trusted-host pypi.company.com crackz
# Or add CA cert:
pip install --cert /path/to/ca-bundle.crt crackz
```

- **Mixed dependencies**: If some packages aren't in your private index:

```
pip install crackz --extra-index-url https://pypi.org/simple/
```

Option D: Standalone Executables (No Python Required)

For users who **don't have Python installed** or prefer standalone applications, PyInstaller executables are available:

Download: 1. Visit the GitHub Releases page 2. Download the appropriate executable for your platform: - **Windows**: `crackz.exe` (CLI) or `crackz-gui.exe` (GUI) - **Linux**: `crackz` (CLI) or `crackz-gui` (GUI) - **macOS**: `crackz` (CLI) or `crackz-gui` (GUI)

Installation:

Windows:

```
REM Download the executables from GitHub Releases
REM Move them to a permanent location
mkdir C:\Program Files\Crackz
move crackz.exe "C:\Program Files\Crackz\"
move crackz-gui.exe "C:\Program Files\Crackz\"

REM Add to PATH (optional, for CLI access from anywhere)
setx PATH "%PATH%;C:\Program Files\Crackz"
```

Linux/macOS:

```
# Download the executables from GitHub Releases
chmod +x crackz crackz-gui

# Move to a system directory (optional)
sudo mv crackz /usr/local/bin/
sudo mv crackz-gui /usr/local/bin/

# Or keep in a local directory and add to PATH
mkdir -p ~/Applications/crackz
mv crackz crackz-gui ~/Applications/crackz/
echo 'export PATH="$HOME/Applications/crackz:$PATH"' >> ~/.bashrc
source ~/.bashrc
```

Usage:

```
# Run CLI directly (no Python needed)
crackz --help
crackz init --name myproject

# Run GUI by double-clicking crackz-gui.exe (Windows)
# Or run from command line:
crackz-gui
```

Notes about standalone executables: - ✅ **No Python installation required** - everything is bundled - ✅ **Includes PyTorch runtime** - works out of the box - ⚠️ **Larger file size** - ~180-250 MB (CPU) or ~250-350 MB (CUDA) - ⚠️ **Platform-specific** - download the version built for your OS - ⚠️ **CUDA executables** require NVIDIA GPU and compatible drivers - 📦 **Self-contained** - all dependencies are included

GPU-enabled executables:

For GPU acceleration, download the CUDA-enabled executables (if available): - Filename pattern: `crackz-cuda.exe` or `crackz-gui-cuda.exe` - Requires: NVIDIA GPU with CUDA 11.8+ drivers - Verify GPU support: Run the executable and check logs for "Running inference on cuda device"

Troubleshooting executables: - **Windows Defender warning**: Unsigned executables may trigger SmartScreen. Click "More info" → "Run anyway" - **Slow first run**: Initial startup extracts bundled files to temp directory (~30 seconds) - **Antivirus false positive**: PyInstaller executables sometimes trigger AV. Add to exclusions if needed - **Missing DLL errors**: Ensure Visual C++ Redistributable is installed (Windows)

Verify:

```
crackz --help
python -c "import crackz; import importlib.metadata as m; print(m.version('crackz'))"
```

3. GPU / CUDA Install

Crackz provides **separate wheels for different PyTorch variants**:

- `crackz-VERSION-py3-none-any.whl` - **CPU version** (includes PyTorch CPU)
- `crackz_cu118-VERSION-py3-none-any.whl` - **CUDA 11.8** version (PyTorch NOT included)
- `crackz_cu128-VERSION-py3-none-any.whl` - **CUDA 12.8** version (PyTorch NOT included)

⚠️ **IMPORTANT**: CUDA wheel variants do NOT include PyTorch. You must install PyTorch with CUDA support separately first.

CPU Installation (One Step)

For CPU (default):

```
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl
```

CUDA Installation (Two Steps Required)

For CUDA 11.8 (RTX 20/30 series, older drivers, Windows recommended):

```
# Step 1: Install PyTorch with CUDA 11.8
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118

# Step 2: Install crackz CUDA variant
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz_cu118-VERSION-py3-none-any.whl
```

For CUDA 12.8 (RTX 40 series, newer drivers, Linux recommended):

```
# Step 1: Install PyTorch with CUDA 12.8
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128

# Step 2: Install crackz CUDA variant
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz_cu128-VERSION-py3-none-any.whl
```

Check Your CUDA Version

To determine which version to install, check your CUDA toolkit version:

Windows:

```
nvidia-smi
```

Look for "CUDA Version" in the output (e.g., "CUDA Version: 12.4" means use cu128)

Linux:

```
nvcc --version
# Or check driver
nvidia-smi | grep "CUDA Version"
```

Compatibility guide: - CUDA 11.x drivers → use `crackz_cu118` - CUDA 12.x drivers → use `crackz_cu128` - No NVIDIA GPU → use `crackz` (CPU version)

Using GitHub CLI (Recommended)

For CPU:

```
# Login first
gh auth login

# Download and install CPU version (one step)
gh release download vVERSION --repo OWNER/crackz --pattern 'crackz-*.whl'
pip install crackz-VERSION-py3-none-any.whl
```

For CUDA 12.8:

```
# Login first
gh auth login

# Step 1: Install PyTorch with CUDA 12.8
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128

# Step 2: Download and install crackz CUDA variant
gh release download vVERSION --repo OWNER/crackz --pattern 'crackz_cu128-*.whl'
pip install crackz_cu128-VERSION-py3-none-any.whl
```

Verify GPU Support

After installation, verify that PyTorch detects your GPU:

```
# Check CUDA availability
python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}')"

# Check PyTorch version (should include +cu118 or +cu128 suffix)
python -c "import torch; print(f'PyTorch version: {torch.__version__}')"

# Check GPU device name
python -c "import torch; print(f'Device: {torch.cuda.get_device_name(0) if torch.cuda.is_available() else \"CPU\"}')"
```

Expected output with GPU:

```
CUDA available: True
PyTorch version: 2.5.0+cu128
Device: NVIDIA GeForce RTX 4090
```

⚠️ **If PyTorch version doesn't show +cu118 or +cu128 suffix**, you installed the CPU version by mistake. Reinstall PyTorch with the correct index URL (see Step 1 above).

Alternative PyTorch Versions (Advanced)

If you need a specific PyTorch build not covered by cu118/cu128:

```
# Step 1: Install specific PyTorch version
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121

# Step 2: Install crackz CUDA variant (cu118 or cu128, either works)
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz_cu118-VERSION-py3-none-any.whl
```

See: https://pytorch.org/get-started/locally/ for all available PyTorch CUDA versions.

Note: Since CUDA wheel variants don't include PyTorch, any PyTorch version ≥2.5.0 will work. The wheel variant name (cu118/cu128) is just a recommendation.

4. Optional Extras (Deprecated)

⚠️ **DEPRECATED**: The `cpu` and `cu128` extras from older versions no longer work correctly and have been removed. Use the two-step installation process described in Section 3 instead.

Old method (DO NOT USE):

```
# This installs CPU PyTorch even with the cu128 extra - BROKEN!
pip install "crackz-VERSION-py3-none-any.whl[cu128]"
```

New method (CORRECT):

```
# Step 1: Install PyTorch with CUDA
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu128

# Step 2: Install crackz
pip install crackz_cu128-VERSION-py3-none-any.whl
```

5. Optional Features

Secure Credential Storage (keyring)

For enhanced security when storing API keys (e.g., for the AI Chat Assistant), you can install the optional `keyring` library. This provides OS-level encrypted credential storage:

Installation:

```
# With uv
uv add keyring

# With pip
pip install keyring
```

What it provides: - **Windows**: Integration with Windows Credential Manager - **macOS**: Integration with Keychain - **Linux**: Integration with Secret Service (GNOME Keyring, KWallet, etc.)

Usage:

```
from crackz_gui.state.secure_storage import (
    store_api_key,
    retrieve_api_key,
    is_secure_storage_available,
)

# Check availability
if is_secure_storage_available():
    store_api_key("your-api-key")
    api_key = retrieve_api_key()
```

When to use: - ✅ **Enterprise deployments** - Required for production environments - ✅ **Shared machines** - Prevents API key exposure - ✅ **Automation scripts** - Secure credential management - ⚠️ **Development/personal use** - Optional but recommended

Note: Without keyring installed, API keys will be stored in plaintext in the application settings file. See AI Chat documentation for details.

6. Install From a Downloaded Wheel (Air-gapped / Offline)

1. On a machine with internet, download the Crackz wheel from GitHub Releases:

   ```
   mkdir crackz_wheels
   cd crackz_wheels

   # Download Crackz wheel
   wget https://github.com/Anaxiatech-AB/crackz/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl

   # Download all dependencies
   pip download -r <(pip show -f crackz | grep Requires) -d .
   # Or download from the wheel itself:
   pip download crackz-VERSION-py3-none-any.whl -d .
   ```

2. Transfer the directory to the offline machine.
3. Install locally:

   ```
   pip install --no-index --find-links crackz_wheels crackz-VERSION-py3-none-any.whl
   ```

7. Install From Source (no build system tooling)

If you cloned the repository (runtime only):

```
pip install .
```

This performs a build via the backend defined in `pyproject.toml`.

8. Upgrading

```
# Download the new version from GitHub Releases
pip install --upgrade https://github.com/Anaxiatech-AB/crackz/releases/download/vNEW_VERSION/crackz-NEW_VERSION-py3-none-any.whl

# Or if using a private index:
pip install -U --index-url https://pypi.company.com/simple/ crackz
```

9. Uninstall

```
pip uninstall crackz
```

(You may remove generated project folders / logs manually.)

10. Post-Install Sanity Checks

```
crackz --version
crackz init --name sanity-test
crackz detect run --project sanity-test  # (Will use default config; may download model weights)
```

11. Common Issues

Symptom	Cause	Fix
`torch.__version__` shows 2.x.x without +cu suffix	Installed CPU PyTorch instead of CUDA	Uninstall torch and reinstall with --index-url (see 'CUDA Installation (Two Steps Required)' in Section 3)
torch / torchvision mismatch	Mixed CUDA versions	Reinstall both from same index (see 'CUDA Installation (Two Steps Required)' in Section 3)
`No module named crackz`	Wrong interpreter / venv	Activate correct virtual environment
`CUDA not available` even with CUDA wheel	CPU PyTorch installed	Check `torch.__version__` for +cu suffix; reinstall PyTorch with correct index URL
`ModuleNotFoundError: No module named 'torch'` with CUDA wheel	Forgot to install PyTorch first	CUDA wheels don't include PyTorch - see 'CUDA Installation (Two Steps Required)' in Section 3
Slow first run	Model + dataset cache warmup	Subsequent runs faster
Permission denied writing logs	Read-only directory	Set `CRACKZ_LOG_DIR` env var to writable path
Version shows unexpected version string (e.g., hardcoded or fallback value)	Package metadata not found or corrupted	Reinstall package: `pip install --force-reinstall crackz-VERSION.whl`; verify installation and metadata. If using a hardcoded fallback, update the version in `pyproject.toml` or the relevant source file.

12. GPU/CUDA Usage FAQ

How do I know if CUDA is working?

After installing with CUDA support, verify it's working:

```
# Check if CUDA is available
python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}')"
python -c "import torch; print(f'CUDA device: {torch.cuda.get_device_name(0)}')"
```

Expected output:

```
CUDA available: True
CUDA device: NVIDIA GeForce RTX 3080
```

How do I start the CUDA version?

You don't need to do anything special! Once you've installed Crackz with CUDA support (see Section 3), it will automatically detect and use your GPU when available.

The same commands work for both CPU and GPU:

```
# These commands automatically use GPU if available:
crackz detect train --project my_project
crackz detect run --project my_project
crackz-gui  # GUI will use GPU automatically
```

How do I force GPU or CPU usage?

Crackz automatically uses your GPU if available, but you can override this:

In the GUI: 1. Open Project → Settings 2. Go to the "AI Model" tab 3. Set **Device** to: - `cuda` - Force GPU (fails if no GPU) - `mps` - Force Apple Metal (macOS only) - `cpu` - Force CPU (slower but works everywhere)

In the CLI:

```
# Force GPU usage (will error if CUDA not available)
crackz detect train --project my_project --device cuda

# Force CPU usage (useful for debugging)
crackz detect train --project my_project --device cpu

# Let Crackz auto-detect (default)
crackz detect train --project my_project
```

In project YAML:

Edit `<project>/<project>_project.yaml` :

```
ai:
  model:
    device: cuda  # or cpu, or mps
```

How do I verify GPU is being used during training?

Check the logs:

```
crackz detect train --project my_project
```

Look for output like:

```
GPU available: True, used: True
TPU available: False, using: 0 TPU cores
IPU available: False, using: 0 IPUs
HPU available: False, using: 0 HPUs
```

Monitor GPU usage:

```
# NVIDIA GPUs
watch -n 1 nvidia-smi

# Should show Python process using GPU memory and compute
```

What CUDA version do I need?

For Windows: - Install CUDA 11.8 (most compatible) - Download `crackz_cu118-VERSION-py3-none-any.whl` from releases

For Linux: - Modern GPUs: CUDA 12.8 (recommended) - Older GPUs: CUDA 11.8 - Download `crackz_cu128-VERSION-py3-none-any.whl` or `crackz_cu118-VERSION-py3-none-any.whl`

Check your installed CUDA version:

```
nvcc --version  # If CUDA toolkit installed
nvidia-smi  # Shows driver version (different from CUDA toolkit)
```

Note: You can use PyTorch with CUDA support even without installing the CUDA toolkit - PyTorch bundles what it needs!

Troubleshooting GPU Issues

"CUDA not available" but I have a GPU:

1. **First, verify you installed the CUDA variant of PyTorch (most common mistake):**

   ```
   python -c "import torch; print(torch.__version__)"
   # Should show: 2.5.0+cu118 or 2.5.0+cu128
   # If it shows just 2.5.0, you have CPU-only version!
   ```

2. **If you have CPU version, reinstall PyTorch with CUDA support:**

   ```
   pip uninstall torch torchvision
   pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118
   ```

3. **Then verify CUDA is available:**

   ```
   python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}')"
   ```

"CUDA out of memory" errors:

Reduce batch size in project settings:

```
# GUI: Project → Settings → Training → Batch Size
# CLI:
crackz detect train --project my_project --batch-size 2
```

GPU slower than expected:

- Check GPU isn't throttling: `nvidia-smi` (temperature, power limit)
- Ensure you're not running other GPU applications
- Try increasing batch size (GPU utilization might be low)

Performance Expectations

Training speed comparison (approximate): - CPU (8-core): ~5-10 images/second - GPU (RTX 3060): ~30-50 images/second - GPU (RTX 4090): ~100-200 images/second

Detection speed comparison: - CPU: ~2-5 images/second - GPU: ~20-100 images/second (depending on model/GPU)

If you're not seeing these speeds, verify GPU is actually being used (see above).

13. Environment Variables (runtime)

Variable	Purpose	Default
CRACKZ_DATABASE_URL	Postgres connection string (required before first use)	*(none — must be set)*
CRACKZ_LOG_DIR	Log directory	logs
PYTHONUNBUFFERED	Immediate stdout	1

Database required before first use

Set `CRACKZ_DATABASE_URL` to a reachable Postgres instance, then run `crackz db init` (or `alembic upgrade head`) before issuing other `crackz` commands. See Database setup for setup options.

14. Security / Integrity Notes

Example override:

```
CRACKZ_LOG_DIR=my_logs crackz detect run --project myproj
```

(On Windows PowerShell: `$Env:CRACKZ_LOG_DIR='my_logs'` before running the command.)

14. Security / Integrity Notes

- Use `pip install --require-hashes -r requirements.txt` if you export a locked set of dependencies.
- Prefer virtual environments to isolate dependencies.

15. Next Steps

- Follow the Getting Started guide to set up a project.
- Review Configuration for dataset paths & model options.
- Consult CLI Cookbook for common command patterns.

16. Migrating Existing Projects After Upgrade

Older projects (created before schema_version field) can be upgraded in-place:

```
crackz migrate --project ./my_project        # applies changes + backup
crackz migrate --project ./my_project --dry-run  # preview only
```

In CI set `CRACKZ_AUTO_MIGRATE=1` to apply automatically when commands load the project.

Need a Docker or GUI setup instead? See the dedicated Docker and GUI pages.

Quick Start

Getting Started with Crackz

Welcome to Crackz! This guide will help you get up and running quickly, whether you're an infrastructure manager, inspector, or technical user.

What is Crackz?

Crackz is an AI-powered crack detection and analysis solution designed for harbor facilities, bridges, and critical infrastructure. It combines cutting-edge deep learning with a production-ready workflow to help you:

- **Reduce inspection costs** by up to 70%
- **Improve safety** by minimizing physical inspections in hazardous locations
- **Detect issues early** before they become critical and costly
- **Ensure compliance** with complete documentation trails

Who Should Use Crackz?

For Infrastructure Managers & Asset Owners

- Process thousands of images in hours instead of weeks
- Identify structural issues before they become critical
- Complete audit documentation for regulatory requirements
- Proven ROI: typical savings of $25,000-$75,000 annually per inspection program

For Technical Teams

- Reproducible project structure with self-contained folders
- Fast experimentation with one-line training overrides
- Scriptable CLI and modern GUI (dark mode by default)
- Container-friendly with Docker images for all deployment scenarios

Quick Start (GUI First, ~5 Minutes)

1. Install Crackz

Choose the installation method that works best for you:

Option A: Simple Install (Recommended)

```
# Install UV package manager
curl -LsSf https://astral.sh/uv/install.sh | sh  # macOS/Linux
# OR
irm https://astral.sh/uv/install.ps1 | iex       # Windows PowerShell

# Authenticate with GitHub
gh auth login

# Download and install Crackz
gh release download --repo anaxiatech/crackz --pattern '*.whl'
uv pip install crackz-*.whl

# Verify installation
crackz --version
```

Option B: Using Docker (No Python Required)

```
# Pull the Docker image
docker pull ghcr.io/anaxiatech-ab/crackz:latest

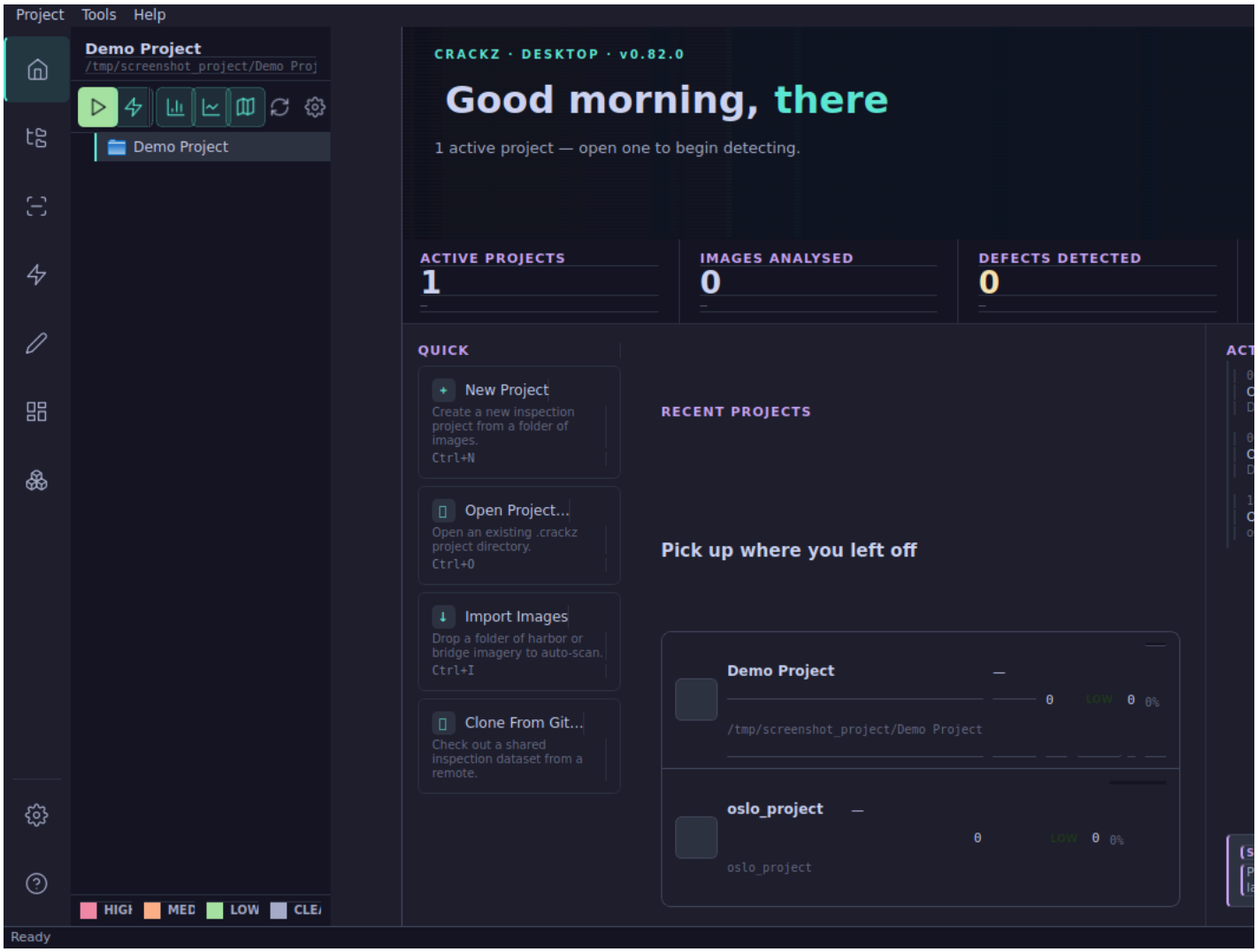
# Verify it works
docker run --rm ghcr.io/anaxiatech-ab/crackz:latest --version
```

Need more options? See the Installation Guide for detailed instructions, GPU setup, and standalone executables.

2. Launch the App

```
crackz-gui
```

On first launch, start from the welcome page:



3. Create Your First Project

In the GUI:

1. Select **New Project**.
2. Enter a project name (for example: `my-first-project`).
3. Choose an annotation template:
4. **Crack Only** (default, binary)
5. **Harbor Defects** (multi-class)
6. **Survey General** (multi-class)
7. Finish the wizard.

4. Import Your Images

In the GUI:

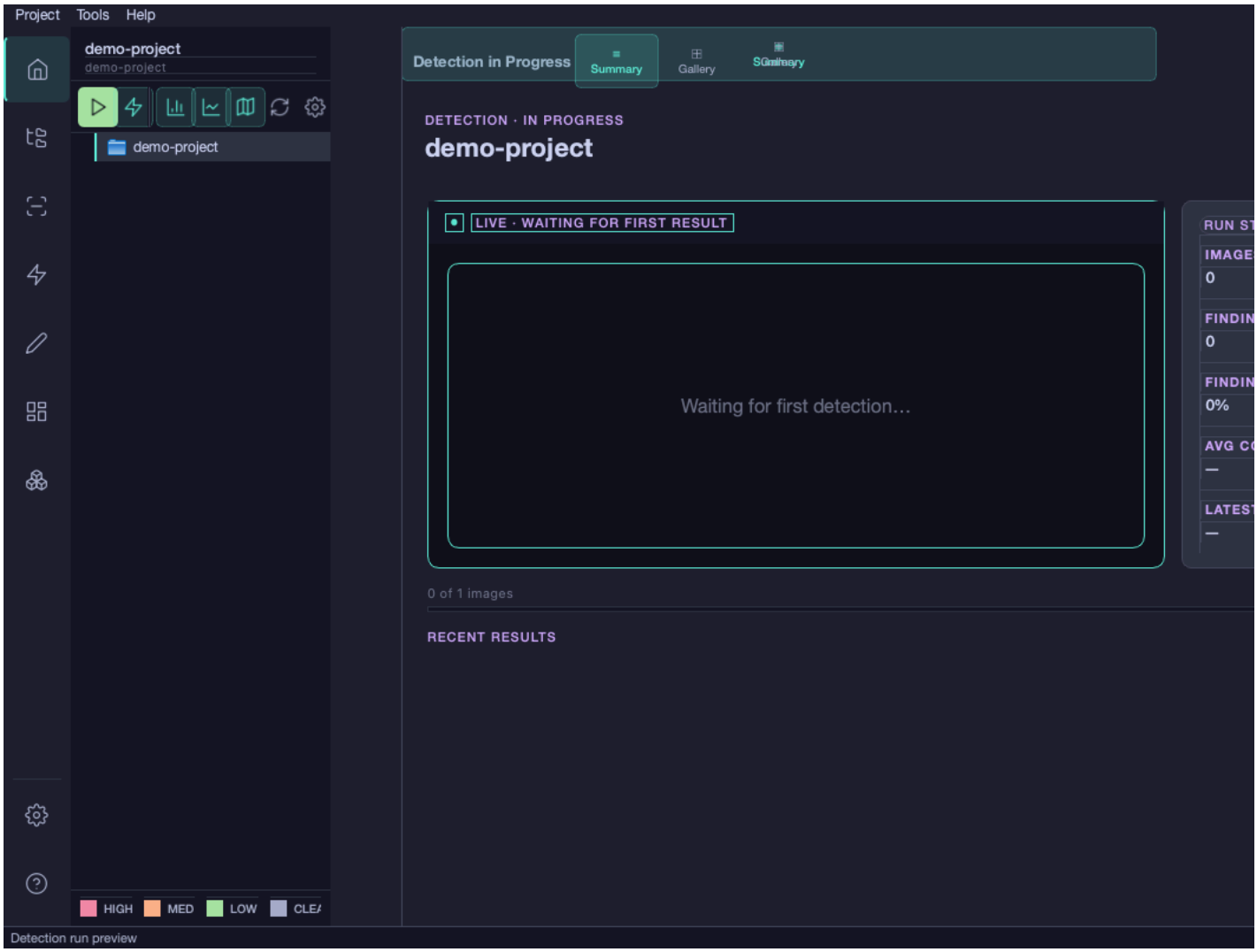
1. Open your project.
2. Go to **Import Images**.
3. Select your image folder.
4. Import to **Raw** split (or split to train/val/test if you're preparing training data).



5. Run Crack Detection

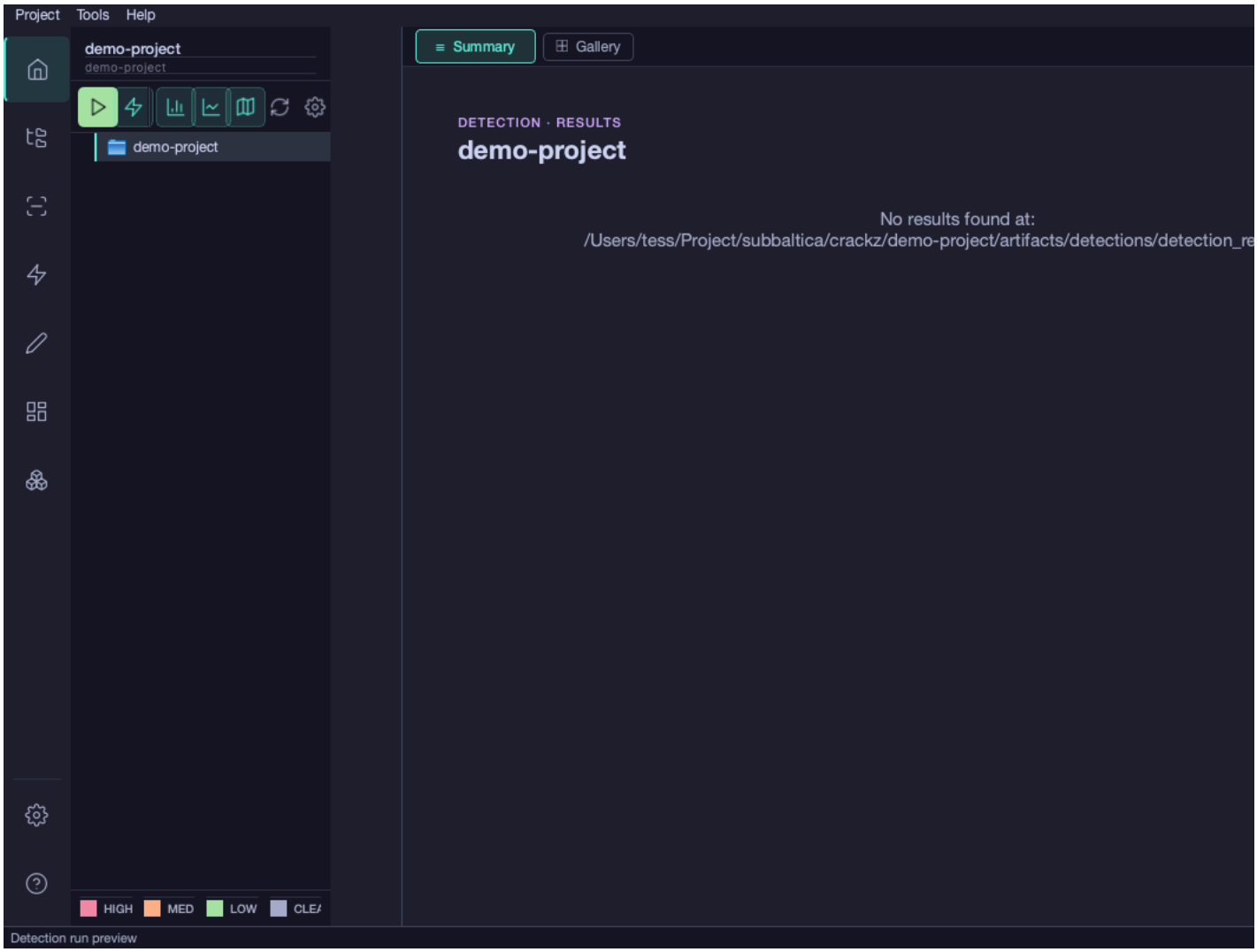
In the GUI:

1. Open **Detection**.
2. Start a run with the current model.
3. Wait for processing to finish (progress and logs update in-app).



6. Review Results

Use the results gallery and summary views to inspect detections, confidence, and image-level outputs.



Result files are also saved to: - `my-first-project/artifacts/detections/visualizations/` - Annotated images - `my-first-project/artifacts/detections/detection_results.json` - Detection statistics

Two Workflows: Pre-Trained vs Custom Training

Workflow 1: Use Pre-Trained Model (Fastest)

Best for: Quick deployment, standard crack detection, proof-of-concept

GUI flow:

1. Create/open project.
2. Import inspection images to `raw`.
3. Ensure a valid model/checkpoint is selected in project settings.
4. Run detection and review results in the gallery.

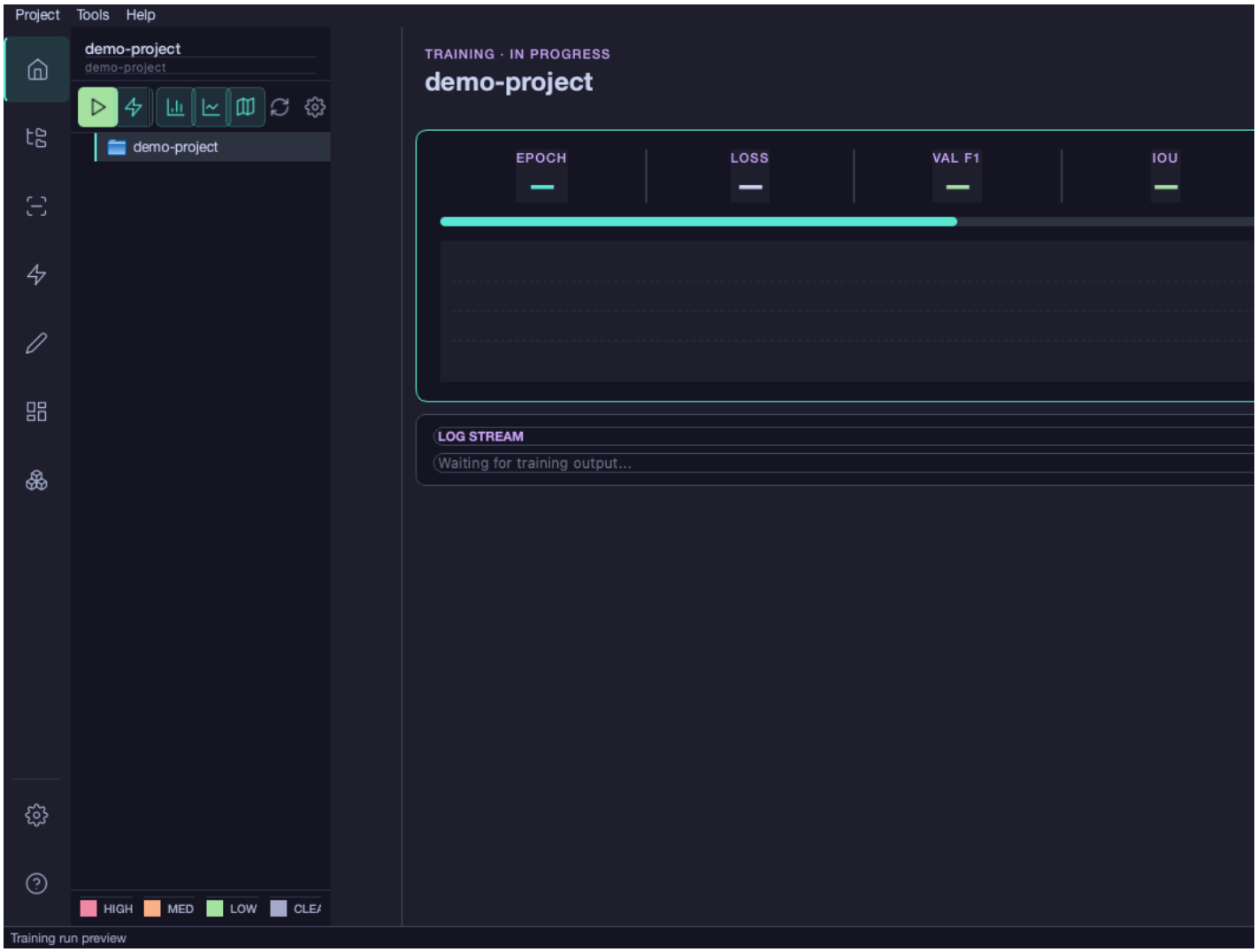
Workflow 2: Train Custom Model (Most Accurate)

Best for: Domain-specific imagery, unique defect types, maximum accuracy

Requirements: Labeled training data (images with corresponding crack masks)

GUI flow:

1. Import labeled images and masks into `train` and `val`.
2. Open training settings and select model/backbone preferences.
3. Start a training run and monitor metrics in-app.
4. Use the trained model for detection runs.



Need training data? See the Customer Guide for detailed instructions on creating masks with CVAT, Labelme, or other annotation tools.

Prefer the Command Line?

If you automate workflows or run batch jobs, use the dedicated CLI Cookbook. It contains command patterns for:

- Project initialization
- Image/mask import
- Training and detection
- Cleaning and maintenance
- Troubleshooting flags and reproducible runs

Understanding Project Structure

After initialization, your project looks like this:

```
my-first-project/
├── images/
│   ├── raw/          # Original images for detection
│   ├── train/        # Training images (if training a model)
│   ├── val/          # Validation images
│   └── test/         # Test images
├── masks/
│   ├── train/        # Ground truth masks for training
│   ├── val/          # Validation masks
│   └── test/         # Test masks
├── artifacts/
│   ├── models/       # Model checkpoints (best.ckpt, last.ckpt)
│   └── runtime/      # Detection results (images + JSON), training metrics
```

```
├── logs/           # Project-specific logs
└── my-first-project_project.yaml  # Configuration file (schema v6)
```

Common Next Steps

Adjust Detection Sensitivity

If you're getting too many false positives or missing real cracks, adjust the threshold:

Edit `my-first-project/my-first-project_project.yaml`:

```
ai:
  model:
    threshold: 0.5  # Increase to 0.7 for fewer false positives
                    # Decrease to 0.3 to catch more cracks
```

Batch Process Multiple Projects

```
# Process several inspection campaigns
for project in site-A site-B site-C; do
    crackz detect run --project $project
done
```

Use Docker for Consistent Environments

```
# Run detection in container
docker run --rm \
  -v $(pwd)/my-first-project:/home/app \
  ghcr.io/anaxiatech-ab/crackz:latest \
  detect run --project /home/app
```

Deploy on Azure for Large Datasets

For processing thousands of images across multiple VMs, see the Azure Batch Guide.

Return on Investment (ROI)

Typical Cost Savings:

Scenario	Manual Inspection	With Crackz	Annual Savings
Small facility (1,000 images/year)	$6,000-$18,000	$1,000-$3,000	$5,000-$15,000
Medium facility (5,000 images/year)	$30,000-$90,000	$5,000-$15,000	$25,000-$75,000
Large facility (10,000+ images/year)	$60,000-$180,000	$10,000-$30,000	$50,000-$150,000

Additional Benefits: - Early detection prevents costly emergency repairs - Extended asset lifespan through proactive maintenance - Reduced liability with documented inspection protocols - Faster regulatory approval with comprehensive documentation

Troubleshooting

Installation Issues

"python: command not found" - Install Python 3.11+ from python.org - Check "Add Python to PATH" during installation

"CUDA not available" (for GPU users)

```
# Verify CUDA installation
nvidia-smi

# Check PyTorch CUDA support
python -c "import torch; print(torch.cuda.is_available())"

# Reinstall with CUDA support (see Installation Guide)
```

Detection Issues

No images found - Verify images are in `images/raw/` directory - Check file extensions: `.jpg`, `.jpeg`, `.png`, `.bmp`, `.tif`, `.tiff`

Detection results look wrong - Adjust threshold in `project.yaml` (see "Adjust Detection Sensitivity" above) - Ensure images are similar to training data - Consider training a custom model on your specific data

Out of memory during training

```
# Reduce batch size
crackz detect train --project my-project --batch-size 1

# Or reduce image size during import
crackz import-images --project my-project --src ./data --type train --size 256 256
```

Getting Help

- **GUI Guide**: Desktop workflow documentation
- **CLI Reference**: Command-line cookbook
- **GUI Guide**: Graphical interface
- **Configuration**: Detailed configuration options
- **Installation**: Advanced installation methods

Commercial Support: - Email: theresia.sofia.snow@gmail.com - For licensing, proof-of-concept projects, and custom training

Next Steps

For End Users: 1. Complete this getting started guide 2. Read the GUI Guide for desktop workflows 3. Explore the CLI Cookbook for common patterns 4. Learn about Configuration options

For Developers: 1. Review Architecture for system design 2. Check the API Reference for programmatic access 3. Read Contributing Guidelines to contribute

For Commercial Deployment: 1. Read the Customer Guide for detailed use cases 2. Review Distribution Guide for packaging options 3. Contact theresia.sofia.snow@gmail.com for licensing

---

Ready to transform your inspection program? Start with `crackz init --name my-first-project` and follow the steps above!

GUI Overview

GUI Guide

The Crackz GUI (`crackz-gui`) is a modern desktop application built with PySide6 (Qt 6), providing a visual interface for project management, image import, model training, and crack detection. It features dark mode via Catppuccin Mocha theming and real-time updates during long-running operations.

Installation & Launch

Launch the GUI

```
# After installing Crackz
crackz-gui
```

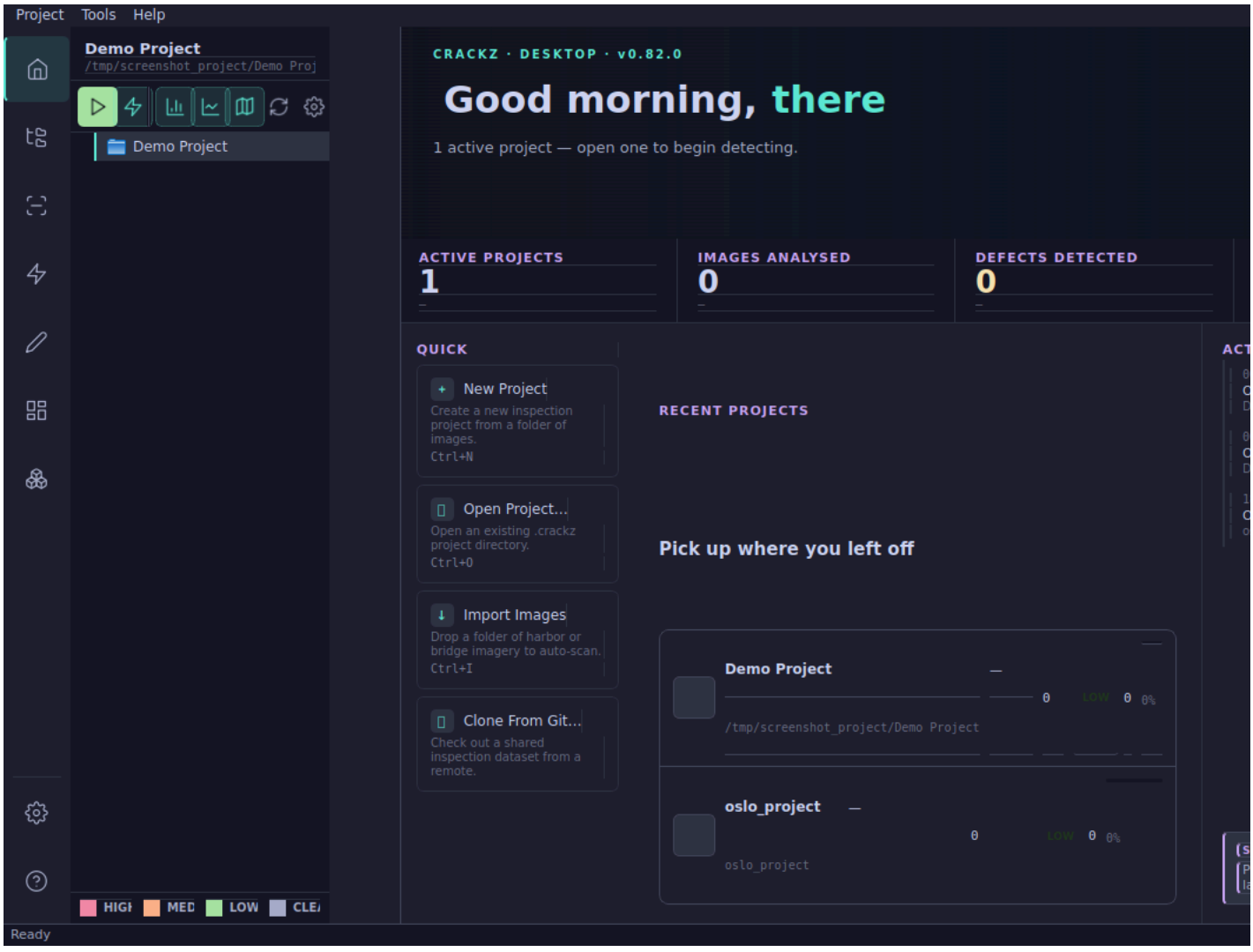
Or use the standalone executable from PyInstaller builds: - **Windows**: Extract `crackz-gui-<version>-windows.zip` , then double-click `crackz-gui.exe` - **macOS/Linux**: Extract the `.tar.gz` , then run `./crackz-gui`

These archives are only attached to draft and prerelease releases; see the availability note in Installation.

> **Note:** Standalone executables are provided via GitHub Releases for major versions. If you do not see the executable for your platform, you may need to build it yourself or use the standard Python installation.

First Launch

On first launch, you'll see the welcome screen with options to: - Create a new project (Cmd/Ctrl+N) - Open an existing project (Cmd/Ctrl+O) - Access recent projects from the list



Project Management

Creating a New Project

1. Click **File → New Project** (Cmd/Ctrl+N) or use the welcome screen
2. Enter a project name and choose a base folder
3. (Optional) Select an **annotation template** to pre-configure defect categories:
4. **Crack Only** – Single binary crack category (default for simple projects)
5. **Harbor Defects** – Crack, Spall, Corrosion, Joint Damage (multi-class harbor inspection)
6. **Survey General** – Crack, Spall, Corrosion, Water Intrusion, Biological Growth
7. Click **Create** — the GUI creates the standard project structure automatically

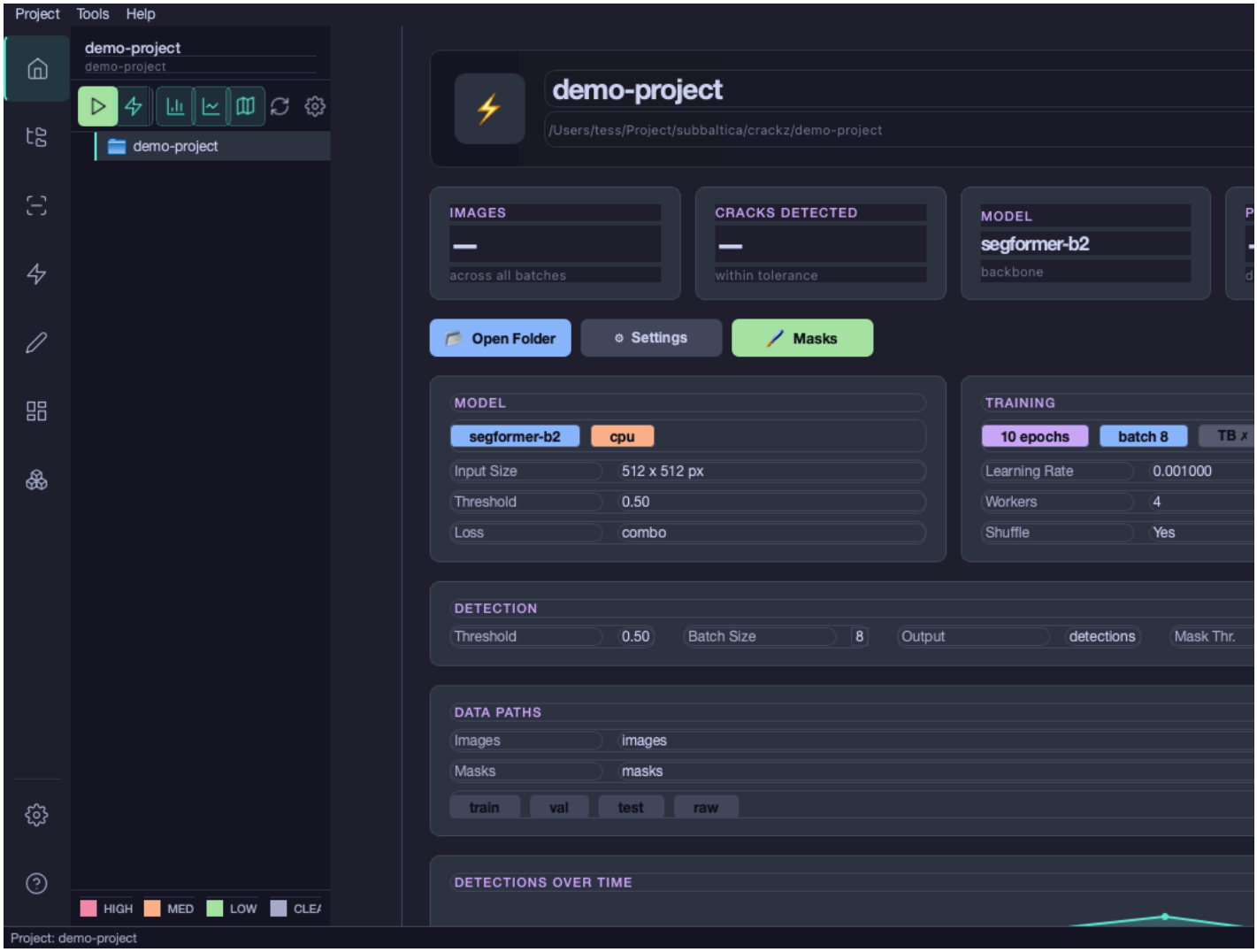
   Custom categories: After creation, open **Project → Settings → Categories** to edit the annotation schema, rename categories, or add new ones before importing masks.

Opening a Project

1. Click **File → Open Project** (Cmd/Ctrl+O)
2. Navigate to the project folder containing `project.yaml`
3. Select the folder and click **Open**

Recent projects appear in the welcome screen and **File → Recent Projects** menu for quick access.

After opening a project, Crackz shows a summary dashboard with the active model, training setup, detection settings, and data paths.



Automatic Migration Support

When opening a project with an outdated configuration or file structure, the GUI automatically detects migration needs and provides user-friendly dialogs:

Schema Migration

- **Automatic Detection**: When project schema is outdated (v0, v1, v2), a dialog prompts for migration
- **Safe Process**: Backup is always created before migration
- **User Choice**: Option to migrate now, skip, or cancel

File Migration (NEW)

- **Structure Detection**: Identifies when artifacts directory uses old structure (checkpoints/, training/, etc.)
- **Informational Dialog**: Explains benefits of new organization and safety measures
- **User Options**: "Migrate" (reorganizes files), "Skip" (continue with old structure), or "Cancel"
- **Progress Tracking**: Shows migration progress with detailed feedback
- **Error Handling**: Automatic rollback if migration fails

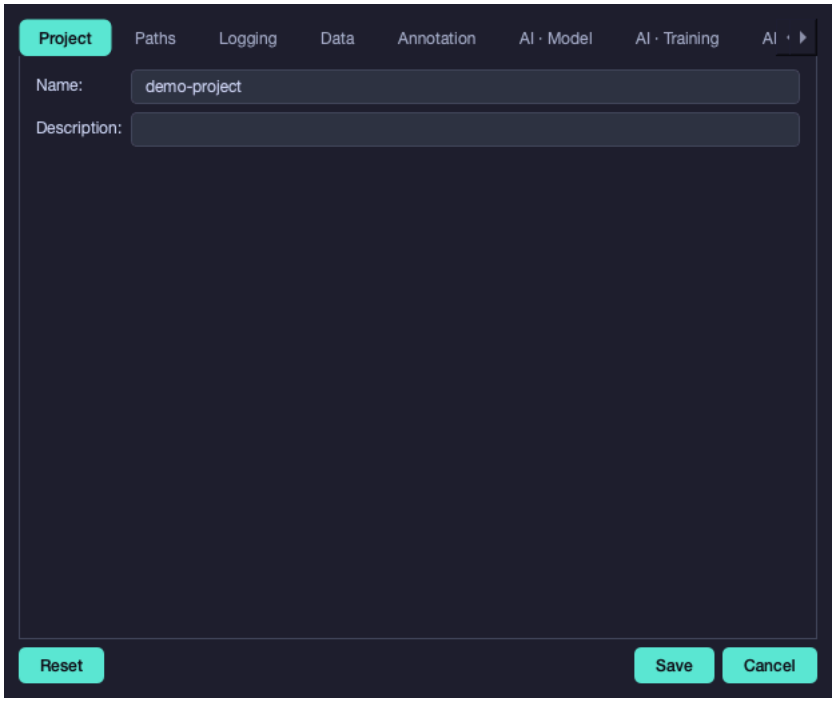
This ensures your projects always use the latest, most organized structure without requiring manual CLI commands.

Project Settings

Access project settings via **Project → Settings** or by clicking the settings icon. Settings are organized into tabs:

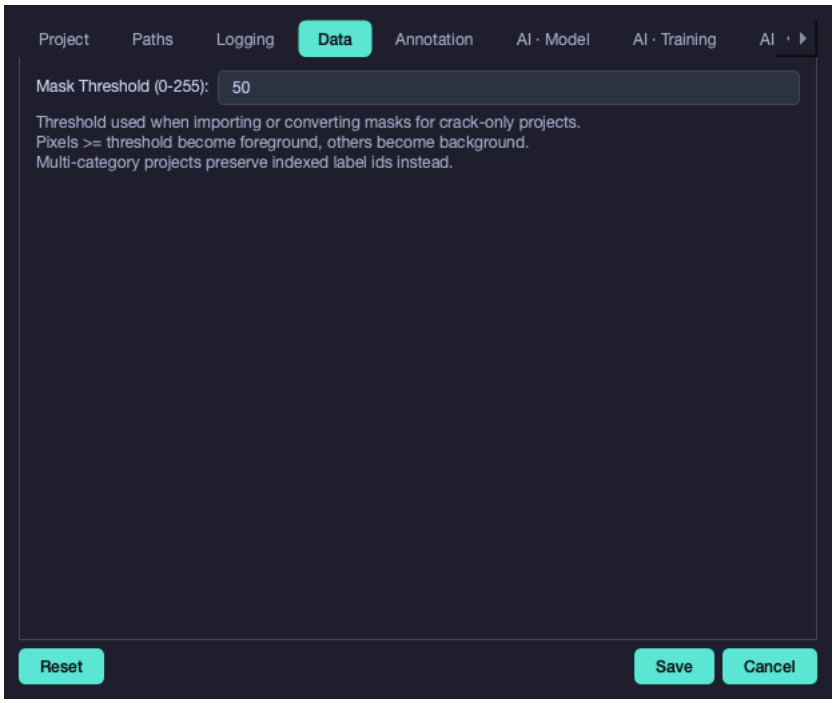
Project Tab

Basic project information and metadata.



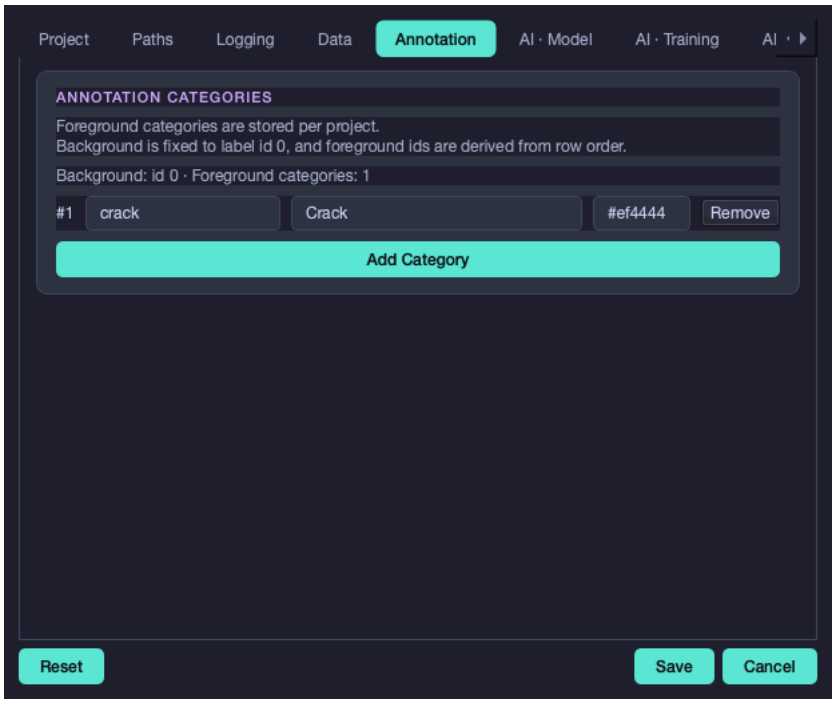
Data Tab

Configure data preprocessing options (image size, quality, normalization).



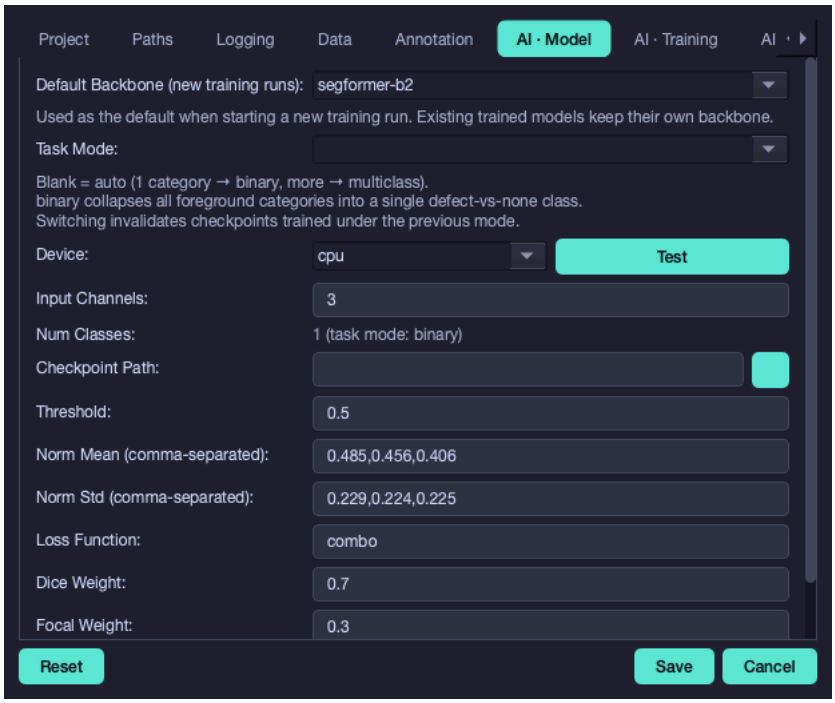
AI Model Tab

Select model architecture, encoder, and advanced model options.



Training Tab

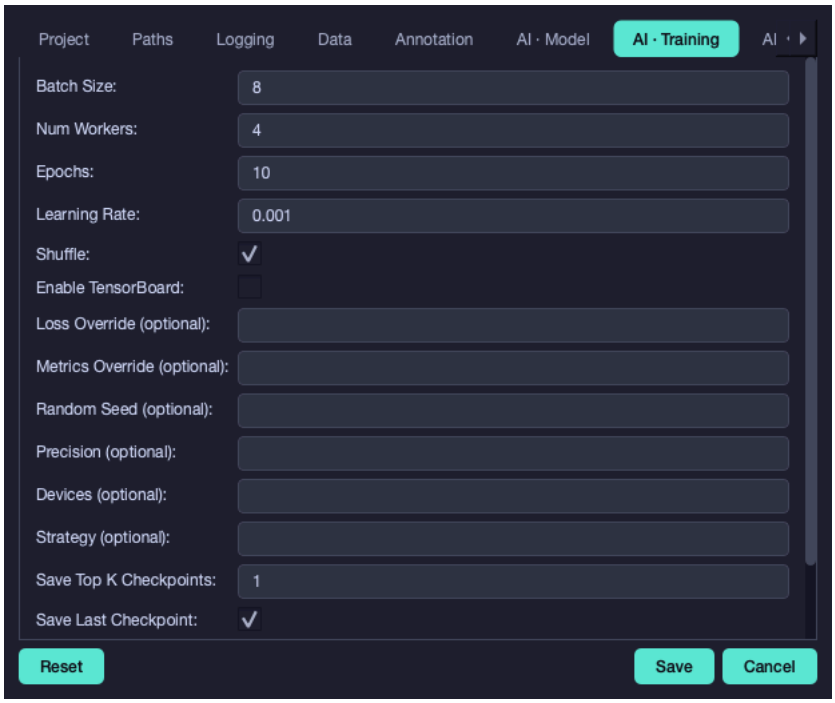
Configure training hyperparameters (epochs, batch size, learning rate, optimizer).



Detection Tab

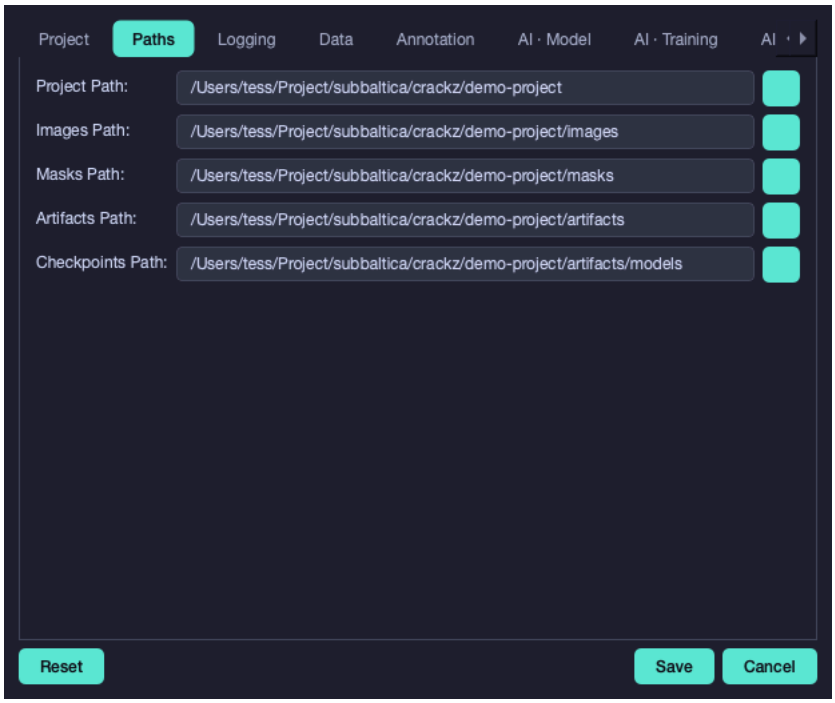
Configure detection-specific options:

- **Batch Size**: Number of images processed per detection batch
- **Severity Thresholds**: Customize crack severity categorization
- Min Confidence (default 0.005): Minimum crack area % to save detections
- Medium Confidence (default 0.02): "Probable Crack" threshold (2% area)
- High Confidence (default 0.05): "Crack Detected" threshold (5% area)
- **Min Crack Area**: Suppress detections smaller than N pixels (speckle noise filter)



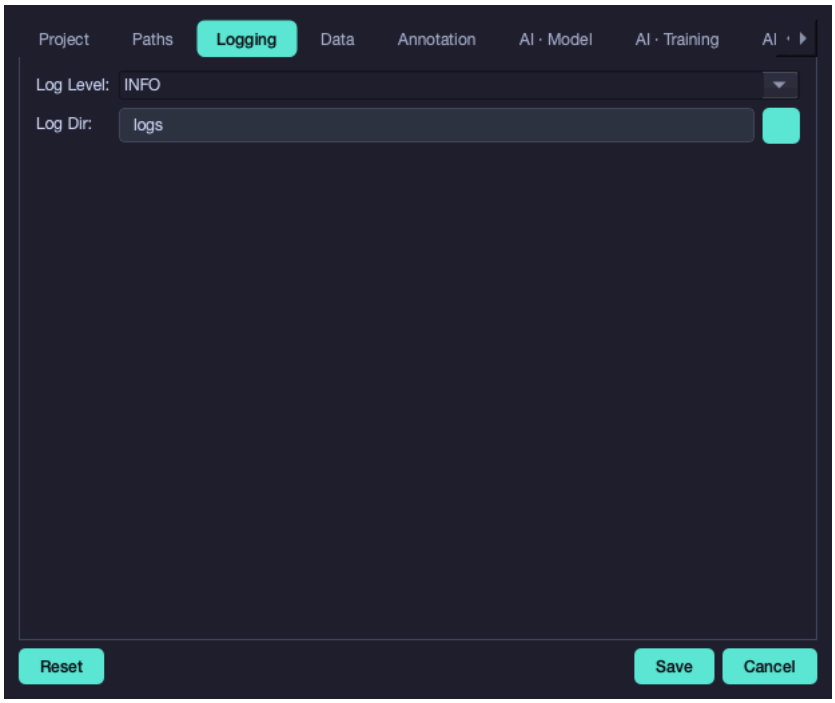
Paths Tab

View and customize project directory structure.



Logging Tab

Configure logging behavior (verbosity, log rotation, paths).



Categories Tab

Define and manage the **project annotation schema** — the set of defect categories used for mask editing, training, and detection results.

What you can do: - **Add categories**: Click **Add Category** to append a new row. Enter a slug-like key (e.g. `crack`, `spall`), a display name, and a hex color for overlays. - **Remove categories**: Click **Remove** on any row. At least one foreground category is required. - **Edit color**: Type a `#RRGGBB` hex color for each category — this controls the overlay chip in the mask editor and detection galleries. - **Reorder categories**: Move rows up/down; Crackz re-assigns contiguous ids automatically.

The annotation note at the bottom shows the derived task mode and number of model output classes:

- **1 category** → `binary` task mode, 1 model output channel (sigmoid)

- **2+ categories** → `multiclass` task mode, `1 + N` output channels (softmax)

Compatibility warning: Changing categories after training will invalidate existing checkpoints. Crackz will warn you and block the save if incompatible checkpoints exist. When existing masks are present, Crackz offers to **remap mask label IDs** automatically so your annotation data stays consistent with the new schema.

> **Tip:** Use the New Project dialog to choose from built-in annotation templates (Crack Only, Harbor Defects, Survey General) when starting a fresh project — this avoids having to configure categories manually.

Image Import

Import Images

Access image import via **Project → Import Images…**.

Import with Automatic Split

Automatically distribute images across train/val/test splits:

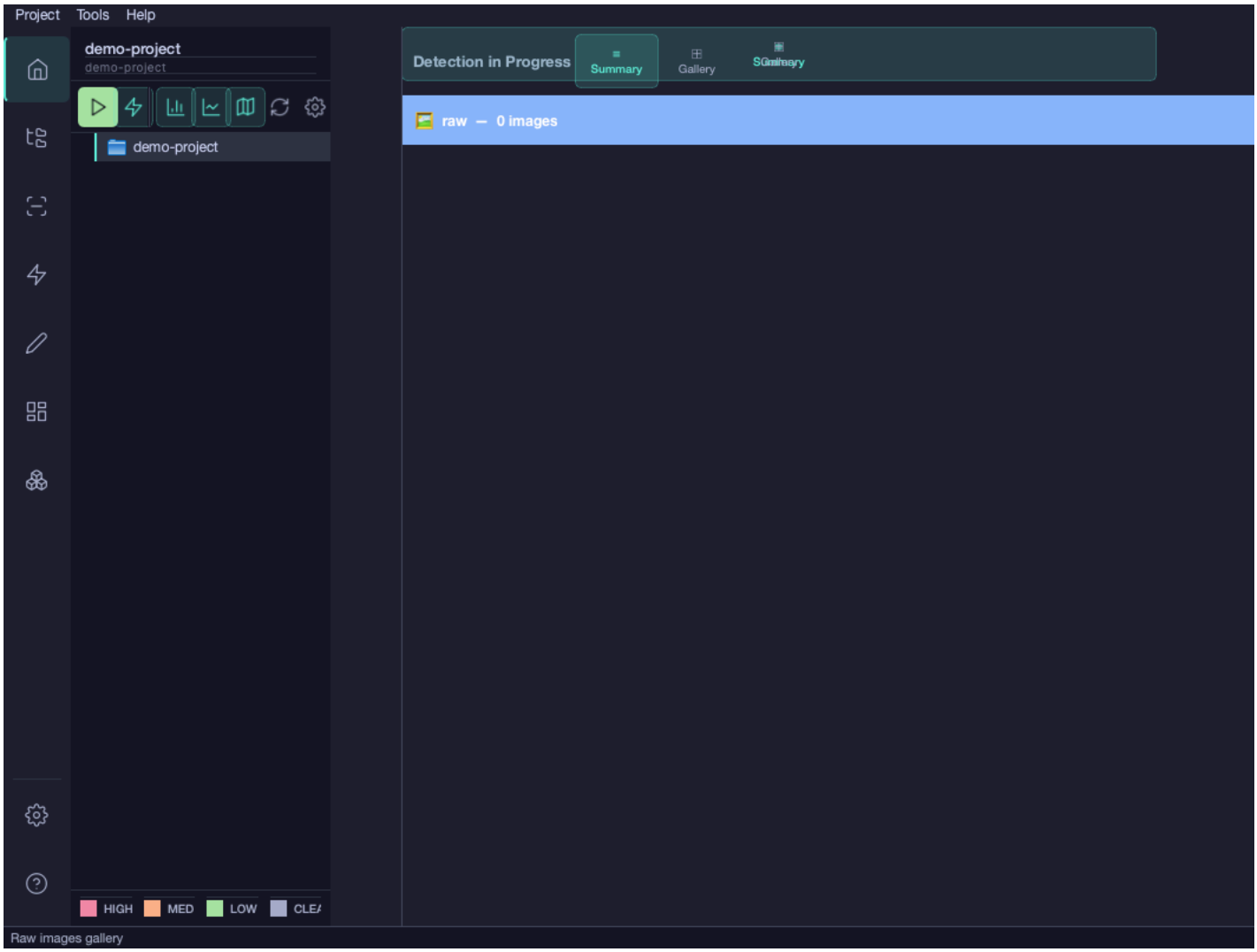
1. Click **Project → Import Images…**
2. Select source directory containing images (with optional `masks/` subdirectory)
3. Set split ratios (default: 70% train, 15% val, 15% test)
4. Optionally set a random seed for reproducible splits
5. Configure image size and JPEG quality
6. Click **Import**

Progress is shown with a real-time progress bar.



Viewing Imported Images

After import, view images in each split via the project tree:



Creating and Editing Masks

Mask Editor Overview

The interactive mask editor allows you to create and edit defect masks for training data.

- **Binary projects** (one category): Masks are black/white images where white pixels (value 255) indicate the defect and black pixels (0) indicate background.
- **Multi-class projects** (two or more categories): Masks use indexed color encoding — pixel value 0 is background; values 1, 2, 3... match the category IDs from your annotation schema. The toolbar shows a **category selector** (color-coded chips) so you can switch the active label before painting.

Opening the Mask Editor

There are three ways to open the mask editor:

1. Context Menu (Browser Tree) - Right-click any image file in the `images/` folder (train/val/test/raw) - Select 🎨 **Edit Mask** from the context menu

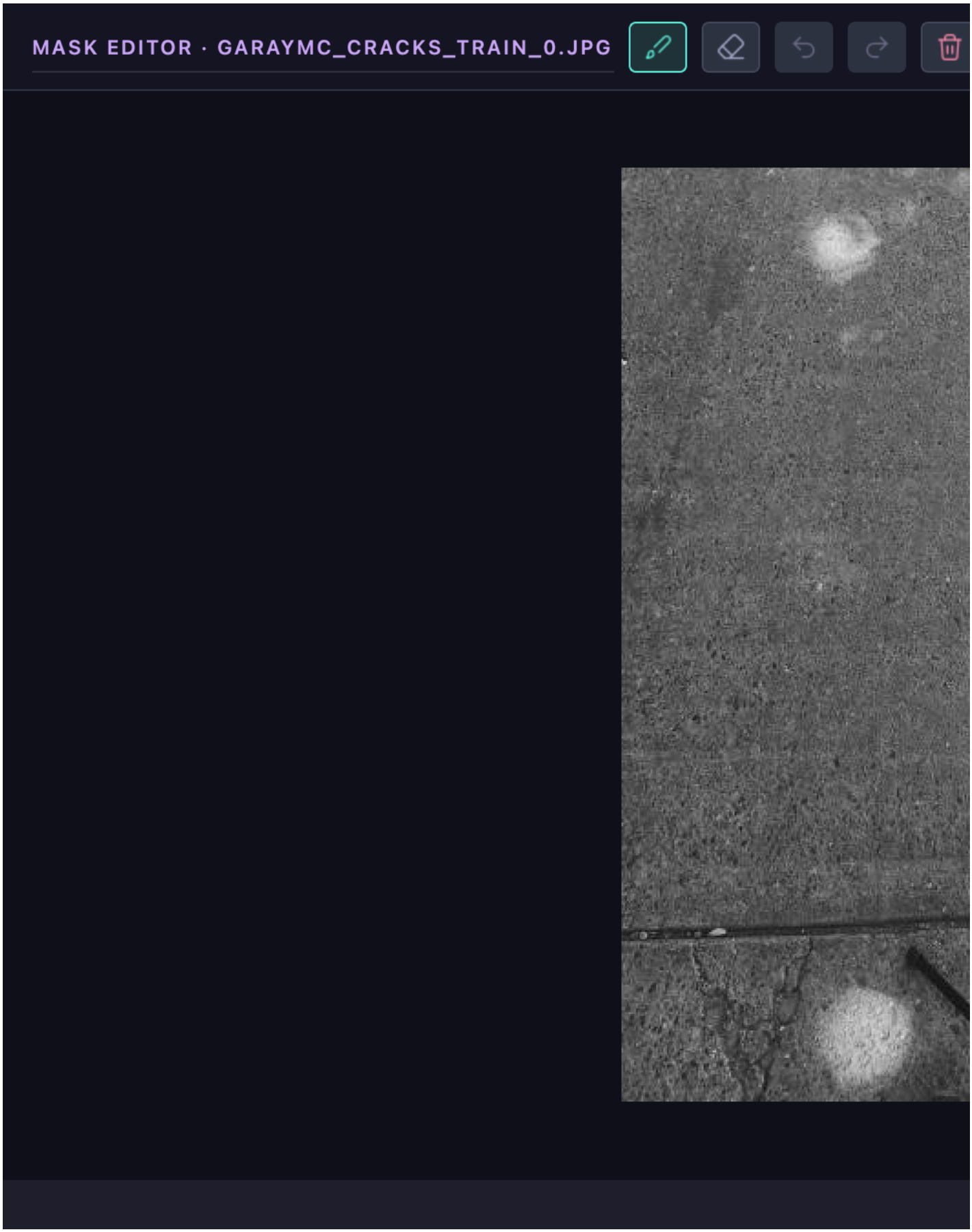
2. Badge Icon (Thumbnail View) - Browse to any images folder in the project tree - Click the 🎨 badge shown below image thumbnails - Badge appears only for images in the `images/` directory

3. Image Preview (Right Pane) - Click any image in the browser to open its preview - Click the 🎨 **Edit Mask** button in the Actions section - Available for all images in the `images/` directory

Using the Mask Editor

The mask editor provides an intuitive interface for painting crack masks with professional drawing tools and visual feedback.

User Interface



The mask editor window features a **two-row toolbar** for optimal layout and control visibility:

Row 1: Tools and Actions - 🎨 **Title** - Shows current image name - 🖌️ **Brush Tool** - Paint crack areas (white/255) - 🧹 **Eraser Tool** - Remove painted areas (black/0) - ↶ **Undo** - Undo last action (up to 20 steps) - ↷ **Redo** - Redo previously undone action - 🗑️ **Clear** - Reset entire mask (with confirmation) - 🔄 **Propagate to Similar** - Find and annotate similar images (right-aligned)

Row 2: Settings - **Brush Size Slider** - Adjust brush size (5-100 pixels) - **Opacity Slider** - Control mask overlay transparency (0.1-1.0)

Bottom Toolbar: - 💾 **Save Mask** - Save as binary PNG to `masks/` directory - × **Cancel** - Close without saving changes

Visual Feedback

The mask editor provides **real-time visual cursor feedback** to help you paint precisely:

- **Brush Cursor**: A colored circle follows your mouse cursor, showing the exact brush size
- **Green circle** when Brush tool is active
- **Red circle** when Eraser tool is active
- **Dynamic sizing**: Circle size updates immediately when you adjust the Brush Size slider
- **Preview before painting**: See exactly where your stroke will go before clicking

Controls and Keyboard Shortcuts

Action	Mouse	Keyboard	Description
Paint	Left-click + drag	—	Paint crack areas with brush
Erase	Left-click + drag (eraser mode)	—	Remove painted areas
Switch to Brush	Click 🖌️ button	`B`	Activate brush tool
Switch to Eraser	Click 🧹 button	`E`	Activate eraser tool
Undo	Click ↶ button	`Ctrl+Z` / `Cmd+Z`	Undo last action
Redo	Click ↷ button	`Ctrl+Y` / `Cmd+Shift+Z`	Redo undone action
Clear All	Click 🗑️ button	—	Reset entire mask (confirms first)
Save	Click 💾 button	`Ctrl+S` / `Cmd+S`	Save mask and close
Cancel	Click × button	`Esc`	Close without saving

Workflow Tips

For precise annotation: 1. **Start with large brush**: Use 50-100px to quickly cover large crack areas 2. **Reduce for details**: Switch to 5-20px for fine cracks and edges 3. **Adjust opacity**: Lower opacity (0.3-0.5) to see the original image through the mask 4. **Use undo liberally**: Don't worry about mistakes—you have 20 undo levels 5. **Watch the cursor**: The colored circle helps you plan your strokes before clicking

For efficient workflow: - **Save frequently**: Click Save Mask after completing each annotation - **Use Clear for restarts**: If you need to start over, use Clear instead of manually erasing everything - **Take breaks**: Long annotation sessions can cause fatigue—save and return later

For quality masks: - **Paint smoothly**: Slow, deliberate strokes produce cleaner masks - **Cover cracks fully**: Make sure the entire crack width is painted white - **Include fine details**: Don't skip hairline cracks—they matter for training - **Check at low opacity**: Lower opacity to 0.2-0.3 to verify you didn't miss any cracks

Mask File Structure

Masks are automatically saved to match your image structure:

```
project/
├── images/
│   └── train/
│       └── crack_001.jpg        # Original image
└── masks/
    └── train/
        └── crack_001.png        # Indexed mask (auto-created)
```

Mask Properties (multi-class projects): - **Format**: PNG (8-bit grayscale) - **Values**: 0 = background, 1 = first category, 2 = second category, ... - **Size**: Same dimensions as source image - **Naming**: Same filename as image (with .png extension)

Mask Properties (binary / crack-only projects): - **Values**: 0 = background / no crack, 255 = crack foreground

Binary Mask Conversion

If you have existing masks with grayscale values, use the **Masks → Convert** command to convert them to binary format required for training.

Mask Gallery

The **Mask Gallery** provides a project-wide overview of all annotation masks, organized by data split. Access it from the sidebar (mask icon) or via **Data → View Masks**.

Features: - Grid view of all masks across `train/`, `val/`, `test/`, and `raw/` splits - Color-coded category chips showing which defect categories are present in each mask - **Missing mask indicators** — images without a mask are shown with a placeholder badge - Click any thumbnail to open the mask in the editor

Generating Missing Masks

When images exist but masks are absent, the Mask Gallery surfaces a **"Generate Missing Masks"** flow: 1. Open the Mask Gallery 2. Click **Generate Missing Masks** (shown when any split has unannotated images) 3. Choose the split and image size settings 4. The missing mask files are created as empty (all-background) templates 5. Open each generated mask in the editor to annotate it

This flow is designed to help you bootstrap annotation from scratch without manually creating blank mask files.

Phase 2B: Semi-Automated Mask Creation

The Phase 2B feature set introduces powerful **semi-automated workflows** to accelerate mask annotation. Instead of manually annotating every image, you can:

1. **Manually annotate a few representative images** (5-10 images)
2. **Propagate masks to similar images** using feature matching
3. **Batch review and refine** propagated masks quickly
4. **(Coming Soon) Active learning** to prioritize the most valuable images for annotation

These workflows can reduce annotation time by **50-80%** for large datasets with similar crack patterns.

Mask Propagation Workflow

Mask propagation uses computer vision (SIFT/ORB feature matching) to find images similar to the one you just annotated, then transfers your mask to those images automatically.

When to Use Mask Propagation

✅ **Good use cases:** - Harbor infrastructure with **repetitive patterns** (same concrete texture, consistent lighting) - Large datasets where **cracks appear in similar locations** (e.g., joints, edges) - Images from the **same structure or session** (consistent viewpoint) - **Initial dataset creation** where you need many training examples quickly

❌ **Less effective for:** - Highly variable images (different structures, angles, lighting) - Unique or one-off crack patterns - Datasets with significant perspective changes

Step-by-Step Propagation

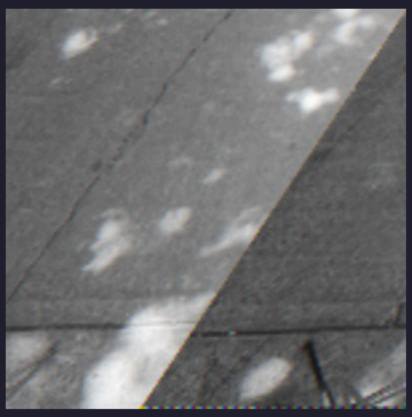
1. Annotate a Representative Image - Open the mask editor on an image with a typical crack pattern - Paint the mask carefully (this will be your "template") - Click **Save Mask** when finished

2. Open Propagation Dialog - With the same image still selected, click 🔄 **Propagate to Similar** in the mask editor toolbar - Or right-click the image in the browser and select 🔄 **Propagate Mask to Similar Images**

3. Configure Propagation Settings

Mask Propagation — GarayMC_cracks_train_0.jpg

Reference Image



Reference Mask



Similarity Threshold:

Min Matches: 10

Feature Detector: SIFT

Find Similar Images

Similar Images (select to propagate)

Select All | Deselect All

The propagation dialog shows: - **Source image preview** with your annotated mask - **Similarity threshold slider** (0.0 - 1.0) - **Higher values** (0.7-1.0): Only very similar images, fewer false positives - **Lower values** (0.3-0.6): More images, but may include less similar ones - **Default**: 0.5 (balanced) - **Feature matching method**: SIFT (default, more accurate) or ORB (faster) - **Gallery of candidate images** sorted by similarity score

4. Review and Select Candidates - Scroll through the **similarity gallery** showing all candidate images - Each thumbnail shows: - **Similarity score** (0.0-1.0) indicating match quality - **Checkbox** to include/exclude from batch - **Select high-quality matches**: Check images with similarity > 0.6 - **Deselect poor matches**: Uncheck images that look different

5. Propagate Masks - Click **Propagate to Selected** to transfer your mask to all checked images - A progress bar shows transformation and saving progress - When complete, the batch review interface opens automatically

How Propagation Works (Technical)

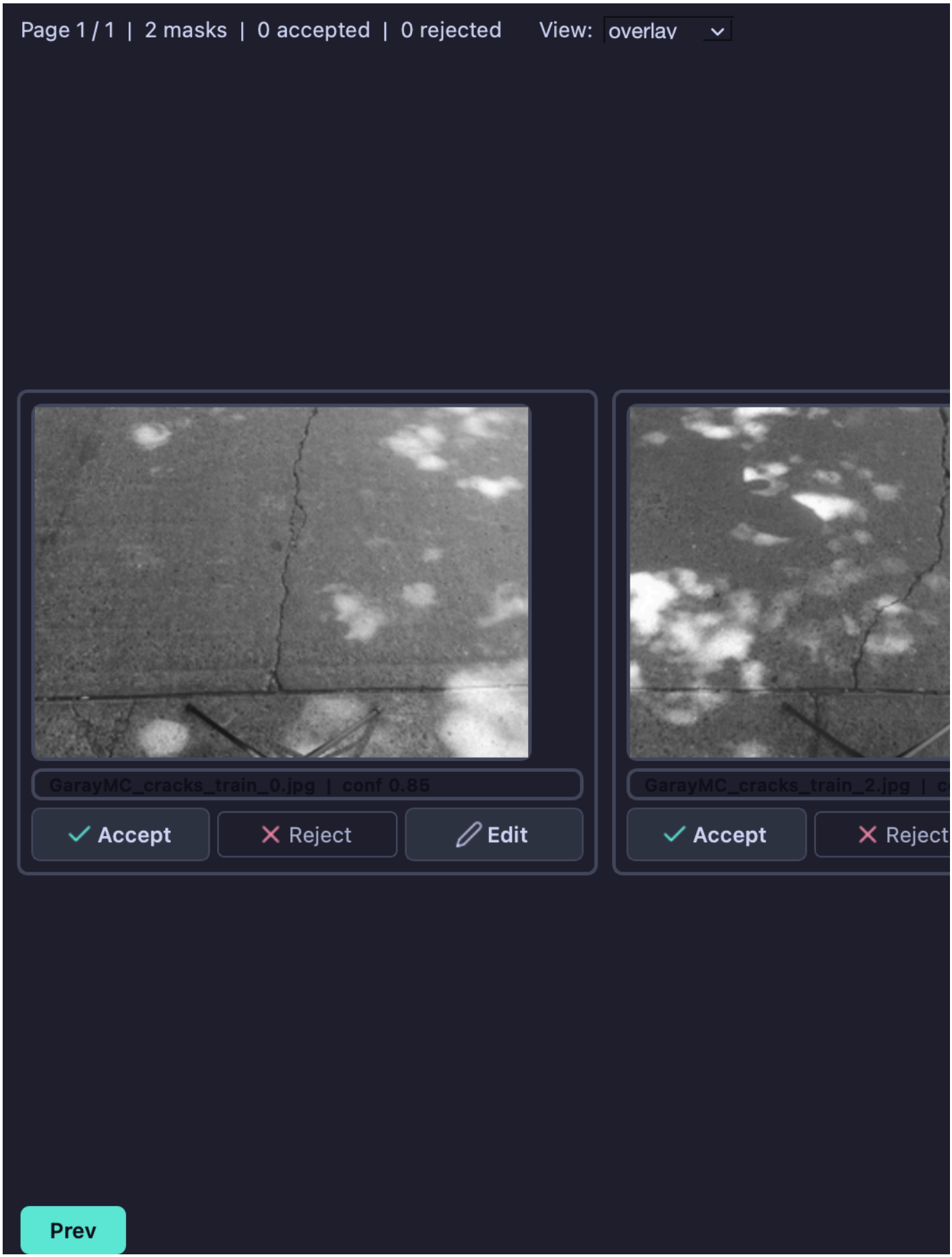
Mask propagation uses **feature-based image alignment**:

1. **Feature Detection**: Extracts keypoints (corners, edges) from both source and target images
2. **Feature Matching**: Finds corresponding keypoints between images
3. **Homography Estimation**: Calculates geometric transformation (rotation, scale, perspective)
4. **Mask Warping**: Applies transformation to your mask, aligning it with the target image
5. **Quality Filtering**: Only propagates if sufficient feature matches are found (>10 matches)

This approach works well for: - Images of the same structure from different angles - Sequential images with camera movement - Images with consistent structural features (corners, edges, patterns)

Batch Review Interface

After mask propagation, the **Batch Review Interface** helps you quickly verify and refine the propagated masks.


Page 1 / 1 | 2 masks | 0 accepted | 0 rejected
View: overlay
GarayMC_cracks_train_0.jpg | conf 0.85
Accept
Reject
Edit
GarayMC_cracks_train_2.jpg | c
Accept
Reject
Prev

Interface Layout

- **3×3 Grid**: Shows 9 images at a time with their propagated masks overlaid
- **Image info**: Each thumbnail shows:
- Original image + mask overlay
- **Confidence badge** (Good/Medium/Low based on similarity score)
- Image filename
- **Navigation controls**: Previous/Next buttons to browse through batches
- **Keyboard shortcuts**: Fast review without using the mouse

Batch Review Workflow

1. Review Propagated Masks - The batch review interface opens automatically after propagation - Review each image in the 3×3 grid - Look for: - ✅ **Correct alignment**: Mask covers the crack accurately - ⚠️ **Partial alignment**: Mask is close but needs adjustment - ❌ **Poor alignment**: Mask is in the wrong location or shape

2. Accept, Reject, or Edit

For each image, you have three options:

Action	Keyboard	Description
Accept	`A`	Keep the mask as-is (good quality)
Reject	`R`	Delete the mask (poor quality)
Edit	`E`	Open mask editor for manual refinement

3. Navigate Through Images

Action	Keyboard	Description
Next image	`Right Arrow`	Move to next image in grid
Previous image	`Left Arrow`	Move to previous image in grid
Next batch	Click "Next" button	Load next 9 images
Previous batch	Click "Previous" button	Load previous 9 images

4. Complete Review - When you've reviewed all images, click **Close** - Accepted masks are saved in the `masks/` directory - Rejected masks are deleted - Edited masks are saved after you close the mask editor

Confidence Badges

Each image shows a **confidence badge** to help prioritize review:

- 🟢 **Good** (>0.7 similarity): High confidence, likely correct
- 🟡 **Medium** (0.5-0.7 similarity): Moderate confidence, review carefully
- 🔴 **Low** (<0.5 similarity): Low confidence, likely needs editing or rejection

Tip: Focus your attention on Medium and Low confidence images—Good ones usually need minimal review.

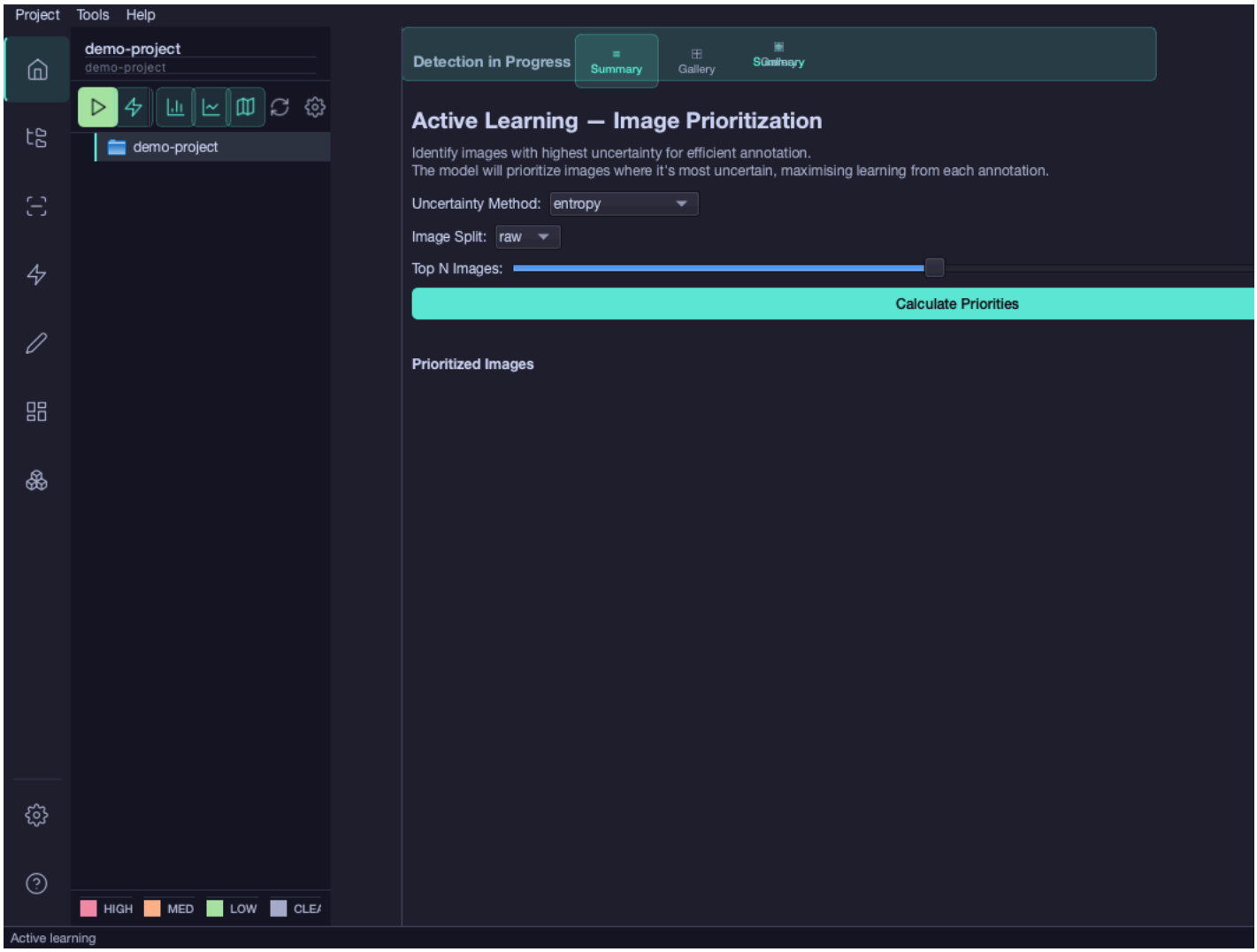
Efficiency Tips

For fast review: 1. **Start with Good badges**: Quickly accept high-confidence images with `A` 2. **Reject obvious failures**: Use `R` for clearly misaligned masks 3. **Edit Medium badges**: Press `E` on medium-confidence images to refine 4. **Use keyboard shortcuts**: Much faster than clicking buttons 5. **Batch process**: Review in batches of 9 to maintain context

For quality: - Don't skip images—every mask affects training quality - When in doubt, choose Edit over Accept - Reject rather than accept poor masks (bad data hurts training) - Aim for 80%+ acceptance rate (if lower, adjust propagation threshold)

Active Learning

Active learning prioritizes which images to annotate next by identifying: - Images with high model uncertainty (where the model is least confident) - Representative samples that maximize training efficiency - Edge cases that improve model robustness



How to use: 1. Open a project that has been trained at least once 2. Navigate to the **Active Learning** view via the sidebar 3. Select an **Uncertainty Method** (entropy, max_probability, variance, or mean_confidence) 4. Choose the **Image Split** to prioritize (raw, train, val, or test) 5. Adjust **Top N Images** using the slider (10–100) 6. Click **Calculate Priorities** to rank images by uncertainty score 7. Annotate the top-ranked images first for maximum training efficiency

Uncertainty Viewer

Coming in a future release

The Uncertainty Viewer is available for testing but not yet part of the main workflow.

The **Uncertainty Viewer** provides visual heatmaps showing where the model is most uncertain in its predictions. This helps: - Identify difficult regions requiring more training data - Prioritize annotation efforts on ambiguous areas - Validate model confidence visually

---

Using Mask Editor in Raw Folder

The mask editor is **enabled for all data splits**, including the `raw/` folder. This is intentional and supports important workflows:

Valid Use Cases for Raw Folder Masks

1. Active Learning Workflow - Run detection on raw images to identify cracks - Review detections and manually refine masks in the editor - Move validated images to train/val splits for retraining

2. Inference Result Refinement - Export detection results from raw folder - Manually correct false positives/negatives - Use corrected masks for reporting or further analysis

3. Pre-Annotation for Future Training - Annotate images in raw folder as you review them - Later move batches of annotated images to train split - Maintains separation between "to be annotated" and "ready for training"

4. Quality Control - Manually verify automated detections before committing to training data - Ensures only high-quality masks enter the training pipeline

Moving Images Between Splits

After annotating images in the `raw/` folder, you may want to move them to training splits:

Using the CLI:

```
# Move annotated images from raw to train split
crackz move-images --project ./my-project --source raw --target train --pattern "crack_*.jpg"
```

Using the GUI: 1. Open the project in the GUI 2. Navigate to `images/raw/` in the browser 3. Select annotated images (hold Ctrl/Cmd to multi-select) 4. Right-click → **Move to Split** → Choose `train`, `val`, or `test` 5. Confirm the move operation

Note: When moving images, their corresponding masks are automatically moved as well.

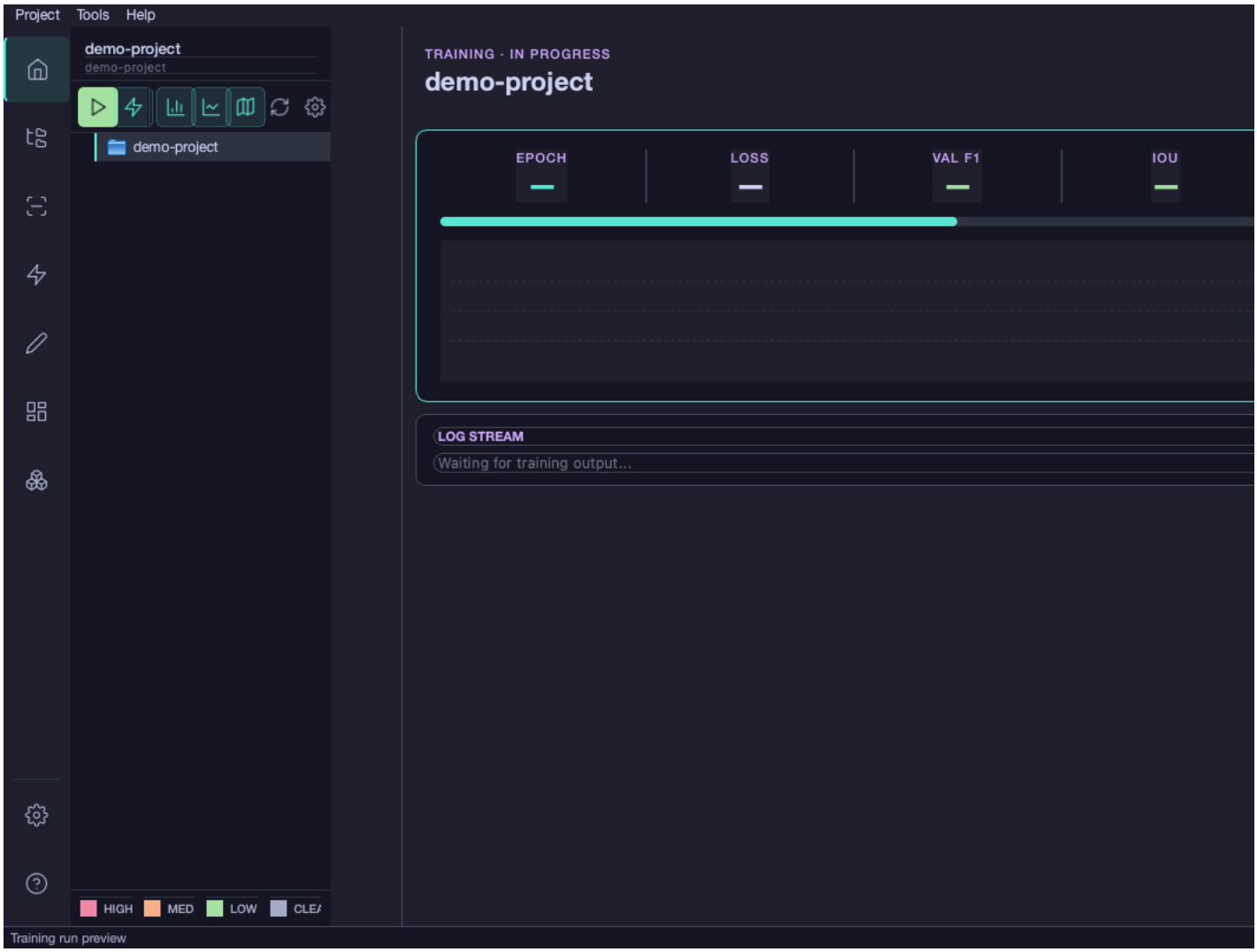
Model Training

Starting Training

1. Click **Train → Start Training** or press **Cmd/Ctrl+T**
2. Review the configuration summary (model architecture, epochs, batch size) and click **Start**
3. Training begins with real-time progress updates

Training Run View

During training, the GUI shows the **Training Run View** with live updates:

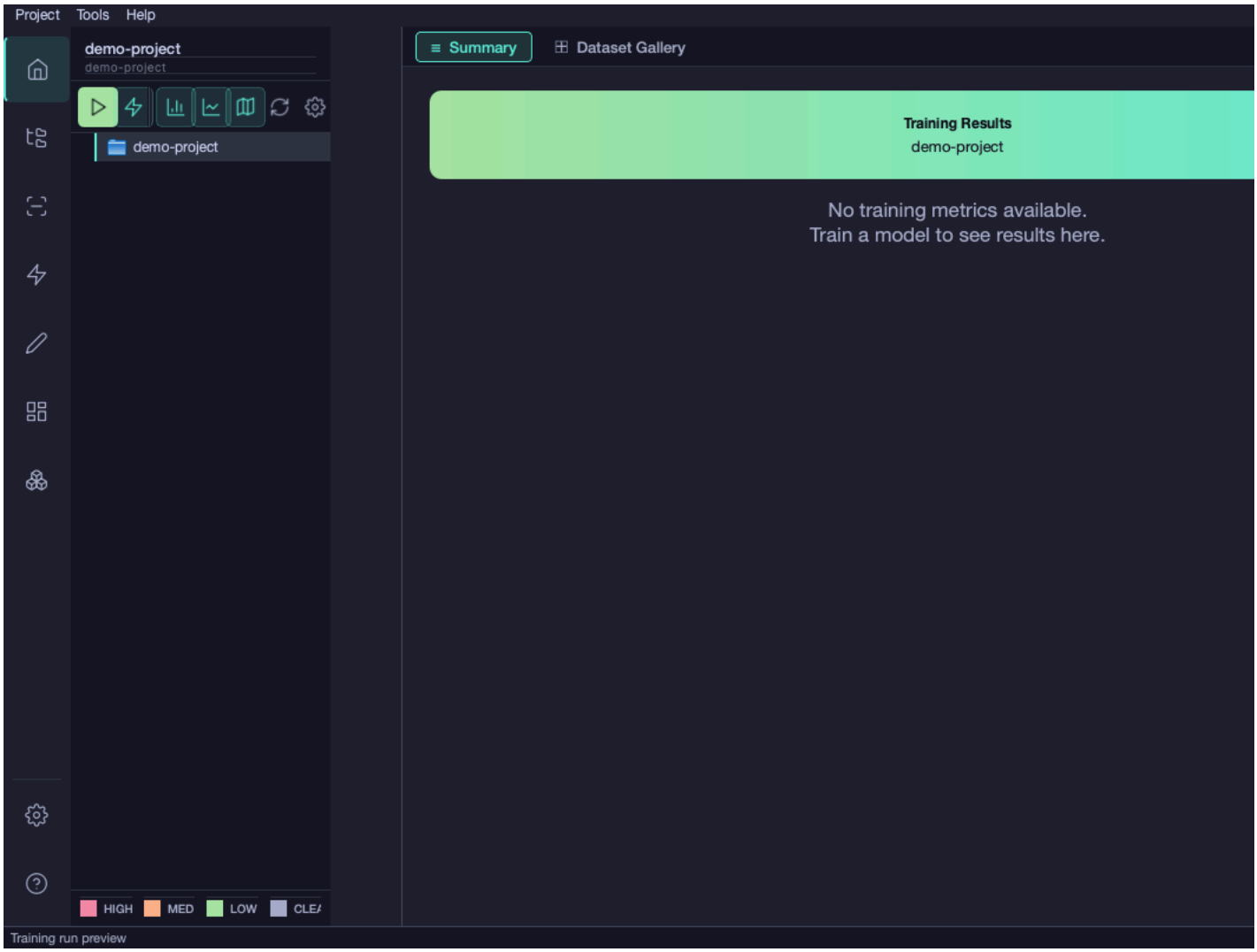


Features: - Real-time loss and metric charts - Current epoch and batch progress - Elapsed time and estimated time remaining - Live validation metrics - Hero section highlighting best validation results - **Navigate freely**: Switch to other views and return anytime using the progress footer

The view updates automatically as training progresses without requiring manual refresh. You can navigate to other views during training and easily return by clicking the progress bar, status label, or "View Progress" button in the progress footer.

Training Results

After training completes, view comprehensive results:



Available Information: - Training/validation loss curves - Metrics over epochs (accuracy, IoU, F1-score; per-class IoU shown for multiclass projects) - Task mode header: **Binary Segmentation** or **Multiclass Segmentation** - Best checkpoint location - Final model performance statistics - Training time and hardware information

---

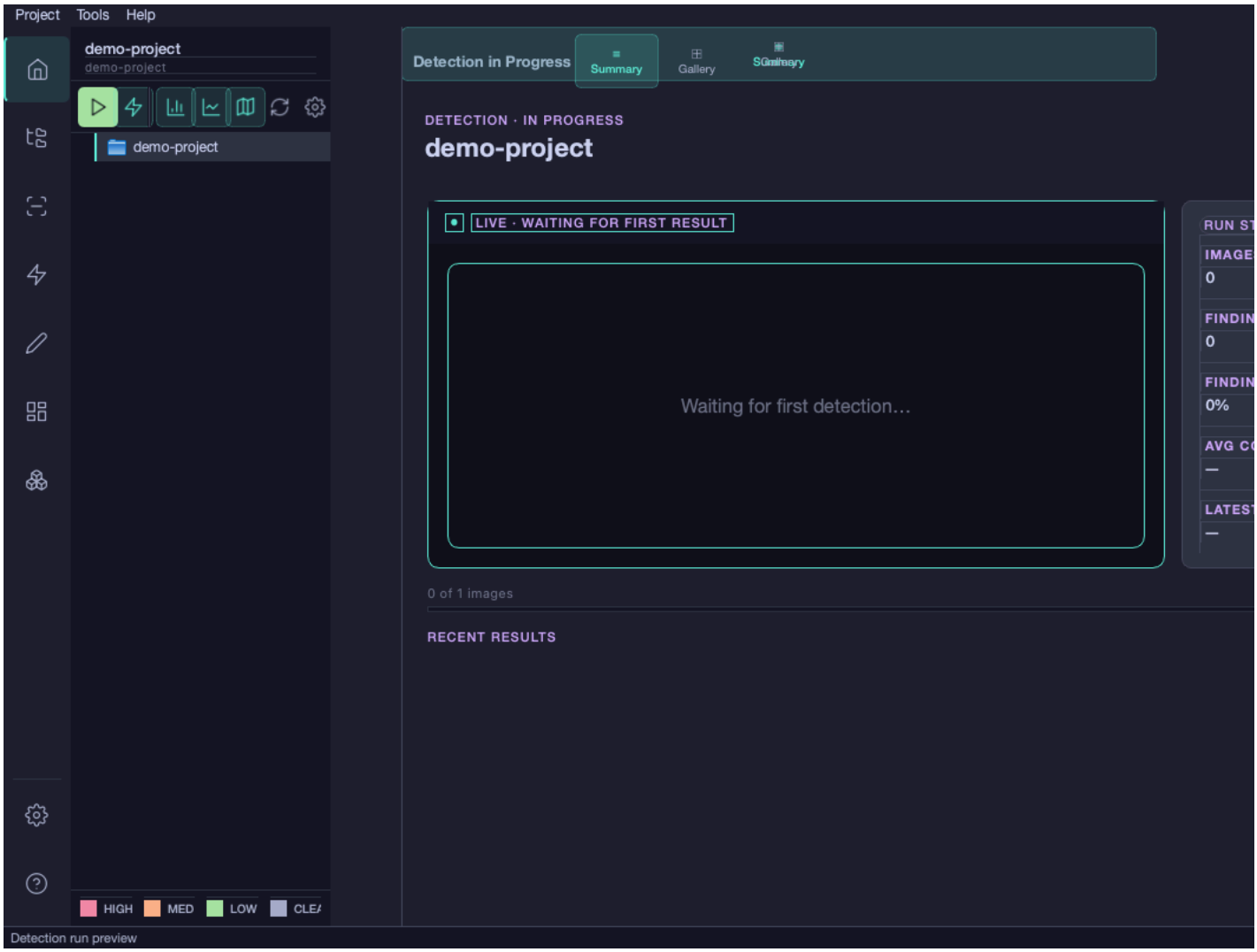
Crack Detection

Running Detection

1. Ensure you have images in the `raw` split or use a custom checkpoint
2. Click **Detect → Run Detection** or press **Ctrl+R**
3. Optional: Select a specific model checkpoint (or use latest)
4. The model selector greys out checkpoints whose taxonomy does not match the project's current annotation schema
5. Detection begins with real-time updates

Detection Run View

During detection, the GUI shows the **Detection Run View** with live gallery updates:

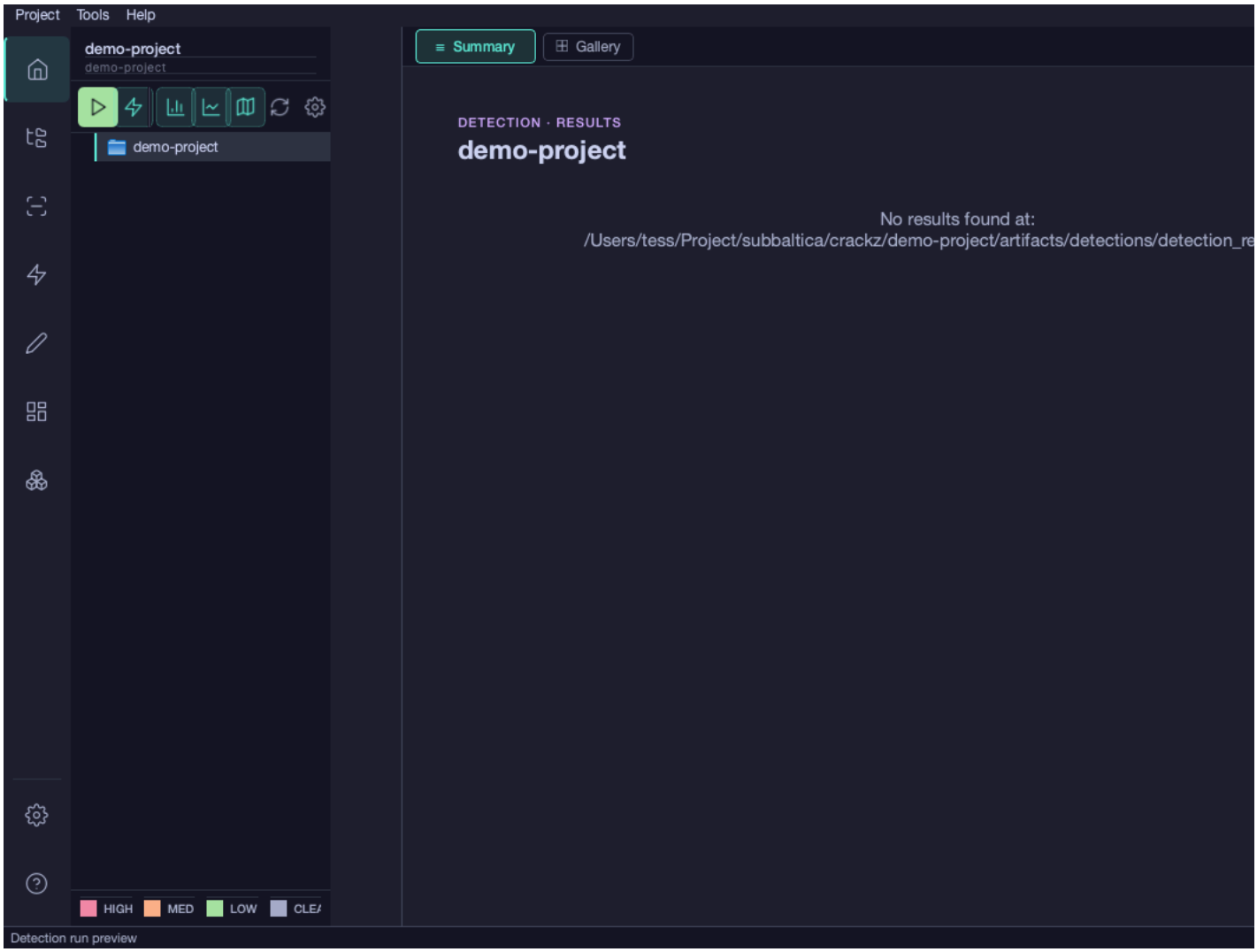


Features: - **Hero Section**: Displays the latest crack detection with dramatic CSI-style presentation - **Live Statistics**: Real-time updates showing: - Images processed count - Cracks detected count - Detection rate percentage - **Real-time Updates**: Statistics refresh automatically as each image is processed (every 2 seconds) - **Progress Tracking**: Shows elapsed time and estimated time remaining - **No Flickering**: Optimized to update only changed elements, preventing UI flicker - **Navigate freely**: Switch to other views and return anytime using the progress footer

The detection run view uses direct callbacks from the detection runner for instant updates, bypassing file polling for better performance. You can navigate to other views during detection and easily return by clicking the progress bar, status label, or "View Progress" button in the progress footer.

Detection Results

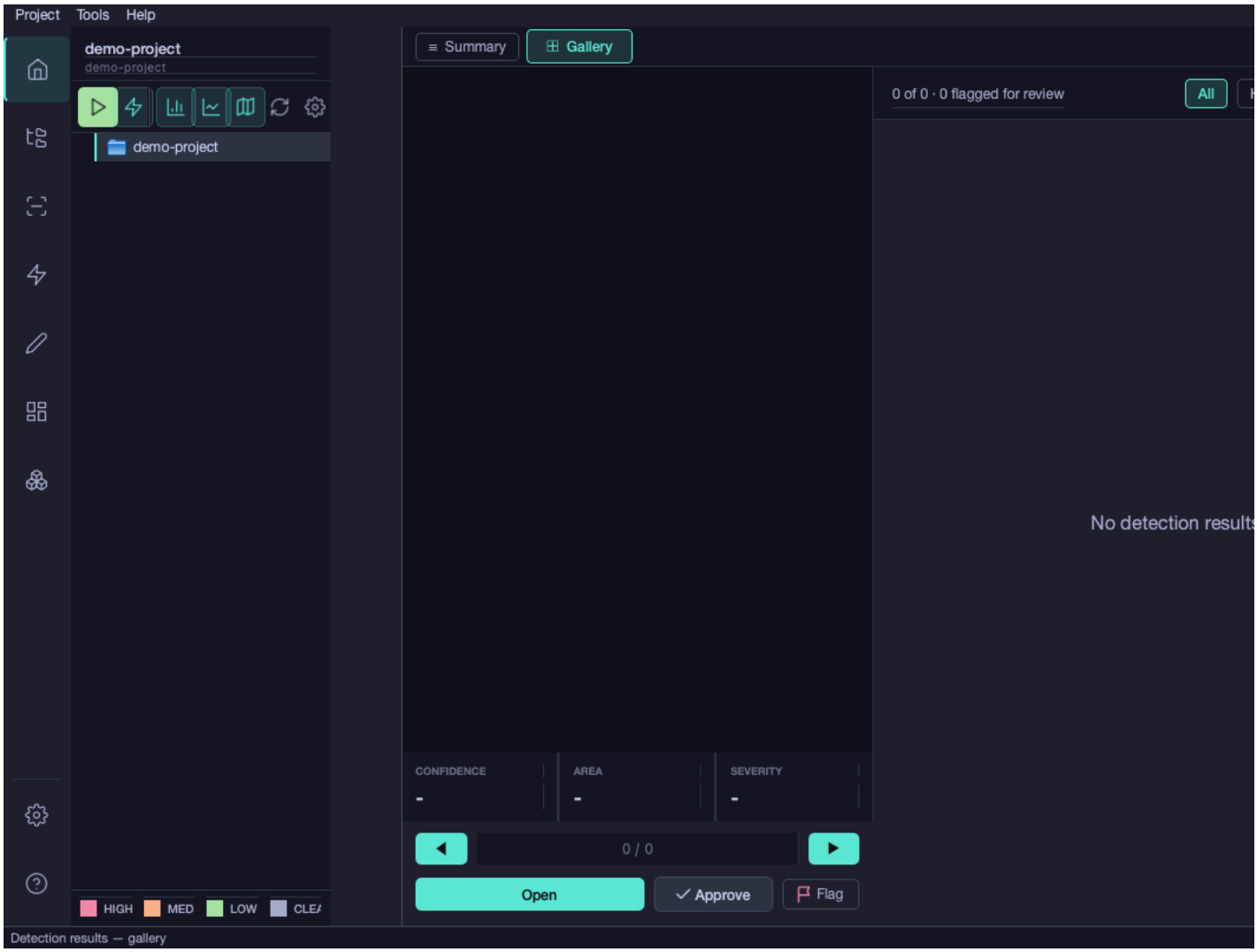
After detection completes, click **View Summary** to see the full gallery:



Available Information: - Grid gallery of all processed images - Color-coded markers (red = crack detected, green = no crack) - **Category findings panel** (multiclass projects): per-category severity breakdown (e.g. how many images contain Crack, Spall, Corrosion) - Click any image to view detailed overlay - Export options for results

Detailed Image View

Click on any detected image to see the detailed overlay view:

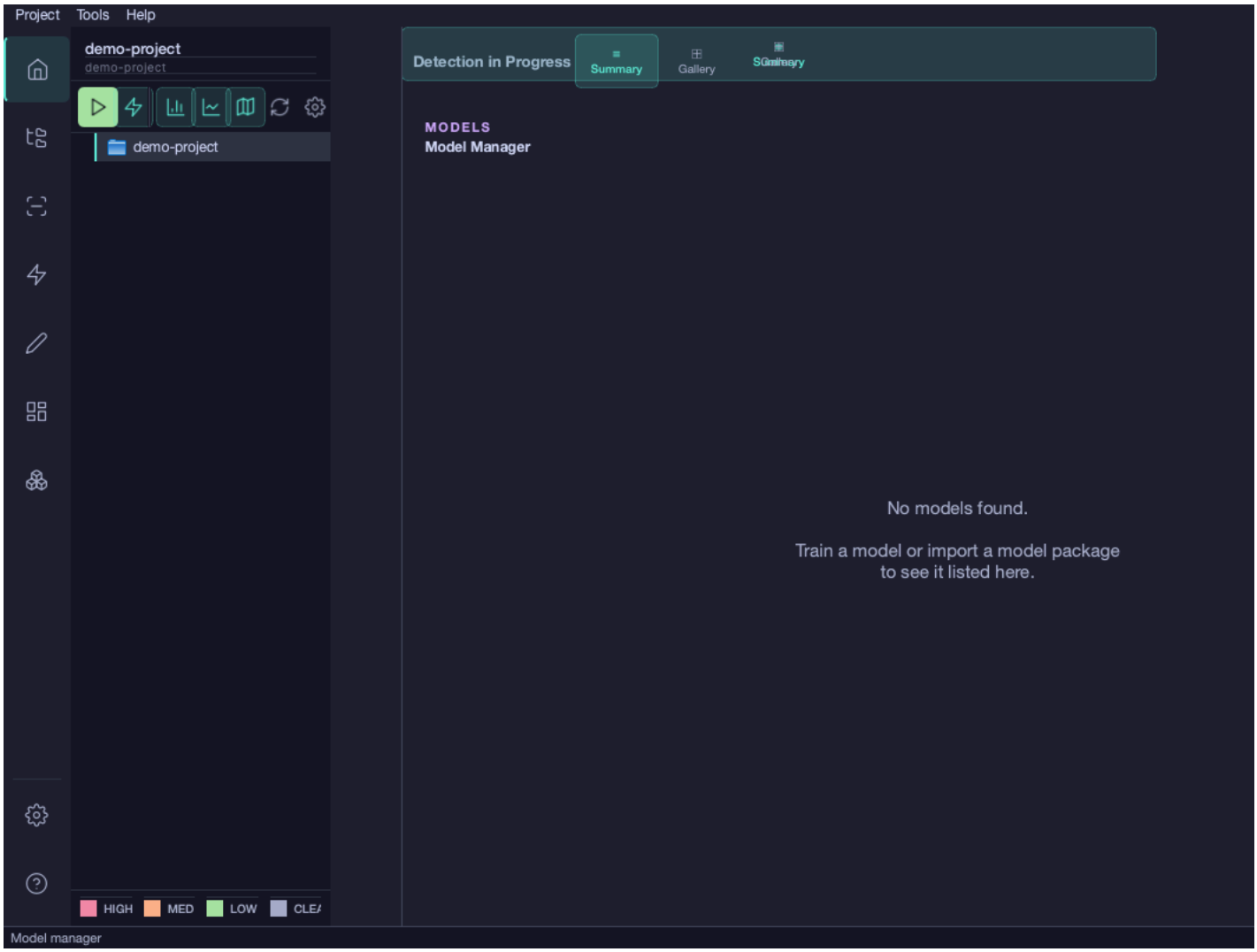


Features: - Original image alongside annotated version - Crack masks highlighted in red overlay - Confidence scores and detection metadata - Image-specific statistics (crack area, coverage percentage)

Model Management

Model Manager

The Model Manager provides a comprehensive overview of all trained models registered to your project. Access it by clicking the **Models** icon (sixth item) in the sidebar.



Features: - List of all registered model checkpoints with metadata (backbone, task mode, categories, input size) - **Taxonomy compatibility**: Models whose annotation schema does not match the current project's categories are greyed out and cannot be selected for detection - Default model indicator — the model used for the next detection run - Per-model notes for documenting experiments - Set any checkpoint as the default with one click - Rename or delete model records - Import unregistered checkpoints found in the models directory

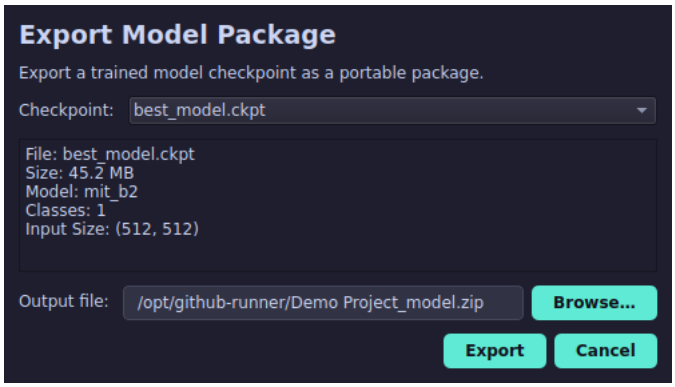
Taxonomy columns: - **Task Mode**: `Binary Segmentation` or `Multiclass Segmentation` - **Categories**: Comma-separated list of category keys the model was trained on

Workflow: 1. Open a project with at least one trained model 2. Click the **Models** sidebar button 3. Select a model from the list to view its details (including task mode and categories) 4. Click **Set as Default** to use it for detection 5. Use the notes field to record training observations

Exporting Models

Share trained models or deploy to other systems:

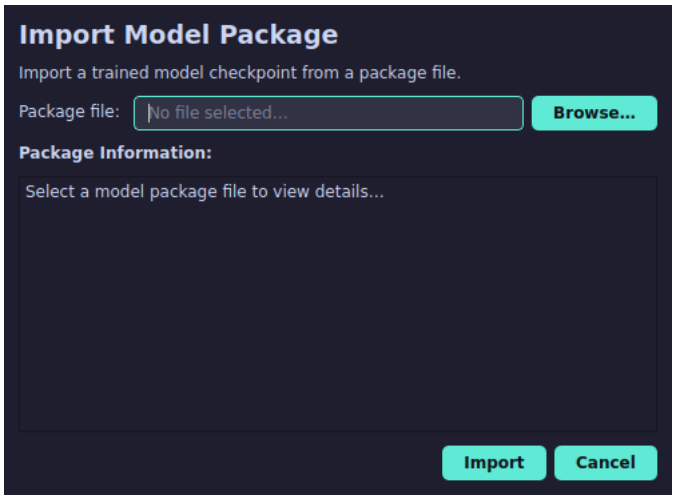
1. Click **Project → Export Model Package…**
2. Select the checkpoint to export from the dropdown
3. Choose the output file path
4. Click **Export**



Importing Models

Use pre-trained models from external sources:

1. Click **Project → Import Model Package...**
2. Browse to and select a model package file (`.zip`)
3. Review the package information
4. Click **Import**



Project Maintenance

Clean Project

> **Note:** The Clean Project feature is planned for a future release. Use the CLI command to clean project artifacts in the meantime:

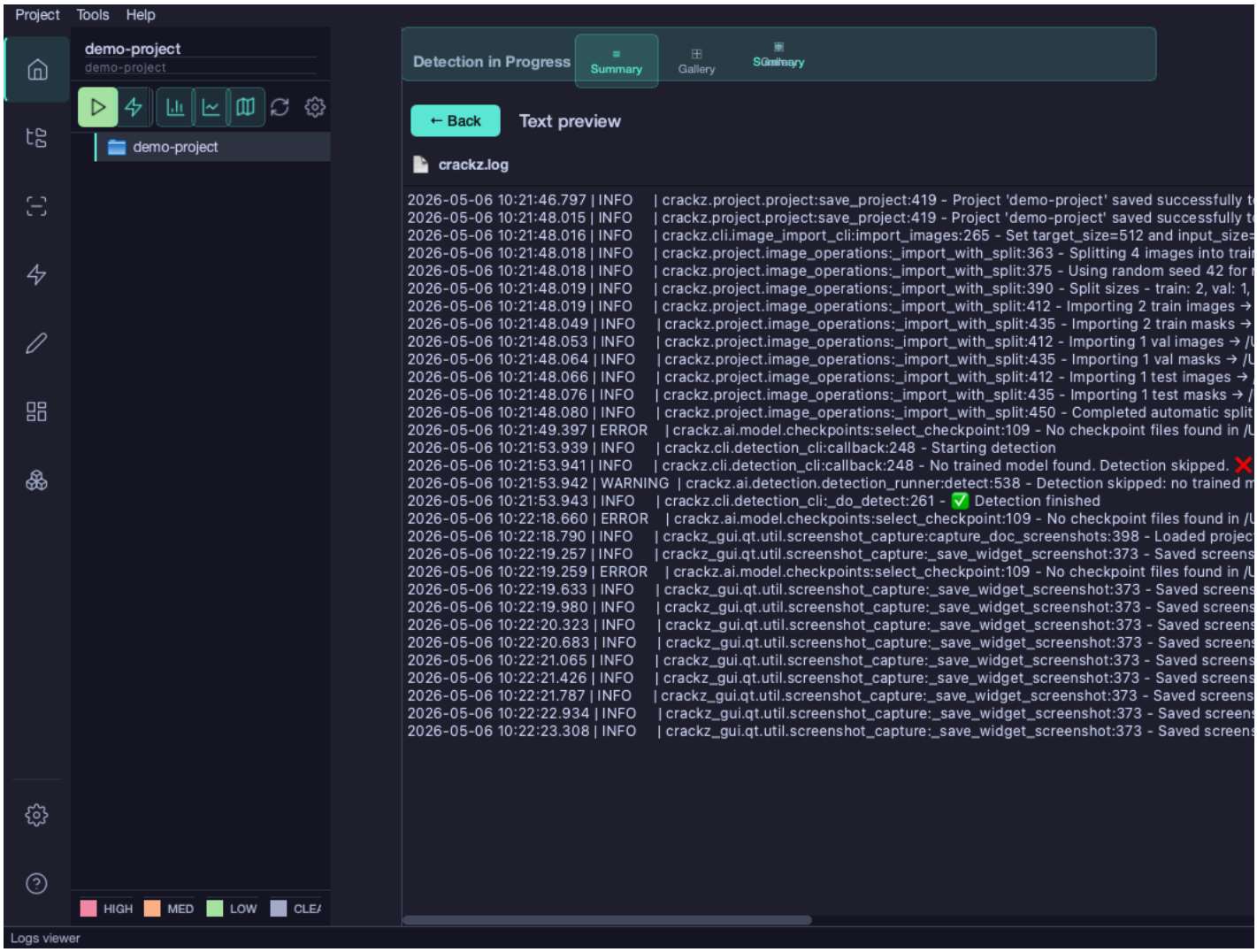
```
crackz clean --project ./my-project
```

Project Information

> **Note:** The Project Information dialog is planned for a future release. Use **Project → Project Settings...** to review and edit project configuration, or inspect the `project.yaml` file directly in the project folder.

Viewing Logs

Access project logs via the project tree:



Features: - Color-coded log levels (DEBUG, INFO, WARNING, ERROR) - Search and filter capabilities - Real-time log streaming during operations - Export logs for troubleshooting

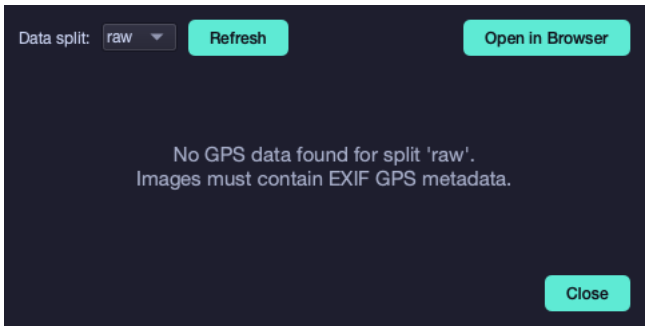
---

Map Viewer

Viewing Geotagged Images on a Map

For projects with geotagged images, the Map Viewer provides an interactive geographic visualization:

1. Click the 🗺️ **Map** button in the toolbar or **Tools → View Images on Map**
2. Select the data split (raw, train, val, test) to visualize
3. Click **"Open in Browser"** to view the interactive map



Features: - **Interactive Leaflet.js maps**: Full zoom, pan, and marker functionality - **OpenStreetMap base layer**: Detailed geographic context - **Color-coded markers**: Visual distinction between different image types or detection results - **Marker popups**: Click markers to view image thumbnails and metadata - **Browser-based**: Opens in your default web browser for optimal performance - **Large dataset support**: Efficiently handles projects with many geotagged images

Note: The map viewer automatically extracts GPS coordinates from image EXIF data. Images without geotags will not appear on the map.

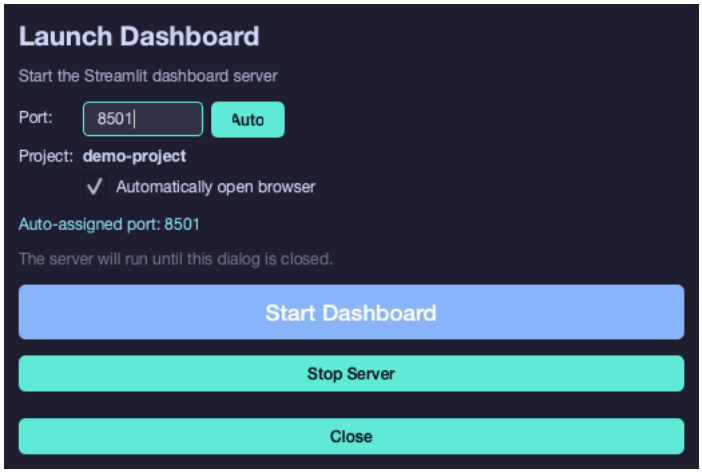
For more details on the map viewer feature, see the Map Viewer Guide.

Dashboard

Launching the Streamlit Dashboard

The GUI provides a convenient way to launch the interactive Streamlit dashboard for advanced data analysis and visualization:

1. Click **Tools → Launch Dashboard** from the menu bar
2. A dialog appears with dashboard launch options:
3. **Automatic port selection**: The GUI finds an available port automatically
4. **Custom port**: Specify a port if needed (default: 8501)
5. **Project validation**: Ensures the current project is suitable for dashboard analysis
6. Click **Start Dashboard** to launch the server
7. The dashboard opens automatically in your default web browser



Dashboard Features Available from GUI: - **Real-time server status**: Visual indicators show when the dashboard is running - **Global state tracking**: Dashboard status is tracked across the entire GUI application - **Automatic cleanup**: Server stops gracefully when GUI is closed - **Error handling**: Clear error messages if the dashboard fails to start - **Port conflict resolution**: Automatically finds alternative ports if the default is busy

Dashboard Integration Benefits

Launching the dashboard through the GUI provides several advantages over the CLI:

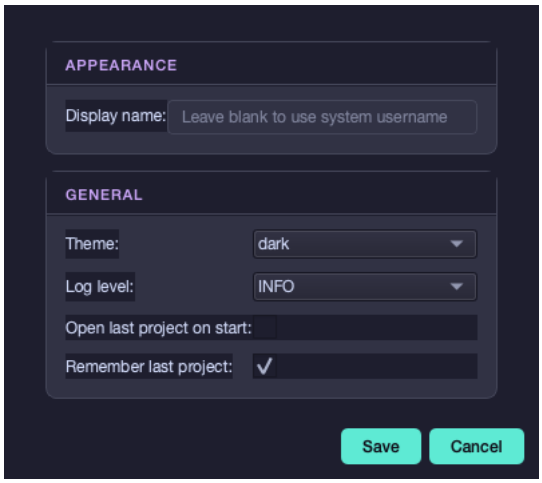
- **Project context**: Automatically uses the currently open project
- **State synchronization**: GUI knows when dashboard is running and can coordinate operations
- **Simplified workflow**: No need to remember CLI commands or manage terminal windows
- **Visual feedback**: Progress indicators and status updates in the GUI
- **Error handling**: User-friendly error messages with suggested solutions

The dashboard provides advanced features for: - Interactive detection result analysis - Training metrics visualization - GPS mapping of geotagged images - Parameter tuning with real-time feedback - Export capabilities for reports and visualizations

For complete dashboard documentation, see the Dashboard Guide.

Application Settings

Access application-wide preferences via **Settings → Application Settings** (`Cmd/Ctrl+,`).



Settings groups:

Setting	Description
Display name	Your name shown in the title bar (leave blank to use system username)
Theme	`dark`, `light`, or `system` — Catppuccin Mocha is used for dark mode
Log level	Controls verbosity: DEBUG, INFO, WARNING, ERROR
Open last project on start	Automatically reopen the most recently used project
Remember last project	Persist the last project path across restarts

Changes take effect immediately after clicking **Save**.

Keyboard Shortcuts

Shortcut	Action
Cmd/Ctrl+N	New Project
Cmd/Ctrl+O	Open Project
Cmd/Ctrl+S	Save Project Settings
Cmd/Ctrl+R	Run Detection
Cmd/Ctrl+T	Train Model
Cmd/Ctrl+I	Import Images
Cmd/Ctrl+,	Open Settings
Cmd/Ctrl+Q	Quit Application
Cmd/Ctrl+D	Launch Dashboard
`§` or `~`	Toggle AI Chat Assistant
Cmd/Ctrl+/	Toggle AI Chat Assistant (alternative)

AI Chat Assistant

The GUI includes an integrated AI-powered chat assistant to help with your crack detection workflows. Access it via: - Keyboard: `§` (Swedish/Nordic), `~` (US), or **Cmd/Ctrl+/** - Menu: **Help → AI Chat Assistant**

The assistant provides: - Real-time help with detection parameter tuning - Project configuration analysis and optimization suggestions - Guidance on using Crackz features - Contextual advice based on your current project state

See the full [AI Chat Assistant Guide] for detailed documentation.

Chat History & Prompt Templates

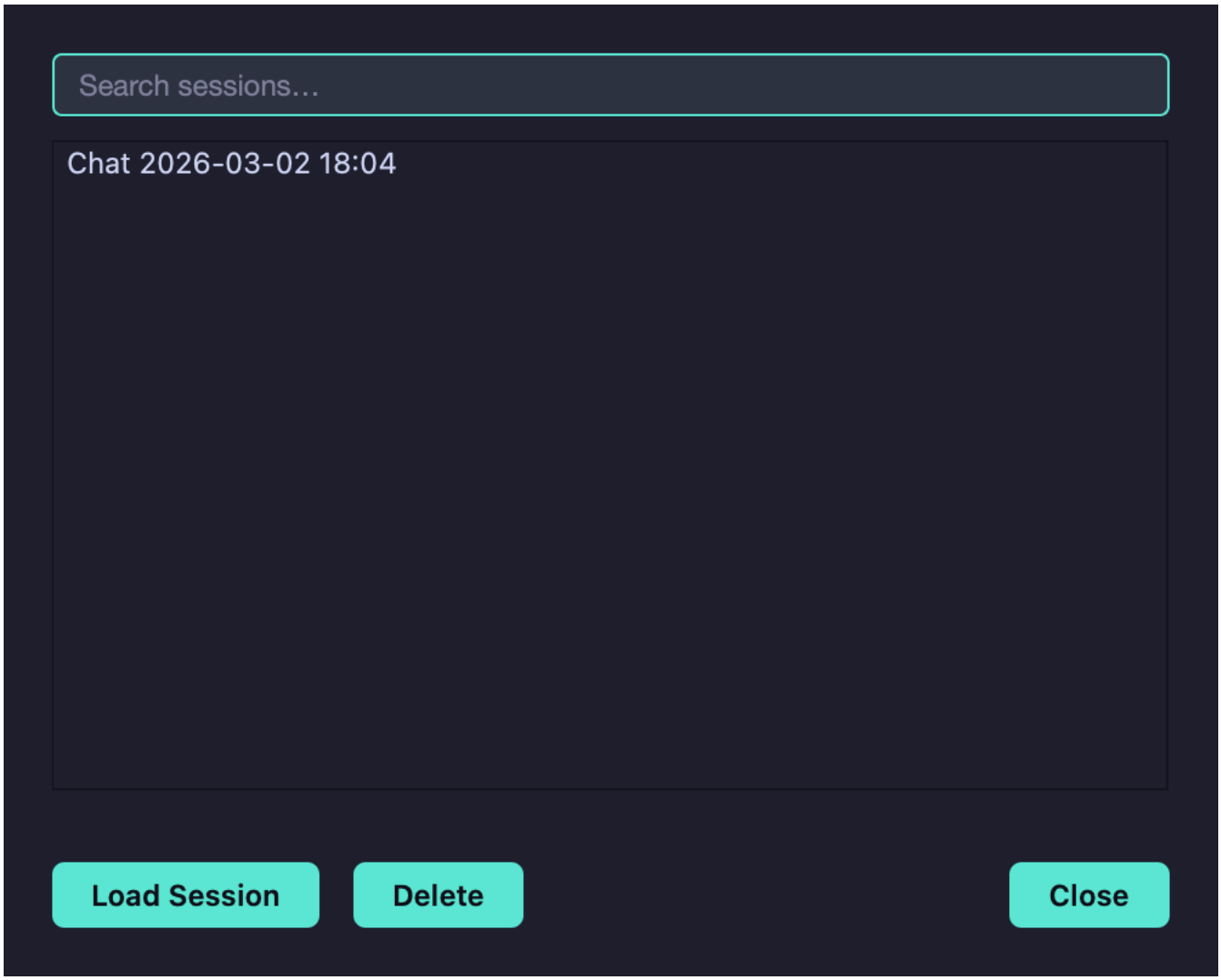
The chat assistant includes powerful features to help you maintain context and reuse effective prompts across sessions:

Chat History

All conversations with the AI assistant are automatically saved to a local SQLite database, allowing you to:

- **Review past conversations**: Access previous discussions to recall advice and recommendations

- **Resume interrupted sessions**: Continue where you left off if you close the chat window
- **Learn from history**: Reference successful parameter tuning discussions from previous projects
- **Maintain context**: Keep track of the assistant's suggestions over time



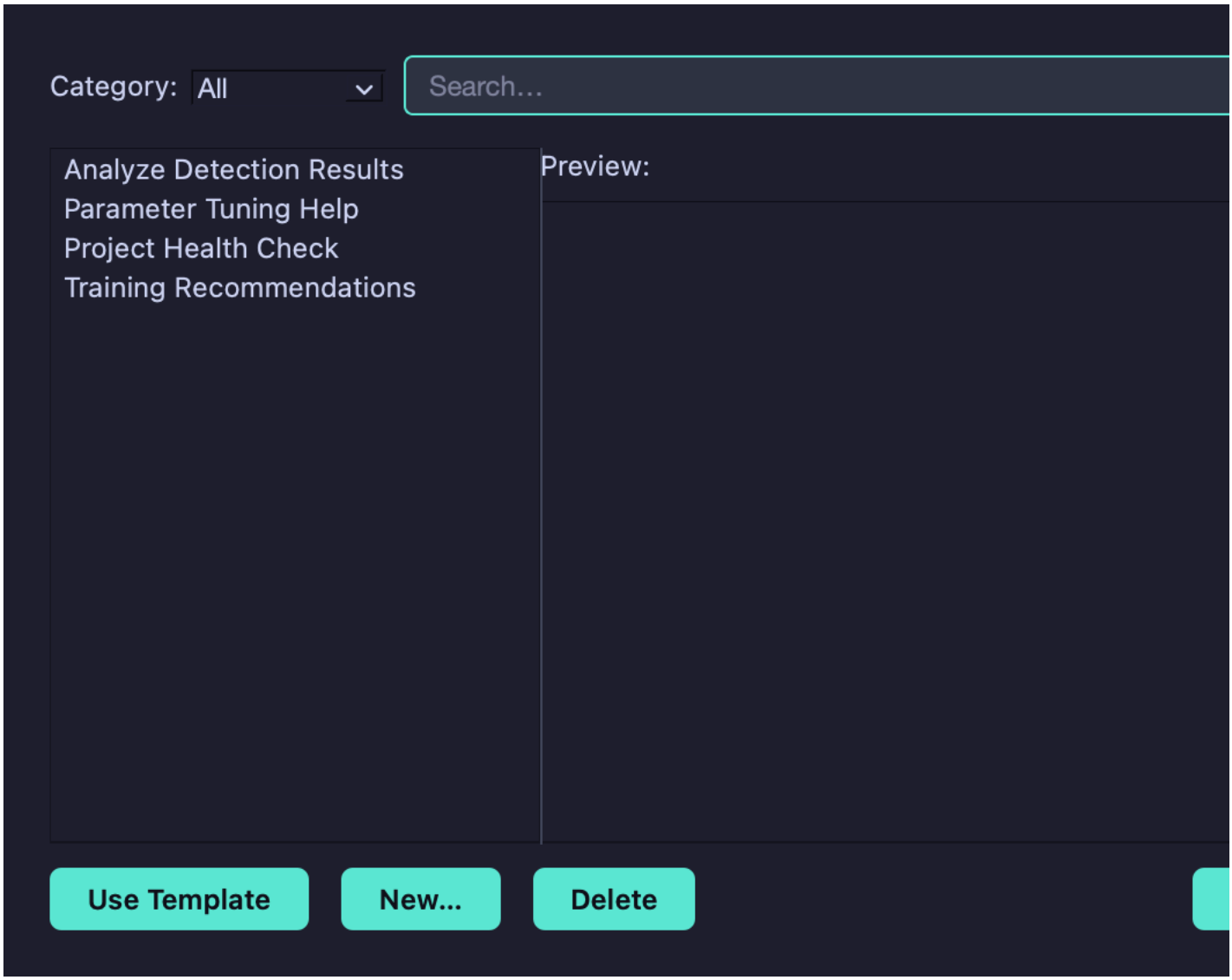
Accessing Chat History: 1. Open the chat assistant (`§` , `~` , or `Cmd/Ctrl+/`) 2. Click the 📜 **History** button in the top-right corner 3. Browse conversations by session: - Sessions are listed with timestamps and message counts - Preview shows the first message of each conversation - Double-click a session to load its full conversation 4. Click **Load Session** to restore a previous conversation into the chat window

Chat History Features: - **Persistent storage**: Conversations survive application restarts - **Fast search**: Find specific discussions by content or date - **Session metadata**: View message counts, timestamps, and conversation previews - **One-click restore**: Load any previous session with a single click - **Privacy**: All history stored locally on your machine

Prompt Templates

Prompt templates help you quickly access common questions and workflows without retyping:

Built-in Templates: - **Optimize Training**: Get recommendations for training hyperparameter tuning - **Detection Setup**: Configure detection parameters for optimal results - **Data Preparation**: Guidance on dataset organization and split ratios - **Model Selection**: Advice on choosing the right architecture and encoder - **Troubleshooting**: Help diagnosing common issues and errors



Using Prompt Templates: 1. Open the chat assistant 2. Click the 📋 **Templates** button in the top-right corner 3. Browse available templates by category 4. Click a template to insert its prompt into the chat input 5. Optionally edit the prompt before sending

Custom Templates — Coming in a future release

In a future update you will be able to create, organize, and share your own prompt templates.

Database Storage

All chat history and templates are stored in a SQLite database:

Location: `~/.crackz/chat_history.db` (user home directory)

Database Schema: - **Sessions table**: Conversation metadata (timestamps, message counts) - **Messages table**: Individual message content (user/assistant, timestamps) - **Templates table**: Reusable prompt templates with categories

Privacy & Security: - All data stored locally on your machine - No cloud synchronization or external access - Database secured with standard file permissions - Easy to delete or export for backup

Database Management: - Automatic schema migrations when updates are released - Efficient indexing for fast search and retrieval - Automatic cleanup of old sessions (configurable retention period) - Export functionality for backing up important conversations

---

Performance Tips

GPU Acceleration

The GUI automatically detects and uses NVIDIA GPUs when available. Check **Help → System Info** to verify GPU detection.

Background Operations

Long-running operations (training, detection, import) run in background threads, keeping the GUI responsive. You can: - Navigate to other views during operations - Cancel operations at any time - Queue multiple operations (they run sequentially)

Return to Run View

When training or detection is in progress, you can navigate to other views (e.g., browse project files, check settings) without losing access to the live progress updates. The GUI provides multiple ways to return to the active run view:

Return Methods: - **"View Progress" Button**: A blue button appears in the progress footer between the progress bar and cancel button - **Click Progress Bar**: Click anywhere on the progress bar itself (cursor changes to hand pointer) - **Click Status Label**: Click the status text showing current operation details

This feature is particularly useful for: - Checking project settings while training runs - Reviewing logs during long detection runs - Browsing previous results while new detection is in progress - Multitasking without losing track of running operations

Note: When the operation completes, the progress footer automatically hides and the tracked view is cleared.

File Watching

The GUI includes intelligent file watching that automatically refreshes views when files change on disk. This is particularly useful when: - Using CLI commands alongside the GUI - Collaborating with team members - Running external scripts that modify project data

File watching is temporarily suspended during detection runs to prevent interference with real-time updates.

Troubleshooting

GUI Won't Launch

```
# Check installation
python -c "from PySide6.QtWidgets import QApplication; print('PySide6 OK')"

# Launch with debug output
crackz-gui --debug
```

Slow Performance

- **Large datasets**: Consider filtering or limiting the number of images displayed in the project view
- **Background operations**: Check if multiple tasks are queued
- **GPU issues**: Verify CUDA drivers if using GPU acceleration

Display Issues

- **Dark mode problems**: Some systems may have rendering issues. Try **View → Theme → Light** mode
- **Scaling**: Adjust DPI scaling in **Settings → Appearance**

Log Files

GUI logs are written to: - **Application logs**: `CRACKZ_LOG_DIR/gui_app.log` - **Project logs**: `<project>/logs/gui_operations.log`

CLI Integration

The GUI and CLI share the same project format and configuration. You can: - Create projects in the GUI, process via CLI - Train models via CLI, view results in GUI - Use CLI for automation, GUI for interactive work

Example workflow:

```
# Create and configure in GUI
crackz-gui

# Batch process via CLI
```

```
crackz detect run --project my-project --batch-size 16

# Review results in GUI
crackz-gui
```

See CLI Cookbook for more command-line examples.

Next Steps

- **New to Crackz?** Start with the Getting Started Guide
- **Training models?** See Training Configuration
- **Need automation?** Check CLI Cookbook
- **Docker deployment?** Read the Docker Guide

AI Chat Assistant

AI Chat Assistant

The Crackz GUI includes an integrated AI chat assistant with **multi-provider support**. Choose between: - **Anthropic (Direct)**: Direct API access to Claude models - **OpenRouter**: Unified gateway to 200+ models from 20+ providers

This feature provides real-time help with crack detection workflows, parameter tuning, and project management.

Overview

The AI chat assistant is a floating window that provides contextual help based on your current project state. It can:

- Analyze your project configuration and suggest optimizations
- Help tune detection parameters for better results
- Explain crack detection concepts and workflows
- Provide guidance on using Crackz features
- Answer questions about your specific project

Getting Started

Prerequisites

You need an API key from **one** of the following providers:

Option 1: Anthropic (Direct) - Get API key from Anthropic Console - Direct access to Claude models - Pricing: $3-15/MTok input, $15-75/MTok output

Option 2: OpenRouter (Recommended) - Get API key from OpenRouter - Access to 200+ models from 20+ providers - **Cost savings**: Up to 75% cheaper than direct (e.g., Claude Haiku: $0.25/MTok vs $1/MTok) - Additional models: GPT-4o, Llama 3.1, Gemini Pro, and more

Both require: - Internet connection for API communication

Security Notice

API Key Storage

Your Anthropic API key is currently stored in **plaintext** in the application settings file.

For Development/Personal Use: - Keep the settings file secure with appropriate file permissions - Never commit the settings file to version control - Consider using environment variables for CI/CD

For Enterprise/Production Use: - Implement secure credential storage (Windows Credential Manager, macOS Keychain, etc.) - Use key rotation policies - Consider encrypted storage or external secret management systems - Review your organization's security policies before deployment

Secure Storage with Keyring (Optional)

Crackz provides optional integration with the keyring library for secure credential storage. This uses your operating system's native secure storage:

- **Windows**: Windows Credential Manager
- **macOS**: Keychain
- **Linux**: Secret Service (GNOME Keyring, KWallet, etc.)

Installation:

```
# Install keyring support
uv add keyring

# Or with pip
pip install keyring
```

Usage:

Once keyring is installed, you can use it programmatically in your Python scripts:

```
from crackz_gui.state.secure_storage import (
    store_api_key,
    retrieve_api_key,
    is_secure_storage_available,
)
```

```
# Check if keyring is available
if is_secure_storage_available():
    # Store API key securely
    store_api_key("your-api-key-here")

    # Retrieve API key
    api_key = retrieve_api_key()

    # Delete API key
    from crackz_gui.state.secure_storage import delete_api_key
    delete_api_key()
else:
    print("Keyring not available - falling back to plaintext storage")
```

Benefits:

- ✅ **Encrypted storage** using OS-level encryption
- ✅ **No plaintext** API keys in settings files
- ✅ **Cross-platform** support (Windows/macOS/Linux)
- ✅ **Optional dependency** - fallback to plaintext if not installed
- ✅ **Zero configuration** - uses OS defaults automatically

Note: The GUI currently stores API keys in plaintext settings. Keyring integration is available for custom scripts and automation workflows. Full GUI integration is planned for a future release.

Initial Setup

1. **Open Chat Window**:
2. Press `§` (Swedish/Nordic keyboards)
3. Press `~` (US keyboards)
4. Press `Ctrl+/` (Windows) or `Cmd+/` (Mac)

5. Or use menu: **Help → AI Chat Assistant**

6. **Configure Provider & API Key**:

7. Click the ⚙ **Settings** button in the chat window
8. **Select Provider**:
    - Choose **Anthropic** for direct Claude access
    - Choose **OpenRouter** for multi-model access (recommended)
9. **Enter API Key**:
    - Paste your provider-specific API key
    - Format: `sk-ant-...` (Anthropic) or `sk-or-...` (OpenRouter)
10. **Select Model**: Choose from provider-specific catalog
11. Click **Save**

!!! tip "OpenRouter Recommended" OpenRouter provides up to 75% cost savings and access to 200+ models while maintaining Claude quality. Get a free API key at openrouter.ai/keys.

1. **Start Chatting**:
2. Type your question in the input box
3. Press **Enter** to send
4. Use **Shift+Enter** or **Ctrl+Enter** for multi-line messages

Keyboard Shortcuts

Opening/Closing Chat

- `§` - Toggle chat window (Swedish/Nordic keyboards)
- `~` - Toggle chat window (US keyboards)
- `Ctrl+/` or `Cmd+/` - Toggle chat window (all keyboards)

In Chat Window

- **Enter** - Send message
- **Shift+Enter** - Insert newline (multi-line message)
- **Ctrl+Enter** - Insert newline (multi-line message)

Available Models

Choose the model that best fits your needs in the settings dialog. Available models depend on your selected provider:

Anthropic (Direct Access)

Direct API access to Claude models with premium support:

Claude Sonnet 4.5 (Recommended)

- **Best balance** of intelligence, speed, and cost
- **Excellent** for coding and agentic tasks
- **200K token** context window
- **Pricing**: $3/MTok input, $15/MTok output

Claude Haiku 4.5

- **Fastest** model with near-frontier intelligence
- **Most economical** Anthropic option
- **200K token** context window
- **Pricing**: $1/MTok input, $5/MTok output

Claude Opus 4.1

- **Most capable** model for complex reasoning
- **Premium** intelligence and accuracy
- **200K token** context window
- **Pricing**: $15/MTok input, $75/MTok output

OpenRouter (Multi-Provider Access)

Up to 75% cost savings with access to 200+ models from 20+ providers:

Claude Models (via OpenRouter)

- **claude-sonnet-4-5** - $2.25/MTok input, $11.25/MTok output (25% savings)
- **claude-haiku-4-5** - **$0.25/MTok** input, $1.25/MTok output (**75% savings**)
- **claude-opus-4-1** - $11.25/MTok input, $56.25/MTok output (25% savings)
- **claude-3.5-sonnet** - Previous generation, similar pricing

Additional Models (OpenRouter Only)

- **openai/gpt-4o** - OpenAI's flagship multimodal model ($2.50/MTok in, $10/MTok out)
- **meta-llama/llama-3.1-70b-instruct** - Open source, high performance ($0.52/MTok in, $0.75/MTok out)
- **google/gemini-pro-1.5** - Google's latest model ($1.25/MTok in, $5/MTok out)

Comparison Table

Model	Provider	Input Cost	Output Cost	Savings	Best For
Claude Haiku 4.5	Anthropic	$1.00/MTok	$5.00/MTok	-	Fast queries
Claude Haiku 4.5	**OpenRouter**	**$0.25/MTok**	**$1.25/MTok**	**75%**	**Best value**
Claude Sonnet 4.5	Anthropic	$3.00/MTok	$15.00/MTok	-	Balanced
Claude Sonnet 4.5	OpenRouter	$2.25/MTok	$11.25/MTok	25%	Savings + quality
GPT-4o	OpenRouter	$2.50/MTok	$10.00/MTok	N/A	OpenAI alternative
Llama 3.1 70B	OpenRouter	$0.52/MTok	$0.75/MTok	N/A	Open source option

Recommendation

For most users: Use **Claude Haiku 4.5 via OpenRouter** for the best combination of: - ✅ 75% cost savings vs direct Anthropic - ✅ Near-frontier intelligence - ✅ Fast responses - ✅ 200K token context

For complex tasks: Use **Claude Sonnet 4.5** (either provider) for deeper reasoning and analysis.

Project Context

The AI assistant automatically has access to your current project context, including:

- Project name and description
- AI configuration (model, training parameters)
- Detection settings (threshold, confidence, NMS)
- Data splits (train/val/test image counts)
- Training history and metrics

This allows the assistant to provide specific, actionable advice for your project.

Example Use Cases

Tune Detection Parameters

```
You: My detection is finding too many false positives. How can I reduce them?

AI: Based on your project, I recommend:
1. Increase confidence_threshold from 0.25 to 0.4
2. Adjust iou_threshold to 0.5 for stricter overlap filtering
3. Consider retraining with more negative examples

Would you like me to explain how to adjust these in your project?
```

Training Guidance

```
You: How many epochs should I train for?

AI: For your dataset with 150 training images:
- Start with 50 epochs as a baseline
- Monitor validation loss - it should decrease steadily
- If validation loss plateaus before 50 epochs, you can stop early
- If still improving at 50, try 75-100 epochs

Your current batch_size of 8 is good for this dataset size.
```

Project Analysis

```
You: Analyze my project configuration

AI: Here's what I found in your oslo_project:
✅ Good splits: 70/20/10 train/val/test distribution
⚠️  Low image count (150 total) - consider data augmentation
✅ Reasonable batch size (8) for dataset
💡 Suggestion: Enable data augmentation in training config
💡 Suggestion: Consider collecting more diverse training data
```

Feature Questions

```
You: How do I export detection results to CSV?

AI: To export detection results:
1. Run detection: `crackz detect run <project>`
2. Results are saved in project/artifacts/detections/
3. Use export command: `crackz export --format csv <project>`

Or in GUI:
1. Click "Run Detection" button
2. Results appear in Detection Results panel
3. Click "Export" → "CSV Format"
```

Character Limits

The input box has a character limit to ensure efficient API usage:

- **Up to 3,500 characters**: Green indicator (optimal)
- **3,500-3,800 characters**: Orange indicator (warning)
- **Over 3,800 characters**: Red indicator (too long)

If your message is too long, consider: - Breaking it into multiple smaller questions - Removing unnecessary details - Being more concise in your description

Privacy & Security

API Key Storage

- API keys are stored **locally** in your application settings
- Keys are **never** transmitted except to Anthropic's API
- Consider using environment variables for production

Data Sent to Claude

When you send a message, the following data is transmitted to Anthropic: - Your message text - Conversation history (previous messages in current session) - Project context (configuration, not images or training data)

Not sent: - Image files - Training data - Detection results (unless explicitly mentioned in your message)

Best Practices

- Don't include sensitive information in messages
- Use project-specific terminology to get better help
- Clear conversation history when switching projects

Troubleshooting

"API key not configured"

Solution: Click the ⚙ Settings button and enter your Anthropic API key.

"Error code: 401 - invalid x-api-key"

Solution: Your API key is invalid or expired. Get a new key from Anthropic Console.

"Error code: 404 - model not found"

Solution: The selected model doesn't exist. Open settings and select a valid model: - `claude-sonnet-4-5-20250929` - `claude-haiku-4-5-20251001` - `claude-opus-4-1-20250805`

Chat window won't open

Solutions: - Try alternative keyboard shortcuts: `§`, `~`, or `Ctrl+/` - Use menu: Help → AI Chat Assistant - Check if window is behind main window (click taskbar icon)

Slow responses

Causes: - Network latency - Model processing time (Opus is slower than Sonnet/Haiku) - Long conversation history

Solutions: - Clear conversation history (🗑️ Clear button) - Switch to Claude Haiku 4.5 for faster responses - Check your internet connection

No project context in responses

Solution: Make sure a project is loaded in the main GUI. The AI can only provide project-specific advice when a project is active.

API Costs

Using the AI chat assistant incurs costs based on your selected provider and model:

Pricing Comparison (November 2024)

Anthropic (Direct): - **Claude Sonnet 4.5**: $3/MTok input, $15/MTok output - **Claude Haiku 4.5**: $1/MTok input, $5/MTok output - **Claude Opus 4.1**: $15/MTok input, $75/MTok output

OpenRouter (Recommended for Cost Savings): - **Claude Sonnet 4.5**: $2.25/MTok input, $11.25/MTok output (25% savings) - **Claude Haiku 4.5**: $0.25/MTok input, $1.25/MTok output (75% savings ⭐) - **Claude Opus 4.1**: $11.25/MTok input, $56.25/MTok output (25% savings) - **GPT-4o**: $2.50/MTok input, $10/MTok output - **Llama 3.1 70B**: $0.52/MTok input, $0.75/MTok output

Cost Management Tips

1. **Use OpenRouter** for up to 75% savings vs direct Anthropic
2. **Use Haiku** for simple questions (cheapest option)
3. **Clear history** regularly to reduce context size
4. **Be concise** in your messages
5. **Monitor usage**:
6. Anthropic: Console
7. OpenRouter: Usage Dashboard

Typical Usage Examples

With Claude Haiku via OpenRouter (75% savings): - Simple question (~100 tokens): **$0.000025** (was $0.0001) - Detailed analysis (~500 tokens): **$0.000125** (was $0.0005) - Extended conversation (5 messages): **$0.000625** (was $0.0025)

With Claude Sonnet (either provider): - Simple question: $0.0003 - $0.0004 - Detailed analysis: $0.0015 - $0.002 - Extended conversation: $0.0075 - $0.01

Cost Optimization Strategy

Maximum Savings (Recommended): 1. Use **OpenRouter** as provider 2. Select **Claude Haiku 4.5** as default model 3. Switch to **Claude Sonnet 4.5** only for complex tasks 4. Result: **~75% cost reduction** vs direct Anthropic

Architecture

Components

ChatWindow (`chat_window.py`)

- Floating Tkinter window with always-on-top behavior
- Message display with auto-scroll
- Input box with character counter
- Settings and Clear buttons

AIChatService (`ai_chat.py`)

- Manages conversation history
- Handles async API calls to Claude
- Provides project context extraction
- Thread-safe callback mechanism

ChatSettingsDialog (`chat_settings_dialog.py`)

- API key configuration
- Model selection
- Settings persistence

Key Features

Async Communication

- Runs API calls in background thread
- Non-blocking UI during API requests
- Typing indicator shows activity

Context Awareness

- Automatically extracts project information
- Includes configuration in system prompt
- Maintains conversation history

Error Handling

- Graceful API error recovery
- User-friendly error messages
- Fallback for missing API key

Development

Adding New Context

To add more project context to the AI assistant:

1. Edit `ai_chat.py` method `_get_context()`:

```
def _get_context(self) -> str:
    if self.app_state is None or self.app_state.project is None:
```

```
        return "No project loaded"

    project = self.app_state.project

    # Add your new context here
    return f"""
    Current project: {project.name}

    Your new context:
    - Additional info: {project.additional_info}
    """
```

2. Context is automatically included in every message to Claude

Testing

Run GUI tests:

```
uv run task tests
```

Manual testing checklist: - [ ] Open/close chat window with keyboard shortcuts - [ ] Configure API key in settings - [ ] Send message and receive response - [ ] Test multi-line messages with Shift+Enter - [ ] Clear conversation history - [ ] Test error handling (invalid API key) - [ ] Verify project context in responses

Related Documentation

- GUI User Guide - Main GUI documentation
- Configuration - Project settings
- Detection Guide - Running crack detection

Support

For issues or questions: - **Documentation**: Check docs/ - **Issues**: GitHub Issues - **Email**: theresia.lundgren@anaxiatech.se

---

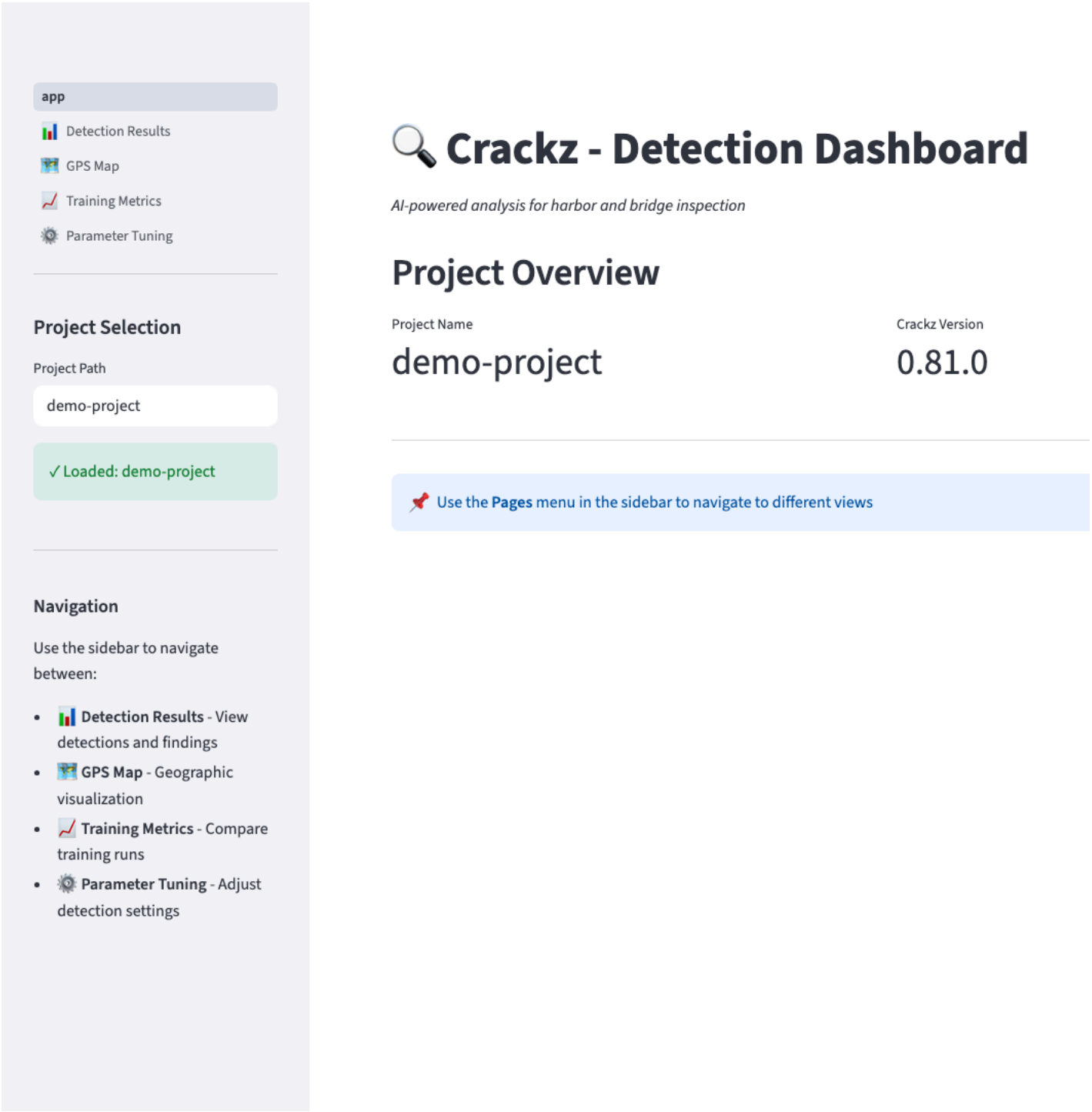
Dashboard

Streamlit Dashboard

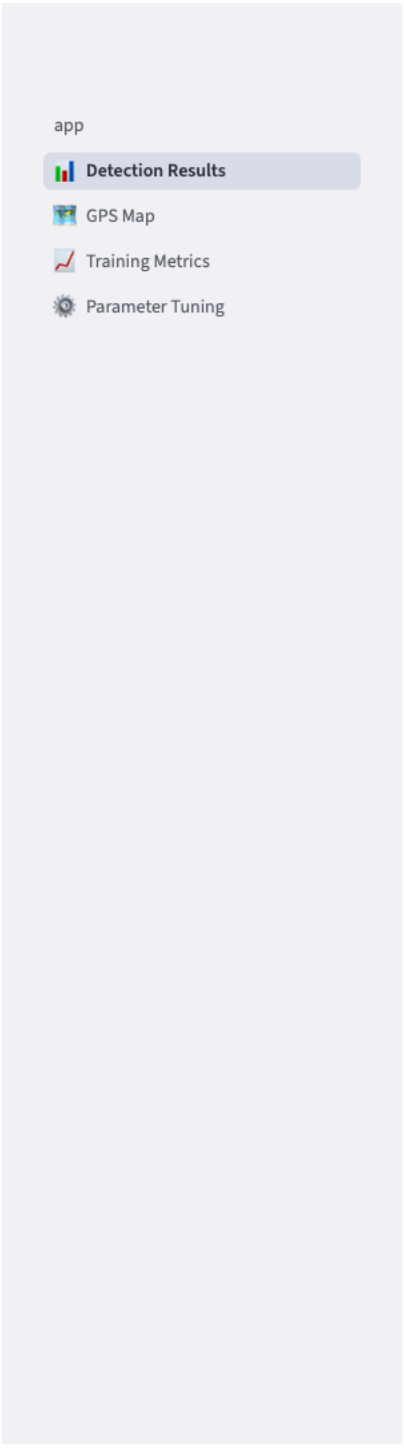
The Crackz Streamlit dashboard provides an interactive web-based interface for analyzing crack detection results, visualizing training metrics, and exploring GPS-tagged crack locations.

Dashboard Screenshots

Home Page



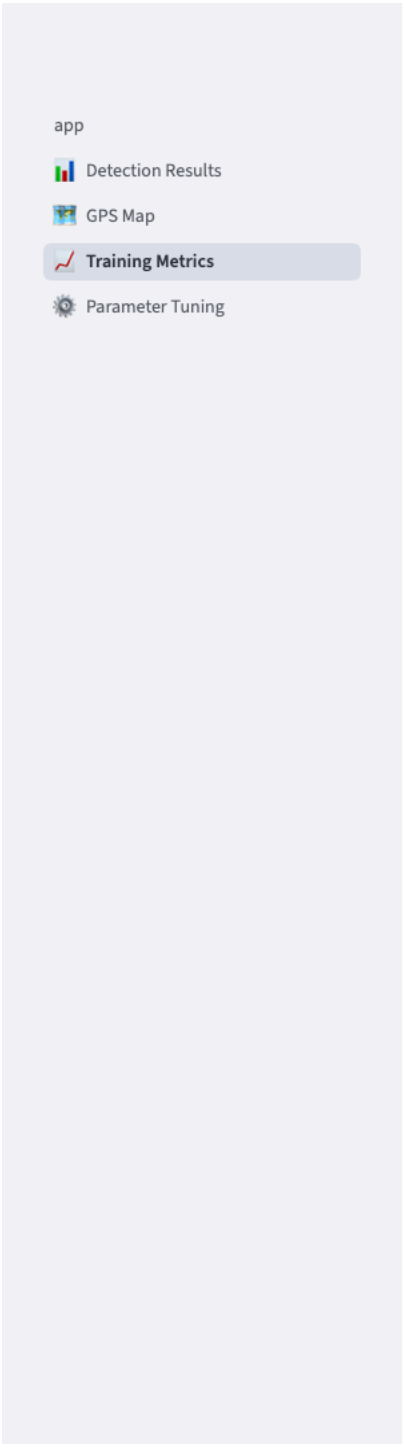
Detection Results


app
Detection Results
GPS Map
Training Metrics
Parameter Tuning
Detection Results
View detection results with visualizations and statistics
No project loaded. Please select a project from the home page.

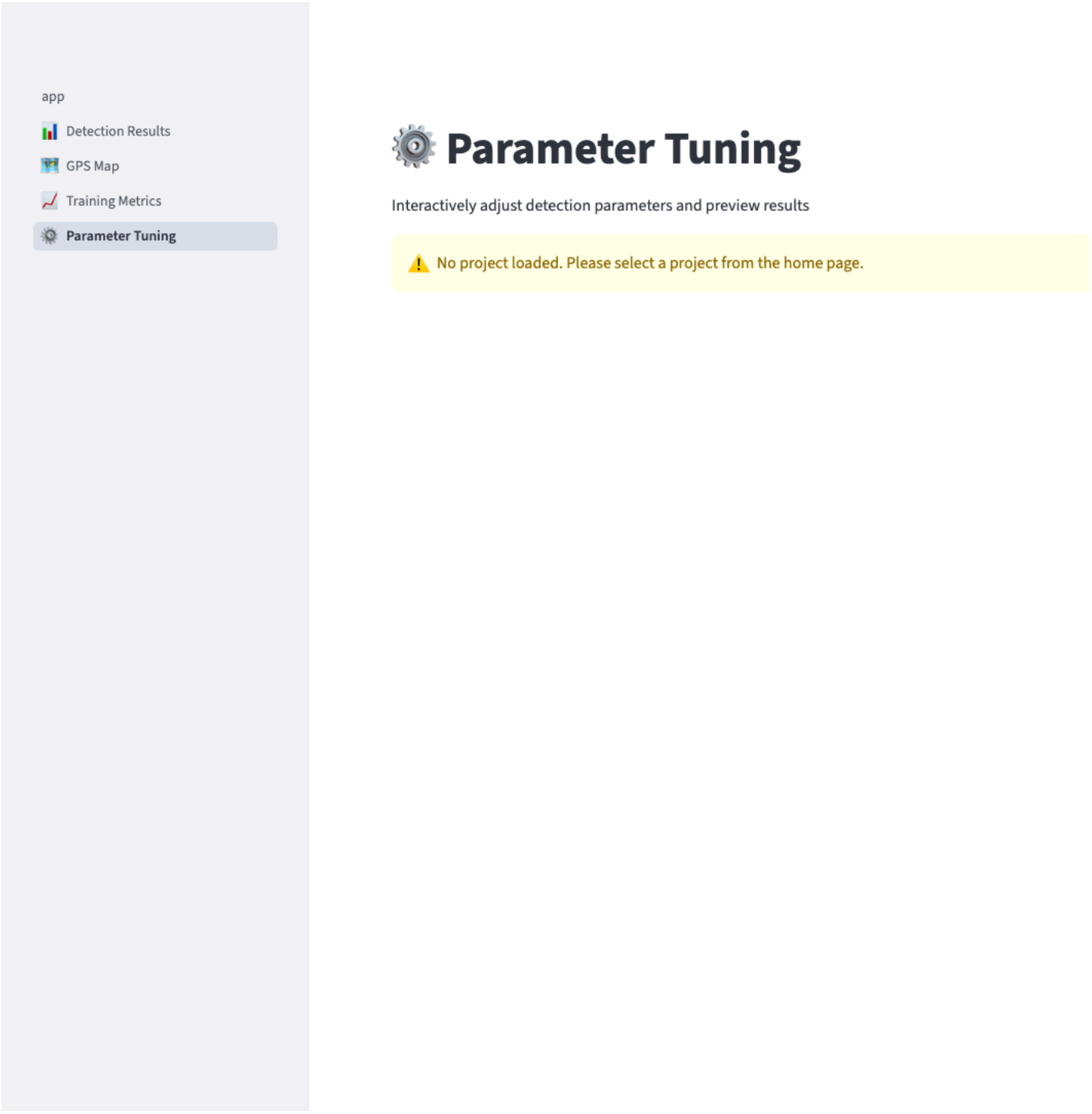
GPS Map



Training Metrics



Parameter Tuning



Overview

The dashboard is a comprehensive visualization tool that complements the CLI and GUI, offering:

- **Interactive data exploration** with filters and sorting
- **Rich visualizations** using Plotly charts
- **GPS mapping** with Folium for geographic visualization
- **Real-time parameter tuning** for detection thresholds
- **Export capabilities** for results and metrics

Quick Start

Launch from CLI

```
# Launch with a specific project
crackz dashboard --project my-project

# Launch with custom port
crackz dashboard --project my-project --port 8502
```

Launch from GUI

The GUI provides the most convenient way to launch the dashboard:

1. Open your project in the Crackz GUI (`crackz-gui`)
2. Click **Tools → Launch Dashboard** from the menu bar
3. Configure launch options in the dialog:
4. **Automatic port selection** (recommended) or custom port
5. **Project validation** ensures compatibility
6. Click **Start Dashboard** to launch

GUI Integration Benefits: - **Automatic project context**: Uses currently open project - **Real-time status tracking**: Visual indicators show server status - **Error handling**: User-friendly error messages and solutions - **Simplified workflow**: No CLI commands to remember

The dashboard will automatically open in your default web browser at the specified port.

Alternative Launch Methods

You can also launch the dashboard directly with Streamlit:

```
streamlit run core/src/crackz/dashboard/app.py -- --project my-project
```

Or programmatically from Python:

```
from crackz.dashboard import launch_dashboard

launch_dashboard(project_name="my-project", port=8501)
```

Dashboard Pages

The dashboard consists of four main pages accessible from the sidebar menu:

1. Detection Results 📊

View and analyze crack detection results with comprehensive visualizations and statistics.

Features: - Summary metrics (total detections, average confidence, average IoU, crack count) - Interactive charts: - Detection counts (cracks vs no cracks) - Confidence score distribution - IoU score distribution (when ground truth available) - Crack severity distribution - Filterable and sortable results table - Paginated image gallery showing detection visualizations - Export results as CSV or JSON

Usage: 1. Select a project from the home page 2. Navigate to "Detection Results" page 3. Use filters to refine results (crack presence, confidence threshold) 4. Sort by various metrics (batch, confidence, IoU) 5. Browse detection visualizations in the image gallery

2. GPS Map 🗺️

Visualize crack locations on an interactive map using GPS coordinates from image EXIF metadata.

Features: - Interactive Folium maps with marker clusters - Color-coded markers: - 🔴 Red: Cracks detected - 🟢 Green: No cracks - 🔵 Blue: Unknown status - Heatmap visualization showing crack concentration - Filters: - Show all locations / cracks only / no cracks - Minimum confidence threshold - Location details table with GPS coordinates - Export GPS data as CSV

GPS Data Requirements: - Images must contain EXIF GPS metadata - GPS coordinates are automatically extracted from image files - Supports images from test/val/train/raw splits

Usage: 1. Ensure your images have GPS EXIF metadata 2. Navigate to "GPS Map" page 3. Choose between markers or heatmap view 4. Apply filters to focus on specific detections 5. Click markers for detailed information

Troubleshooting: If no GPS data is shown: - Verify images have GPS metadata: `crackz geo extract-gps --project <name>` - Check that images are in your project's data directories - Use GPS-enabled cameras or phones to capture images

3. Training Metrics 📈

Monitor model training progress with detailed metrics and visualizations.

Features: - Training summary (total epochs, final losses, best validation loss) - Training and validation loss plots over epochs - Additional metrics visualization (accuracy, precision, recall, etc.) - Epoch-by-epoch metrics table - Training configuration display - Export metrics as CSV or JSON

Data Source: Metrics are loaded from `artifacts/training/training_metrics.json`

Usage: 1. Train a model: `crackz train --project <name>` 2. Navigate to "Training Metrics" page 3. Review loss curves and metrics 4. Compare training vs validation performance 5. Export metrics for further analysis

4. Parameter Tuning ⚙️

Interactively adjust detection parameters and preview results in real-time.

Features: - Adjustable parameters: - Detection threshold (0.0 - 1.0) - Minimum confidence (0.0 - 1.0) - Overlay transparency (0.0 - 1.0) - Overlay color (RGB color picker) - View modes: - Original image - Overlay with mask - Side-by-side comparison (original | mask | overlay) - Real-time detection statistics: - Crack coverage percentage - Crack pixel count - Image dimensions - Parameter export as JSON - Image selection from multiple sources

Usage: 1. Navigate to "Parameter Tuning" page 2. Select an image from the dropdown 3. Adjust parameters using sliders and color picker 4. Switch between view modes to compare 5. Export optimal settings for production use

Project Structure

The dashboard is organized as follows:

```
core/src/crackz/dashboard/
├──   init  .py          # Launch function
├── app.py              # Main dashboard app
├── pages/              # Dashboard pages
│   ├── 1_📊_Detection_Results.py
│   ├── 2_🗺️_GPS_Map.py
│   ├── 3_📈_Training_Metrics.py
│   └── 4_⚙️_Parameter_Tuning.py
└── utils/              # Utility modules
    ├── data_loader.py  # Data loading functions
    ├── image_utils.py  # Image processing utilities
    ├── map_utils.py    # GPS mapping utilities
    └── viz_utils.py    # Visualization functions
```

Data Locations

The dashboard reads data from these project locations:

Data Type	Location
Detection visualizations	`artifacts/detections/visualizations/`
Detection results JSON	`artifacts/detections/detection_results.json`
Training metrics JSON	`artifacts/training/training_metrics.json`
Source images	`data/{test,val,train,raw}/`
GPS coordinates	Extracted from image EXIF metadata

Configuration

Port Configuration

Change the default port (8501) when launching:

```
crackz dashboard --project my-project --port 8080
```

Streamlit Configuration

Create `.streamlit/config.toml` in your project root to customize Streamlit:

```
[server]
port = 8501
headless = true

[theme]
primaryColor = "#FF4B4B"
backgroundColor = "#FFFFFF"
secondaryBackgroundColor = "#F0F2F6"
```

```
textColor = "#262730"
font = "sans serif"
```

See Streamlit configuration docs for more options.

Dependencies

The dashboard requires these dependencies (automatically installed):

- `streamlit>=1.30.0` - Dashboard framework
- `streamlit-folium>=0.15.0` - Map integration
- `folium>=0.15.0` - Interactive maps
- `plotly>=5.18.0` - Interactive charts
- `pandas>=2.0.0` - Data manipulation

Best Practices

For Detection Results

1. **Run detection first**: Generate visualizations before viewing in dashboard

   ```
   crackz detect --project my-project
   ```

2. **Use meaningful thresholds**: Filter results with appropriate confidence thresholds
3. **Export data**: Download CSV/JSON for further analysis or reporting

For GPS Visualization

1. **Verify GPS metadata**: Check images have GPS before expecting map data

   ```
   crackz geo extract-gps --project my-project
   ```

2. **Use heatmap for patterns**: Switch to heatmap view to identify crack concentration areas
3. **Filter by confidence**: Focus on high-confidence detections for more reliable mapping

For Training Metrics

1. **Monitor regularly**: Check training metrics periodically during long training runs
2. **Compare epochs**: Use the epoch table to identify the best checkpoint
3. **Watch for overfitting**: Look for divergence between training and validation loss

For Parameter Tuning

1. **Start with defaults**: Begin with threshold=0.5, confidence=0.3
2. **Test multiple images**: Verify parameters work across different scenarios
3. **Export settings**: Save optimal parameters for batch processing

Troubleshooting

Dashboard Won't Start

Issue: Error when launching dashboard

Solutions: - Ensure Streamlit is installed: `uv sync` or `pip install streamlit` - Check port availability: Try a different port with `--port 8502` - Verify project path is correct

No Detection Images

Issue: Image gallery is empty

Solutions: - Run detection: `crackz detect --project <name>` - Check visualizations directory: `artifacts/detections/visualizations/` - Verify project has source images in data directories

No GPS Data

Issue: Map shows no markers

Solutions: - Verify images have GPS metadata - Use `crackz geo extract-gps` to check for GPS data - Ensure images are in supported directories (test/val/train/raw)

No Training Metrics

Issue: Training page shows no data

Solutions: - Train a model first: `crackz train --project <name>` - Check metrics file exists: `artifacts/training/training_metrics.json` - Verify training completed successfully

Performance Tips

1. **Use pagination**: The image gallery auto-paginates for better performance with many images
2. **Filter early**: Apply filters before loading large datasets
3. **Close unused tabs**: Navigate to specific pages rather than keeping all open
4. **Limit image size**: Large images may slow down rendering; resize if needed

Integration with CLI

The dashboard complements CLI workflows:

```
# 1. Initialize project
crackz init my-project

# 2. Import images
crackz import-images --source ./photos --project my-project

# 3. Train model
crackz train --project my-project

# 4. Run detection
crackz detect --project my-project

# 5. Launch dashboard to view results
crackz dashboard --project my-project
```

API Reference

Python API

```
from crackz.dashboard import launch_dashboard

# Launch dashboard programmatically
launch_dashboard(
    project_name="my-project",  # Optional: project to load
    port=8501                   # Optional: server port
)
```

CLI Reference

```
# Show help
crackz dashboard --help

# Launch with project
crackz dashboard --project my-project

# Launch with custom port
crackz dashboard --project my-project --port 8080

# Launch without initial project
crackz dashboard
```

Example Workflow

Here's a complete workflow using the dashboard:

```
# 1. Create and setup project
crackz init harbor-inspection
crackz import-images --source ./drone-photos --project harbor-inspection

# 2. Train model
crackz train --project harbor-inspection --epochs 50

# 3. View training progress
crackz dashboard --project harbor-inspection
# Navigate to "Training Metrics" page to review

# 4. Run detection on test set
crackz detect --project harbor-inspection --split test

# 5. Analyze results
crackz dashboard --project harbor-inspection
# Navigate to "Detection Results" page
# Navigate to "GPS Map" page to see locations

# 6. Tune parameters if needed
# Use "Parameter Tuning" page to optimize thresholds

# 7. Re-run with optimized parameters
crackz detect --project harbor-inspection --threshold 0.6

# 8. Generate final report
crackz report generate --project harbor-inspection --format pdf
```

Related Documentation

- CLI Reference - Command-line interface documentation
- GUI Guide - Desktop GUI application guide
- Getting Started - Initial setup and first steps
- Map Viewer - GPS visualization in the GUI

Support

For issues or questions about the dashboard:

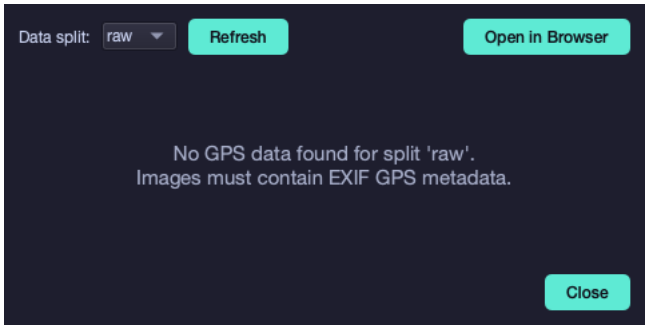
1. Check Troubleshooting section above
2. Review Streamlit documentation
3. Open an issue on GitHub with the `dashboard` label

Map Viewer

Map Viewer Feature

Overview

The Map Viewer provides an interactive map display for geotagged images by opening Leaflet.js maps in your default web browser.



Features

Map Display

- Opens interactive maps in your default web browser
- Full Leaflet.js functionality and performance
- Reliable loading of all map components
- Handles datasets of any size

Note: Maps always open in your default web browser for optimal performance and full Leaflet.js functionality. The browser displays an interactive Leaflet.js map with geotagged images shown as clickable markers on OpenStreetMap.

Interactive Maps

- Interactive Leaflet.js maps with full functionality (zoom, pan, markers, popups)
- OpenStreetMap tiles for base layer
- Color-coded markers for different image types

Technical Details

Dependencies

- **Leaflet.js**: JavaScript mapping library (loaded via CDN)
- **OpenStreetMap**: Map tile provider
- **webbrowser**: Python standard library module for opening URLs

Implementation

Maps are displayed in your default web browser for the best user experience.

Technical Implementation

- File: `gui/src/crackz_gui/components/map_viewer.py`
- Generates Leaflet.js HTML map with geotagged image markers
- Saves HTML to a temporary file
- Opens the map in your default web browser using `webbrowser.open()`
- Browser provides full JavaScript functionality for interactive maps

Usage

From GUI

1. Open a project in the Crackz GUI
2. Click the 🗺️ **Map** button in the toolbar (or navigate to **View > Map View**)
3. Select the data split (raw, train, val, test)
4. Click **"Open in Browser"** to view the interactive map
5. Map opens instantly in your default web browser

From Code

```
from pathlib import Path
from crackz_gui.qt.views.map_viewer_view import MapViewerDialog
from crackz.project.project import load_project

# Load a project
project = load_project(Path("path/to/project"))

# Note: MapViewerDialog requires a parent GUI app instance and is typically
# used within the Crackz GUI application. For command-line access to GPS data, use:
from crackz.project.geoposition import get_project_geopositions

# Get GPS data for all images in the project
gps_data = get_project_geopositions(project)
for image_path, geo_pos in gps_data.items():
    print(f"{image_path}: {geo_pos.latitude}, {geo_pos.longitude}")
```

Map Features

Markers

- **Blue markers**: Images without detections
- **Red markers**: Images with crack detections
- Click markers for image details

Legend

- Shows marker color meanings
- Displays total image count
- Located in bottom-right corner

Controls

- **Zoom**: Mouse wheel or +/- buttons
- **Pan**: Click and drag
- **Popups**: Click markers to see image names

GPS Data Requirements

Images must contain EXIF GPS metadata to appear on the map: - Latitude and Longitude in GPS IFD - Standard EXIF format supported - Test images in `tests/test_data/` and `oslo_project/` include GPS data for Västervik harbor

Test Data

All test images now include GPS coordinates for Västervik harbor, Sweden: - **Latitude**: 57.758611° N (57°45'31.0"N) - **Longitude**: 16.637222° E (16°38'14.0"E)

Updated Locations

- `tests/test_data/images/` - 13 images
- `oslo_project/images/` - 15 images (across all splits)

Script

Use `scripts/add_test_gps_data.py` to add GPS data to test images:

```
python scripts/add_test_gps_data.py
```

Troubleshooting

Map Doesn't Open in Browser

1. **Check Default Browser**: Ensure you have a web browser set as default
2. **Check Logs**: Look in `logs/crackz.log` for error messages
3. **Temporary File Issues**: Map HTML is saved to temp directory - ensure write permissions

No GPS Data Found

- Verify images contain EXIF GPS metadata
- Use `crackz.project.geoposition.get_project_geopositions()` to check project GPS data
- Run `python scripts/add_test_gps_data.py` to add GPS data to test images

Map Shows But No Markers

- Check that you selected the correct split (raw/train/val/test)
- Verify images in that split actually have GPS coordinates
- Click "Refresh" button to reload GPS data

Performance Issues

- Browser mode handles any number of markers efficiently
- For 1000+ images, initial map load may take a few seconds
- Use split selector to view subsets of your data

Future Enhancements

Potential improvements: - Clustering markers for large datasets - Image preview on marker hover - Filter by detection status - Custom marker icons - Heatmap overlay for detection density - Export map as static image

Related Files

- `gui/src/crackz_gui/components/map_viewer.py` - Main map viewer implementation
- `gui/src/crackz_gui/components/browser/toolbar.py` - Toolbar with map button
- `core/src/crackz/project/geoposition.py` - GPS extraction utilities
- `scripts/add_test_gps_data.py` - Add GPS data to test images

Image Tiling

Image Tiling

Crackz can slice large photogrammetry photos, aerial mosaics, or any high-resolution image into fixed-size tiles ready for model training and inference. Tiling is available via the GUI (**Project → Tile Images…**) and the CLI (`crackz tile-image`).

Overview

Large images are too memory-intensive to pass directly to a segmentation model. Tiling breaks them into uniform square patches that match the model's expected input size. Key capabilities:

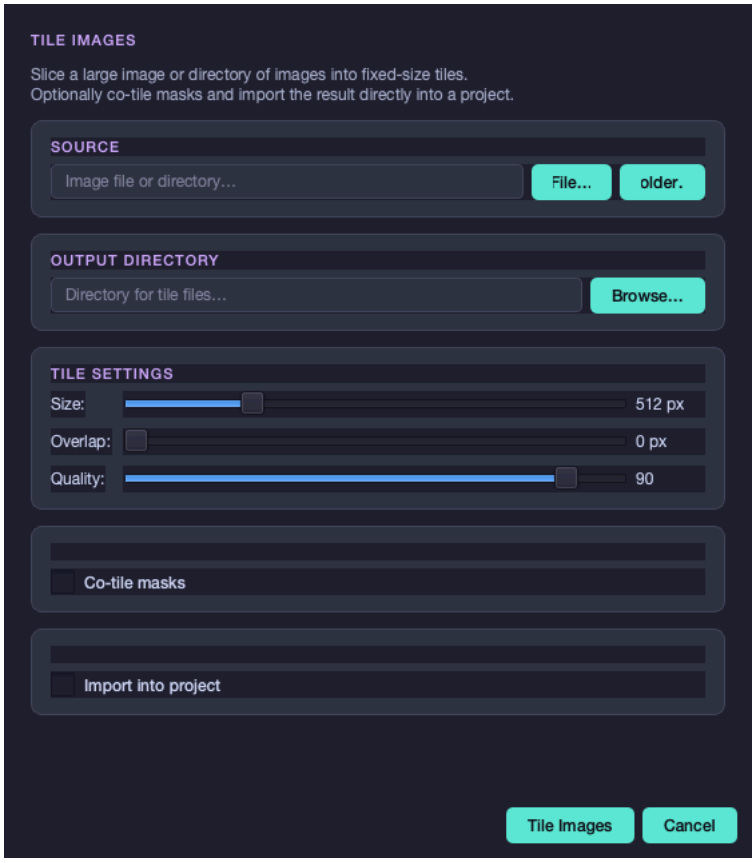
- **Single image or batch** — tile one file or an entire directory
- **Overlap** — extend tiles by N pixels on each edge to reduce boundary artifacts
- **Co-register masks** — tile binary annotation masks in sync with images (lossless PNG output)
- **Project import** — send tiles straight into a project; model input size is updated automatically
- **Auto-split** — randomly distribute tiles into train / val / test sets in one step

Tip: Refresh the tiling screenshots for this page with:

```
uv run python scripts/capture_qt_screenshots.py \
  --scenes tile-images-dialog,tile-images-dialog-masks,tile-images-dialog-import
```

GUI Usage

Open **Project → Tile Images…** from the menu bar.



Source

Click **File…** to select a single image, or **Folder…** to select a directory of images. After selection the dialog shows:

- **Single image** — the pixel dimensions (`2048 × 2048 px`)
- **Directory** — the number of images found

Output Directory

Click **Browse…** to choose where tile files will be written. The GUI picker only accepts existing directories, so create the destination folder first if needed.

Tile Settings

Control	Default	Range	Effect
Size	512 px	16 – 2048 px	Edge length of each square tile
Overlap	0 px	0 – (size − 1) px	Pixels shared between adjacent tiles
Quality	90	1 – 100	JPEG quality for image tiles

Once a single image is loaded, a live estimate appears below the sliders:

```
≈ 16 tiles · 4 × 4 grid · 512 × 512 px each
```

> **Overlap and tile count:** With `size = 512` and `overlap = 64`, adjacent tiles share a 64-pixel border. This increases the tile count but ensures the model sees context at tile edges.

Co-tile Masks

Check **Co-tile masks** to expand the mask section.

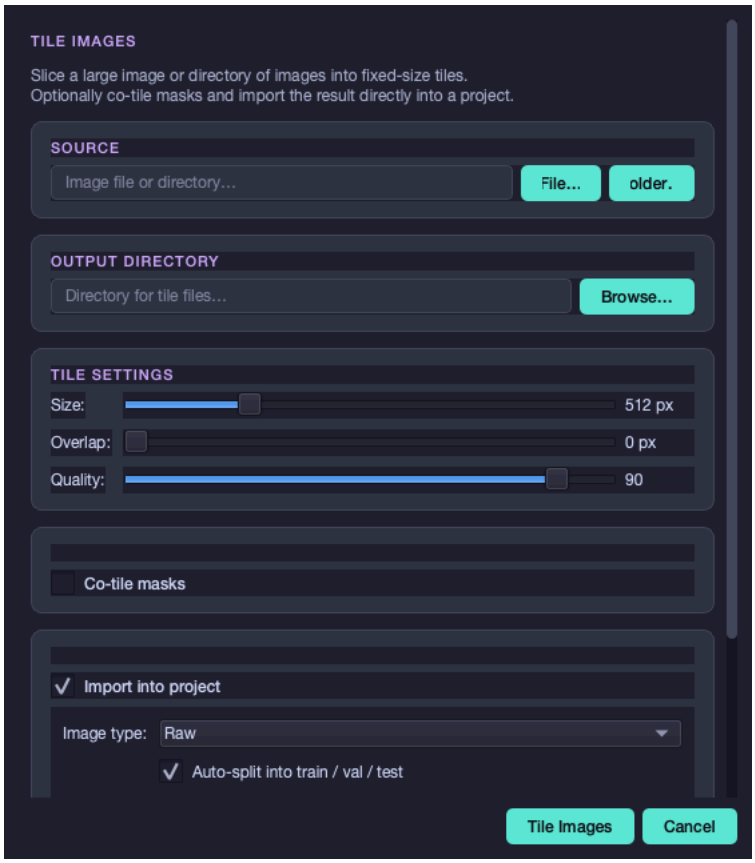


Field	Purpose
Masks source	PNG or JPEG mask file (or directory) matching the source images exactly
Masks output	Directory for the output mask tiles

Mask tiles are saved as lossless PNG regardless of source format to preserve binary pixel values.

Import into Project

Check **Import into project** to import tiles directly after tiling completes. A project must be open.



Field	Default	Purpose
Image type	Raw	Category to import tiles as: Raw, Train, Val, or Test
Auto-split	off	Randomly distribute tiles across train / val / test

When **Auto-split** is on, the Image type selector is disabled and three ratio fields appear:

Ratio	Default
Train	70 %
Val	15 %
Test	15 %

Ratios must sum to 100 %. An optional **Random seed** makes the split reproducible.

After a successful import the model input size is automatically set to the tile dimensions in the project configuration.

Running

Click **Tile Images** to start. A progress bar appears while tiling runs in the background. Keep the dialog open until the operation completes; closing is disabled while tiling is running.

When tiling completes without import enabled, the progress bar shows the tile count summary and the **Tile Images** button re-enables so you can adjust settings and run again.

CLI Usage

```
crackz tile-image --src <source> --out <output> [OPTIONS]
```

Basic examples

Tile a single image into 512 × 512 tiles:

```
crackz tile-image --src mosaic.jpg --out tiles/
```

Tile a directory with 256 × 256 tiles and 32 px overlap:

```
crackz tile-image --src raw_images/ --out tiles/ --tile-size 256 --overlap 32
```

Tile images and their masks together:

```
crackz tile-image \
  --src raw_images/ \
  --out tiles/images/ \
  --masks-src annotations/ \
  --masks-out tiles/masks/ \
  --tile-size 512
```

Tile and import directly into a project:

```
crackz tile-image \
  --src mosaic.jpg \
  --out tiles/ \
  --project ~/projects/harbor_survey \
  --type raw
```

Tile and auto-split into train / val / test:

```
crackz tile-image \
  --src raw_images/ \
  --out tiles/ \
  --masks-src annotations/ \
  --masks-out tiles/masks/ \
  --project ~/projects/harbor_survey \
  --split \
  --train-ratio 0.7 \
  --val-ratio 0.15 \
  --test-ratio 0.15 \
  --random-seed 42
```

All options

Flag	Short	Default	Description
`--src`	`-s`	required	Source image file or directory
`--out`	`-o`	required	Output directory for image tiles
`--masks-src`	`-ms`	—	Mask file or directory (optional)
`--masks-out`	`-mo`	—	Output directory for mask tiles
`--tile-size`	`-z`	`512`	Square tile edge length in pixels
`--overlap`		`0`	Pixel overlap between adjacent tiles
`--quality`	`-q`	`90`	JPEG quality for image tiles (1–100)
`--project`	`-p`	—	Project directory for auto-import
`--cfg`	`-c`	—	YAML config file (alternative to `--project`)
`--type`	`-t`	`raw`	Image type when importing: `raw`, `train`, `val`, `test`
`--split`		false	Auto-split tiles into train / val / test
`--train-ratio`		`0.7`	Training set fraction (requires `--split`)
`--val-ratio`		`0.15`	Validation set fraction (requires `--split`)
`--test-ratio`		`0.15`	Test set fraction (requires `--split`)
`--random-seed`		—	Integer seed for reproducible splits

Tile Naming

Output files follow the pattern `{stem}_{row:04d}_{col:04d}.{ext}` in row-major order (top-left origin).

Example: A 2048 × 2048 image `mosaic.jpg` tiled at 1024 × 1024 (no overlap):

```
mosaic_0000_0000.jpg   mosaic_0000_0001.jpg
mosaic_0001_0000.jpg   mosaic_0001_0001.jpg
```

Mask tiles follow the same naming with `.png` extension:

```
mosaic_0000_0000.png   mosaic_0000_0001.png
mosaic_0001_0000.png   mosaic_0001_0001.png
```

Edge tiles that fall outside the image boundary are zero-padded to the full tile size.

Tile Count Formula

The exact number of tiles for a given image and settings:

```
step = max(tile_size - overlap, 1)
cols = max(ceil((width  - overlap) / step), 1)
```

```
rows = max(ceil((height - overlap) / step), 1)
n_tiles = cols × rows
```

Example — 4096 × 4096 image, tile size 512, overlap 64:

```
step = 512 - 64 = 448
cols = ceil((4096 - 64) / 448) = ceil(9.0) = 9
rows = 9
n_tiles = 81
```

Supported Formats

Input	Output
`.jpg`, `.jpeg`, `.png`, `.tif`, `.tiff`, `.bmp`, `.webp`	Image tiles: `.jpg` (JPEG, quality-controlled)
Mask: `.png`, `.jpg`, `.jpeg`, `.tif`, `.tiff`	Mask tiles: `.png` (lossless)

Python API

The tiling functions are in `crackz.project.tiling`:

```
from crackz.project.tiling import tile_image, tile_image_with_mask, tile_image_dir

# Tile a single image
tiles = tile_image(
    source=Path("mosaic.jpg"),
    dest_dir=Path("tiles/"),
    tile_size=(512, 512),
    overlap=0,
    quality=90,
)

# Tile image + mask together
img_tiles, msk_tiles = tile_image_with_mask(
    source=Path("mosaic.jpg"),
    mask=Path("mask.png"),
    dest_dir=Path("tiles/images/"),
    mask_dest_dir=Path("tiles/masks/"),
    tile_size=(512, 512),
    overlap=32,
    quality=90,
)

# Batch-tile a directory
img tiles, msk tiles = tile image dir(
    source_dir=Path("raw_images/"),
    dest_dir=Path("tiles/images/"),
    tile_size=(512, 512),
    overlap=0,
    quality=90,
    mask_source_dir=Path("annotations/"),  # optional
    mask_dest_dir=Path("tiles/masks/"),    # optional
)
```

Report Generation

Report Generation

Generate professional Word documents with detection results and analysis.

Overview

The reporting feature creates branded Microsoft Word documents (.docx) containing:

- **Project Information**: Name, description, creation date
- **Detection Statistics**: Total images analyzed, detection rate, average confidence
- **Severity Breakdown**: Distribution of crack severity levels
- **Sample Images**: Representative detection results organized by severity
- **Anaxiatech Branding**: Company logo, colors, and contact information

Reports are ideal for sharing inspection results with clients, stakeholders, or archival purposes.

Quick Start

Basic Report Generation

Generate a report for your project:

```
crackz report generate --project demo-project
```

This creates `demo-project_report.docx` in the project directory.

Custom Output Path

Specify where to save the report:

```
crackz report generate --project demo-project --output inspection_report.docx
```

Using Config File

Generate from a YAML configuration:

```
crackz report generate --cfg project_config.yaml --output report.docx
```

Report Contents

1. Header Section

- **Anaxiatech logo** (automatically included if available)
- **Document title**: "Harbor Infrastructure Inspection Report"
- **Subtitle**: "Crack Detection Analysis"
- **Generation date**: Timestamp when report was created

2. Project Information

A table containing: - Project name - Description - Creation date

3. Detection Statistics

Summary metrics including: - Total images analyzed - Number of images with detected cracks - Overall detection rate (%) - Average confidence level (%)

4. Severity Breakdown

Distribution of results by confidence level: - **Crack Detected**: High confidence detections (≥5% default) - **Probable Crack**: Medium confidence (2-5% default) - **Possible Crack**: Low confidence (0.5-2% default) - **No Crack**: Below detection threshold

Each severity level shows: - Number of images - Percentage of total - Color-coded for easy identification

5. Detection Results

Sample images organized by severity (up to 5 images per category): - Original filename - **Detection visualization images** with crack overlays and heatmaps - Confidence percentage - Crack area percentage (when available)

The report includes the actual detection visualization images showing: - Red/yellow overlay highlighting detected cracks - Color-coded intensity based on detection confidence - Visual representation of the AI's crack detection

Note: Images are automatically resized to fit page width while maintaining aspect ratio. Large image sets show a representative sample with a note about complete results.

6. Company Information

- About Anaxiatech section
- Contact information (email, website)
- Disclaimer about AI-generated results

Report Customization

Severity Thresholds

Confidence thresholds are configured in your project's detection settings:

```
ai:
  detection:
    min_confidence: 0.005      # 0.5% - Possible Crack threshold
    medium_confidence: 0.02    # 2% - Probable Crack threshold
    high_confidence: 0.05      # 5% - Crack Detected threshold
```

Adjust these values to control how detections are categorized in reports.

Image Limits

By default, reports include up to 5 sample images per severity level to keep file sizes manageable. This prevents extremely large documents when processing hundreds or thousands of images.

For complete results, refer to the project's `detections/` directory.

Use Cases

Client Deliverables

Share inspection results with harbor authorities or infrastructure managers:

```
crackz report generate --project harbor_inspection_2024 \
  --output "Harbor_Inspection_Report_2024-Q4.docx"
```

Progress Reports

Document ongoing monitoring campaigns:

```
# Monthly reports
crackz report generate --project monthly_monitoring \
  --output "reports/2024-11_monitoring.docx"
```

Compliance Documentation

Create records for regulatory compliance:

```
crackz report generate --project compliance_check \
  --output "compliance/inspection_$(date +%Y%m%d).docx"
```

Internal Reviews

Generate reports for internal quality assurance:

```
crackz report generate --project qa_validation \
  --output "internal/qa_review.docx"
```

Technical Details

Output Format

- **Format**: Microsoft Word (.docx) - Office 2007 and later
- **Compatibility**: Microsoft Word, LibreOffice, Google Docs, Apple Pages
- **Styling**: Professional theme with Anaxiatech brand colors
- **Images**: Embedded at appropriate resolution (max 6 inches width)

Requirements

- Detection must be run first (`crackz detect run`)
- Detection results JSON must exist in project
- Sufficient disk space for Word document (typically 5-50 MB depending on images)

Performance

Report generation typically takes: - **Small projects** (10-50 images): 1-5 seconds - **Medium projects** (50-200 images): 5-15 seconds - **Large projects** (200+ images): 15-30 seconds

Large projects show only sample images to maintain reasonable generation times.

Troubleshooting

No Detection Results

Problem: "No detection results found for project"

Solution: Run detection first:

```
crackz detect run --project demo-project
```

Images Not Appearing

Problem: Report shows "⚠️ Detection image not found" messages

Causes: - Detection visualizations were deleted or moved - Detection hasn't been run yet - Project directory structure is incorrect

Solution: Verify detection visualization images exist:

```
# Check for visualization images (new structure)
ls -la demo-project/artifacts/detections/visualizations/

# Or legacy location (older projects)
ls -la demo-project/artifacts/detections/*.png
```

Re-run detection if needed:

```
crackz detect run --project demo-project
```

Note: The report uses detection visualization images from `artifacts/detections/visualizations/` which show the original images with crack overlays. These are automatically generated during detection.

Large File Size

Problem: Report file is very large (>100 MB)

Cause: High-resolution detection images

Solutions: 1. The report automatically limits to 5 images per severity level 2. For smaller files, reduce detection image resolution in project config:

```
data:
  size: [512, 512]  # Use smaller images
```

Permission Denied

Problem: Cannot save report file

Solutions: - Check write permissions on output directory - Close the file if it's open in Word or another application - Specify a different output path with `--output`

Best Practices

Naming Conventions

Use descriptive, dated filenames:

```
crackz report generate --project harbor_main \
  --output "Harbor_Main_Inspection_2024-11-05.docx"
```

Batch Processing

Generate reports for multiple projects:

```
for project in project1 project2 project3; do
  crackz report generate --project "$project" \
    --output "reports/${project}_$(date +%Y%m%d).docx"
done
```

Archive with Project

Keep report alongside project data:

```
crackz report generate --project demo-project  # Creates demo-project/demo-project_report.docx
```

Version Control

Track report parameters in project config:

```
# Add to project notes
project:
  notes: "Report generated 2024-11-05 with default thresholds"
```

See Also

- Detection Workflow - Running crack detection
- Configuration - Adjusting detection parameters
- CLI Cookbook - Common workflow examples
- Project Layout - Understanding project layout

CLI Reference

app

Main entrypoint for Crackz CLI.

Usage:

```
[OPTIONS] [COMMAND] [ARGS]...
```

Options:

```
 -v, --version         Show version and exit
--debug                Enable debug logging
--quiet                Only warnings and errors
--log-dir DIRECTORY    Central log directory (precedence: env CRACKZ_LOG_DIR
                       > --log-dir > system temp). Created if missing.
                       Project logs are always under <project>/logs.
--telemetry            Opt in to local telemetry: append structured events to
                       logs/events.jsonl. No image data, no PII. Also enabled
                       by CRACKZ_TELEMETRY_ENABLED=1. Off by default. See
                       ADR-021.
--install-completion   Install completion for the current shell.
--show-completion      Show completion for the current shell, to copy it or
                       customize the installation.
```

Dashboard

The `dashboard` command launches an interactive web-based dashboard for visualizing detection results, training metrics, and GPS locations.

Basic Usage

Launch the dashboard with a project:

```
crackz dashboard --project my-project
```

The dashboard will open automatically in your web browser at `http://localhost:8501`.

Custom Port

Specify a different port:

```
crackz dashboard --project my-project --port 8080
```

Without Initial Project

Launch the dashboard and select a project from the UI:

```
crackz dashboard
```

Features

The dashboard provides four main pages:

1. **Detection Results** (📊)
2. View detection statistics and charts
3. Browse detection visualizations
4. Filter and sort results
5. Export data as CSV/JSON
6. **GPS Map** (🗺️)
7. Interactive map with detection markers
8. Heatmap visualization
9. Location-based filtering
10. Export GPS data
11. **Training Metrics** (📈)
12. Training/validation loss curves
13. Metrics comparison across epochs
14. Export training data

15. **Parameter Tuning** (⚙️)

16. Interactive threshold adjustment
17. Real-time overlay preview
18. Export optimal parameters

Common Use Cases

View detection results:

```
crackz detect --project my-project
crackz dashboard --project my-project
# Navigate to "Detection Results" page
```

Monitor training:

```
crackz train --project my-project --epochs 50
crackz dashboard --project my-project
# Navigate to "Training Metrics" page
```

Visualize crack locations:

```
crackz dashboard --project my-project
# Navigate to "GPS Map" page
```

Optimize parameters:

```
crackz dashboard --project my-project
# Navigate to "Parameter Tuning" page
# Adjust thresholds and preview results
```

For detailed dashboard documentation, see the Dashboard Guide.

Project Cleaning

The `clean` command removes temporary files, logs, and training artifacts from a project while preserving important data like images and masks.

Basic Usage

Clean a project using the project directory:

```
crackz clean --project ./my_project
```

Short flag is also supported:

```
crackz clean -p ./my_project
```

What Gets Cleaned

By default, the clean command removes:

- **Log files** (`*.log` in the project's log directory)
- **TensorBoard logs** (training visualization data)
- **Model checkpoints** (saved model states during training) - stored in `models/` or `checkpoints/` for older projects
- **Detection results** (inference outputs)
- **Training metrics** (training statistics and metrics)

Always preserved: - Images (train, test, val, raw) - Masks (train, test, val, raw) - Project configuration files - Non-log files in the log directory

Dry Run (Preview Before Cleaning)

See what would be cleaned without actually deleting anything:

```
crackz clean --project ./my_project --dry-run
```

Example output:

```
Dry run: The following would be cleaned:
  • /path/to/project/logs/crackz.log
  • /path/to/project/artifacts/training/tensorboard
  • /path/to/project/artifacts/models
  • /path/to/project/artifacts/detections

Re-run without --dry-run to apply changes.
```

Logs Only Mode

Clean only log files while preserving all artifacts:

```
crackz clean --project ./my_project --logs-only
```

This removes: - Log files (`*.log`) - TensorBoard logs

But preserves: - Model checkpoints (in `models/` or legacy `checkpoints/` directory) - Detection results - Training metrics

Example Output

```
$ crackz clean --project ./my_project

Project cleaned! Removed 5 items (logs and artifacts).
```

With `--logs-only` :

```
$ crackz clean --project ./my_project --logs-only

Project cleaned! Removed 2 items (logs).
```

Common Use Cases

After training: Clean up old model files and logs to save disk space:

```
crackz clean -p ./my_project
```

Keep artifacts, clean logs: Useful when debugging or reviewing training metrics:

```
crackz clean -p ./my_project --logs-only
```

Safe preview: Always check what will be deleted first:

```
crackz clean -p ./my_project --dry-run
```

Project Migration (Schema & File Upgrades)

The `migrate` command upgrades older project YAML files and optionally migrates the artifacts directory structure to the latest organization.

Schema Migration

Upgrades project configuration to the current schema version. Current migration paths: - **Schema v0 → v1**: Adds `schema_version: 1` and ensures `ai.training.tensorboard` exists (default `false`) - **Schema v1 → v2**: Adds `data.mask_threshold: 1` for binary mask conversion - **Schema v2 → v3**: Adds `crackz_version` field with current version

File Migration (NEW)

Optionally reorganizes the artifacts directory structure for better organization: - `checkpoints/` to `models/` (model files in dedicated directory) - `training/tensorboard/` to `runtime/tensorboard/` (runtime data separation) - `training/lightning_logs/` to `runtime/lightning_logs/` (PyTorch Lightning logs) - `detections/*.png` to `detections/visualizations/` (visualization files organized)

Basic Usage

Schema migration only (default):

```
crackz migrate --project ./my_project
```

Schema + file migration:

```
crackz migrate --project ./my_project --migrate-files
```

Or using the config file:

```
crackz migrate --cfg ./my_project/my_project_project.yaml --migrate-files
```

Short flags are also supported:

```
crackz migrate -p ./my_project --migrate-files
crackz migrate -c ./my_project/my_project_project.yaml --migrate-files
```

Dry Run

Preview both schema and file changes without making modifications:

```
# Schema changes only
crackz migrate --project ./my_project --dry-run

# Schema + file changes
crackz migrate --project ./my_project --migrate-files --dry-run
```

Displays the steps that *would* be performed without modifying files.

Safety Features

Automatic Backups: By default, backups are created before any changes: - Schema files: `.v{version}.{timestamp}.bak` format - File migration: Complete artifacts directory backup before moving files

Skip Backup (not recommended):

```
crackz migrate --project ./my_project --no-backup
```

Interactive Confirmation: File migration requires user confirmation before proceeding, showing: - What files will be moved - Backup location - Rollback instructions if needed

Output Example

First run (migration needed):

```
[migrated] my_project schema 0 -> 1 (current=1) steps: 0->1: added tensorboard default + schema_version=1
```

Subsequent runs (idempotent) show:

```
[up-to-date] my_project schema 1 -> 1 (current=1) steps: none
```

Status Meanings

Status	Meaning
migrated	One or more migration steps applied
up-to-date / noop	Already at current schema version
(dry-run)	Only reported actions; no file changes

> Future versions will add additional incremental steps; each will remain backward compatible with prior saved projects using automatic default insertion.

Continuous Integration

The repository defines multiple GitHub Actions workflows:

Workflow	Purpose
`build.yml`	Lockfile check, unit tests, linting
`integration-tests.yml`	Runs `pytest -m integration` (Docker + end-to-end) on PRs
`release.yml`	Semantic release (version bump, changelog, GitHub release)
`docker-build.yml`	Docker image publishing (manual trigger only)
`smoke-tests.yml`	Quick runtime tests across platforms (manual trigger only)
`benchmark-tests.yml`	Runs `pytest -m benchmark` performance tests on PRs
`docs.yml`	Builds & deploys documentation (only on doc file changes)

Integration test marker usage locally:

```
pytest -m integration -vv
```

Skip them (default already does):

```
pytest -m "not integration"
```

Benchmark test marker usage locally:

```
pytest -m benchmark --benchmark-only
```

To add new CLI commands, extend `crackz/cli/*_cli.py` then re-build docs; mkdocs-typer auto-regenerates this reference page at build time.

Project Schema Migration (CLI)

When a command loads a project with an outdated or missing `schema_version`, Crackz will:

- Interactive shells: prompt to migrate in-place (backup `.bak` created)
- Non-interactive shells: print guidance unless `CRACKZ_AUTO_MIGRATE=1` is set

For manual migration, use:

```
crackz migrate --project <project-dir>
```

Options: `--dry-run`, `--no-backup`

See `migration.md` for full details.

Model Export and Import

The `model` subcommand provides tools for packaging and sharing trained models.

Export a Trained Model

Export a trained model checkpoint as a portable zip package:

```
crackz model export --project ./my_project --out my_model.zip
```

Short flags:

```
crackz model export -p ./my_project -o my_model.zip
```

What gets exported: - Model checkpoint file (best.ckpt, last.ckpt, or final.ckpt) - Model metadata (encoder, weights, architecture config) - Training configuration for reproducibility

Default behavior: - Automatically selects the best available checkpoint (best > last > final) - Output file defaults to `<project_name>_model.zip`

Export specific checkpoint:

```
crackz model export -p ./my_project -c final -o final_model.zip
```

Available checkpoint options: `best`, `last`, `final`

Import a Pre-trained Model

Import a model package into a project:

```
crackz model import --project ./my_project --model my_model.zip
```

Short flags:

```
crackz model import -p ./my_project -m my_model.zip
```

What happens during import: - Extracts checkpoint to project's models directory (or checkpoints in older projects) - Displays model configuration information - Validates the package format

Overwrite existing checkpoint:

```
crackz model import -p ./my_project -m my_model.zip --force
```

By default, import fails if a checkpoint with the same name already exists. Use `--force` to overwrite.

Use Cases

Share models with team members:

```
# Export from training project
crackz model export -p ./training_project -o harbor_crack_v1.zip

# Import into deployment project
crackz model import -p ./deployment_project -m harbor_crack_v1.zip
```

Transfer learning workflow:

```
# Export pre-trained model
crackz model export -p ./pretrained_project -o base_model.zip

# Import into new project for fine-tuning
crackz model import -p ./finetuning_project -m base_model.zip
```

Backup trained models:

```
# Export after successful training
crackz model export -p ./my_project -o backups/model_$(date +%Y%m%d).zip
```

Package Format

Model packages are standard zip files containing: - `<checkpoint_name>.ckpt` - PyTorch Lightning checkpoint - `metadata.json` - Model configuration and metadata

Example metadata:

```
{
  "project_name": "harbor_detection",
  "checkpoint_name": "best.ckpt",
  "model_config": {
    "encoder": "resnet34",
    "encoder_weights": "imagenet",
    "in_channels": 3,
    "num_classes": 1,
    "threshold": 0.5,
    "loss_name": "combo",
    "dice_weight": 0.7,
    "focal_weight": 0.3
  }
}
```

CLI Cookbook

CLI Cookbook

Task-focused examples for common workflows.

Conventions

Project directory layout (after `crackz init --name demo-project`):

```
demo-project/
 images/
   raw/ train/ val/ test/
 masks/
 artifacts/
 detections/
 checkpoints/
 project.yaml
```

General Tips

- Run `crackz --help` and append `--help` after any subcommand for options.
- Use `--project` or `--cfg` (YAML) – one is required for most run/train operations.
- Prefer relative paths kept inside the project folder for reproducibility.
- Use `--debug` for verbose logging, `--quiet` for minimal output.
- Supported image extensions (import + dataset loader): **.jpg, .jpeg, .png, .bmp, .tif, .tiff**

Initialize Project

```
crackz init --name demo-project
```

Overwrite existing (dangerous):

```
crackz init --name demo-project --force
```

Annotation Categories and Task Mode

Manage the project's annotation categories without editing the project YAML (works headless, e.g. over SSH). All commands take `--project`.

```
crackz config categories list --project demo-project
crackz config categories add rust --color "#aa5500" --name "Rust" --project demo-project
crackz config categories rename rust --key corrosion --name "Corrosion" --project demo-project
crackz config categories recolor corrosion "#112233" --project demo-project
crackz config categories remove corrosion --project demo-project
crackz config task-mode show --project demo-project
```

- `add` appends the category with the next free id. Keys are lowercase slugs (`a-z`, `0-9`, `_`, `-`, starting with a letter); colors are `#RRGGBB`.
- Ids are contiguous `1..N` and never renumbered, because masks refer to them. So `remove` only accepts the highest id, and it refuses while any mask in `train`, `val`, `test` or `raw` still contains that id. Remap those masks with `crackz masks remap`, or pass `--force`.
- Adding a second category to a single-category (binary) project switches it to multiclass. Binary masks store the defect as `255`, which is not a valid category id, so `add` warns when existing masks contain such values. Remap them before training, for example `crackz masks remap --label-map 255=1 --project demo-project`.
- Renaming a key (`rename --key`) does not rewrite trained models or stored detections, which keep the old key; `rename` prints a warning. A name-only rename is unaffected.
- `remove` skips unreadable files (for example `.DS_Store`) with a warning, and does not scan masks at all with `--force` or when the project is in binary mode, where masks carry no category ids.
- Each change recomputes `num_classes` and the resolved task mode (one category is binary, several is multiclass unless `task-mode set` overrides it).
- `--project` is required; `config` commands do not infer the project from the current folder.

Clean Project

Remove temporary files, logs, and training artifacts to save disk space:

```
# Preview what will be cleaned (safe)
crackz clean --project demo-project --dry-run

# Clean logs and artifacts
```

```
crackz clean --project demo-project

# Clean only logs, keep checkpoints and detections
crackz clean --project demo-project --logs-only
```

What gets removed: - Log files (`*.log`) - TensorBoard training logs - Model checkpoints (unless `--logs-only`) - Detection results (unless `--logs-only`) - Training metrics (unless `--logs-only`)

Always preserved: - Images (all splits: train/val/test/raw) - Masks - Project configuration

When to clean: - After training completes to free disk space - Before archiving a project - When troubleshooting to start fresh with logs

Import Images

Import raw images (resizing & quality normalization) into a split. Supported `--type` : raw, train, val, test.

```
crackz import-images \
  --project demo-project \
  --src ./samples \
  --type raw \
  --size 512 512 \
  --quality 90
```

Multiple splits (run per split):

```
crackz import-images --project demo-project --src ./samples --type train --size 512 512
crackz import-images --project demo-project --src ./samples --type val   --size 512 512
crackz import-images --project demo-project --src ./samples --type test  --size 512 512
```

Automatic split (NEW): Import and automatically split images into train/val/test:

```
crackz import-images \
  --project demo-project \
  --src ./samples \
  --split \
  --train-ratio 0.7 \
  --val-ratio 0.15 \
  --test-ratio 0.15 \
  --size 512 512
```

The `--split` option automatically shuffles and distributes images across train/val/test directories. Default ratios are 70%/15%/15%. When `--split` is used, the `--type` option is ignored. Masks are automatically matched and distributed with their corresponding images.

For reproducible splits (useful for experiments), add `--random-seed` :

```
crackz import-images --project demo-project --src ./samples --split --random-seed 42
```

For photogrammetry or video frames, consecutive images overlap heavily, so a per-file random split puts near-copies in both train and validation and inflates the scores. Use `--group-by` so every image of a group lands in the same split:

```
# one group per source folder
crackz import-images --project demo-project --src ./samples --split --group-by folder
# consecutive files in blocks of 10 (natural filename order)
crackz import-images --project demo-project --src ./samples --split --group-by sequence-block --group-size 10
```

Other modes: `filename-prefix` (names equal after dropping trailing numbers) and `tile-source` (tiles written by `crackz tile` , `{stem}_{row}_{col}` , stay with their source photo). Groups are assigned whole, so the split ratios are approximate when groups are large. `--group-by` requires `--split` ; `--group-size` applies only to `sequence-block` .

Include masks (auto-detected from `--src/masks`), or specify:

```
crackz import-images --project demo-project --src ./samples --masks_src ./samples/masks --type train
```

Overwrite processed images:

```
crackz import-images --project demo-project --src ./samples --type train --overwrite
```

> Training requires populated `images/train` (and ideally `val` , `test`). If you only imported to `raw/` , re-import with `--type train` or copy those images into `train/` before `crackz detect train` . Alternatively, use `--split` to automatically distribute images across all splits.

Import Masks Only (Utility Script)

For cases where you need to import masks separately from images, or when masks are organized in a split directory structure, use the `scripts/import_masks.py` utility:

```
# Basic usage - import from organized mask directories
python scripts/import_masks.py <project_dir> <masks_source_dir>

# Example: Import masks for skytteren_projekt from smasks directory
python scripts/import_masks.py ./skytteren_projekt ./smasks
```

```
# Custom size and quality
python scripts/import_masks.py ./my_project ./mask_sources --size 1024 1024 --quality 95
```

Directory Structure Expected: The `masks_source_dir` should contain subdirectories for each split:

```
masks_source_dir/
  train/
    mask_001.jpg
    mask_002.png
    ...
  val/
    mask_101.jpg
    ...
  test/
    mask_201.png
    ...
```

What It Does: - Imports masks from split-organized directories (train/val/test) - Resizes masks to specified dimensions (default: 512x512) - Converts masks to binary PNG format using `data.mask_threshold` from project config - Deletes original mask files after successful conversion - Automatically normalizes filenames (removes `_mask` suffix if present)

Options: - `--size WIDTH HEIGHT`: Target size for imported masks (default: 512 512) - `--quality QUALITY`: JPEG/PNG quality 1-100 (default: 90)

When to Use: - Masks are stored separately from images - Masks need to be added to an existing project - Re-importing masks with different size/quality settings - Masks are organized by split but images are already imported

Notes: - The script uses the project's `data.mask_threshold` setting for binary conversion - Masks are matched to images by filename stem (without extension) - Missing splits (e.g., no `val/` directory) are safely skipped - Binary conversion: pixels ≥ threshold → 255 (white), others → 0 (black)

Creating and Editing Masks

Interactive Mask Editor (CLI)

Open the GUI mask editor from the command line to create or edit crack masks:

```
# Basic usage - edit an image (mask path auto-derived)
crackz masks edit images/train/crack_001.jpg

# With custom mask path
crackz masks edit photo.jpg --mask output/my_mask.png

# With project context (smart path derivation)
crackz masks edit images/raw/inspection.jpg --project my_harbor_project
```

What It Does: - Opens interactive GUI mask editor from terminal - Allows painting crack masks with brush tool - Supports eraser, undo/redo, adjustable brush size/opacity - Saves binary PNG masks (0=background, 255=crack) - Works standalone without launching full GUI application

Mask Path Derivation: If `--mask` is not specified, the save path is intelligently derived:

1. **Project Structure** (if `--project` provided):

   ```
   Image:  project/images/train/crack_001.jpg
   Mask:   project/masks/train/crack_001.png  ← Auto-derived
   ```

2. **Adjacent Directory**:

   ```
   Image:  data/images/train/photo.jpg
   Mask:   data/masks/train/photo.png  ← Auto-derived
   ```

3. **Local Subdirectory** (fallback):

   ```
   Image:  /tmp/photo.jpg
   Mask:   /tmp/masks/photo.png  ← Auto-derived
   ```

Requirements: - Display server (X11, Wayland, or macOS GUI) - GUI dependencies (`pyside6` package) - Mouse or trackpad for drawing

Use Cases: - Quick mask creation without full GUI - Batch processing in scripts:

```
for img in images/train/*.jpg; do
    crackz masks edit "$img" --project my_project
done
```

- Remote editing via SSH with X forwarding:

```
ssh -X user@server
crackz masks edit /data/harbor/crack.jpg
```

Tips: - Command blocks until editor is closed (modal operation) - Use ↶ Undo / ↷ Redo for corrections (up to 20 levels) - Adjust brush size for large areas vs fine details - Cancel closes editor without saving

Converting Masks to Binary Format

If you have existing grayscale masks, convert them to binary format required for training:

```
# Convert all masks in a project (uses project threshold)
crackz masks convert --project my_harbor_project

# Convert with custom threshold
crackz masks convert --project my_harbor_project --threshold 128
```

What It Does: - Finds all non-PNG mask files in train/val/test/raw directories - Converts them to binary PNG (only 0 and 255 values) - Deletes original non-PNG files after conversion - Uses project's configured threshold or custom value

Threshold Logic: - Pixels ≥ threshold → 255 (crack/foreground) - Pixels < threshold → 0 (background) - Default from project config (typically 1 for JPEG masks)

When to Use: - After importing masks from external sources - When masks contain grayscale values (not binary) - Before training (binary masks are required) - JPEG compression created grayscale artifacts

---

Understanding Dataset Splits (train / val / test / raw)

Split	Purpose	Used When	Contains Labels?	Typical Size %
train	Optimizes model parameters (backprop / gradient descent)	Every epoch	Yes (masks)	60–80
val	Monitors generalization, selects best checkpoint, tunes thresholds	End of epoch (or mid-epoch)	Yes (masks)	10–20
test	Final, unbiased performance estimate AFTER model selection	Once after training	Yes (masks)	10–20
raw	Unlabeled / production imagery for inference & detection output	Inference only	No (optional)	N/A

Key Points

- Never tune hyperparameters or pick “best” epoch on the test set—reserve it strictly for a final report.
- The validation set guides early stopping, model selection, and threshold adjustment (e.g. segmentation threshold).
- If you lack enough images for a full split, prefer a small validation set over none; skip a test set only for rapid prototyping.
- Masks (ground-truth) are required for `train`, `val`, and `test`. Missing masks there will degrade learning or metrics. The loader will fabricate a zero mask if absent, which is only meaningful for `raw`.
- The `raw/` split is ideal for real-world inference runs that should not influence training statistics.

Recommended Workflow

1. Import or curate labeled data into `train`, `val`, and `test` (add masks with matching filename stems → `image_01.jpg` / `image_01.png`).
2. Place any unlabeled deployment imagery into `raw/`.
3. Run `crackz detect train` to fit the model; best checkpoint is picked via validation IoU.
4. Run `crackz detect run --project <p>` on `raw/` to generate predictions for field data.
5. Only after all tuning, evaluate on `test` to produce final performance metrics (IoU, Dice, etc.).

Minimum Viable Split (Small Datasets)

If images are scarce (< ~100): - Use ~80% train, 20% val; defer a true test set until more data arrives. - Report validation metrics, but label them clearly as not a final hold-out.

Common Pitfalls

Pitfall	Consequence	Prevention
Using test as validation	Inflated metrics / leakage	Keep test unseen until the end
Missing masks in train	Poor learning, near-zero gradients	Ensure mask filenames match image stems
Only raw images imported	Training fails (no supervised data)	Re-import with `--type train`
Highly imbalanced splits	Unstable validation metrics	Stratify if possible / check counts

Creating image masks

Masks (ground-truth) are required for `train`, `val`, and `test` splits. Below is a concise guide for end users on preparing masks that work well with Crackz.

1. Tools: choose a labeling tool such as `labelme`, CVAT, Roboflow, or a generic editor (Photoshop/GIMP). For quick starts, `labelme` is lightweight: `pip install labelme`.
2. Binary masks: export or save a single-channel mask per image where background = 0 (black) and crack pixels = 255 (white). Avoid anti-aliased/partial-alpha edges that create intermediate values.
3. Filenames: masks must share the same filename stem as the image (e.g. `image_001.png` → `images/train/image_001.jpg` and `masks/train/image_001.png`).
4. Location: place masks under `masks/<split>/` (e.g. `masks/train/`, `masks/val/`, `masks/test/`). The import tool will pair masks with images by filename.
5. Formats: prefer lossless formats such as PNG or TIFF. Do not use JPEG for mask files because compression introduces artifacts.
6. Dimensions: mask pixel dimensions must match the corresponding image exactly.
7. Importing: when using `crackz import-images`, either provide `--masks_src` or place masks inside the source folder under `masks/`. When using `--split`, masks are matched and split automatically.

Tips: - If you have polygon annotations (GeoJSON/LabelMe), export them to filled masks using the labeling tool's export function. - For bulk conversion from grayscale annotations, a thresholding step with OpenCV (`cv2.threshold`) plus morphological cleaning typically yields good binary masks.

Run Detection

```
crackz detect run --project demo-project
```

With a config file instead of `--project`:

```
crackz detect run --cfg config/detect.yaml
```

Model Selection

By default, detection automatically selects the best available checkpoint (`best.ckpt` > `last.ckpt` > `final.ckpt`). You can override this with the `--model` option:

```
# Use specific checkpoint by name
crackz detect run --project demo-project --model best
crackz detect run --project demo-project --model last
crackz detect run --project demo-project --model final

# Use specific epoch checkpoint (e.g., epoch-0.9234.ckpt)
crackz detect run --project demo-project --model epoch-0.9234
```

Auto-selection priority (when `--model` not specified): 1. `best.ckpt` - Highest validation IoU during training 2. `last.ckpt` - Most recent training checkpoint 3. `final.ckpt` - Final training checkpoint

Use cases: - **Default (auto-select)**: Production inference with best available model - **`--model best`**: Explicitly use highest-performing checkpoint - **`--model last`**: Test most recent training state (useful during iterative development) - **`--model epoch-X`**: Compare performance across different training epochs

Train Model

```
crackz detect train --project demo-project
```

(Uses `train/val/test` splits; ensure they are populated.)

Training Overrides (Fast Experiments)

For quick smoke tests or CI you can override epochs and batch size at runtime:

```
crackz detect train --project demo-project --epochs 1 --batch-size 1
```

Notes: - Overrides do NOT persist in the project YAML. - Useful for validating pipelines and container builds. - Combine with a tiny dataset for sub-minute feedback.

Model Export and Import

Share trained models between projects, team members, or deployment environments using portable model packages.

Export a Trained Model

Export your trained model as a portable zip package:

```
# Auto-selects best checkpoint, outputs to <project-name>_model.zip
crackz model export --project demo-project

# Specify output path
crackz model export --project demo-project --out models/harbor_v1.zip

# Export specific checkpoint (best, last, or final)
crackz model export --project demo-project --checkpoint best --out best_model.zip
```

Short flags:

```
crackz model export -p demo-project -o my_model.zip
crackz model export -p demo-project -c final -o final_model.zip
```

What gets packaged: - Model checkpoint file (`.ckpt`) - Metadata (encoder, architecture, training config) - Model configuration for reproducibility

Default behavior: - Automatically selects best available checkpoint: `best.ckpt` > `last.ckpt` > `final.ckpt` - Output defaults to `<project_name>_model.zip` if `--out` not specified

Import a Pre-trained Model

Import a model package into your project:

```
# Import model package
crackz model import --project demo-project --model harbor_v1.zip

# Overwrite existing checkpoint with same name
crackz model import --project demo-project --model harbor_v1.zip --force
```

Short flags:

```
crackz model import -p demo-project -m my_model.zip
crackz model import -p demo-project -m my_model.zip --force
```

What happens: - Extracts checkpoint to `checkpoints/` directory - Displays model configuration (encoder, weights, classes) - Validates package format

Note: Import fails if a checkpoint with the same name already exists. Use `--force` to overwrite.

Common Workflows

Share with team members:

```
# Team member A: Export trained model
crackz model export -p training-project -o shared/harbor_crack_v1.zip

# Team member B: Import into their project
crackz model import -p deployment-project -m shared/harbor_crack_v1.zip
```

Transfer learning:

```
# Export base model trained on general cracks
crackz model export -p general-cracks -o pretrained/base_model.zip

# Import into specialized project for fine-tuning
crackz model import -p harbor-specific -m pretrained/base_model.zip

# Fine-tune on harbor-specific data
crackz detect train -p harbor-specific --epochs 10
```

Backup before experiments:

```
# Backup current best model before trying new hyperparameters
crackz model export -p demo-project -c best -o backups/before_experiment.zip

# Run experiment
crackz detect train -p demo-project --epochs 50

# If results are worse, can import the backup
crackz model import -p demo-project -m backups/before_experiment.zip --force
```

Version control for models:

```
# Export with timestamp
crackz model export -p demo-project -o models/model_2025-10-11.zip

# Or use git-style versioning
crackz model export -p demo-project -o models/v1.0.0.zip
```

Deployment workflow:

```
# Export from training environment
crackz model export -p training-project -o production/model.zip

# Transfer to production server and import
crackz model import -p production-project -m production/model.zip
```

```
# Run detection in production
crackz detect run -p production-project
```

Package Contents

Model packages are standard zip files containing: - `<checkpoint_name>.ckpt` - PyTorch Lightning checkpoint with model weights - `metadata.json` - Model configuration and training metadata

You can inspect the package manually:

```
# Linux/Mac
unzip -l my_model.zip

# Windows PowerShell
Expand-Archive -Path my_model.zip -DestinationPath temp -Force
cat temp\metadata.json
```

Example `metadata.json`:

```
{
  "project_name": "harbor_detection",
  "checkpoint_name": "best.ckpt",
  "model_config": {
    "encoder": "resnet34",
    "encoder weights": "imagenet",
    "in_channels": 3,
    "num_classes": 1,
    "threshold": 0.5,
    "loss_name": "combo",
    "dice_weight": 0.7,
    "focal_weight": 0.3
  }
}
```

Geographic Data (GPS/EXIF)

Crackz can extract GPS coordinates from image EXIF data and visualize detection results on interactive maps or export to GeoJSON for GIS analysis.

Create Interactive Map

Generate an interactive HTML map showing image locations with detection status:

```
# Generate map for raw split (default)
crackz geo create-map --project demo-project --output harbor_map.html

# Map for specific split (train/val/test)
crackz geo create-map --project demo-project --output train_map.html --split train
```

Short flags:

```
crackz geo create-map -p demo-project -o map.html -s raw
```

What it does: - Extracts GPS coordinates from image EXIF data - Creates standalone HTML file with Leaflet.js map - Blue markers: Images without detection results - Red markers: Images with detection results - Interactive: Click markers to see image names - No internet required after generation

Use cases: - Visualize drone survey coverage - Track detection locations in harbors - Quality check GPS data before GIS export - Share results with stakeholders (open in any browser)

Export to GeoJSON

Export detection results to GeoJSON format for GIS software (QGIS, ArcGIS, etc.):

```
# Export all images with GPS data
crackz geo export-geojson --project demo-project --output detections.geojson

# Export specific split
crackz geo export-geojson --project demo-project --output train.geojson --split train
```

Short flags:

```
crackz geo export-geojson -p demo-project -o data.geojson -s raw
```

GeoJSON structure:

```
{
  "type": "FeatureCollection",
  "features": [
    {
      "type": "Feature",
      "geometry": {
        "type": "Point",
```

```
        "coordinates": [longitude, latitude]
      },
      "properties": {
        "image_name": "IMG_0001.jpg",
        "image path": "/path/to/image.jpg",
        "has_detection": true,
        "latitude": 59.123456,
        "longitude": 10.123456
      }
    }
  ],
  "metadata": {
    "project": "demo-project",
    "split": "raw",
    "total_images": 42,
    "images_with_detections": 15
  }
}
```

Use cases: - Import results into QGIS/ArcGIS for spatial analysis - Combine with existing GIS datasets - Generate heat maps or density visualizations - Create reports with geographic context

GPS Data Requirements

Images must contain GPS EXIF metadata: - **Drone images**: Most DJI drones embed GPS automatically - **Smartphones**: Enable location services when taking photos - **Supported tags**: GPSLatitude, GPSLongitude, GPSLatitudeRef, GPSLongitudeRef - **Formats**: Works with all supported image formats (.jpg, .jpeg, .png, .bmp, .tif, .tiff)

Check GPS data:

```
# Linux/Mac
exiftool image.jpg | grep GPS

# Python (Pillow)
from PIL import Image
img = Image.open("image.jpg")
exif = img.getexif()
gps_ifd = exif.get_ifd(0x8825)  # 0x8825 is GPS IFD tag
print(gps_ifd)
```

Workflow example:

```
# 1. Import drone photos (GPS data preserved in EXIF)
crackz import-images --project harbor --src ./drone_photos --type raw --size 512 512

# 2. Run detection
crackz detect run --project harbor

# 3. Generate map to visualize results
crackz geo create-map --project harbor --output harbor_results.html

# 4. Export to GeoJSON for GIS analysis
crackz geo export-geojson --project harbor --output harbor.geojson

# 5. Open map in browser to review
# (or import GeoJSON into QGIS/ArcGIS)
```

Defect Similarity Search (Embeddings)

Find visually similar defects across a project using DINOv2 embeddings stored in pgvector. Requires a reachable Postgres (see ADR-023); GPS filtering additionally requires PostGIS.

Backfill embeddings for every detection result, then query:

```
# 1. Embed all un-embedded detection results (idempotent; re-run safely)
crackz embed run --project harbor

# 2. Find defects similar to a stored detection result
crackz embed similar --project harbor --detection-id 42 -k 10

# 3. Find defects similar to an arbitrary query image
crackz embed search --project harbor --image query.jpg -k 5
```

GPS-filtered ranking

Pass `--radius-m` to restrict results to a geographic radius and rank them by a blended visual + proximity score (`final_score`). For `embed similar` , the queried detection's own GPS anchors the search; for `embed search` , supply the query location with `--lat` / `--lon` :

```
# Visually similar cracks within 50 m of detection 42
crackz embed similar --project harbor --detection-id 42 --radius-m 50

# Tune how much GPS proximity weighs against visual similarity (default 0.15)
crackz embed similar --project harbor --detection-id 42 --radius-m 50 --gps-weight 0.3

# Image search needs an explicit query location for a radius
crackz embed search --project harbor --image query.jpg \
    --radius-m 50 --lat 59.334 --lon 18.063
```

When `--radius-m` is set, the results table gains **Distance (m)** and **Score** columns. If GPS filtering can't run — PostGIS unavailable, or the query has no GPS coordinates — the command prints a note and falls back to pure-visual results (defects with no stored GPS are excluded only when a radius is applied).

> The same search is available in the GUI: right-click a detection and choose 🔍 **Find similar defects** to open the side-by-side comparison view.

Defect Progression (Tracks)

Track how one physical defect evolves across separate inspection runs using stored defect tracks. Requires a reachable Postgres (see ADR-023); GPS tiebreak is Haversine in Python and does **not** require PostGIS.

Propose track links

Auto-link detections across runs by visual similarity (and GPS proximity when available). The command is idempotent — confirmed links are never overwritten, and re-runs only reprocess `proposed`/unlinked rows:

```
# Auto-propose links for all detections in the project (uses config defaults)
crackz track link --project harbor

# Override the similarity threshold and GPS radius
crackz track link --project harbor --sim-threshold 0.90 --radius-m 30
```

Output: `Proposed N link(s) across M track(s).`

Matching rule: a pair of detections from different runs is linked when their DINOv2 cosine similarity is at or above `embedding_track_sim_threshold` (default 0.85). When both detections have GPS, the distance must also fall within `embedding_track_radius_m` (default 50 m) and proximity is used as a tiebreak. When either side has no GPS (e.g. underwater), matching falls back to visual-only — the GPS clause is skipped, not failed. Transitive groups (A↔B and B↔C) collapse into a single track.

View a defect's history

Show one detection's full inspection timeline — ordered chronologically by run date, with visual drift relative to the earliest member:

```
# Print the progression table for detection 42
crackz track history 42 --project harbor
```

The table columns are: `run_at`, `category`, `confidence`, `defect_sev`, `status` (`proposed` / `confirmed` / `rejected`), and `drift` (cosine similarity vs the earliest member in the track). When the detection has no track yet, the command prints `not linked` and exits cleanly.

Confirm or reject proposed links

Auto-proposed links are reviewed in the **GUI** — right-click a defect in the gallery and choose 📈 **Show progression**, then use the inline **Confirm** / **Reject** buttons on each timeline card. (There is no `crackz track confirm` / `reject` CLI command in this slice; confirmation is an interactive, per-member decision.) Confirmed links survive subsequent `track link` re-runs; rejected links stay rejected and are not re-proposed.

Typical workflow

```
# 1. Embed all detection results (idempotent)
crackz embed run --project harbor

# 2. Auto-propose track links
crackz track link --project harbor

# 3. Inspect a defect's history (CLI), or open the GUI to confirm/reject
crackz track history 42 --project harbor
```

> The same progression is available in the GUI: right-click a defect in the gallery and choose 📈 **Show progression** to open the ProgressionView with a timeline scrubber and inline confirm/reject controls.
>
> **Known limitation:** a worsening defect that drifts visually below the similarity threshold will not be auto-linked. Manual addition (split/merge) is deferred to a future anomaly-evolution slice.

Logging & Verbosity

```
crackz --debug detect run --project demo-project
crackz --quiet detect run --project demo-project
crackz --log-dir run_logs detect run --project demo-project
```

Environment override:

```
CRACKZ_LOG_DIR=logs_debug crackz detect run --project demo-project
```

Progress Reporting

Detection & training show a progress bar; additional log messages appear with `--debug`.

Cancelling Long-Running Operations

Press **Ctrl+C** to cancel detection or training operations:

```
# Start training
crackz detect train --project demo-project

# Press Ctrl+C to cancel at any time
# The operation will be cancelled gracefully
# Partial results (e.g., checkpoints) are preserved
```

The CLI will: - Intercept the interrupt signal (Ctrl+C) - Stop the operation at the next checkpoint - Exit with code 130 (standard for Ctrl+C) - Preserve any partial progress (e.g., model checkpoints saved before cancellation)

Using a YAML Config

Store unified hyperparameters and paths:

```
crackz detect run --cfg configs/detect.yaml
crackz detect train --cfg configs/train.yaml
```

Batch / Script Execution

Example bash loop (different projects or configs):

```
for P in demo-project other-project; do
  crackz detect run --project "$P" || echo "Failed: $P" >&2
done
```

Docker (CPU) Updated Examples

Container WORKDIR is now the user home `/home/app`.

```
# Init project (host ./project mapped to /home/app)
mkdir -p project
 docker run --rm -v %cd%\project:/home/app crackz-cli init --name demo-project

# Import images (mount samples to /data)
 docker run --rm -v %cd%\project:/home/app -v %cd%\samples:/data \
  crackz-cli import-images --project /home/app/demo-project --src /data --type raw --size 512 512

# Detection
 docker run --rm -v %cd%\project:/home/app crackz-cli detect run --project /home/app/demo-project
```

GPU variant:

```
docker run --gpus all --rm -v %cd%\project:/home/app crackz-cli:gpu detect run --project /home/app/demo-project
```

Troubleshooting

Issue	Resolution
Missing `--project` or `--cfg`	Supply one of them; required for detect/train
No images imported	Check `--src` path and file extensions (.jpg/.jpeg/.png/.bmp/.tif/.tiff)
Training says no images found	Ensure you imported into `images/train` (not only `raw`)
Empty detections	Verify model checkpoint & threshold settings
CUDA not used	Ensure GPU variant & driver; check `torch.cuda.is_available()`
Permission errors	Adjust log or output directory permissions

Generate Reports

Create professional Word documents with detection results:

```
# Generate report with default name (project_name_report.docx)
crackz report generate --project demo-project

# Specify custom output path
crackz report generate --project demo-project --output inspection_report.docx

# Use config file instead of project path
crackz report generate --cfg project.yaml --output report.docx
```

Report contents: - Project information and metadata - Detection statistics (total images, detection rate, confidence) - Severity breakdown (Crack Detected, Probable, Possible, None) - Sample images organized by severity (up to 5 per category) - Anaxiatech branding and contact information

Requirements: - Detection must be run first (`crackz detect run`) - Output format: Microsoft Word (.docx) - Compatible with: Word, LibreOffice, Google Docs, Pages

For detailed documentation, see Report Generation.

Active Learning (Phase 2)

Prioritize unlabeled images by uncertainty to maximize annotation efficiency.

Find Top Priority Images

Identify which images would most benefit from annotation:

```
# Find top 50 most uncertain images
uv run crackz active-learning run \
  --project demo-project \
  --top-n 50 \
  --method entropy \
  --out priorities.json
```

Available uncertainty methods: - `entropy` - High entropy indicates uncertain predictions (default) - `max_probability` - Lower confidence = higher uncertainty - `variance` - High disagreement across pixels - `mean_confidence` - Distance from decision boundary

View Method Descriptions

```
uv run crackz active-learning info
```

Process Specific Split

```
# Prioritize images in 'raw' split (default)
uv run crackz active-learning run --project demo-project --split raw

# Prioritize images in 'train' split
uv run crackz active-learning run --project demo-project --split train
```

Output Format

JSON output includes ranked images with uncertainty scores:

```
{
  "method": "entropy",
  "total_images": 500,
  "prioritized_count": 50,
  "images": [
    {
      "path": "image_042.jpg",
      "uncertainty_score": 0.8234,
      "rank": 1
    }
  ]
}
```

Workflow: 1. Train initial model on small labeled dataset 2. Run active learning on unlabeled images 3. Annotate top-N prioritized images 4. Retrain model with new annotations 5. Repeat until desired accuracy

Mask Propagation (Phase 2)

Copy masks from one image to similar images using feature matching.

Propagate to Similar Images

```
# Find similar images and propagate mask
uv run crackz propagate-masks run \
  --project demo-project \
  --reference wall_01.jpg \
  --mask wall_01_mask.png \
  --split raw \
  --threshold 0.8 \
  --out ./propagated_masks
```

Parameters: - `--threshold` - Similarity cutoff (0.6-0.95, default: 0.8) - `--min-matches` - Required feature points (default: 10) - `--sift` / `--orb` - Feature detector (SIFT = accurate, ORB = faster)

Find Similar Images Only

Preview which images are similar without propagating:

```
uv run crackz propagate-masks find-similar \
  --project demo-project \
  --reference wall_01.jpg \
  --split raw \
  --out similarities.json
```

Advanced Options

```
# More permissive matching (more propagations, less accurate)
uv run crackz propagate-masks run \
  --reference ref.jpg \
  --mask ref.png \
  --project demo-project \
  --threshold 0.6 \
  --min-matches 5

# Stricter matching (fewer propagations, more reliable)
uv run crackz propagate-masks run \
  --reference ref.jpg \
  --mask ref.png \
  --project demo-project \
  --threshold 0.9 \
  --min-matches 20 \
  --sift
```

Use ORB Features (Faster)

```
uv run crackz propagate-masks run \
  --reference ref.jpg \
  --mask ref.png \
  --project demo-project \
  --orb
```

When to use mask propagation: - Same structure photographed from different angles - Sequential frames from video walkthrough - Adjacent sections of the same structure - Temporal monitoring (same location over time)

Output: - Propagated masks saved to specified directory - Confidence scores included for quality assessment - Only images above similarity threshold are processed

Next

- See `api.md` for programmatic usage examples (load project + call `detect()` / `train()`).

  Need high performance or reproducibility? See Advanced Performance and Configuration – Checkpoint Management. Export tuned configs with `crackz export-config`.

Shell Completion

Shell Completion

Crackz uses Typer, which provides built-in shell completion generation for common shells. This page shows how to enable and verify completion for the `crackz` CLI.

Supported Shells

- Bash
- Zsh
- Fish
- PowerShell (Core / Windows)

Quick One-Off Completion (Any Shell)

You can preview completion script output without installing it permanently:

```
crackz --show-completion bash
crackz --show-completion zsh
crackz --show-completion fish
crackz --show-completion powershell
```

Pipe it into `source` / `Invoke-Expression` for a temporary session (examples below).

Persistent Installation

Typer exposes a helper flag:

```
crackz --install-completion SHELL
```

Replace `SHELL` with one of: `bash`, `zsh`, `fish`, `powershell`. After installation open a new shell (or source your updated profile) and test with:

```
crackz det<TAB>
```

It should expand to:

```
crackz detect
```

Bash

```
crackz --install-completion bash
# New shell OR
source ~/.bash_completion
```

If using Git Bash on Windows ensure `bash-completion` is available; otherwise fall back to `--show-completion` and manual sourcing.

Zsh

```
crackz --install-completion zsh
# Then restart shell or:
autoload -U compinit && compinit
```

If the script is placed in a directory not in `$fpath`, move it or append that location to `fpath`.

Fish

```
crackz --install-completion fish
# Typically installs into ~/.config/fish/completions/
# Reload:
exec fish
```

PowerShell (Windows / Cross-Platform)

Recommended: Use the helper script we ship:

```
pwsh -File scripts/setup_completion.ps1
# Or force re-install:
pwsh -File scripts/setup_completion.ps1 -Force
```

This will: 1. Ensure `crackz` is callable. 2. Run `crackz --install-completion powershell`. 3. Append an evaluation line to your PowerShell profile if missing:

```
crackz --show-completion powershell | Out-String | Invoke-Expression
```

4. Instruct you to reload your session (`. $PROFILE`).

Manual (without script):

```
crackz --install-completion powershell
# If the profile wasn't updated, add manually:
Add-Content $PROFILE "crackz --show-completion powershell | Out-String | Invoke-Expression"
. $PROFILE
```

Temporary Session Activation Examples

Shell	Command
Bash	`crackz --show-completion bash > /tmp/crackz.sh && source /tmp/crackz.sh`
Zsh	`source <(crackz --show-completion zsh)`
Fish	`crackz --show-completion fish | source`
PowerShell	`crackz --show-completion powershell | Out-String | Invoke-Expression`

Troubleshooting

Symptom	Possible Cause	Resolution
No suggestions after install	Profile not reloaded	Open new shell or source profile (`. $PROFILE`)
Command not found	Virtual env not active	Activate venv or use `uv run crackz --install-completion ...` then adjust profile line to call `uv run crackz --show-completion ...`
Duplicate entries appended	Script re-run without `--Force` cleanly	Use `-Force` with the PowerShell helper or manually prune profile
Zsh errors about compinit	compinit not initialized	Run `autoload -U compinit && compinit`
Bash no `_completion_loader` found	Missing bash-completion package	Install bash-completion or source manually from `--show-completion` output

Uninstalling Completion

Manually remove the added line from your shell profile (e.g., `~/.bashrc` , `~/.zshrc` , PowerShell `$PROFILE`) and delete any generated completion file (Fish: remove from `~/.config/fish/completions/`).

Programmatic Invocation (CI / Ephemeral)

To enable completion for an ephemeral CI step (e.g., for smoke tests that rely on completion logic executing):

```
crackz --show-completion bash > .crackz_comp.sh
source .crackz_comp.sh
# run tests that exercise completion
```

Next

- See the CLI Cookbook for command usage examples.
- Visit Configuration and Advanced Performance for tuning.

Project Configuration

Configuration Reference

This page explains how configuration is structured, loaded, and overridden when using Crackz.

Layers & Precedence

Configuration values come from these sources (last one wins): 1. Built-in defaults (Pydantic model defaults) 2. Project YAML (project file saved by `crackz init` or programmatic creation) 3. Environment variables (prefixed or direct where supported) 4. CLI flags / command arguments

Effective precedence:

```
CLI flag > ENV var > Project YAML > Code default
```

Project YAML Structure

A project created with `crackz init --name demo-project` saves a file named: `demo-project/_project.yaml` (pattern: `<name>_project.yaml`).

Example (abridged, schema v6):

```yaml
schema_version: 6
project:
  name: demo-project
  description: Demo project
  created: 2025-01-01T12:00:00+00:00
annotation:
  background:
    id: 0
    name: Background
    color: "#000000"
  categories:
    - id: 1
      key: crack
      name: Crack
      color: "#ef4444"
paths:
  project_path: demo-project
  image_path: demo-project/images
  image_mask_path: demo-project/masks
  artifacts_path: demo-project/artifacts
logging:
  level: INFO
  log_dir: logs
ai:
  model:
    backbone: segformer-b2
    active_checkpoint: best
    device: cpu
    in_channels: 3
    input_size: [1024, 1024]
    threshold: 0.5
    loss_name: combo
    dice_weight: 0.7
    focal_weight: 0.3
  training:
    batch_size: 8
    num_workers: 4
    epochs: 10
    learning_rate: 0.001
    shuffle: true
    augmentation: true
  detection:
    threshold: 0.5
    batch_size: 8
    output_dir_name: detections
data:
  mask_threshold: 50
  image_quality: 90
  target_size: 1024
```

> **Schema migrations**: Projects created with older versions are automatically migrated when opened. Run `crackz migrate --project <path>` to migrate without opening the GUI. A timestamped backup is created before any migration. See the Migration Guide for details.

Unified ProjectConfig (Scaffold)

For programmatic work you can use the unified wrapper `ProjectConfig`:

```
from pathlib import Path
from crackz.config.project_config import ProjectConfig
```

```
pc = ProjectConfig(
    name="demo-project",
    project_path=Path("demo-project"),
    image_path=Path("demo-project/images"),
    image mask path=Path("demo-project/masks"),
    artifacts_path=Path("demo-project/artifacts"),
)
pc.save(Path("project_config.yaml"))

# Later
loaded = ProjectConfig.load(Path("project_config.yaml"))
project = loaded.to_project()
```

You can also serialize a live `Project` back to a config:

```
from crackz.config.project_config import ProjectConfig
pc = ProjectConfig.from_project(project)
pc.save(Path("backup_config.yaml"))
```

AI Configuration Sections

Section	Model	Purpose
ai.model	`ModelConfig`	Architecture + encoder + device + loss weights
ai.training	`TrainingConfig`	Hyperparameters for training loop
ai.detection	`DetectionConfig`	Inference batch & threshold settings

ModelConfig Fields

Field	Default	Notes
backbone	segformer-b2	Model backbone architecture (see backbone options below)
active_checkpoint	best	Checkpoint used for detection ("best", "last", or filename)
task_mode	null	Explicit task override: `binary` or `multiclass`. Null auto-derives from annotation schema
device	cpu	cpu / cuda / mps / xla (validated)
in_channels	3	Number of input channels (3 for RGB, 1 for grayscale)
num_classes	1	**Derived** at runtime from annotation schema — do not set manually
input_size	(1024,1024)	WxH expected by model/transforms
threshold	0.5	Applied to sigmoid output (binary) or confidence threshold (multiclass)
normalization.mean	[0.485, 0.456, 0.406]	ImageNet normalization mean values
normalization.std	[0.229, 0.224, 0.225]	ImageNet normalization std values
loss_name	combo	Abstract label; extendable (combo, dice, focal)
dice_weight	0.7	Weighted contribution in combo loss
focal_weight	0.3	Weighted contribution in combo loss

> **Note on `num_classes`**: As of schema v6, `num_classes` is derived automatically from the project annotation schema (1 category → binary mode with 1 output channel; 2+ categories → multiclass mode with `1 + num_categories` output channels). Do not set this field manually.
>
> **Note on `backbone`**: The `encoder` / `encoder_weights` fields from schema ≤ v4 are superseded by `backbone`. Projects are auto-migrated. Supported values include any `segmentation-models-pytorch` encoder **plus** SegFormer variants (`segformer-b0` through `segformer-b5`). See backbone options below.

Supported Backbones

Crackz supports two model families as the backbone:

SegFormer (Recommended)

SegFormer is the **default** model family for new projects. It uses a hierarchical Transformer encoder with a lightweight MLP decoder, giving strong accuracy without the large memory footprint of heavier transformer models.

Backbone	Parameters	Speed	Accuracy	Best For
`segformer-b0`	3.7M	Very Fast	Good	Edge deployment, real-time
`segformer-b1`	13.7M	Fast	Very Good	Balanced — lighter GPU
`segformer-b2`	**25.4M**	**Medium**	**Excellent**	**⭐ Default — best value**
`segformer-b3`	45.2M	Slow	Excellent	Large datasets, GPU recommended
`segformer-b4`	64.1M	Slower	Superior	High accuracy, ample GPU budget
`segformer-b5`	84.7M	Slow	Best	Maximum accuracy benchmark

U-Net with SMP Encoders

The classic U-Net decoder with any `segmentation-models-pytorch` encoder. Useful for binary crack-only projects, smaller datasets, or environments where SegFormer weights cannot be downloaded.

Backbone (encoder only)	Parameters	Speed	Accuracy	Best For
`efficientnet-b0`	4M	Medium	Excellent	⭐ **Best value** - efficient & accurate
`efficientnet-b3`	10M	Slow	Excellent	⭐ **Recommended** - production crack detection
`efficientnet-b7`	64M	Very Slow	Best	Maximum accuracy, state-of-the-art
`resnet18`	11M	Fast	Good	Quick prototyping, edge devices
`resnet34`	21M	Medium	Very Good	Balanced performance
`resnet50`	23M	Slow	Excellent	Large datasets, maximum accuracy
`mobilenet_v2`	2M	Very Fast	Fair	Edge deployment, real-time

Encoder Weights Options (U-Net only) - `imagenet`: ImageNet-1K pretrained (recommended for transfer learning) - `ssl`: Self-supervised learning weights - `swsl`: Semi-weakly supervised learning weights - `None` or `null`: Random initialization (train from scratch)

Model Type Examples

Binary Segmentation — SegFormer (Default)

```
ai:
  model:
    backbone: segformer-b2   # Default SegFormer model
    input_size: [1024, 1024]
    threshold: 0.5
annotation:
  categories:
    - id: 1
      key: crack
      name: Crack
      color: "#ef4444"
```

Multi-Class Segmentation — Harbor Defects

```
annotation:
  categories:
    - id: 1
      key: crack
      name: Crack
      color: "#ef4444"
    - id: 2
      key: spall
      name: Spall
      color: "#f59e0b"
    - id: 3
      key: corrosion
      name: Corrosion
      color: "#3b82f6"
ai:
  model:
    backbone: segformer-b2
    # num_classes is derived automatically: 1 + 3 categories = 4 output channels
```

Binary Segmentation — U-Net with EfficientNet (legacy or fallback)

```
ai:
  model:
    backbone: efficientnet-b3   # U-Net decoder, EfficientNet-B3 encoder
    input_size: [512, 512]
    threshold: 0.5
```

Grayscale Input

```
ai:
  model:
    backbone: segformer-b2
    in_channels: 1          # Grayscale images
    normalization:
      mean: [0.5]
      std: [0.5]
```

Force binary task mode with multiple annotation categories

```
ai:
  model:
    backbone: segformer-b2
    task_mode: binary       # All categories collapse into one defect-vs-background class
```

See Architecture for use case examples and backbone selection guidance.

TrainingConfig Fields

Field	Default	Notes
batch_size	8	Reduce for memory constraints
num_workers	4	Data loader worker processes

Field	Default	Notes
epochs	10	Total training passes
learning_rate	0.001	Optimizer LR (single value)
shuffle	true	Shuffle training set each epoch
loss	None	Optional explicit loss override
metrics	None	Optional metrics spec string
seed	None	Global seed for reproducibility (torch, numpy, random)
precision	None	Mixed precision mode (e.g. 16, "16-mixed", "bf16-mixed")
devices	None	Number of devices (e.g. GPUs) for distributed runs
strategy	None	PyTorch Lightning strategy (ddp, ddp_spawn, etc.)
tensorboard	false	Enable TensorBoard logging
save_top_k	1	Number of best checkpoints to keep (by val_iou)
save_last	true	Whether to save the last checkpoint
auto_prune_checkpoints	false	Automatically prune checkpoints after training
checkpoint_keep	3	Checkpoints to keep when auto-pruning

TrainingConfig Contract & Compatibility

The `TrainingConfig` model is part of Crackz's persisted project schema (YAML). Projects saved on disk are expected to remain loadable across MINOR and PATCH releases.

Contract summary: - Stability: Existing fields will not be removed or have semantics changed in a PATCH or MINOR release. - Additions: New optional fields must provide a safe default (so older YAML still loads). - Renames / Removals: Only allowed with a MAJOR version and MUST include a migration shim (e.g. read old field, map to new, warn). Prefer soft-deprecate first. - Types: Field type changes are considered breaking unless widening (e.g. `int` -> `int | str`). - Serialization: Only include primitive / Pydantic-friendly types to keep YAML diff-friendly.

`tensorboard` Field Rationale: - Added after initial releases to optionally enable structured log output for TensorBoard under `artifacts/runtime/tensorboard/`. - Older project YAML won't contain the key; code must treat missing attribute as `False`. We enforce this via `getattr(project.ai.training, "tensorboard", False)` in the trainer code.

Backward Compatibility Pattern (example):

```
# Safe access pattern inside runtime code
use_tb = getattr(project.ai.training, "tensorboard", False)
if use_tb:
    ... # create logger
```

Adding a New Training Field – Checklist: 1. Pick a clear, concise snake_case name. 2. Provide a conservative default that preserves existing behavior. 3. Document in this table + add a short note if interaction with other fields matters. 4. Update or add a focused unit test asserting the default value. 5. Use `getattr(..., default)` for transitional reads if legacy YAML may lack the field. 6. (Optional) Add a validation rule if only a narrow value set is valid.

Deprecating a Field (soft): 1. Leave field in the model; mark in docs table as "(deprecated)". 2. Emit a one-time warning on access (guard with an env flag or module-level set to avoid spam). 3. After at least one MINOR release cycle, remove only in next MAJOR with a migration note.

Testing Guarantees: - Unit test `tests/unit/crackz/ai/test_detection.py::test_tensorboard_logger_disabled_by_default` asserts the default remains `False`. - Schema drift detection (optional future enhancement) may snapshot `model_fields`.

Recommended Future Enhancements: - Provide additional migration steps for future schema versions. - Add automatic snapshot tests for config model field sets.

If you extend `TrainingConfig`, update this section and add tests; otherwise CI may allow a silent change without guide coverage.

DetectionConfig Fields

Field	Default	Notes
threshold	0.5	Binarization for predicted mask
learning_rate	0.001	For loading / fallback model init
batch_size	8	Inference batch size
shuffle	true	Shuffle images in detection loop
output_dir_name	detections	Relative to artifacts path
seed	None	Optional seed for deterministic detection (transforms/order)
min_crack_area_px	0	Minimum crack area in pixels to suppress speckle noise
min_confidence	0.005	Minimum confidence (0.5% crack area) to save detections
medium_confidence	0.02	Threshold for "Probable Crack" severity (2% crack area)
high_confidence	0.05	Threshold for "Crack Detected" severity (5% crack area)

Severity Thresholds

The `min_confidence`, `medium_confidence`, and `high_confidence` fields control crack severity categorization:

- **min_confidence** (default 0.005): Minimum crack area percentage to save a detection. Images with lower confidence are filtered out entirely.
- **medium_confidence** (default 0.02): Threshold between "Possible Crack" and "Probable Crack" categories (2% crack area).
- **high_confidence** (default 0.05): Threshold for "Crack Detected" category (5% crack area).

Severity Categories:

Confidence Range	Severity Category	Typical Crack Coverage
>= high_confidence (0.05)	Crack Detected	5%+ crack area
>= medium_confidence (0.02)	Probable Crack	2-5% crack area
>= min_confidence (0.005)	Possible Crack	0.5-2% crack area
< min_confidence (0.005)	(filtered out)	Not saved

Note: The "confidence" metric represents **crack area percentage** (mean probability over the whole image), not maximum pixel confidence. Typical values for images with cracks range from 0.5-8% (0.005-0.08 as decimal).

Example Configuration:

```
ai:
  detection:
    min_confidence: 0.005     # Save detections with ≥0.5% crack area
    medium_confidence: 0.02   # "Probable Crack" at ≥2% crack area
    high_confidence: 0.05     # "Crack Detected" at ≥5% crack area
    min_crack_area_px: 100    # Suppress detections smaller than 100 pixels
```

Data Configuration Section

The `data` section controls data preprocessing and validation behavior.

Section	Model	Purpose
data	`DataConfig`	Data preprocessing settings (mask conversion, validation)

DataConfig Fields

Field	Default	Notes
mask_threshold	50	Threshold for converting masks to binary format during import
image_quality	90	JPEG quality for saving/resizing images during import (1-100)
target_size	1024	Image resize dimension (NxN pixels) applied during import (64-4096)

Mask Threshold Behavior

The `mask_threshold` parameter controls how imported masks are converted to binary format:

- **Value**: Integer threshold (0-255) for binarization
- **Default**: 50 (filters out noise and faint annotations)
- **Usage**: During mask import, any pixel value ≥ threshold becomes 255 (white/foreground), others become 0 (black/background)

Common Threshold Values:

Threshold	Behavior	Use Case
1	Any non-zero pixel → foreground	Preserves all marked pixels including noise
50	Light gray and brighter → foreground	**Default** - filters noise while preserving annotations
128	Mid-gray and brighter → foreground	Stricter filtering of uncertain annotations
255	Only pure white → foreground	Very strict - only definite annotations

Example Configuration:

```
data:
  mask_threshold: 50  # Default - balanced filtering
```

When to Adjust: - Masks contain grayscale uncertainty values → use higher threshold (e.g., 128) - Masks are noisy with faint artifacts → leave at default (50) or increase - Masks are already binary or you want to preserve all pixels → use lower threshold (1) - Very clean masks with only definite annotations → can use higher threshold (128-255)

The threshold is applied automatically during: - `crackz import-images --masks_src <path>` (CLI import) - `project.import_images()` (programmatic import) - The `scripts/import_masks.py` utility script

Migration Note: Projects created before schema version 2 will not have this field. Running `crackz migrate` will add `mask_threshold: 50` to use the recommended default.

Annotation Schema

The `annotation` section (added in schema v6) defines the **project-owned label schema** — the set of defect categories used by the mask editor, training pipeline, detection results, and model registry.

```
annotation:
  background:
    id: 0
    name: Background
    color: "#000000"
  categories:
    - id: 1
      key: crack
      name: Crack
      color: "#ef4444"
    - id: 2
      key: spall
      name: Spall
      color: "#f59e0b"
    - id: 3
      key: corrosion
      name: Corrosion
      color: "#3b82f6"
```

AnnotationBackground Fields

Field	Default	Notes
id	0	Always 0; non-configurable
name	Background	Human-readable name
color	#000000	Hex color for overlays

AnnotationCategory Fields

Field	Required	Notes
id	yes	Stable integer, must be contiguous `1..N`
key	yes	Lowercase slug (e.g. `crack`, `spall`). Used in JSON, CLI, and model metadata
name	yes	Human-readable display name (e.g. `Crack`, `Spall`)
color	yes	Hex color `#RRGGBB` for mask overlays and gallery chips

Rules

- **Background is always id 0** and is reserved; you cannot add it as a foreground category.
- **Category ids must be contiguous** (`1, 2, 3, …N`). Gaps are not permitted.
- **Category keys must be unique** within a project.
- **Single category → binary task mode**: a project with one foreground category uses the one-channel sigmoid path. The model output classes will be `1`.
- **Multiple categories → multiclass task mode**: a project with N > 1 foreground categories uses the softmax path. The model output classes will be `1 + N` (background + N foregrounds).
- `task_mode: binary` **override**: set this in `ai.model` to collapse all foreground categories into a single defect-vs-background class even when multiple categories exist.

Editing Categories from the CLI

`crackz config categories list | add | rename | recolor | remove` edits this schema without hand-editing the YAML; see the CLI cookbook. The rules above are enforced: ids stay contiguous and are never renumbered, so `remove` only accepts the highest id.

Annotation Templates

New projects can start from a built-in template via the GUI New Project dialog or the CLI:

Template	Categories	Use Case
`crack-only`	Crack	Binary crack detection (default)
`harbor-defects`	Crack, Spall, Corrosion, Joint Damage	Harbor infrastructure inspection
`survey-general`	Crack, Spall, Corrosion, Water Intrusion, Biological Growth	Broad surface condition survey

```
# CLI: initialize a project with a harbor-defects annotation template
crackz init --name harbor-project --annotation-template harbor-defects
```

Schema Compatibility

Checkpoints are **taxonomy-aware**: the model head size is derived from the annotation schema at training time and stored in the checkpoint metadata. If you change your annotation categories after training, existing checkpoints will be flagged as incompatible in the **Model Manager** and cannot be used for detection until replaced.

When editing categories in Project Settings, Crackz will: 1. **Warn** if existing checkpoints would be invalidated. 2. **Offer to remap** existing mask label IDs if masks already exist for the project. 3. **Block** saves that would make checkpoint and mask data inconsistent.

Advanced Training Fields

Additional optional performance & reproducibility keys: | Field | Location | Purpose | |-------|----------|---------| | seed | ai.training / ai.detection | Set global RNG seed(s) | | precision | ai.training | Mixed precision (e.g. 16-mixed, bf16-mixed) | | devices | ai.training | Number of devices (e.g. GPUs) for distributed runs | | strategy | ai.training | Lightning strategy (ddp, ddp_spawn, etc.) |

Export a tuned unified config snapshot:

```
crackz export-config --project demo-project --out tuned_config.yaml
```

Programmatic:

```
from crackz.config.project_config import ProjectConfig
pcfg = ProjectConfig.from_project(project)
pcfg.save(Path("tuned_config.yaml"))
```

See Advanced Performance for deep guidance.

Environment Variables

Some settings are exposed through Pydantic / explicit env usage: | Variable | Effect | Default | |----------|--------|---------| | CRACKZ_LOG_DIR | Logging output directory | logs | | CRACKZ_LOG_LEVEL (indirect via CrackzSettings) | Logging level override | INFO | | PYTHONUNBUFFERED | Immediate stdout/stderr flush | 1 |

Example (Linux/macOS):

```
CRACKZ_LOG_DIR=run_logs crackz detect run --project demo-project
```

Windows (PowerShell):

```
$env:CRACKZ_LOG_DIR='run_logs'; crackz detect run --project demo-project
```

Device Selection

The chosen device originates from `ai.model.device` unless empty; fallback auto-detection occurs (via `resolve_device_and_accelerator`). To run on GPU: 1. Install CUDA-enabled torch (or use GPU Docker image) 2. Set `ai.model.device: cuda` 3. Verify with: `python -c "import torch; print(torch.cuda.is_available())"`

TensorBoard Integration

Crackz supports TensorBoard logging during training without requiring code modifications. When enabled, training metrics (loss, IoU) are automatically logged to TensorBoard.

Enabling TensorBoard

Set `tensorboard: true` in the training configuration:

```
ai:
  training:
    tensorboard: true
    epochs: 10
    batch_size: 8
```

Or programmatically:

```
from crackz import Project

project = Project.load(Path("demo-project"))
project.ai.training.tensorboard = True
project.save()
```

Viewing TensorBoard Logs

After training starts, TensorBoard logs are saved to:

```
<artifacts_path>/runtime/tensorboard/
```

Launch TensorBoard to view metrics:

```
tensorboard --logdir demo-project/artifacts/runtime/tensorboard
```

Then open http://localhost:6006 in your browser.

What Gets Logged

When TensorBoard is enabled, the following metrics are automatically logged: - Training loss (per step and per epoch) - Training IoU (per epoch) - Validation loss (per epoch) - Validation IoU (per epoch) - Test loss (per epoch) - Test IoU (per epoch)

Docker Usage

When using Docker, expose TensorBoard's port and mount the project directory:

```
docker run --rm -v $(pwd)/demo-project:/workspace -p 6006:6006 crackz-cli \
  tensorboard --logdir /workspace/artifacts/runtime/tensorboard
```

Overriding Strategy Examples

Goal	Approach
Quick threshold test	CLI: `crackz detect run --project p --cfg cfg.yaml` then modify YAML threshold or set env override & rerun
Faster iteration	Lower `training.epochs` & `batch_size` in YAML for a dev profile
High throughput detection	Increase `detection.batch_size` until GPU RAM saturates

Checkpoint Management

Checkpoints live under: `<artifacts_path>/models/` and may include: - `best.ckpt` (highest monitored metric) - `last.ckpt` (latest epoch) - Patterned files: `epoch={N}-val_iou={score}.ckpt` (e.g., `epoch=9-val_iou=0.8234.ckpt`)

Note: Legacy projects may have checkpoints in `<artifacts_path>/checkpoints/` - both locations are supported for backward compatibility.

Configuration Options

Control checkpoint saving behavior in your project YAML:

```
ai:
  training:
    save_top_k: 1          # Keep N best checkpoints (by val_iou), default: 1
    save_last: true        # Always save last.ckpt, default: true
    auto_prune_checkpoints: false  # Auto-prune after training, default: false
    checkpoint_keep: 3     # Checkpoints to keep when auto-pruning, default: 3
```

Disk Space Optimization Strategies:

1. **Minimal (default)**: `save_top_k: 1, save_last: true` → 2-3 checkpoints max
2. **Conservative**: `save_top_k: 3, auto_prune_checkpoints: true, checkpoint_keep: 3`
3. **Aggressive**: `save_top_k: 1, save_last: false, auto_prune_checkpoints: true, checkpoint_keep: 1`

Manual Pruning

Prune old checkpoints keeping N best (heuristic):

```
crackz detect prune-checkpoints --project demo-project --keep 3
```

Docker (GPU example):

```
docker run --gpus all --rm -v %cd%\project:/workspace crackz-cli:gpu \
  detect prune-checkpoints --project /workspace/demo-project --keep 2
```

Programmatic:

```
from crackz.ai.checkpoints import prune_checkpoints
removed = prune_checkpoints(project, keep=2)
```

Checkpoint Scoring: Checkpoints are ranked by: 1. `best.ckpt` (score: 2.0) 2. Epoch files by embedded val_iou (e.g., `003-0.8234.ckpt` → score: 0.8234) 3. `last.ckpt` (score: 0.5)

Lowest-scored checkpoints are pruned first.

Detection Results & Metrics

Detection writes JSON metrics to:

```
<artifacts_path>/detections/detection_results.json
```

Load & summarize:

```
from crackz.ai.results import load_detection_results
from crackz.ai.eval import basic_summary
results = load_detection_results(project)
print(basic_summary(results))
```

Crack Severity Categorization: Detection results include a `crack_severity` field that categorizes detections based on confidence levels: - **"Crack Detected"** (confidence ≥ 0.7): High confidence detection - **"Probable Crack"** (confidence ≥ 0.5): Medium confidence detection - **"Possible Crack"** (confidence < 0.5): Low confidence detection

This categorization helps reduce false positives by distinguishing between confident detections and uncertain ones. The GUI displays these categories with color coding for easy visual identification.

Reproducibility Tips (Planned Enhancements)

Tip	Description
Pin versions	Use lockfile / requirements export for deterministic builds
Seed control	(Future) Add `seed` field to configs and set torch / numpy seeds
Store config snapshot	Commit YAML + ProjectConfig exports with model artifacts

Minimal Quick Reference

```
# project config.yaml (unified example)
name: demo-project
description: Example project
project_path: demo-project
image_path: demo-project/images
image_mask_path: demo-project/masks
artifacts_path: demo-project/artifacts
logging:
  level: INFO
  log dir: logs
model:
  encoder: resnet34
  device: cpu
  input_size: [512, 512]
training:
  epochs: 5
  batch size: 4
  learning_rate: 0.0005
  tensorboard: false  # Set to true to enable TensorBoard logging
detection:
  threshold: 0.55
  batch_size: 4
```

Upgrading Projects (Schema Migration)

Older project YAML files (created before `schema_version` existed) can be safely upgraded in-place using the CLI:

```
crackz migrate --project ./my_project
```

This will: - Detect absence of `schema_version` (treat as version 0) - Add `schema_version: 1` - Insert missing training field `tensorboard: false` if absent - Create a `.bak` backup copy by default

Alternatively, use the config file directly:

```
crackz migrate --cfg ./my_project/my_project_project.yaml
```

Dry-run (no changes written):

```
crackz migrate --project ./my_project --dry-run
```

Skip backup (NOT recommended unless under version control):

```
crackz migrate --project ./my_project --no-backup
```

Idempotent: re-running on an already upgraded project reports `up-to-date` / `noop`.

Automation tip: If you maintain many archived projects, you can batch migrate:

```
for d in projects/*; do
  [ -d "$d" ] && crackz migrate --project "$d" --no-backup;
done
```

See the CLI reference (Migrate section) for status meanings.

If you need a field not yet exposed, open an issue or extend `ProjectConfig` / the Pydantic models and regenerate the project YAML.

Training Configuration

TrainingConfig Contract

`TrainingConfig` defines the core knobs for model training. It lives in `crackz.project.config` and is embedded under `ai.training` in a project.

Field	Type	Default	Description
`batch_size`	int	8	Samples per optimizer step.
`num_workers`	int	4	DataLoader worker processes.
`epochs`	int	10	Total training epochs.
`learning_rate`	float	1e-3	Optimizer base LR.
`shuffle`	bool	True	Shuffle training dataset each epoch.
`loss`	str | None	None	Optional explicit loss override (framework default if None).
`metrics`	str | None	None	Optional metrics spec string.
`seed`	int | None	None	Global seed for reproducibility (torch, numpy, random).
`precision`	int | str | None	None	Mixed precision mode (16, "16-mixed", "bf16-mixed", etc.).
`devices`	int | None	None	Number of accelerator devices (e.g. GPUs) to use.
`strategy`	str | None	None	PyTorch Lightning strategy (e.g. "ddp", "ddp_spawn").
`tensorboard`	bool	False	Enable TensorBoard logging to `artifacts/training/tensorboard`.

Snapshot Test

A snapshot test (`tests/unit/crackz/project/test_training_config_schema.py`) guards against accidental field drift. If you intentionally change `TrainingConfig`:

1. Update this document.
2. Run the tests to see the snapshot failure.
3. Update the snapshot (e.g. regenerate expected structure) per test instructions.
4. Bump `CURRENT_SCHEMA_VERSION` and add a migration step if persisted YAML needs changes.

Adding Fields Safely

1. Add the field with a sensible default.
2. Update migration logic to inject the new default for legacy projects.
3. Document the field here.
4. Consider whether CLI flags are warranted for runtime overrides.

Runtime Overrides

CLI training overrides currently support `--epochs` and `--batch-size`. Runtime changes are not persisted: they affect only the active training invocation.

To add a new override: - Extend CLI option definitions. - Apply the override after project load and (if needed) after migration.

TensorBoard

When `tensorboard=True`, logs are written under `<project>/artifacts/training/tensorboard`. The directory is created lazily.

Reproducibility

Setting `seed` triggers seeding helper logic that affects random, NumPy, and PyTorch to the extent supported by selected device / strategy.

Data Splits & Import

Data Splits and Import Guide

A comprehensive guide to understanding and using Crackz's data organization system for training and detection.

Table of Contents

1. Understanding Data Splits
2. Import with Automatic Split
3. Complete Training Workflow
4. Best Practices
5. Troubleshooting

Understanding Data Splits

Crackz organizes images into four different directories based on their purpose. Understanding these splits is essential for effective model training and deployment.

The Four Split Types

Split	Purpose	Masks Required?	Used During Training?	When to Use
raw	Production inference	❌ No	❌ No	Real-world images you want to analyze
train	Model training	✅ Yes	✅ Yes	Teaching the model crack patterns
val	Validation	✅ Yes	✅ Yes (monitoring)	Checking training progress
test	Final evaluation	✅ Yes	❌ No	Measuring real-world performance

Detailed Explanation

Raw Split (`images/raw/`)

Purpose: Detection on new, unlabeled images

Characteristics: - No masks required - Used for production deployments - Perfect for batch processing real-world images - Model outputs predictions without ground truth comparison

Example Use Case:

> You have 1,000 photos of harbor infrastructure. You want to detect cracks automatically. Put these images in `raw/` and run detection.

How to Import:

```
crackz import-images --project my-project --src ./photos --type raw --size 512 512
```

Train Split (`images/train/` + `masks/train/`)

Purpose: Training the model

Characteristics: - Masks **required** - each image must have a corresponding mask - The model learns crack patterns from these examples - Typically the largest split (60-80% of labeled data) - Used during every training epoch

What Happens During Training: - Model sees each train image - Makes a prediction - Compares prediction to the ground truth mask - Adjusts weights to improve accuracy

Example Use Case:

> You manually labeled 700 images with crack annotations. These teach the model what cracks look like.

Mask Requirements: - Filename must match image (e.g., `img001.jpg` → `img001.png`) - Binary format: 0 (black) = background, 255 (white) = crack - Same dimensions as the image - PNG or TIFF format (lossless)

Validation Split (`images/val/` + `masks/val/`)

Purpose: Monitoring training progress

Characteristics: - Masks **required** - Used to evaluate model during training (not for training itself) - Helps prevent overfitting - Guides hyperparameter tuning decisions - Typically 15-20% of labeled data - **Minimum recommended: 10-20 images**

What Happens During Training: - After each epoch, model evaluates on validation set - Loss and accuracy metrics are computed - Early stopping triggers if validation performance degrades - **Model never updates weights based on validation data**

Example Use Case:

> You held out 150 labeled images. After each training epoch, the model checks its performance on these to ensure it's not just memorizing the training data.

Why It's Important: - Prevents overfitting (model performs well on train but poorly on new data) - Helps choose best checkpoint during training - Provides feedback for hyperparameter tuning (batch size, learning rate, etc.)

Test Split (`images/test/` + `masks/test/`)

Purpose: Final, unbiased evaluation

Characteristics: - Masks **required** - **Never** used during training or validation - Kept completely separate until final evaluation - Provides realistic performance estimate - Typically 10-15% of labeled data

What Happens During Testing: - Run **after** training is complete - Model makes predictions on test images (seen for the first time) - Predictions compared to ground truth masks - Final metrics (IoU, precision, recall) computed

Example Use Case:

> You held out 150 labeled images that the model has never seen. After training completes, you test on these to get an honest measure of real-world performance.

Why It's Important: - Provides unbiased performance estimate - Simulates real-world deployment - Cannot be used for tuning (would bias the results) - Essential for reporting final model performance

---

Recommended Split Ratios

Choose ratios based on your dataset size:

Dataset Size	Train	Val	Test	Reasoning
Tiny (<50 images)	60%	25%	15%	Need larger val set for reliable feedback
Small (50-100)	70%	20%	10%	Balance between training and validation
Medium (100-1000)	70%	15%	15%	Standard research practice
Large (>1000)	80%	10%	10%	More data for training, smaller % still gives good validation

Absolute Minimums: - Validation: At least 10 images (preferably 20+) - Test: At least 10 images - Train: At least 30-50 images for basic training

---

Import with Automatic Split

The **Import with Split** feature automatically distributes your images into train/val/test directories, eliminating manual file organization.

When to Use

✅ **Use Import with Split when you:** - Have labeled images with corresponding masks - Want to train a custom model - Need automatic random distribution - Want reproducible splits

❌ **Don't use Import with Split for:** - Production images for detection (use `--type raw` instead) - Images without masks - Re-organizing already split data

CLI Usage

Basic Import with Split

```
crackz import-images \
  --project my-project \
  --src ./labeled-data/images \
  --src-masks ./labeled-data/masks \
```

```
  --split-ratios 0.7 0.15 0.15 \
  --size 512 512
```

What happens: 1. Discovers all images in source directory 2. Matches masks by filename 3. Randomly shuffles files 4. Splits according to ratios (70% train, 15% val, 15% test) 5. Resizes images to 512×512 6. Copies to appropriate directories 7. Displays summary of split results

Reproducible Split (with Seed)

```
crackz import-images \
  --project my-project \
  --src ./labeled-data/images \
  --src-masks ./labeled-data/masks \
  --split-ratios 0.7 0.15 0.15 \
  --seed 42 \
  --size 512 512
```

Why use a seed: - Same split every time you import - Reproducible experiments - Consistent with team members - Re-import if you need to adjust image size

Custom Ratios

```
# Small dataset: larger validation set
crackz import-images \
  --project my-project \
  --src ./small-dataset/images \
  --src-masks ./small-dataset/masks \
  --split-ratios 0.6 0.25 0.15 \
  --size 512 512

# Large dataset: more for training
crackz import-images \
  --project my-project \
  --src ./large-dataset/images \
  --src-masks ./large-dataset/masks \
  --split-ratios 0.8 0.1 0.1 \
  --size 512 512
```

Overlapping frames (photogrammetry, video, tiles)

A per-file random split puts near-duplicate neighbouring frames on both sides of the split, which inflates validation and test scores. Add `--group-by` (`folder`, `sequence-block` with `--group-size N`, `filename-prefix`, or `tile-source`) so every image of a group stays in one split. Beware that a flat folder, or camera counters like `DJI_0001`, collapse into a single group: Crackz warns when a split ends up empty. See the CLI cookbook for examples.

GUI Usage

The GUI provides a visual, user-friendly interface for importing with split.

Step-by-Step Guide

1. **Open the Import Dialog**
2. Right-click on the project root in the file browser

3. Select "📊 Import Images with Split..."

4. **Select Your Files**

5. Click "📁 Select Images..."
6. Browse to your images directory
7. Click "📁 Select Masks..."
8. Browse to your masks directory

9. Status shows: "Found X images, Y masks"

10. **Configure Split Ratios**

11. Drag the sliders to set percentages
12. **Train slider**: Percentage for training (default 70%)
13. **Val slider**: Percentage for validation (default 15%)
14. **Test slider**: Percentage for testing (default 15%)

15. Watch the **Total indicator**:

    - Green ✓ = Sum equals 100% (ready to import)
    - Red ✗ = Sum doesn't equal 100% (adjust sliders)

16. **Optional: Set Random Seed**

17. Check "Use random seed" for reproducible splits
18. Enter a number (e.g., 42)

19. Same seed = same split every time
20. **Configure Image Settings**
21. Width and height (default 512×512 pixels)
22. Quality slider (default 90%)
23. **Start Import**
24. Click "Import" button
25. Progress bar shows status
26. Success message when complete

Visual Indicators

- **File count**: Shows discovered images and masks
- **Ratio total**: Must be green ✓ to proceed
- **Progress bar**: Real-time import progress
- **Status messages**: Shows which split is being processed

Complete Training Workflow

Here's a complete end-to-end example showing how to use data splits effectively.

Scenario

You have 200 labeled images of harbor cracks and want to train a custom detection model.

Step 1: Create Project

```
crackz init --name harbor-inspection
cd harbor-inspection
```

Result: Empty project structure created

Step 2: Import with Automatic Split

```
crackz import-images \
  --project . \
  --src ~/datasets/harbor-cracks/images \
  --src-masks ~/datasets/harbor-cracks/masks \
  --split-ratios 0.7 0.15 0.15 \
  --seed 42 \
  --size 512 512
```

Result: - 140 images → `images/train/` (70%) - 30 images → `images/val/` (15%) - 30 images → `images/test/` (15%) - Matching masks in corresponding `masks/` directories

Step 3: Verify Split

```
# Count images
ls images/train/ | wc -l   # Should show 140
ls images/val/ | wc -l     # Should show 30
ls images/test/ | wc -l    # Should show 30

# Count masks (should match images)
ls masks/train/ | wc -l    # Should show 140
ls masks/val/ | wc -l      # Should show 30
ls masks/test/ | wc -l     # Should show 30
```

Step 4: Train Model

```
crackz detect train --epochs 50 --batch-size 8
```

What happens during training: - Model learns from `images/train/` and `masks/train/` - After each epoch, validates on `images/val/` and `masks/val/` - Saves best checkpoint based on validation performance - **Test set remains completely unused**

Output: - Training metrics in `artifacts/metrics.json` - Loss curves in `artifacts/loss_curve.png` - Best model in `checkpoints/best.ckpt`

Step 5: Evaluate on Test Set

```
crackz detect run --split test
```

What happens: - Loads best checkpoint - Runs detection on `images/test/` (first time model sees these) - Compares to `masks/test/` for ground truth - Generates performance metrics

Output: - Test results in `detections/` - Final metrics showing real-world performance

Step 6: Deploy on Production Data

```
# Import new unlabeled images
crackz import-images \
  --project . \
  --src ~/new-harbor-photos \
  --type raw \
  --size 512 512

# Run detection
crackz detect run --split raw
```

Result: Annotated images in `detections/` showing detected cracks

Best Practices

Do's ✅

1. **Use reproducible seeds** when experimenting

   ```
   --seed 42  # Same split every time
   ```

2. **Keep backups** of original data before importing

   ```
   cp -r original-data backup/
   ```

3. **Verify split balance** - check all splits have similar crack/no-crack ratios

   ```
   # Check if splits are representative
   ls images/train/ | wc -l
   ls images/val/ | wc -l
   ```

4. **Use minimum validation size** - at least 10-20 images

   ```
   # For 50 images: 0.6, 0.25, 0.15 ensures val gets ~13 images
   --split-ratios 0.6 0.25 0.15
   ```

5. **Document your split** - record seed and ratios in project notes

   ```
   # In project documentation
   split_info:
     seed: 42
     ratios: [0.7, 0.15, 0.15]
     date: 2025-10-19
   ```

Don'ts ❌

1. **Don't tune on test data** - never use test set for hyperparameter decisions
2. **Don't re-split frequently** - this breaks reproducibility and wastes time
3. **Don't skip validation** - training without validation often leads to overfitting
4. **Don't use tiny val sets** - fewer than 10 images gives unreliable feedback
5. **Don't mix splits** - keep train/val/test completely separate
6. **Don't forget masks** - train/val/test splits require corresponding masks

Data Quality Checklist

Before importing with split, verify:

- [ ] All images have corresponding masks
- [ ] Mask filenames match image filenames exactly
- [ ] Masks are binary (0 and 255 only)
- [ ] Masks are same dimensions as images
- [ ] At least 30 images total (preferably 100+)
- [ ] Diverse conditions (lighting, angles, crack types)
- [ ] Clean annotations (no mislabeled cracks)

Troubleshooting

Common Issues and Solutions

Issue: "Ratios must sum to 1.0"

Symptom: Error when trying to import with split

Cause: Train + val + test ratios don't equal 1.0 exactly

Solution:

```
# Wrong: 0.7 + 0.2 + 0.2 = 1.1 ❌
# Right: 0.7 + 0.15 + 0.15 = 1.0 ✅
--split-ratios 0.7 0.15 0.15
```

Issue: Validation split is empty

Symptom: After import, `images/val/` has no files

Cause: Too few images or val ratio too small

Solution:

```
# If you have 10 images and use 0.7/0.15/0.15:
# train=7, val=1, test=2 (works)

# If you have 5 images and use 0.8/0.1/0.1:
# train=4, val=0, test=1 (val is empty!)

# Solution: Use larger val ratio or more images
--split-ratios 0.6 0.25 0.15  # Ensures val gets at least 1
```

Issue: Masks not found for some images

Symptom: Warning about missing masks during import

Cause: Mask filenames don't match image filenames

Solution:

```
# Image: images/crack_001.jpg
# Mask must be: masks/crack_001.png (same stem)

# Check your filenames:
ls images/ > image_list.txt
ls masks/ > mask_list.txt
# Compare the stems (filename without extension)
```

Issue: Different split on each import

Symptom: Files go to different splits each time

Cause: No seed specified, random shuffle differs

Solution:

```
# Add --seed for reproducible splits
--seed 42
```

Issue: Training fails with "not enough validation data"

Symptom: Training crashes or warns about validation set

Cause: Validation split too small (< 10 images)

Solution:

```
# Increase validation ratio
--split-ratios 0.65 0.25 0.1  # Gives more to validation
```

Issue: Out of memory during training

Symptom: CUDA out of memory or system crash

Cause: Batch size too large or images too big

Solution:

```
# Import with smaller size
--size 256 256  # Instead of 512 512

# Or reduce batch size during training
crackz detect train --batch-size 2  # Instead of 8
```

Advanced Topics

Stratified Splitting

For datasets with imbalanced classes (many no-crack, few crack images), consider manual stratification:

```
# Separate images by class first
mkdir temp/crack temp/no-crack

# Move files to class folders
# ... organize manually or with script ...

# Import each class separately with same ratio
crackz import-images --src temp/crack --split-ratios 0.7 0.15 0.15 --seed 42
crackz import-images --src temp/no-crack --split-ratios 0.7 0.15 0.15 --seed 42
```

Cross-Validation

For small datasets, consider k-fold cross-validation:

```
# Import full dataset to train
crackz import-images --src data --type train --size 512 512

# Manually create folds by moving files between train/val
# Train 5 models with different train/val splits
# Average results for robust performance estimate
```

Incremental Training

Adding new data to existing splits:

```
# Import new batch with same seed and ratios
crackz import-images \
  --src new-data \
  --split-ratios 0.7 0.15 0.15 \
  --seed 42 \
  --size 512 512

# WARNING: This adds to existing splits
# Consider fresh re-import of all data for clean splits
```

Quick Reference

Split Summary Table

Split	Dir	Masks	Used in Training	Used in Testing	Purpose
Raw	`images/raw/`	No	No	No	Production detection
Train	`images/train/`	Yes	Yes (learning)	No	Teaching model
Val	`images/val/`	Yes	Yes (monitoring)	No	Preventing overfit
Test	`images/test/`	Yes	No	Yes	Final evaluation

Common Commands

```
# Import for training (with split)
crackz import-images --src data/images --src-masks data/masks \
  --split-ratios 0.7 0.15 0.15 --seed 42 --size 512 512

# Import for detection (no split)
crackz import-images --src photos --type raw --size 512 512

# Train on train split, validate on val split
crackz detect train --epochs 50 --batch-size 8

# Test on test split
crackz detect run --split test

# Detect on raw split
crackz detect run --split raw
```

Recommended Ratios

- **Standard**: 70% train, 15% val, 15% test
- **Small dataset**: 60% train, 25% val, 15% test
- **Large dataset**: 80% train, 10% val, 10% test

Further Reading

- GUI Guide - Using the graphical interface
- CLI Cookbook - Command-line recipes
- Training Configuration - Advanced training settings
- Configuration Guide - Project configuration details

Docker

Docker Usage

This page summarizes containerized workflows for the Crackz CLI. For deeper context see the repository README.

Registry

Pre-built images are available from GitHub Container Registry:

```
# Pull from registry
docker pull ghcr.io/anaxiatech-ab/crackz:latest
docker pull ghcr.io/anaxiatech-ab/crackz:slim
docker pull ghcr.io/anaxiatech-ab/crackz:gpu

# Run from registry
docker run --rm ghcr.io/anaxiatech-ab/crackz:latest --help
```

Registry URL: https://github.com/anaxiatech?tab=packages&repo_name=crackz

Images

Tag	Dockerfile	Purpose	Python Version
`latest`	`Dockerfile`	Full multi-stage build (uv-based wheel build)	3.13
`slim`	`Dockerfile.slim`	Minimal runtime (uv-based wheel build)	3.13
`gpu`	`Dockerfile.gpu`	CUDA 12.8 runtime (requires NVIDIA Container Toolkit)	3.13

Build locally:

```
# Full
docker build -t crackz-cli .
# Slim
docker build -f Dockerfile.slim -t crackz-cli:slim .
# GPU (CUDA 12.8)
docker build -f Dockerfile.gpu -t crackz-cli:gpu .
```

All images default to Python 3.13. You can override at build time:

```
docker build --build-arg PYTHON_VERSION=3.13 -t crackz-cli .
```

Build Strategy

All images follow a two-stage build pattern for optimized builds:

1. **Builder stage**: Uses `ghcr.io/astral-sh/uv:bookworm-slim` to install managed Python and build a wheel with `uv build --wheel`.
2. **Runtime stage**:
3. **Full/Slim**: Starts from `python:3.13-slim` (Debian-based)
4. **GPU**: Starts from `nvidia/cuda:12.8.0-runtime-ubuntu22.04` with Python 3.13 installed via deadsnakes PPA
5. **Installation**: Creates a virtual environment and installs the pre-built wheel
6. **Optional PyTorch index override** via `--build-arg TORCH_INDEX_URL=...` to select specific CUDA wheels (e.g., for older CUDA versions)

Benefits:

- **Faster builds**: `uv build` creates optimized wheels quickly
- **Better layer caching**: Wheel build is cached separately from dependencies
- **Smaller runtime**: Build tools are not included in the final image
- **Flexibility**: Can override PyTorch index for specific CUDA versions

GPU Image Details

The GPU image is specifically configured for NVIDIA GPU deployment with CUDA 12.8:

- **Base**: `nvidia/cuda:12.8.0-runtime-ubuntu22.04`
- **Python Installation**: Python 3.13 installed from deadsnakes PPA (Ubuntu 22.04 ships Python 3.10 by default)
- **System Dependencies**: Includes `git`, `libglib2.0-0`, `libjpeg-turbo8`, `libgomp1`, and `ca-certificates`
- **CUDA Support**: CUDA 12.8 runtime
- **PyTorch**: By default, installs PyTorch 2.8.0+ with CUDA 12.8 support from PyPI

PyTorch CUDA Version Compatibility

The GPU image uses CUDA 12.8 to match PyTorch's cu128 wheels. If you need a different CUDA version:

```
# Default: CUDA 12.8 (recommended, matches PyTorch cu128)
docker build -f Dockerfile.gpu -t crackz-cli:gpu .

# Override for CUDA 12.1
docker build -f Dockerfile.gpu \
  --build-arg TORCH_INDEX_URL=https://download.pytorch.org/whl/cu121 \
  -t crackz-cli:gpu-cu121 .

# Override for CUDA 12.4 (if needed for compatibility)
docker build -f Dockerfile.gpu \
  --build-arg TORCH_INDEX_URL=https://download.pytorch.org/whl/cu124 \
  -t crackz-cli:gpu-cu124 .
```

Important: Leave `TORCH_INDEX_URL` empty (default) to use CUDA 12.8, which provides the best compatibility with the latest PyTorch releases.

GPU Build Verification

During the build, the image verifies PyTorch installation:

```
#18 [runtime 7/9] RUN /app/.venv/bin/python3 -c 'import torch; print("Torch:", torch.__version__, "CUDA available:", torch.cuda.is_available())'
#18 0.699 Torch: 2.8.0+cpu CUDA available: False
```

Note: `CUDA available: False` during build is expected when building on macOS or systems without NVIDIA GPUs. CUDA will be available when the container runs on a Linux system with NVIDIA drivers and the NVIDIA Container Toolkit.

Platform Note: The GPU image targets `linux/amd64`. When building on Apple Silicon (ARM64), you may see platform warnings—this is expected and correct.

Basic Commands

Container now starts in the non-root user home (`/home/app`). Mount your host project directory there to persist projects.

```
# Help
docker run --rm crackz-cli --help

# Init project (creates /home/app/demo-project inside container)
mkdir -p project
docker run --rm -v "$(pwd)/project:/home/app" crackz-cli init --name demo-project

# Import images (mount samples to /data)
mkdir -p samples
# (populate samples/ with images and optional masks/ first)
docker run --rm \
  -v "$(pwd)/project:/home/app" \
  -v "$(pwd)/samples:/data" \
  crackz-cli import-images --project /home/app/demo-project --src /data --type raw --size 512 512

# Detection
docker run --rm -v "$(pwd)/project:/home/app" crackz-cli detect run --project /home/app/demo-project
```

GPU variant:

```
docker run --gpus all --rm -v "$(pwd)/project:/home/app" crackz-cli:gpu detect run --project /home/app/demo-project
```

Volumes & Persistence

Path in Container	Purpose	Suggested Host Mount
`/home/app`	Project (images, artifacts, config, logs)	`project/`
`/data`	Input image dataset (import source)	`samples/`

A separate `/app` directory still contains the installed package; user content lives under `/home/app`.

Environment Variables

Variable	Effect	Default
`CRACKZ_LOG_DIR`	Log directory (relative to current working dir unless absolute)	`logs`
`PYTHONUNBUFFERED`	Unbuffered stdout/stderr	`1`

Override on run (writes under `/home/app/central_logs` in this example):

```
docker run --rm -e CRACKZ_LOG_DIR=central_logs -v "$(pwd)/project:/home/app" \
  crackz-cli detect run --project /home/app/demo-project
```

Makefile Shortcuts

Provided for POSIX shells (now mounting to `/home/app`):

```
make build        # full image
make build-slim   # slim image
make build-gpu    # gpu image (CUDA 12.8)
make init NAME=demo-project              # creates project under /home/app
make import PROJECT=demo-project DATA_DIR=./samples
make detect PROJECT=demo-project
make detect-gpu PROJECT=demo-project     # GPU detection
make train-gpu PROJECT=demo-project      # GPU training
```

Testing with Act

You can test the Docker build workflow locally using `act` (GitHub Actions local runner):

```
# Install act (macOS)
brew install act

# Test GPU build with make targets
make act-gpu-dryrun          # Dry-run to validate workflow
make act-gpu                 # Actually run the GPU build locally
make act-docker-build        # Test all Docker builds (full, slim, gpu)
```

When to Use Which Image

Scenario	Recommended Tag
Local experimentation	`latest`
CI quick smoke tests	`slim`
Training / GPU inference	`gpu` (CUDA 12.8)
Older GPU hardware	`gpu` with custom `TORCH_INDEX_URL`

Troubleshooting

Symptom	Cause	Fix
Permission errors	Host FS perms	Ensure mounted dir writable by your user (UID mismatch typically OK because container runs as created system user)
Missing logs	Log dir not created yet	Ensure `CRACKZ_LOG_DIR` points inside writable mounted path
CUDA not detected in container	Driver / toolkit mismatch	Align CUDA version, update drivers, use `--gpus all`, verify NVIDIA Container Toolkit installed
CUDA version mismatch	Wrong PyTorch wheel	Rebuild with appropriate `TORCH_INDEX_URL` or use default CUDA 12.8

Next Steps

- See Getting Started for an initial workflow.
- See CLI Cookbook for advanced CLI patterns.
- Explore API Reference for programmatic usage.

Azure VM (GPU)

Azure Windows VM with CUDA for Crackz

Deploy GPU-accelerated Windows VMs on Azure for running Crackz AI inference workloads.

Overview

The `azure_vm_cuda.sh` script creates GPU-enabled Windows 11 Pro virtual machines optimized for Crackz machine learning workloads. It automatically configures:

- ✅ **GPU-optimized VM sizes** (NC-series with NVIDIA Tesla GPUs)
- ✅ **CUDA Toolkit 11.8** installation
- ✅ **Python 3.13 + UV package manager**
- ✅ **Crackz with GPU support**
- ✅ **Development tools** (VS Code, Git)
- ✅ **Network security** (RDP + ML ports)

Quick Start

```
# Deploy default GPU VM
./scripts/azure/azure_vm_cuda.sh

# Deploy with custom configuration
./scripts/azure/azure_vm_cuda.sh \
  --name ml-workstation \
  --size Standard_NC12s_v3 \
  --location westus2

# Connect via RDP and verify
# RDP to the public IP, then run:
nvidia-smi
cd C:\Crackz && .venv\Scripts\Activate.ps1
python -m crackz --version
```

VM Sizes & GPU Options

Recommended Sizes

VM Size	GPU	VRAM	CPU Cores	RAM	Cost/Hour*	Use Case
`Standard_NC6s_v3`	1x V100	16GB	6	112GB	~$3.06	Standard inference
`Standard_NC12s_v3`	2x V100	32GB	12	224GB	~$6.12	Batch processing
`Standard_NC24s_v3`	4x V100	64GB	24	448GB	~$12.24	Large-scale inference

Budget Options

VM Size	GPU	VRAM	CPU Cores	RAM	Cost/Hour*	Use Case
`Standard_NC6s_v2`	1x P100	16GB	6	112GB	~$2.07	Development
`Standard_NV6`	1x M60	8GB	6	56GB	~$1.20	Testing

*Approximate East US pricing (subject to change; check Azure for current rates)

Command Reference

Basic Usage

```
# Default deployment (V100 GPU, East US)
./scripts/azure/azure_vm_cuda.sh

# Custom name and location
./scripts/azure/azure_vm_cuda.sh --name gpu-crackz --location westeurope

# Specific GPU configuration
./scripts/azure/azure_vm_cuda.sh --size Standard_NC12s_v3 --location northeurope
```

Options

Option	Description	Default
`--name NAME`	VM name	`crackz-gpu-vm`
`--location REGION`	Azure region	`eastus`
`--size SIZE`	VM size	`Standard_NC6s_v3`
`--user USERNAME`	Admin username	`crackzuser`
`--password PASSWORD`	Admin password	`CrackzGpu2024!`
`--mode MODE`	Deployment mode	`standard`
`--data-disk-size SIZE`	Data disk size in GB	`1024`
`--storage-type TYPE`	Storage type	`Premium_LRS`
`--no-data-disk`	Skip creating separate data disk	Create data disk
`--no-cuda`	Skip CUDA installation	Install CUDA
`--no-crackz`	Skip Crackz installation	Install Crackz
`--enable-nested-virt`	Enable nested virtualization	Disabled
`--destroy`	Destroy VM and resources	Create
`--help`	Show help	-

Deployment Examples

```
# Production inference server
./scripts/azure/azure_vm_cuda.sh \
  --name crackz-prod \
  --size Standard_NC12s_v3 \
  --location westus2 \
  --mode production

# Development environment with large storage
./scripts/azure/azure_vm_cuda.sh \
  --name crackz-dev \
  --size Standard_NV6 \
  --mode development \
  --data-disk-size 2048 \
  --storage-type Premium_LRS

# Budget setup without data disk
./scripts/azure/azure_vm_cuda.sh \
  --name crackz-simple \
  --size Standard_NV6 \
  --no-data-disk

# Manual setup (no auto-install)
./scripts/azure/azure_vm_cuda.sh \
  --name crackz-manual \
  --size Standard_NC6s_v3 \
  --no-cuda \
  --no-crackz

# Docker development (with nested virtualization)
./scripts/azure/azure vm_cuda.sh \
  --name crackz-docker \
  --size Standard_NC6s_v3 \
  --enable-nested-virt

# Clean up resources
./scripts/azure/azure_vm_cuda.sh --name crackz-gpu-vm --destroy
```

Storage Configuration

Data Disk Options

The script automatically creates and configures a separate data disk for ML projects:

Parameter	Description	Default	Options
`--data-disk-size SIZE`	Data disk size in GB	1024	32-32767
`--storage-type TYPE`	Storage performance tier	Premium_LRS	Premium_LRS, Standard_LRS
`--no-data-disk`	Skip data disk creation	false	Creates separate disk

Storage Layout

```
C:\ (OS Disk - 127 GB)
├── Windows/
├── Program Files/
├── Users/crackzuser/Desktop/Crackz/   # Crackz installation
└── ...

D:\ (Data Disk - configurable size)
├── Projects/                          # ML project workspace
│   ├── datasets/                     # Training datasets
│   ├── models/                       # Model checkpoints
│   ├── outputs/                      # Detection results
```

```
│   └── exports/                    # Exported models
└── temp/                           # Temporary processing
```

Storage Performance

Type	Performance	Use Case	Cost/Month*
Premium_LRS	Up to 20,000 IOPS	Production ML workloads	~$150/TB
Standard_LRS	Up to 2,000 IOPS	Development/testing	~$50/TB

*Costs for East US region, subject to change

Best Practices

```
# Production: High-performance storage
./scripts/azure/azure_vm_cuda.sh \
  --name ml-prod \
  --data-disk-size 2048 \
  --storage-type Premium_LRS

# Development: Cost-effective storage
./scripts/azure/azure_vm_cuda.sh \
  --name ml-dev \
  --data-disk-size 512 \
  --storage-type Standard_LRS

# Simple setup: No separate disk
./scripts/azure/azure_vm_cuda.sh \
  --name ml-simple \
  --no-data-disk
```

What Gets Installed

Automatic Installation (default)

1. **NVIDIA Drivers & CUDA 11.8**
2. CUDA Toolkit with cuDNN
3. Environment variables configured

4. Compatible with PyTorch

5. **Python Environment**

6. Python 3.13.x (latest)
7. UV package manager

8. Virtual environment at `C:\Crackz\.venv`

9. **Crackz with GPU Support**

10. Latest release wheel
11. CUDA-enabled PyTorch

12. Desktop shortcut created

13. **Development Tools**

14. Visual Studio Code
15. Git for Windows
16. Windows Terminal (via Chocolatey)

Network Configuration

- **RDP (3389)**: Remote desktop access
- **File Upload Web (8080)**: Browser-based image upload interface
- **SFTP (22)**: Secure file transfer protocol
- **SMB/CIFS (445)**: Windows file sharing
- **Jupyter (8888)**: For notebook development
- **TensorBoard (6006)**: For training visualization
- **Static Public IP**: Consistent connectivity

Post-Deployment Setup

File Upload Methods (Offline Capable)

The VM provides multiple ways to upload images for analysis:

Method 1: Web Upload Interface (Recommended)

```
# 1. Connect via RDP to the VM
# 2. Double-click "File Upload Server" desktop shortcut
# 3. Open browser and go to: http://localhost:8080
# 4. Or access remotely: http://<VM-PUBLIC-IP>:8080
```

Features: - ✅ Easy drag-and-drop file upload - ✅ Multiple file selection - ✅ Progress tracking - ✅ Automatic directory organization - ✅ Works offline once VM is running

Method 2: Network File Share

```
# From your local computer:
# Windows: \\<VM-PUBLIC-IP>\CrackzUploads
# Mac/Linux: smb://<VM-PUBLIC-IP>/CrackzUploads

# Credentials: crackzuser / CrackzGpu2024!
```

Method 3: SFTP (Secure File Transfer)

```
# Using any SFTP client (WinSCP, FileZilla, etc.)
sftp crackzuser@<VM-PUBLIC-IP>
# Upload to: /d/Projects/uploads/images/
```

File Organization

```
D:\Projects\uploads\
├── images/          # Individual images for analysis
└── batch/          # Batch processing folders
```

1. Connect via RDP

```
# Use Windows built-in Remote Desktop
mstsc /v:<PUBLIC_IP>

# Or download RDP file
az vm show -d -g crackz-gpu-rg -n crackz-gpu-vm --query publicIps -o tsv
```

2. Verify GPU Setup

```
# Check GPU status
nvidia-smi

# Test CUDA availability
python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}')"
python -c "import torch; print(f'GPU count: {torch.cuda.device_count()}')"
```

3. Verify Crackz Installation

```
# Activate virtual environment
cd C:\Crackz
.\.venv\Scripts\Activate.ps1

# Check Crackz version
python -m crackz --version

# Test GPU detection
python -m crackz gui  # Should detect CUDA device
```

4. Run Analysis on Uploaded Images

```
# Activate virtual environment
cd C:\Crackz
.\.venv\Scripts\Activate.ps1

# Option 1: Direct analysis on uploaded images
python -m crackz detect run --images "D:\Projects\uploads\images" --device cuda

# Option 2: Create project and import uploaded images
python -m crackz init harbor-inspection
python -m crackz import-images "D:\Projects\uploads\images" harbor-inspection/images/raw
python -m crackz detect run harbor-inspection --device cuda

# Option 3: Batch processing multiple folders
python -m crackz detect run --images "D:\Projects\uploads\batch\*" --device cuda

# View results
explorer "harbor-inspection\artifacts\detection_results"
```

5. Traditional Workflow (Local Images)

```
# Create test project
python -m crackz init test-project

# Import sample images
python -m crackz import-images C:\path\to\images test-project/images/raw

# Run GPU-accelerated detection
python -m crackz detect run test-project --device cuda
```

Best Practices

Cost Optimization

- ✅ **Use spot instances** for non-critical workloads (-60-90% cost)
- ✅ **Auto-shutdown**: Configure automatic shutdown during idle periods
- ✅ **Right-size VMs**: Start with smaller sizes and scale up as needed
- ✅ **Deallocate when idle**: `az vm deallocate` to stop compute charges

Performance Optimization

- ✅ **Premium SSD storage** for faster I/O (automatically configured)
- ✅ **Place in same region** as your data storage
- ✅ **Use proximity placement groups** for multi-VM deployments
- ✅ **Enable accelerated networking** for high-throughput workloads

Security

- 🚨 **CRITICAL: Change default password** immediately after first login
- ✅ **Use strong passwords** (12+ characters, mixed case, numbers, symbols)
- ✅ **Configure Azure Key Vault** for password management in production
- ✅ **Use Azure Bastion** for secure access without public IPs
- ✅ **Enable disk encryption** for sensitive data
- ✅ **Set up automatic security updates** via Windows Update
- ✅ **Monitor access logs** and set up alerts for unusual activity

Default Password Warning

The script uses a default password (`CrackzGpu2024!`) for initial setup convenience. This password: - ❌ Should **NEVER** be used in production - ❌ Is visible in deployment logs and command history - ❌ Could be compromised if logs are accessed

Immediate Actions After Deployment: 1. Connect via RDP using the default credentials 2. Change the administrator password immediately 3. Enable Windows Defender and automatic updates 4. Consider disabling the built-in administrator account

Troubleshooting

Common Issues

CUDA Installation Failed

```
# Manually install CUDA
# Download from: https://developer.nvidia.com/cuda-11-8-0-download-archive
# Choose Windows x86_64 local installer
```

Crackz Installation Failed

```
# Manual installation
cd C:\Crackz
.\.venv\Scripts\Activate.ps1
uv pip install --find-links https://download.pytorch.org/whl/cu118 torch torchvision
# Download wheel from GitHub releases and install
```

GPU Not Detected

```
# Check drivers
nvidia-smi

# Verify CUDA installation
nvcc --version

# Check PyTorch CUDA
python -c "import torch; print(torch.version.cuda)"
```

Performance Issues

- Check GPU utilization: `nvidia-smi -l 1`
- Monitor system resources in Task Manager
- Verify SSD storage performance
- Check network bandwidth to data sources

Logs and Monitoring

```
# Windows Event Logs
eventvwr.msc

# CUDA installation logs
Get-Content C:\ProgramData\NVIDIA Corporation\CUDA Installer\log.txt

# Crackz logs
Get-Content $env:TEMP\crackz-*.log
```

Regional Availability

GPU VM Availability by Region

Region	NC-series	NV-series	Recommended
East US	✅	✅	✅ Primary
West US 2	✅	✅	✅ Primary
North Europe	✅	✅	✅ EU option
Southeast Asia	✅	✅	✅ APAC option
West Europe	⚠️ Limited	✅	Backup
Central US	⚠️ Limited	✅	Backup

Check current availability:

```
az vm list-sizes --location eastus --query "[?contains(name, 'NC')]"
```

Cost Estimation

Monthly Costs (24x7 usage)

Configuration	VM Cost	Storage	Total	Use Case
NC6s_v3 (1x V100)	~$2,203	~$30	~$2,233	Production
NC6s_v2 (1x P100)	~$1,490	~$30	~$1,520	Development
NV6 (1x M60)	~$864	~$30	~$894	Testing

Cost Optimization Tips

```
# Use spot instances (60-90% discount)
az vm create --priority Spot --max-price -1 ...

# Auto-shutdown schedule
az vm auto-shutdown --name crackz-gpu-vm --resource-group crackz-gpu-rg --time 1800

# Deallocate when not in use
az vm deallocate --name crackz-gpu-vm --resource-group crackz-gpu-rg
```

Related Documentation

- Azure Batch Deployment - Multi-VM distributed processing
- Installation Guide - General Crackz installation
- Configuration - Crackz configuration options
- CLI Cookbook - Common command patterns

Support

For issues specific to Azure deployment: 1. Check VM size availability in your region 2. Verify Azure subscription GPU quotas 3. Review Azure Activity Log for deployment errors 4. Contact Azure support for infrastructure issues

For Crackz-specific issues: - GitHub Issues - Check logs in `%TEMP%\crackz-*.log` - Verify GPU detection with test commands above

Azure Batch

Azure Batch Deployment

This guide explains how to deploy and use Crackz on Azure Batch for distributed, scalable crack detection processing.

Overview

Azure Batch allows you to run large-scale parallel and high-performance computing (HPC) applications efficiently in the cloud. The Crackz Azure Batch deployment enables:

- **Distributed Processing**: Process large datasets across multiple compute nodes
- **Scalability**: Automatically scale compute resources based on workload
- **Cost Optimization**: Use low-priority VMs to reduce costs
- **Container Support**: Run Crackz Docker images directly on Batch nodes

> **Note**: For smaller datasets or interactive development, consider using Azure VM deployment instead. See the VM vs Batch Comparison to help you decide which option is best for your use case.

Prerequisites

1. **Azure CLI**: Install the Azure CLI
2. **Azure Subscription**: Active Azure subscription with appropriate permissions
3. **Docker Image**: Built Crackz Docker image (pushed to a container registry)
4. **Azure Login**: `az login` to authenticate

Quick Start

1. Create a Batch Pool

The simplest way to create a Batch pool:

```
./scripts/azure/azure_batch.sh \
  --batch-account mycrackzaccount \
  --storage-account mycrackzstorage
```

This creates: - A Batch account named `mycrackzaccount` - A storage account named `mycrackzstorage` - A pool with 2 compute nodes (Standard_D2s_v3) - A blob container for data storage

2. Upload Your Data

Upload your Crackz project to Azure Storage:

```
# Get storage account key
STORAGE_KEY=$(az storage account keys list \
  --account-name mycrackzstorage \
  --resource-group subbaltica-crackz-rg \
  --query "[0].value" -o tsv)

# Upload project directory
az storage blob upload-batch \
  --account-name mycrackzstorage \
  --account-key "$STORAGE_KEY" \
  --destination crackz-data \
  --source ./my-project
```

3. Submit a Job

Create and submit a Batch job:

```
# Create job
az batch job create \
  --id crackz-detection-job \
  --pool-id crackz-pool

# Add tasks to the job
az batch task create \
  --job-id crackz-detection-job \
  --task-id detect-task-1 \
  --command-line "docker run crackz-cli:latest detect run --project /mnt/batch/tasks/fsmounts/crackz-data/my-project"
```

Configuration Options

Pool Configuration

Option	Description	Default
`--batch-account`	Azure Batch account name (required)	-
`--resource-group`	Azure resource group name	`subbaltica-crackz-rg`
`--location`	Azure region	`swedencentral`
`--pool-id`	Unique pool identifier	`crackz-pool`
`--vm-size`	VM size for compute nodes	`Standard_D2s_v3`
`--node-count`	Number of dedicated nodes	`2`
`--image`	Docker image to use	`crackz-cli:latest`
`--storage-account`	Storage account for data	-
`--container`	Blob container name	`crackz-data`

VM Sizes for Different Workloads

VM Size	vCPUs	Memory	Use Case	Hourly Cost*
`Standard_D2s_v3`	2	8 GB	Small datasets, testing	~$0.10
`Standard_D4s_v3`	4	16 GB	Medium datasets	~$0.20
`Standard_D8s_v3`	8	32 GB	Large datasets	~$0.40
`Standard_NC6s_v3`	6	112 GB	GPU acceleration (V100)	~$3.06
`Standard_NC4as_T4_v3`	4	28 GB	GPU acceleration (T4)	~$0.53

* Approximate costs as of 2024, subject to change

Advanced Usage

Custom Pool Configuration

Create a pool with custom settings:

```
./scripts/azure/azure_batch.sh \
  --batch-account crackz-batch \
  --pool-id high-performance-pool \
  --vm-size Standard_D8s_v3 \
  --node-count 10 \
  --storage-account crackzstore
```

GPU-Enabled Pool

For GPU acceleration, use GPU-capable VM sizes:

```
./scripts/azure/azure_batch.sh \
  --batch-account crackz-batch-gpu \
  --pool-id gpu-pool \
  --vm-size Standard_NC4as_T4_v3 \
  --node-count 2 \
  --image crackz-cli:gpu
```

Auto-Scaling Pools

For dynamic workloads, configure auto-scaling by modifying the pool after creation:

```
# Enable auto-scaling
az batch pool autoscale enable \
  --pool-id crackz-pool \
  --account-name mycrackzaccount \
  --auto-scale-formula '
    startingNumberOfVMs = 2;
    maxNumberofVMs = 10;
    pendingTaskSamplePercent = $PendingTasks.GetSamplePercent(180 * TimeInterval_Second);
    pendingTaskSamples = pendingTaskSamplePercent < 70 ? startingNumberOfVMs : avg($PendingTasks.GetSample(180 * TimeInterval_Second));
    $TargetDedicatedNodes = min(maxNumberofVMs, pendingTaskSamples);
  '
```

Workflow Examples

Batch Processing Multiple Projects

Process multiple crack detection projects in parallel:

```
#!/bin/bash
# Create job
az batch job create --id multi-project-job --pool-id crackz-pool

# Submit tasks for each project
for project in project1 project2 project3; do
  az batch task create \
    --job-id multi-project-job \
    --task-id "detect-$project" \
    --command-line "docker run crackz-cli:latest detect run --project /data/$project"
done
```

Training and Detection Pipeline

Run a complete training and detection pipeline:

```
# Training task
az batch task create \
  --job-id pipeline-job \
  --task-id train \
  --command-line "docker run crackz-cli:latest detect train --project /data/my-project --epochs 50"

# Detection task (depends on training)
az batch task create \
  --job-id pipeline-job \
  --task-id detect \
  --depends-on train \
  --command-line "docker run crackz-cli:latest detect run --project /data/my-project"
```

Batch Job with Output Files

Configure tasks to upload outputs back to storage:

```
az batch task create \
  --job-id output-job \
  --task-id detect-with-output \
  --command-line "docker run -v /mnt/output:/output crackz-cli:latest detect run --project /data/project" \
  --resource-files '[{"httpUrl":"https://mystorage.blob.core.windows.net/input/data.zip","filePath":"data.zip"}]' \
  --output-files '[{"filePattern":"/mnt/output/**/*","destination":{"container":{"containerUrl":"https://mystorage.blob.core.windows.net/results"}},"uploadOptions":{"uploadCo
```

Monitoring and Management

Check Pool Status

```
az batch pool show \
  --pool-id crackz-pool \
  --account-name mycrackzaccount \
  --query "{State:allocationState, DedicatedNodes:currentDedicatedNodes}"
```

List Running Jobs

```
az batch job list \
  --account-name mycrackzaccount \
  -o table
```

Monitor Task Progress

```
az batch task list \
  --job-id crackz-detection-job \
  --account-name mycrackzaccount \
  -o table
```

Get Task Logs

```
az batch task file download \
  --job-id crackz-detection-job \
  --task-id detect-task-1 \
  --file-path stdout.txt \
  --destination ./task-stdout.txt \
  --account-name mycrackzaccount
```

Cost Optimization

Using Low-Priority VMs

Low-priority VMs can reduce costs by up to 80%:

```
# After pool creation, modify to use low-priority nodes
az batch pool resize \
```

```
  --pool-id crackz-pool \
  --account-name mycrackzaccount \
  --target-dedicated-nodes 0 \
  --target-low-priority-nodes 4
```

Note: Low-priority VMs may be preempted, so ensure your tasks are resilient.

Pool Management

Delete pools when not in use:

```
./scripts/azure/azure batch.sh \
  --batch-account mycrackzaccount \
  --pool-id crackz-pool \
  --destroy
```

Troubleshooting

Common Issues

Issue	Solution
Pool creation fails	Check quota limits: `az batch location quotas show --location <region>`
Tasks stuck in "Running"	Check task logs: `az batch task file download`
Container image not found	Ensure image is in an accessible registry
Authentication errors	Run `az batch account login` before operations

Debug Task Failures

```
# Get task details
az batch task show \
  --job-id crackz-detection-job \
  --task-id detect-task-1 \
  --account-name mycrackzaccount

# Download stderr
az batch task file download \
  --job-id crackz-detection-job \
  --task-id detect-task-1 \
  --file-path stderr.txt \
  --destination ./task-stderr.txt \
  --account-name mycrackzaccount
```

Increase Batch Quotas

If you hit quota limits, request an increase:

```
# Check current quotas
az batch location quotas show --location swedencentral

# Submit quota increase request through Azure Portal
# Support > New support request > Issue type: Service and subscription limits
```

Integration with CI/CD

GitHub Actions Example

```
name: Azure Batch Processing
on:
  push:
    branches: [main]

jobs:
  batch-process:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v5

      - name: Azure Login
        uses: azure/login@v1
        with:
          creds: ${{ secrets.AZURE_CREDENTIALS }}

      - name: Create Batch Pool
        run: |
          ./scripts/azure/azure_batch.sh \
            --batch-account ${{ secrets.BATCH_ACCOUNT }} \
            --storage-account ${{ secrets.STORAGE_ACCOUNT }}

      - name: Submit Batch Job
        run: |
```

```
        az batch job create --id ci-job-${{ github.run_number }} --pool-id crackz-pool
        az batch task create \
          --job-id ci-job-${{ github.run_number }} \
          --task-id detect \
          --command-line "docker run crackz-cli:latest detect run --project /data/project"
```

Security Best Practices

1. **Use Managed Identities**: Avoid storing credentials in code
2. **Private Container Registries**: Use Azure Container Registry with authentication
3. **Network Isolation**: Deploy pools in Virtual Networks
4. **Encryption**: Enable encryption for storage accounts
5. **RBAC**: Use role-based access control for Batch resources

Performance Tuning

Optimize Task Slots

Adjust `taskSlotsPerNode` for better resource utilization:

```
# Modify pool to allow multiple tasks per node
az batch pool set \
  --pool-id crackz-pool \
  --account-name mycrackzaccount \
  --json-file pool-config.json
```

Where `pool-config.json` contains:

```
{
  "taskSlotsPerNode": 4
}
```

Batch Size Configuration

For detection tasks, adjust batch size based on VM memory:

VM Size	Recommended Batch Size	Notes
Standard_D2s_v3 (8 GB)	4-8	Conservative for 512x512 images
Standard_D4s_v3 (16 GB)	16-32	Good balance
Standard_D8s_v3 (32 GB)	32-64	Large batches

Modify project configuration before uploading:

```
ai:
  detection:
    batch_size: 16  # Adjust based on VM size
```

Related Documentation

- VM vs Batch Comparison - Help choosing between VM and Batch
- Configuration Reference - Project configuration options
- Docker Usage - Docker image details
- Advanced Performance - GPU and multi-device training
- Azure VM Deployment - Single VM deployment alternative

References

- Azure Batch Documentation
- Azure Batch Pricing
- Azure CLI Reference

VM vs Batch

Azure Deployment Comparison

This document helps you choose between Azure VM and Azure Batch deployment options for Crackz.

Quick Decision Matrix

Criterion	Azure VM	Azure Batch
Dataset Size	Small to medium (<1000 images)	Large (1000+ images)
Processing Time	Minutes to hours	Hours to days
Parallelism	Single instance	Multiple nodes (2-100+)
Cost	Predictable (fixed VM cost)	Variable (pay per compute time)
Setup Complexity	Simple	Moderate
Use Case	Development, testing, small projects	Production, batch processing, research
GUI Support	Yes (RDP available)	No (headless only)
Best For	Interactive work, model development	Large-scale automated processing

Azure VM Deployment

When to Use

- **Development & Testing**: Interactive environment with RDP access
- **Small Datasets**: Processing up to 1000 images
- **GUI Required**: Need to use Crackz GUI for manual inspection
- **Predictable Costs**: Fixed hourly rate for VM
- **Learning**: Experimenting with Crackz features

Pros

✓ Simple setup (one script, one command)
✓ RDP access for interactive work
✓ Predictable hourly costs
✓ Full Windows environment
✓ Good for development and debugging

Cons

✗ Limited to single VM performance
✗ Manual scaling required
✗ Not cost-effective for large datasets
✗ Idle time still incurs costs

Example Usage

```
# Create Windows 11 VM
./scripts/azure/azure_vm.sh \
  --name crackz-dev \
  --size Standard_D4s_v3

# Connect via RDP and use GUI
# IP address displayed after creation
```

Cost Example

- **Standard_D4s_v3** (4 vCPUs, 16 GB RAM): ~$0.20/hour
- **8 hours of processing**: ~$1.60
- **Monthly (730 hours)**: ~$146 if left running

Azure Batch Deployment

When to Use

- **Large Datasets**: Processing thousands of images

- **Parallel Processing**: Need to process multiple projects simultaneously
- **Cost Optimization**: Want to pay only for actual compute time
- **Automation**: Fully automated pipelines without human interaction
- **Scale**: Need to scale from 2 to 100+ nodes dynamically

Pros

✓ Massive parallelism (100+ nodes)
✓ Pay only for compute time
✓ Automatic task distribution
✓ Low-priority VMs (up to 80% cost savings)
✓ Built-in retry and failure handling
✓ Integration with Azure Storage

Cons

✗ More complex setup
✗ Headless only (no GUI)
✗ Requires understanding of Batch concepts
✗ Initial learning curve

Example Usage

```
# Create batch pool with 10 nodes
./scripts/azure/azure_batch.sh \
  --batch-account mybatchaccount \
  --storage-account mystorage \
  --node-count 10 \
  --vm-size Standard_D4s_v3

# Submit 100 detection jobs
for i in {1..100}; do
  az batch task create \
    --job-id detection-job \
    --task-id "task-$i" \
    --command-line "docker run crackz-cli:latest detect run --project /data/project-$i"
done
```

Cost Example (10 nodes)

- **Standard_D4s_v3**: ~$0.20/hour per node
- **10 nodes for 2 hours**: ~$4.00
- **Low-priority pricing**: ~$0.80 (80% savings)

Hybrid Approach

For many workflows, combining both approaches works best:

Recommended Workflow

1. **Development Phase** (Azure VM)
2. Use VM for model development and tuning
3. Test with small sample dataset
4. Finalize configuration and parameters
5. Export optimized project config
6. **Production Phase** (Azure Batch)
7. Deploy validated configuration to Batch
8. Process full dataset in parallel
9. Collect results in Azure Storage
10. Analyze aggregated metrics

Example

```
# Phase 1: Development on VM
./scripts/azure/azure_vm.sh --name crackz-dev --size Standard_D4s_v3
# RDP in, use GUI to develop and test
# Export config: crackz export-config --project demo --out optimized.yaml

# Phase 2: Production on Batch
./scripts/azure/azure_batch.sh \
  --batch-account prod-batch \
  --pool-id production-pool \
```

```
  --node-count 20 \
  --vm-size Standard_D8s_v3

# Upload optimized config and data to storage
# Submit batch jobs with optimized parameters
```

Feature Comparison

Feature	Azure VM	Azure Batch
Deployment Script	`azure_vm.sh`	`azure_batch.sh`
Max Parallelism	1	Unlimited
Docker Support	Manual	Native
Auto-scaling	No	Yes
Task Retry	Manual	Automatic
Job Dependencies	Manual	Built-in
Output Collection	Manual	Automatic
Monitoring	Azure Portal	Batch API + Portal
Storage Integration	Manual mount	Native

Cost Optimization Tips

For Azure VM

1. Use **B-series** for development (burstable, cheaper)
2. Stop VM when not in use (only pay for storage)
3. Use **spot instances** for non-critical work (up to 90% savings)
4. Resize VM based on workload

For Azure Batch

1. Use **low-priority VMs** for fault-tolerant workloads
2. Enable **auto-scaling** to scale down when idle
3. Delete pools when not needed
4. Use smaller VM sizes with higher node counts
5. Batch similar tasks to reduce overhead

Getting Started

First Time Users

1. Start with **Azure VM** to learn Crackz
2. Process small sample dataset
3. Understand the workflow and configuration
4. Graduate to **Azure Batch** for production

Experienced Users

- Use **Azure VM** for:
- Model experimentation
- Configuration tuning
- Small ad-hoc jobs

- GUI-based analysis

- Use **Azure Batch** for:

- Production pipelines
- Large dataset processing
- Automated workflows
- Cost-sensitive operations

Migration Path

Moving from VM to Batch:

```
# 1. Develop on VM
./scripts/azure/azure_vm.sh --name dev-vm

# 2. Test configuration
crackz detect train --project my-project --epochs 50
crackz detect run --project my-project

# 3. Export configuration
crackz export-config --project my-project --out batch-config.yaml

# 4. Upload to Azure Storage
az storage blob upload-batch \
  --account-name mystorage \
  --destination crackz-data \
  --source my-project

# 5. Create Batch pool
./scripts/azure/azure_batch.sh \
  --batch-account mybatch \
  --storage-account mystorage \
  --node-count 10

# 6. Submit batch jobs
az batch job create --id production-job --pool-id crackz-pool
# ... add tasks
```

Support and Documentation

- **Azure VM**: See `scripts/azure/azure_vm.sh --help`
- **Azure Batch**: See Azure Batch Documentation
- **Configuration**: See Configuration Reference
- **Docker Images**: See Docker Usage

FAQ

Q: Can I use GPU with both options?
A: Yes. Use `--size Standard_NC*` for VM, and similar for Batch pool config.

Q: Which is faster for 5000 images?
A: Batch with 20 nodes can be 20x faster than single VM.

Q: Can I use both simultaneously?
A: Yes! Develop on VM, process on Batch.

Q: What about costs for idle time?
A: VM charges for uptime. Batch only charges for task execution time.

Q: Can I access Batch task outputs?
A: Yes, via Azure Storage or download with `az batch task file download`.

Related Resources

- Azure VM Pricing
- Azure Batch Pricing
- Azure Storage Pricing
- Azure Cost Calculator

Model Evaluation

Model Evaluation

Automated evaluation system to measure crack detection model performance with quantitative metrics.

Quick Start

CLI Evaluation

```
# Run evaluation on test dataset
crackz evaluate run --project my_harbor

# Specific metrics
crackz evaluate run --project my_harbor --metrics accuracy,iou

# Export results
crackz evaluate run --project my_harbor --output results.json
crackz evaluate run --project my_harbor --output results.csv
```

Python API

```
from pathlib import Path
from crackz.ai.evaluation import evaluate_model, export_json

# Run evaluation
results = evaluate_model(
    project_path=Path("my_harbor"),
    metrics=["accuracy", "precision", "recall", "iou"]
)

# Display results
print(results)

# Export to JSON
export_json(results, Path("evaluation_results.json"))
```

Metrics

Accuracy

Overall correctness of crack predictions.

```
Accuracy = (TP + TN) / (TP + TN + FP + FN)
```

Precision

Positive predictive value - measures false positive rate.

```
Precision = TP / (TP + FP)
```

Critical for avoiding unnecessary repairs.

Recall

Sensitivity - measures detection rate.

```
Recall = TP / (TP + FN)
```

Critical for safety - must catch all cracks.

F1 Score

Harmonic mean of precision and recall.

```
F1 = 2 * (Precision * Recall) / (Precision + Recall)
```

Balanced metric considering both false positives and false negatives.

IoU (Intersection over Union)

Segmentation quality metric.

```
IoU = Area of Overlap / Area of Union
```

Measures how well the predicted mask matches the ground truth.

Configuration

Threshold

The classification threshold converts model probability outputs to binary predictions:

```
# Default threshold (0.5)
crackz evaluate run --project my_harbor

# Custom threshold
crackz evaluate run --project my_harbor --threshold 0.7
```

Higher threshold = fewer false positives, more false negatives. Lower threshold = more false positives, fewer false negatives.

Metrics Selection

```
# All metrics (default)
crackz evaluate run --project my_harbor

# Specific metrics
crackz evaluate run --project my_harbor --metrics accuracy,precision,recall
crackz evaluate run --project my_harbor --metrics iou  # Only IoU
```

Available metrics: `accuracy`, `precision`, `recall`, `f1`, `iou`

Output Formats

Console Output

```
Evaluation Results (500 images)
==================================================
Accuracy:     92.3%
Precision:    89.1%
Recall:       94.8%
F1 Score:     91.9%
Mean IoU:     0.847
Threshold:    0.5
```

JSON Export

```
{
  "num_images": 500,
  "threshold": 0.5,
  "metrics": {
    "accuracy": 0.923,
    "precision": 0.891,
    "recall": 0.948,
    "f1_score": 0.919,
    "iou": 0.847
  }
}
```

CSV Export

```
Metric,Value
Num Images,500
Threshold,0.5
Accuracy,0.9230
Precision,0.8910
Recall,0.9480
F1 Score,0.9190
IoU,0.8470
```

Test Dataset

Evaluation requires a test dataset with ground truth masks:

```
project/
├── images/
│   └── test/          # Test images
│       ├── img1.png
│       ├── img2.png
│       └── ...
└── masks/
    └── test/          # Ground truth masks
        ├── img1.png
```

```
        ├── img2.png
        └── ...
```

Image and mask filenames must match (sorted order).

Python API Reference

evaluate_model()

```
def evaluate_model(
    project_path: Path,
    test_images: list[Path] | None = None,
    test_masks: list[Path] | None = None,
    metrics: list[str] | None = None,
    threshold: float = 0.5,
) -> EvaluationResults:
    """Evaluate crack detection model on test dataset."""
```

Parameters: - `project_path` : Path to Crackz project - `test_images` : Optional list of test image paths (default: uses project test split) - `test_masks` : Optional list of ground truth mask paths (default: uses project test split) - `metrics` : List of metrics to calculate (default: all metrics) - `threshold` : Classification threshold (default: 0.5)

Returns: - `EvaluationResults` object with calculated metrics

EvaluationResults

```
@dataclass
class EvaluationResults:
    accuracy: float
    precision: float
    recall: float
    f1_score: float
    iou: float
    num_images: int
    threshold: float
```

Export Functions

```
# Export to JSON
export_json(results, Path("results.json"))

# Export to CSV
export_csv(results, Path("results.csv"))
```

Troubleshooting

No test images found

Error:

```
ValueError: No test images found
```

Solution: Ensure your project has test images in `<project>/images/test/` :

```
ls <project>/images/test/
```

Image/mask count mismatch

Error:

```
ValueError: Image/mask count mismatch: 100 vs 98
```

Solution: Ensure every test image has a corresponding mask with the same filename:

```
# Check counts
ls <project>/images/test/ | wc -l
ls <project>/masks/test/ | wc -l

# Find missing masks
diff <(ls <project>/images/test/) <(ls <project>/masks/test/)
```

No trained model

Error:

```
FileNotFoundError: No trained model checkpoint found
```

Solution: Train a model first:

```
crackz detect train --project <project>
```

Best Practices

1. Representative Test Dataset

- Use images from different locations/conditions
- Include various crack types and severities
- Minimum 100 images for reliable metrics
- Never include test images in training data

2. Ground Truth Quality

- Accurate mask annotations are critical
- Use consistent labeling guidelines
- Review masks for errors before evaluation
- Consider inter-annotator agreement

3. Metric Interpretation

- **High recall** (>95%): Critical for safety - catching all cracks
- **High precision** (>90%): Reduces unnecessary inspections
- **High F1** (>92%): Balanced performance
- **High IoU** (>0.80): Accurate crack segmentation

4. Threshold Selection

- Default 0.5 is reasonable starting point
- Increase for fewer false alarms (higher precision)
- Decrease to catch more cracks (higher recall)
- Use ROC curves for optimal threshold selection

Future Enhancements

- Confusion matrix visualization
- ROC curves and optimal threshold selection
- Per-image results export
- Model comparison functionality
- Confidence interval calculation
- Cross-validation support

See Also

- Training Configuration
- Detection Workflow
- API Reference

Advanced Performance

Advanced Performance & Reproducibility

This page describes optional settings for faster training / inference and more reproducible runs.

Overview

Advanced knobs live under `ai.training` and `ai.detection` in the project YAML (or unified `ProjectConfig`). They are all optional and default to safe, CPU-friendly values.

Area	Field	Location	Purpose
Precision	`precision`	`ai.training`	Mixed precision / bfloat16 acceleration
Devices	`devices`	`ai.training`	Number of devices (GPUs) for distributed training
Strategy	`strategy`	`ai.training`	PyTorch Lightning strategy (e.g. `ddp`)
Reproducibility	`seed`	`ai.training`, `ai.detection`	Sets global RNGs (python/numpy/torch/lightning)

Mixed Precision

Mixed precision can significantly improve throughput on modern GPUs.

Supported values (Lightning compatible): - `16` or `16-mixed` – FP16 mixed precision - `bf16-mixed` – bfloat16 mixed precision (Ampere + some CPUs) - `32` – (default if omitted)

Example:

```
ai:
 training:
   precision: 16-mixed
   batch_size: 16
```

CLI run:

```
crackz detect train --project demo-project
```

(Precision is read from project YAML; no extra flag required.)

Multi-GPU / Distributed Training

Enable multi-GPU training by specifying `devices` > 1 and a suitable `strategy` (usually `ddp`).

```
ai:
 training:
   devices: 2
   strategy: ddp
   batch_size: 8
   precision: bf16-mixed
```

Run (assuming 2 visible GPUs):

```
crackz detect train --project demo-project
```

Docker (GPU):

```
docker run --gpus all --rm -v %cd%\project:/home/app crackz-cli:gpu \
 detect train --project /home/app/demo-project
```

Strategies Cheat Sheet

Strategy	Use Case	Notes
`ddp`	Standard multi-GPU	Best default on Linux
`ddp_spawn`	Windows / Jupyter	Slight overhead
`auto`	Let Lightning decide	Conservative choice

Seeding & Reproducibility

Set `seed` in either training or detection sections:

```
ai:
 training:
```

```
    seed: 1234
  detection:
    seed: 1234
```

Effect: - Sets `PYTHONHASHSEED`, python `random`, numpy, torch (CPU + CUDA), and Lightning worker seeds. - Does NOT force deterministic convolution algorithms (can slow performance). Add those manually if needed:

```
import torch
torch.use_deterministic_algorithms(True)
```

Caveats

- GPU kernels may still introduce nondeterminism (atomic ops, etc.).
- Different hardware / driver stacks can still change results slightly.

Example Comprehensive YAML

```
project:
  name: demo-project
  description: Perf tuned
paths:
  project_path: demo-project
  image_path: demo-project/images
  image_mask_path: demo-project/masks
  artifacts_path: demo-project/artifacts
logging:
  level: INFO
  log_dir: logs
ai:
  model:
    encoder: resnet34
    device: cuda
    input_size: [512, 512]
    threshold: 0.5
  training:
    batch_size: 16
    epochs: 20
    learning_rate: 0.0005
    precision: 16-mixed
    devices: 2
    strategy: ddp
    seed: 4242
  detection:
    batch_size: 8
    seed: 4242
```

Workflow Tips

Goal	Tip
Prototype quickly	Keep CPU defaults; small batch, epochs=1, precision unset
Throughput	Increase batch_size gradually; enable 16-mixed precision
Large model memory	Use bf16-mixed (if supported) or reduce batch_size
Reproducible baseline	Fix seed; export config (`crackz export-config`)

Exporting a Tuned Config

After modifying fields inside the project YAML:

```
crackz export-config --project demo-project --out tuned_config.yaml
```

Share `tuned_config.yaml` for reproducible runs.

Detection Phase Seeds

Providing `ai.detection.seed` ensures preprocessing ordering / transforms (if later added) are deterministic between runs.

Troubleshooting

Issue	Suggestion
OOM errors	Lower batch_size or remove mixed precision
Slower than expected	Ensure CUDA visible; confirm precision active in logs
DDP hang	Use `ddp_spawn` on Windows or for notebook contexts

Issue	Suggestion
Seed ineffective	Check for non-deterministic ops; set `torch.use_deterministic_algorithms(True)`

Next

- See Configuration for base fields
- See CLI Cookbook for usage patterns
- Open an issue for additional tuner hooks (gradient accumulation, profiling)

Migration Guide

Project Schema Migration

Crackz versions project YAML files with a simple integer `schema_version` field. When new optional fields are added (e.g. `ai.training.tensorboard`) the schema version increases and a lightweight in-place migration is available.

When Migration Is Needed

You may need migration if: - Your project YAML has no `schema_version` field (legacy v0) - The value is lower than the current `CURRENT_SCHEMA_VERSION` exposed by `crackz.project.migration`

Migration adds any missing defaulted fields and stamps the new `schema_version`. It never removes user data.

Schema Version History

Version	Changes	Notes
0 (legacy)	No `schema_version` field, no `ai.training.tensorboard`	Original projects created before versioning
1	Added `schema_version: 1` and `ai.training.tensorboard: false`	First versioned schema
2	Added `data` section with `mask_threshold: 1`	Binary mask conversion support
3	New artifacts structure + `project.crackz_version` tracking	**Current version** - Improved organization

Version 3 Changes (Current)

Schema version 3 introduces improved artifacts organization and version tracking:

```
schema_version: 3
project:
  name: my project
  description: My crack detection project
  created: '2025-01-15T10:30:00Z'
  crackz_version: '0.50.0'  # NEW: Tracks Crackz app version
```

What gets added: - `project.crackz_version`: Tracks which Crackz version created/last modified the project - New artifacts directory structure (backward compatible): - `checkpoints/` to `models/` (clearer purpose for model weights) - `runtime/` for temporary artifacts (TensorBoard, Lightning logs) - `detections/visualizations/` for PNG detection outputs

Backward compatibility: - Old projects continue working - `checkpoints_path` checks legacy `checkpoints/` directory first - New writes automatically use improved structure (`models/`, `runtime/`, etc.) - **File migration available** - optionally migrate existing files to new structure - Migration only updates schema version and adds `crackz_version` field by default

Version 2 Changes

Schema version 2 introduced the `data` configuration section:

```
data:
  mask_threshold: 1
schema_version: 2
```

Added `data.mask_threshold` for binary mask conversion during image import.

CLI Auto / Assisted Migration

Most CLI commands that load a project (e.g. `detect train`, `detect run`, image imports, exporting configs, pruning checkpoints) now check the schema version and will:

1. Interactive terminal:
2. Prompt: `Project schema v0 < v1. Migrate now? [y/N]:`
3. On Yes: Runs in-place migration with a `.bak` backup.
4. Non-interactive (CI, pipelines):
5. If `CRACKZ_AUTO_MIGRATE=1` is set, migration runs automatically.
6. Otherwise a guidance message is printed and the old schema is used.

Manual Migration Command

Use the explicit command to inspect or apply migrations:

```
crackz migrate --project <path-to-project-dir>
# or
crackz migrate --cfg <path-to-project-yaml>
```

Common flags: - `--dry-run` : Show what would change without writing. - `--no-backup` : Skip writing `<file>.bak` before overwrite. - `--migrate-files` : **NEW** Also migrate artifacts directory structure (checkpoints to models, etc.)

Schema-Only Migration (Default)

Basic migration only updates the YAML schema and adds missing fields:

```
# Using project directory
crackz migrate --project my_project --dry-run
crackz migrate --project my_project

# Using direct YAML file path
crackz migrate --cfg my_project/my_project_project.yaml
```

File Migration (Optional)

NEW in v3: Optionally migrate the actual artifacts directory structure:

```
# Preview what files would be moved
crackz migrate --project my_project --migrate-files --dry-run

# Migrate both schema AND files
crackz migrate --project my_project --migrate-files
```

File migration moves: - `checkpoints/` to `models/` - `training/tensorboard/` to `runtime/tensorboard/` - `training/lightning_logs/` to `runtime/lightning_logs/` - `detections/*.png` to `detections/visualizations/`

Safety features: - Always creates backup of entire artifacts directory before moving files - Interactive confirmation before proceeding - Automatic rollback if migration fails - Detailed progress reporting

Backups

By default, a **timestamped backup** is created before modifying the original project file. Backup files use the format: `{filename}.v{version}.{timestamp}.bak`

Example:

```
myproject_project.v2.20250115_143022.bak
```

This format includes: - **Version number** (`v2`) - Shows which schema version this backup came from - **Timestamp** (`20250115_143022`) - UTC timestamp ensures unique backups per migration - Multiple migrations create multiple backups (never overwrites previous backups)

To revert a migration: 1. Locate the timestamped backup file (e.g., `myproject_project.v2.20250115_143022.bak`) 2. Remove or rename the current YAML file 3. Rename the backup file to remove the `.v2.20250115_143022.bak` suffix

Benefits: - Never lose migration history (each migration creates a unique backup) - Easy to identify which version a backup came from - Can rollback to any previous migration step if needed

Environment Variable Summary

Variable	Effect
`CRACKZ_AUTO_MIGRATE=1`	Auto-migrate without prompt in non-interactive sessions

Programmatic Migration

You can call the library function directly:

```
from pathlib import Path
from crackz.project.migration import migrate_project_file

result = migrate_project_file(Path("my_project"))  # or path/to_yaml
print(result)
# Example result:
# {
#   'status': 'migrated',
#   'from_version': 2,
#   'to_version': 3,
#   'steps': ['2->3: updated to artifacts structure v3 + added crackz_version tracking'],
#   'path': 'my_project/my_project_project.yaml',
#   'backup_path': 'my_project/my_project_project.v2.20250115_143022.bak',
```

```
#   'dry_run': False
# }
```

The result dict contains: - `status`: "migrated", "noop", or "up-to-date" - `from_version`: Original schema version - `to_version`: Final schema version - `steps`: List of migration step descriptions - `path`: Path to migrated file - `backup_path`: Path to backup file (if created) - `dry_run`: Whether this was a dry run

GUI Migration

Automated Migration Detection

When opening a project in the GUI, automatic detection occurs for:

1. **Schema Migration**: Always prompted and performed when project schema is outdated
2. **File Migration**: Detects when artifacts directory structure needs updating

Schema Migration Dialog

When opening a project with an old schema version, the GUI displays a dialog offering migration. A backup is created automatically; declining leaves the project untouched.

File Migration Dialog (NEW)

When artifacts directory structure is outdated, the GUI displays an informational dialog explaining: - Benefits of the new structure (better organization, clearer separation) - Safety measures (backup creation, rollback capability) - User options: "Migrate", "Skip", or "Cancel"

The migration process shows progress and handles any errors gracefully with automatic rollback.

Safety and Idempotence

Running migration on an already up-to-date project is a no-op. Repeated runs after success make no further changes.

Troubleshooting

Symptom	Cause	Resolution
Prompt appears every run	Write failed, read-only filesystem	Ensure write permissions to project directory
Field still missing after migration	Uncommitted manual edits reverted defaults	Re-run migration or add field manually
Many `.bak` files accumulate	Multiple migrations over time	Safe to delete old backups after verifying current project works
Need to rollback migration	Migration caused issues	Rename timestamped `.bak` file to remove suffix and replace current YAML

Architecture

System Architecture

This document describes the current (implemented) architecture of Crackz, focusing on the **actual code paths** in this repository. Previous diagrams referenced future or hypothetical services (artifact stores, model servers, remote registries) that are not yet part of the codebase; those have been removed or clearly marked as future work.

Workspace Layout

Crackz uses a **uv workspace** with two member packages:

```
crackz/                          # Workspace root
├── pyproject.toml               # Workspace config, dev deps, tool settings (no [project])
├── core/                        # crackz-core package
│   ├── pyproject.toml           # Package metadata and dependencies
│   ├── src/crackz/              # Core library (AI, CLI, config, logging, project)
│   └── tests/                   # Core unit + integration tests
├── gui/                         # crackz-gui package (depends on crackz-core)
│   ├── pyproject.toml           # Package metadata, dependency on crackz-core
│   ├── src/crackz_gui/          # PySide6 Qt GUI application
│   └── tests/                   # GUI unit + integration tests
└── tests/                       # Root-level tests (cross-package, legacy)
```

Key points: - `core/` owns all domain logic, CLI, config, and AI pipeline - `gui/` owns the desktop application and depends on `core/` - Both are installed together via `uv sync --dev` from the workspace root - Import paths: `from crackz.ai import ...` (core), `from crackz_gui.qt import ...` (GUI)

Module Layer Architecture

Crackz follows a strict **four-layer architecture** where dependencies only flow downward. Upper layers may depend on lower layers — never the reverse.

```
┌──────────────────────────────────────────────────────────┐
│  Layer 4 — Presentation / Entry Points                   │
│  crackz.cli · crackz_gui · crackz.dashboard              │
├──────────────────────────────────────────────────────────┤
│  Layer 3 — Application Façade                            │
│  crackz.app                                              │
├──────────────────────────────────────────────────────────┤
│  Layer 2 — Domain / Business Logic                       │
│  crackz.ai · crackz.project · crackz.report              │
├──────────────────────────────────────────────────────────┤
│  Layer 1 — Infrastructure / Support                      │
│  crackz.config · crackz.log · crackz.tracing · crackz.util │
└──────────────────────────────────────────────────────────┘
```

```
graph TD
 subgraph L4["Layer 4 — Presentation"]
   CLI["crackz.cli"]
   GUI["crackz_gui"]
   DASH["crackz.dashboard"]
 end
 subgraph L3["Layer 3 — Application Façade"]
   APP["crackz.app"]
 end
 subgraph L2["Layer 2 — Domain"]
   AI["crackz.ai"]
   PROJECT["crackz.project"]
   REPORT["crackz.report"]
 end
 subgraph L1["Layer 1 — Infrastructure"]
   CONFIG["crackz.config"]
   LOG["crackz.log"]
   TRACING["crackz.tracing"]
   UTIL["crackz.util"]
 end

 CLI --> APP
 GUI --> APP
 DASH --> APP
 APP --> AI
 APP --> PROJECT
 APP --> REPORT
 AI --> CONFIG
 AI --> LOG
 AI --> UTIL
 PROJECT --> CONFIG
 PROJECT --> LOG
 PROJECT --> UTIL
 REPORT --> CONFIG
 REPORT --> LOG
 CONFIG --> LOG
 CONFIG --> UTIL
```

Layer Rules

Layer	Module	May import from	May NOT import from
4 – Presentation	`crackz.cli`	L3, L2, L1, `crackz_gui`, `crackz.dashboard`	—
4 – Presentation	`crackz_gui`	L3, L2, L1, `crackz.dashboard`	`crackz.cli`
4 – Presentation	`crackz.dashboard`	L3, L2, L1	`crackz.cli`, `crackz_gui`
3 – Façade	`crackz.app`	L2, L1	L4
2 – Domain	`crackz.ai`	L1	L3, L4
2 – Domain	`crackz.project`	L1	L3, L4
2 – Domain	`crackz.report`	L1	L3, L4
1 – Infrastructure	`crackz.config`	`crackz.project`, `crackz.log`, `crackz.util`	`crackz.ai`, `crackz.report`, L3, L4
1 – Infrastructure	`crackz.log`	—	everything else
1 – Infrastructure	`crackz.tracing`	`crackz.log`	L2, L3, L4
1 – Infrastructure	`crackz.util`	`crackz.log`	everything else

These rules are enforced automatically by **import-linter** (`.importlinter`) and `scripts/check_architecture.py`, both run as part of `uv run task pre_commit` and CI.

High-Level Overview

Crackz consists of two user-facing entry points sharing a common core library: - **CLI** (Typer) – primary automation interface (import images, train, detect, export config) - **GUI** (PySide6) – exploratory desktop interface (project browsing, import, training & detection triggers, previews)

Core subsystems: - **Project / Configuration**: Pydantic models; YAML + env + defaults merged via `pydantic-settings` - **Image Import & Layout**: Normalizes images into split directories (`raw/ train/ val/ test/`) and masks - **Dataset Layer**: `CrackzDataset` (Torch `Dataset`) + DataLoader factory - **Training & Detection Pipelines**: PyTorch + PyTorch Lightning orchestration (training, validation, testing, inference) - **Performance Utilities**: Global seed, Trainer override extraction - **Logging**: Centralized through `loguru` with structured formatting - **Artifacts & Outputs**: Project filesystem (`artifacts/models/`, `artifacts/runtime/`, metrics JSON, detection PNGs) plus Postgres metadata (via `CRACKZ_DATABASE_URL`)

No network services, model servers, remote registries, or cloud object storage are currently part of runtime behavior.

Local Persistence

> See the **Diagrams** page for draw.io ER and class diagrams. For full database setup, configuration, and operational runbooks see **Database setup**. The decision is recorded in **ADR-0023**.

Crackz uses a single **Postgres** database as the system of record for all projects, models, detection history, training metrics, and defect embeddings. The filesystem continues to hold model weights, detection PNGs, and metric JSON files.

Extension required: pgvector ≥ 0.8 (vector similarity search with iterative scan). **Extension recommended:** PostGIS (geospatial queries on GPS-tagged detections; Crackz falls back to Python Haversine when PostGIS is absent).

Configure the database with a single environment variable, then run migrations:

```
export CRACKZ_DATABASE_URL="postgresql+psycopg://crackz:crackz@localhost:5432/crackz"
crackz db init          # or: alembic upgrade head
```

To bring forward history from legacy per-project SQLite files:

```
crackz db migrate-sqlite   # id-remapped by root_path; idempotent, warn-and-skip
```

Both the GUI welcome-screen aggregates and the core `ModelRegistry` now read this single database. `ModelRegistry`'s public surface is preserved; it is re-implemented on `crackz.db` repository classes internally.

Entity relationship

```
erDiagram
    projects {
        int id PK
        string name
        string root_path
        string task_mode
        string category_keys
        timestamp created_at
    }
    project_tags {
        int id PK
        int project_id FK
        string tag
    }
    models {
        int id PK
```

```
    int project_id FK
    string name
    string encoder
    string checkpoint_name
    timestamp created_at
}
detection_runs {
    int id PK
    int project_id FK
    int model_id FK
    timestamp run_at
    int image count
    int crack_count
}
detection_results {
    int id PK
    int project_id FK
    int detection_run_id FK
    string image
    float confidence
    smallint has defect
    float gps_latitude
    float gps_longitude
    geometry geom
}
training_epochs {
    int id PK
    int project id FK
    int model_id FK
    string run_id
    int epoch
    float val_iou
}
defect_embeddings {
    int id PK
    int project_id FK
    int detection run id FK
    int detection_result_id FK
    int crop_index
    string model_name
    string embedding_version
    int dim
    vector embedding
    geometry geom
    timestamp created_at
}

projects ||--o{ project_tags : "has"
projects ||--o{ models : "owns"
projects ||--o{ detection_runs : "owns"
projects ||--o{ detection_results : "owns"
projects ||--o{ training_epochs : "owns"
projects ||--o{ defect embeddings : "owns"
detection_runs ||--o{ detection_results : "contains"
detection_results ||--o{ defect_embeddings : "has"
```

`embedding vector(768)` — DINOv2 defect embedding stored with pgvector; queried via an HNSW cosine index. `geom geometry(POINT, 4326)` — PostGIS point for GPS-tagged detections and embedding crops; requires the PostGIS extension. All child tables cascade-delete from `projects` (`ON DELETE CASCADE`). Migrations are managed by Alembic: revision `0001` = the six core tables + PostGIS; revision `0002` = `defect_embeddings` + pgvector.

Open Architecture & Model Flexibility

Crackz is built with an **open architecture** that supports multiple model types and use cases:

Segmentation Task Modes

Crackz treats binary segmentation and multiclass semantic segmentation as separate task modes, even when a project has only one foreground category.

Task mode	Model output	Prediction activation	Training target	Primary use
`binary`	1 foreground channel	Sigmoid + configurable threshold	Float mask where any non-zero pixel is foreground	Sparse crack or defect-vs-background detection
`multiclass`	Background + one class per category	Softmax + argmax	Integer indexed mask ids matching the annotation schema	Projects where the defect category of each pixel matters

This is an intentional architecture decision, not just a `num_classes` detail. Binary crack detection is usually extremely sparse, so the one-channel sigmoid path, threshold tuning, and Dice/Focal loss remain better suited than forcing a background-plus-one foreground softmax model. Multiclass mode is used when category identity is part of the output contract.

Mask ingestion follows the same boundary:

- Binary projects threshold masks to black/white semantics. Imported or loaded masks may contain `0`, `1`, `255`, or other grayscale foreground values; all non-zero pixels collapse to the single foreground class for training. When a dataset uses values outside the legacy `{0, 1, 255}` set, Crackz logs one INFO-level breadcrumb per dataset so accidental multiclass-mask reuse is visible without blocking legitimate grayscale foreground masks.
- Multiclass projects preserve indexed label ids and validate them against the project annotation schema. These masks must not be converted to black/white, because the pixel value is the category id.

When a project has multiple annotation categories but `ai.model.task_mode` is set to `binary`, all foreground categories intentionally collapse into one defect-vs-none model. When `task_mode` is omitted, Crackz derives the mode from the annotation schema: one foreground category resolves to `binary`, multiple categories resolve to `multiclass`.

Default Configuration: Segmentation - Binary segmentation (pixel-level classification): crack/no-crack, deviation/no-deviation - Uses U-Net architecture with configurable encoder backbones (ResNet, EfficientNet, MobileNet, etc.) - Suitable for: structural crack detection, surface defect analysis, damage mapping

Alternative Model Types While segmentation is the default, the architecture supports: - **Classification models**: Whole-image categorization (e.g., "has crack" vs "no crack") - **Multi-class segmentation**: Different defect types (cracks, corrosion, spalling) - **Custom architectures**: Bring your own PyTorch Lightning model

Model Acquisition Workflow End users have two primary paths:

1. **Train Your Own Model**
2. Import labeled training data (images + masks)
3. Configure model architecture and training parameters
4. Run `crackz detect train` to create a custom checkpoint

5. Ideal for: domain-specific data, unique defect types, custom requirements

6. **Use Pre-Trained Models**

7. Acquire ready-made checkpoints from model providers or partners
8. Place checkpoint in project's `artifacts/models/` directory
9. Run `crackz detect run` with existing checkpoint
10. Ideal for: quick deployment, standard use cases, proof-of-concept

Encoder Flexibility The segmentation model supports any encoder from `segmentation-models-pytorch`, including: - ResNet family (resnet18, resnet34, resnet50, resnet101) - EfficientNet (efficientnet-b0 through efficientnet-b7) - MobileNet (mobilenet_v2) for edge deployment - DenseNet, VGG, and many others

Configure via `project.yaml`:

```
ai:
  model:
    backbone: segformer-b2   # or segformer-b0/b4, efficientnet-b3, resnet34, etc.
annotation:
  categories:
    - id: 1
      key: crack
      name: Crack
      color: "#ef4444"
```

Common Use Cases & Model Examples

1. Harbor Infrastructure Inspection (Binary Segmentation) - **Task**: Detect and map cracks in concrete harbor walls - **Model**: SegFormer-B2 (default) or U-Net with EfficientNet-B3 - **Annotation**: Single `crack` category → binary task mode - **Input**: 1024×1024 RGB images - **Output**: Binary segmentation mask highlighting crack pixels - **Workflow**: `crackz init --name harbor --annotation-template crack-only` → train → detect

2. Harbor Comprehensive Survey (Multi-Class Segmentation) - **Task**: Identify multiple defect types (cracks, spalling, corrosion, joint damage) - **Model**: SegFormer-B2 with 5 output channels (background + 4 categories) - **Annotation**: `crack`, `spall`, `corrosion`, `joint_damage` categories → multiclass task mode - **Input**: 1024×1024 RGB images - **Output**: Indexed mask (0=background, 1=crack, 2=spall, 3=corrosion, 4=joint_damage) - **Workflow**: `crackz init --name harbor --annotation-template harbor-defects` → train → detect

3. Bridge Surface Analysis (Multi-Class Segmentation) - **Task**: Identify different defect types (cracks, spalling, corrosion) - **Model**: SegFormer-B4 (higher accuracy GPU option) - **Annotation**: 3 categories → multiclass task mode, 4 model output classes - **Input**: High-resolution bridge deck photos - **Output**: Multi-class indexed mask per category - **Workflow**: Collect labeled dataset → configure harbor-defects template → train on GPU

4. Deviation Detection (Binary Segmentation) - **Task**: Identify surface irregularities or deviations from standard - **Model**: SegFormer-B0 or U-Net with MobileNetV2 (for edge deployment) - **Input**: Normalized surface scans - **Output**: Binary deviation mask - **Workflow**: Train on deviation examples → Deploy on mobile/edge devices

5. Custom Domain Application - **Task**: Adapt to specific industry (e.g., aerospace panel inspection) - **Model**: Start from SegFormer-B2 with a custom annotation schema - **Input**: Domain-specific imagery (composite materials, metal surfaces) - **Output**: Custom category masks - **Workflow**: Define custom categories via `annotation.categories` → collect labeled data → train

---

C4 Level 1 (Container View)

```
C4Container
  title Crackz — Container View (Implemented)
  Person(user, "User")
  System_Boundary(crackz, "Crackz Local System") {
```

```
    Container(cli, "CLI", "Typer", "Command parsing & dispatch")
    Container(gui, "GUI", "PySide6 (Qt 6)", "Desktop inspection & invocation")
    Container(core, "Core Library", "Python Packages", "project, ai, config, logging")
    Container(fs, "Local Filesystem", "images/ + artifacts/", "Images, masks, checkpoints, metrics")
    ContainerDb(db, "Crackz DB", "Postgres + pgvector + PostGIS", "Projects, models, detections, training epochs, defect embeddings")
  }
  Rel(user, cli, "Runs commands")
  Rel(user, gui, "Interacts (point & click)")
  Rel(cli, core, "Invokes APIs")
  Rel(gui, core, "Invokes APIs")
  Rel(core, fs, "read/write images, masks, checkpoints, metrics")
  Rel(core, db, "read/write all project metadata")
  Rel(gui, db, "read project index + welcome aggregates")
```

C4 Context (Local Execution)

```
C4Context
  title Crackz — System Context (Local)
  Person(user, "User", "Engineer / Operator")
  System(local, "Crackz Application", "CLI + GUI + Core Library")
  System_Ext(osfs, "Host Filesystem", "Images + Artifacts")
  Rel(user, local, "Invoke CLI / Launch GUI")
  Rel(local, osfs, "read / write")
```

Component Breakdown (Core Library)

```
C4Component
  title Core Library Components (Implemented)
  Container(core, "core", "Python")
  Component(project, "Project & Paths", "pydantic", "Project metadata & path helpers")
  Component(config, "Configuration", "pydantic-settings", "Merge defaults / env / YAML")
  Component(importer, "Image Importer", "Pillow", "Resize, quality, organize splits & masks")
  Component(dataset, "Dataset Layer", "Torch Dataset", "CrackzDataset + DataLoader factory")
  Component(train, "Training Orchestrator", "PyTorch Lightning", "Fit/validate/test + callbacks")
  Component(detect, "Detection Pipeline", "PyTorch", "Batch inference over dataset splits (raw/test)")
  Component(metrics, "Metrics & Checkpoints", "Lightning + JSON", "Save val/test metrics & best.ckpt/last.ckpt")
  Component(perf, "Performance Utilities", "Seeding", "Seed + trainer overrides")
  Component(logging, "Logging", "loguru", "Structured log sink & setup")
  Component(evaluation, "Evaluation Framework", "Metrics", "IoU, precision, recall, F1, accuracy")
  Component(reporter, "Report Generator", "python-docx", "Branded Word reports")
  Component(viz, "Visualization", "Matplotlib", "Detection overlays (3-panel)")
  Component(pipeline_orch, "Pipeline Orchestrator", "Workflow", "Train→detect + cancellation")
  Component(device_mgmt, "Device Management", "Abstraction", "CPU/GPU/MPS selection")
  Component(severity, "Severity Classifier", "Categorization", "3-tier crack severity")
  Component(suggester, "Mask Suggester", "Semi-automation", "Model-assisted annotation")
  Component(perf est, "Performance Estimator", "RuntimeTracker", "Time estimates")
  Component(async_cb, "Async Callbacks", "Janus", "Sync/async bridging")
  Component(tracing_mod, "Tracing", "OpenTelemetry", "AI API observability")
  Component(ai_chat_svc, "AI Chat", "Anthropic", "User assistance")
  Rel(importer, project, "Uses path layout")
  Rel(dataset, project, "Resolves split directories")
  Rel(train, dataset, "Consumes (train/val/test) loaders")
  Rel(detect, dataset, "Consumes detection loader (raw or test)")
  Rel(train, metrics, "Saves checkpoints + metrics")
  Rel(detect, metrics, "Writes detection results JSON/PNGs")
  Rel(project, config, "Initializes from effective settings")
  Rel(train, perf, "Reads overrides / seed")
  Rel(detect, perf, "Sets seed (optional)")
  Rel(dataset, logging, "Reports warnings (missing masks / size mismatch)")
  Rel(detect, evaluation, "Evaluates performance")
  Rel(evaluation, metrics, "Calculates metrics")
  Rel(detect, viz, "Visualizes results")
  Rel(detect, reporter, "Generates reports")
  Rel(train, pipeline_orch, "Orchestrated by")
  Rel(detect, pipeline_orch, "Orchestrated by")
  Rel(train, device_mgmt, "Selects device")
  Rel(detect, device_mgmt, "Selects device")
  Rel(detect, severity, "Categorizes cracks")
  Rel(suggester, detect, "Uses model")
  Rel(train, perf_est, "Tracks runtime")
  Rel(detect, perf_est, "Tracks runtime")
  Rel(train, async_cb, "Reports progress")
  Rel(detect, async_cb, "Reports progress")
  Rel(ai_chat_svc, tracing_mod, "Optionally traced")
```

Evaluation & Reporting Flow

Detection results flow through evaluation and reporting pipelines:

```
flowchart LR
  A["Test Dataset"] --> B["Evaluator"]
  B --> C["Metrics Calculator"]
  C --> D["Evaluation Results"]
  D --> E["JSON/CSV Reporter"]
  D --> F["Report Generator"]
  F --> G["Branded Word Document"]
  I["Detection Results"] --> J["Visualization"]
```

```
 J --> K["Detection Overlays"]
 I --> F
```

Async Callback Architecture

Training/detection progress flows from sync threads to async GUI:

```
flowchart TB
 A["Training Thread (sync)"] --> B["CallbackDispatcher.sync_q"]
 B --> C["Janus Queue"]
 C --> D["CallbackDispatcher.async_q"]
 D --> E["GUI Event Loop (async)"]
 E --> F["UI Update"]
```

Key Features: Thread-safe sync/async bridging, backpressure handling (drops oldest on full queue), non-blocking updates.

Data Flow (Operational)

```
flowchart LR
 A["Import Command / GUI Action"] --> B["Image Importer"]
 B --> C["Split Directories (images/*, masks/*)"]
 C --> D["Train Command"]
 D --> E["Training Orchestrator<br/>(Trainer.fit)"]
 E --> F["Checkpoints &amp;amp; Metrics<br/>(best.ckpt / last.ckpt / JSON)"]
 F --> G["Detect Command"]
 G --> H["Inference Loop<br/>(sigmoid → threshold)"]
 H --> I["Annotated Output<br/>(PNGs + detection_results.json)"]
```

Sequence (Image Import)

```
sequenceDiagram
 participant User
 participant CLI as CLI / GUI
 participant Project
 participant Import as ImageImporter
 participant FS as Filesystem
 User->>CLI: import-images --src ./samples --type train
 CLI->>Project: load_project(path|yaml)
 CLI->>Import: iterate source images
 Import->>FS: resize + normalize + write split file
 Import->>FS: write mask (if provided)
 Import-->>CLI: progress callback
 CLI-->>User: summary (count, skipped)
```

Sequence (Training & Detection)

```
sequenceDiagram
 participant User
 participant CLI as CLI / GUI
 participant Project
 participant DS as CrackzDataset
 participant Trainer as Lightning Trainer
 participant Model
 participant FS as Filesystem
 User->>CLI: detect train --project P
 CLI->>Project: load_project()
 CLI->>DS: build train/val/test datasets
 DS-->>CLI: DataLoaders
 CLI->>Trainer: fit(model, loaders)
 Trainer->>FS: save best.ckpt / last.ckpt
 Trainer-->>CLI: metrics (val/test)
 CLI-->>User: training summary
 User->>CLI: detect run --project P
 CLI->>Project: load project()
 CLI->>Model: load checkpoint (best or last)
 CLI->>DS: build raw dataset (or test)
 DS-->>CLI: DataLoader
 CLI->>Model: forward (batched)
 Model-->>CLI: probabilities
 CLI->>CLI: threshold -> mask
 CLI->>FS: write annotated PNG + JSON metrics
 CLI-->>User: detection summary
```

Configuration Resolution

```
flowchart TD
 Defaults[Built-in Defaults] --> Merge
```

```
YAML[Project YAML] --> Merge
Env[Environment] --> Merge
Merge --> Effective[Effective Settings]
```

Model Selection — Mental Model

Crackz separates *project defaults* (Settings) from the *catalog of trained models* (Model Manager). Each concept has exactly one home:

Concept	Role	Where it lives
Project default backbone	Suggested backbone for new training runs	`ai.model.backbone` in `project.yaml`, editable in **Settings → AI · Model**
Active Crackz model	Trained model used by detection when the user doesn't pick another	`ai.model.active_checkpoint`, set via **Model Manager → Set as Default**
Crackz model catalog	All trained checkpoints + metadata (backbone, num_classes, loss, created_at, notes)	Project SQLite DB at `artifacts/project.db`, browsed in **Model Manager**
Per-run backbone choice	Backbone used for a specific training run (defaults to project default)	Chosen in the *Start Training* dialog; not set in Settings
Per-run model choice	Crackz model used for a specific detection run (defaults to active model)	Chosen in the *Model Selector* dialog (filtered by current project backbone by default)

Detection fast-path: if `active_checkpoint` is set and its backbone matches `ai.model.backbone`, **Run Detection** runs immediately. **Run Detection… (pick model)** always opens the selector for ad-hoc overrides.

Training fast-path: the *Start Training* dialog pre-selects the project default backbone; checking *Save as new project default* persists the choice to `project.yaml`. When resuming from a checkpoint, the checkpoint's backbone wins (an architecture mismatch warning is shown).

Deployment (Local + Optional Docker)

```
flowchart LR
 User([User]) --> CLI["CLI (Typer)"]
 User --> GUI["GUI (PySide6)"]
 CLI --> CORE["Core Library"]
 GUI --> CORE
 CORE --> FS["Filesystem<br/>images / artifacts/models / artifacts/runtime"]
 CORE -.optional.-> GPU[("GPU")]

 subgraph Optional_Docker [Optional Docker]
   IMG["Container Image<br/>(Python 3.13 + deps)"]
 end

 %% Show that container can host CLI/GUI
 IMG --> CLI
 IMG --> GUI
```

Responsibilities Summary

Concern	Module(s)	Notes
Project metadata & paths	`crackz.project.project`	Pydantic models & path helpers
Config merge	`pydantic-settings` integration	YAML + env + defaults
Import / resize	`project.import_images` / CLI wrapper	Resizes, re-encodes, organizes splits & masks
Dataset / batching	`crackz.ai.data`	CrackzDataset + mask fallback + collate
Training orchestration	`crackz.ai.detection.train`	Lightning Trainer + callbacks
Inference / detection	`crackz.ai.detection.detect`	Loads model (checkpoint) + sigmoid + threshold
Model definition	`crackz.ai.model`	Segmentation architecture wrapper
Metrics & checkpoints	`crackz.ai.checkpoints` (callbacks, selection)	best.ckpt / last.ckpt resolution
Device management	`crackz.ai.device`	Device detection and accelerator utilities
General utilities	`crackz.util`	Filesystem operations and cross-cutting concerns
Performance utilities	`crackz.ai.performance`	Global RNG (Generator), seed set, precision/devices
Logging	`crackz.log.crackz_logger`	Central logger setup
GUI state & browsing	`crackz_gui.*`	Project tree, previews, recent projects
Evaluation & metrics	`crackz.ai.evaluation`	IoU, precision, recall, F1, accuracy
Report generation	`crackz.report.generator`	Branded Word documents
Visualization	`crackz.ai.visualization`	Matplotlib 3-panel overlays
Pipeline orchestration	`crackz.ai.pipeline`	Train→detect workflows

Concern	Module(s)	Notes
Cancellation	`crackz.ai.pipeline.cancellation`	Thread-safe cancellation tokens
Severity classification	`crackz.ai.detection.severity`	3-tier categorization
Mask suggestion	`crackz.ai.mask_suggester`	Model-assisted annotation
Performance estimation	`crackz.ai.performance.runtime_estimation`	Runtime tracking
Async callbacks	`crackz.ai.core.async_callbacks`	Janus sync/async bridge
Tracing	`crackz.tracing`	Optional OpenTelemetry
AI chat service	`crackz_gui.services.ai_chat`	Claude API integration

Dataset Splits (Recap)

See the **CLI Cookbook – Understanding Dataset Splits** section for detailed rationale. In brief: - `train/` – learns weights - `val/` – selects checkpoint & thresholds - `test/` – final metric (optional in early prototypes) - `raw/` – inference-only imagery (no labels)

Future (Not Yet Implemented)

Planned Capability	Status	Notes
Remote artifact store (S3 / GCS)	Planned	Abstract storage layer needed
Model registry integration	Planned	Could adopt MLflow-compatible API
Hyperparameter tuning service	Planned	Integrate with Optuna / Ray Tune
Model serving API	Planned	TorchServe / FastAPI wrapper
Advanced augmentation pipeline	Planned	Albumentations / custom transforms
Active learning loop	Planned	Feed predictions back into labeling pipeline

---

Technology Stack

- **Language**: Python 3.13+
- **DL Framework**: PyTorch, PyTorch Lightning, segmentation-models-pytorch, torchmetrics, torchvision
- **Data & Imaging**: Pillow, (optionally HF datasets in the future)
- **CLI**: Typer
- **GUI**: PySide6 (Qt 6)
- **Config**: pydantic, pydantic-settings
- **Logging**: loguru
- **Progress / TUI**: rich
- **Packaging / Build**: uv (workspace mode), hatchling, PyInstaller (GUI binary optional)
- **Testing**: pytest (+ coverage, html, benchmarks markers)
- **Project Layout**: uv workspace with `core/` and `gui/` members

---

Last updated: synchronized with repository implementation on commit timeline of current editing session.

Architecture Guidelines

Architecture Guidelines

> **Developer-focused architecture patterns and principles for Crackz**
>
> This document complements architecture.md (system architecture) and internal/adr/02-decisions.md (ADR decisions) with practical guidelines for developers.

Last Updated: 2025-11-13

Table of Contents

- Overview
- Architectural Principles
- Module Organization
- Design Patterns
- Code Organization Patterns
- Configuration Architecture
- Error Handling Architecture
- Testing Architecture
- Performance Architecture
- Security Architecture
- Extending the Architecture

Overview

Crackz follows a **modular, layered architecture** designed for: - **Enterprise deployment**: Docker containers, air-gapped environments, GPU support - **Offline-first operation**: No required internet connectivity - **Commercial reliability**: Stable APIs, backward compatibility, data privacy - **Developer productivity**: Type safety, fast tests, clear patterns

Core Architectural Layers

```
┌─────────────────────────────────────────┐
│  User Interfaces (CLI, GUI)             │  ← Entry points
├─────────────────────────────────────────┤
│  Business Logic (Commands, Operations)  │  ← Use cases
├─────────────────────────────────────────┤
│  Core Domain (Project, AI, Config)      │  ← Domain models
├─────────────────────────────────────────┤
│  Infrastructure (Logging, Storage)      │  ← Technical concerns
└─────────────────────────────────────────┘
```

Key Insight: Dependencies flow downward. Upper layers depend on lower layers, never the reverse.

Architectural Principles

1. Separation of Concerns (SoC)

What: Each module has a single, well-defined responsibility.

Why: Reduces coupling, improves testability, enables independent changes.

Examples:

```
# ✅ GOOD: Clear separation
# core/src/crackz/cli/detection_cli.py - CLI commands only
# core/src/crackz/ai/detection.py - Detection logic only
# core/src/crackz/project/structure.py - Project layout only

# ❌ BAD: Mixed concerns
# core/src/crackz/everything.py - CLI, detection, project management all in one file
```

In Practice: - CLI modules handle user interaction and input validation - AI modules handle ML operations and model management - Config modules handle settings and validation - Project modules handle directory structure and metadata - GUI modules handle user interface only

2. Dependency Inversion Principle (DIP)

What: Depend on abstractions (interfaces, protocols), not concrete implementations.

Why: Enables testing with mocks, allows swapping implementations.

Examples:

```
# ✅ GOOD: Depend on protocol
from typing import Protocol

class ModelProtocol(Protocol):
    def predict(self, image: torch.Tensor) -> torch.Tensor: ...

def run_detection(model: ModelProtocol, image: torch.Tensor) -> dict:
    """Works with any model implementing the protocol."""
    return {"mask": model.predict(image)}

# ❌ BAD: Depend on concrete class
def run_detection(model: SegmentationModel, image: torch.Tensor) -> dict:
    """Tightly coupled to SegmentationModel."""
    return {"mask": model.predict(image)}
```

In Practice: - Use Pydantic models for configuration (abstracts YAML/dict/env) - Use protocols for AI models (enables mock models in tests) - Use pathlib.Path for filesystem (abstracts OS differences)

3. Single Responsibility Principle (SRP)

What: A class/module should have one reason to change.

Why: Changes in one area don't cascade to unrelated code.

Examples:

```
# ✅ GOOD: Single responsibility
class ProjectLoader:
    """Loads project configuration from disk."""
    def load(self, path: Path) -> ProjectConfig: ...

class ProjectValidator:
    """Validates project structure and configuration."""
    def validate(self, project: ProjectConfig) -> list[str]: ...

# ❌ BAD: Multiple responsibilities
class ProjectManager:
    """Loads, validates, trains models, exports results."""
    def load(self, path: Path) -> ProjectConfig: ...
    def validate(self, project: ProjectConfig) -> list[str]: ...
    def train_model(self, project: ProjectConfig) -> None: ...
    def export_results(self, project: ProjectConfig) -> None: ...
```

4. Open/Closed Principle (OCP)

What: Open for extension, closed for modification.

Why: Add new features without changing existing code.

Examples:

```
# ✅ GOOD: Extensible via configuration
def create_model(config: ModelConfig) -> nn.Module:
    """Factory method - add new encoders via config."""
    if config.encoder in SUPPORTED_ENCODERS:
        return create_segmentation_model(config)
    raise ValueError(f"Unsupported encoder: {config.encoder}")

# ❌ BAD: Requires code changes for new models
def create_model(encoder_name: str) -> nn.Module:
    if encoder_name == "resnet34":
        return ResNet34Model()
    elif encoder_name == "efficientnet":
        return EfficientNetModel()
    # Must modify this function for each new encoder!
```

5. Explicit is Better Than Implicit

What: Make dependencies, assumptions, and behavior explicit.

Why: Code is self-documenting, reduces surprises.

Examples:

```
# ✅ GOOD: Explicit parameters
def train_model(
    project_path: Path,
    epochs: int,
    batch_size: int,
    learning_rate: float,
) -> dict[str, float]:
    """All parameters explicit."""
    pass
```

```
# ❌ BAD: Implicit global state
GLOBAL_CONFIG = {}  # Modified elsewhere

def train_model():
    """What parameters? Where do they come from?"""
    epochs = GLOBAL_CONFIG["epochs"]  # Implicit dependency
```

Module Organization

Directory Structure Philosophy

```
core/src/crackz/          # Core library (crackz-core package)
├── ai/                   # AI/ML pipeline (domain logic)
│   ├── data.py           # Dataset, data loading
│   ├── models.py         # Model definitions
│   ├── training.py       # Training pipeline
│   └── detection.py      # Inference pipeline
├── cli/                  # CLI interface (entry points)
│   ├── options.py        # Shared CLI options
│   ├── exceptions.py     # CLI error handling
│   ├── project_cli.py
│   └── detection cli.py
├── config/               # Configuration (domain models)
│   ├── project.py        # ProjectConfig
│   ├── ai.py             # AIConfig
│   └── logging.py        # LogConfig
├── log/                  # Logging (infrastructure)
│   └── setup.py          # Loguru configuration
└── project/              # Project management (domain logic)
    └── structure.py      # Directory layout

gui/src/crackz_gui/       # GUI package (crackz-gui, depends on crackz-core)
├── qt/                   # PySide6 Qt application
│   ├── app_state.py      # QtAppState (reactive state)
│   └── async_bridge.py   # TaskSignals, run_in_background
├── services/             # External service integrations
├── state/                # State management
└── util/                 # GUI utilities
```

Module Boundaries

Rule: Modules should have **low coupling** and **high cohesion**.

✅ Allowed Dependencies

```
CLI (core)  ──→  Business Logic  ──→  Domain Models  ──→  Infrastructure
GUI (gui)   ──→  (ai/, project/)     (config/)           (log/)

Examples:
- core: cli/detection_cli.py → ai/detection.py → config/ai.py → log/setup.py ✅
- gui:  crackz_gui/qt/panels/train.py → crackz.ai.training → crackz.config ✅
- gui:  crackz-gui depends on crackz-core (declared in gui/pyproject.toml) ✅
```

❌ Forbidden Dependencies

```
Domain ──X→ GUI/CLI     # Domain shouldn't know about UI
Infrastructure ──X→ Domain  # Infrastructure shouldn't depend on business logic
GUI ──X→ CLI            # GUI shouldn't import CLI modules

Examples:
- ai/detection.py → cli/options.py ❌  # Domain shouldn't import from CLI
- log/setup.py → config/project.py ❌  # Infrastructure shouldn't import domain
- crackz_gui → crackz.cli ❌  # GUI should use core APIs, not CLI
```

How to Fix Violations:

```
# ❌ BAD: Domain imports from CLI
from crackz.cli.options import PROJECT_PATH_OPTION
def train_model(project_path: Path = PROJECT_PATH_OPTION): ...

# ✅ GOOD: CLI calls domain with explicit parameters
from crackz.cli.options import PROJECT_PATH_OPTION
from crackz.ai.training import train_model

def train_cli(project_path: Path = PROJECT_PATH_OPTION):
    train_model(project_path)  # Domain function has no CLI dependency
```

Design Patterns

1. Factory Pattern (Model Creation)

Where: `core/src/crackz/ai/models.py`

Why: Centralize model instantiation logic, support multiple model types.

Example:

```
def create_segmentation_model(config: ModelConfig) -> nn.Module:
    """Factory for creating segmentation models."""
    import segmentation_models_pytorch as smp

    return smp.Unet(
        encoder_name=config.encoder,
        encoder_weights=config.encoder_weights,
        in_channels=config.in_channels,
        classes=config.num_classes,
    )
```

Benefits: - Single place to modify model creation - Easy to add new model types - Testable with config variations

2. Strategy Pattern (Detection Backends)

Where: `core/src/crackz/ai/detection.py`

Why: Support different detection strategies (single image, batch, distributed).

Example:

```
class DetectionStrategy(Protocol):
    def detect(self, images: list[Path]) -> dict[Path, np.ndarray]: ...

class SingleImageDetection:
    def detect(self, images: list[Path]) -> dict[Path, np.ndarray]:
        """Process one at a time."""
        return {img: self._detect_one(img) for img in images}

class BatchDetection:
    def detect(self, images: list[Path]) -> dict[Path, np.ndarray]:
        """Process in batches for efficiency."""
        results = {}
        for batch in chunk_list(images, self.batch_size):
            results.update(self._detect_batch(batch))
        return results
```

3. Builder Pattern (Configuration Assembly)

Where: `core/src/crackz/config/project.py`

Why: Complex configuration with defaults, overrides, validation.

Example:

```
class ProjectConfig(BaseModel):
    """Pydantic acts as builder with validation."""
    name: str
    project_path: Path
    training: TrainingConfig = Field(default_factory=TrainingConfig)
    logging: LogConfig = Field(default_factory=LogConfig)

    @classmethod
    def load(cls, path: Path) -> ProjectConfig:
        """Build from YAML with defaults."""
        with open(path) as f:
            data = yaml.safe_load(f)
        return cls(**data)  # Validation + defaults applied
```

4. Command Pattern (CLI Commands)

Where: `core/src/crackz/cli/*.py`

Why: Encapsulate operations as objects, enable undo/logging/queuing.

Example:

```
@app.command(name="train")
def train_command(
    project_path: Path = PROJECT_PATH_OPTION,
    epochs: int | None = EPOCHS_OPTION,
) -> None:
    """Each CLI command is a self-contained operation."""
    try:
        # Load configuration
        config = ProjectConfig.load(project_path / f"{project_path.name}_project.yaml")

        # Override with CLI args
        if epochs:
            config.training.epochs = epochs

        # Execute
        train_model(config)
```

```
        typer.secho("✅ Training completed", fg=typer.colors.GREEN)
    except Exception as e:
        cli_failure(e, "Training")
```

5. Repository Pattern (Project Access)

Where: `core/src/crackz/project/structure.py`

Why: Abstract filesystem access, enable testing with in-memory projects.

Example:

```
class ProjectRepository:
    """Abstracts project storage."""

    def load(self, path: Path) -> ProjectConfig:
        """Load project from filesystem."""
        return ProjectConfig.load(path / f"{path.name}_project.yaml")

    def save(self, project: ProjectConfig) -> None:
        """Save project to filesystem."""
        project.save(project.project_path / f"{project.name}_project.yaml")

    def exists(self, path: Path) -> bool:
        """Check if project exists."""
        return (path / f"{path.name}_project.yaml").exists()
```

6. Persistence Boundary Pattern

Where: - User-level project index: `gui/src/crackz_gui/data/projects_db.py` - Project-level operational DB: `core/src/crackz/ai/model/registry.py`

Why: Keep user-specific GUI state separate from project-owned inspection history.

Rules: - Store the all-projects list in the platform user config directory as `crackz.db`. - Store project-level metadata in `<project>/artifacts/project.db`. - Treat `<project>/artifacts/project.db` as authoritative for model records, detection run summaries, and training epoch metrics. - Treat the user-level `crackz.db` as an index and cache: paths, pins, notes, tags, and welcome-screen aggregate values. - Do not write model, detection, or training history into the user-level DB. - Do not read artifact-level `crackz.db` in steady state; migrate it into `artifacts/project.db`.

Postgres is owned by the `crackz.db` package. Only `crackz.db` imports `sqlalchemy`, `psycopg`, `psycopg2`, `asyncpg` or `alembic` (import-linter contract 13, mirrored by `DB_OWNER` in `scripts/check_architecture.py`). Other layers call a repository, catch `crackz.db.errors.DatabaseError`, and apply migrations with `crackz.db.migrate.upgrade_to_head`. There are no exempted files; a new leak fails CI.

Implementation sketch:

```
# User-level DB: known projects + cached aggregate stats.
projects_db = ProjectsDB()
projects_db.upsert_project(str(project.root), name=project.name)
projects_db.update_project_stats(
    str(project.root),
    image_count=summary.image_count,
    defect_count=summary.defect_count,
    model_count=summary.model_count,
    best_val_iou=summary.best_val_iou,
)

# Project-level DB: authoritative project operation history.
registry = ModelRegistry(project.registry_db_path, project=project)
registry.record_detection_run(detection_run)
registry.upsert_training_epoch(training_epoch)
```

Testing expectations: - User DB tests should use a temporary `ProjectsDB(db_path=...)`. - Project DB tests should use a temporary project root and assert `registry_db_path` resolves to `artifacts/project.db`. - Migration tests should verify artifact-level `crackz.db` is moved into `project.db` and removed.

7. Async/Concurrent Operations Pattern

Where: `core/src/crackz/ai/core/async_callbacks.py`

Problem: PyTorch Lightning training runs in synchronous threads, but GUI updates must be async to remain responsive. Direct async calls from sync code cause `RuntimeError: no running event loop`.

Solution: Use janus queue to bridge sync and async worlds. Janus provides both `sync_q` (thread-safe) and `async_q` (async-safe) interfaces to the same queue.

Implementation:

```
from crackz.ai.core.async_callbacks import CallbackDispatcher

# In training setup (main thread)
dispatcher = CallbackDispatcher(maxsize=100)
```

```
# In PyTorch Lightning callback (training thread)
sync_reporter = dispatcher.create_sync_progress_reporter()
sync_reporter(0.5, "Training epoch 5/10")  # Thread-safe put

# In GUI (async event loop)
async for progress, message in dispatcher.iter_progress():
    update_progress_bar(progress, message)  # Async get
```

Key Features: - **Thread-safe**: sync_q uses thread locks - **Async-safe**: async_q uses asyncio primitives - **Backpressure handling**: Drops oldest item when queue full - **Non-blocking**: GUI never blocks on training updates

When to Use: Any time you need to communicate between sync operations (AI training, file I/O) and async GUI updates.

Benefits: - Decouples AI operations from UI - Prevents GUI freezing during long operations - Handles burst updates gracefully - Type-safe with protocols (`AsyncProgressReporter`, `AsyncMetricsReporter`)

7. Observability & Monitoring Pattern

Where: `core/src/crackz/tracing/`

Problem: Production deployments need visibility into AI Chat API calls (latency, errors, token usage), but adding observability shouldn't break the app if dependencies are missing.

Solution: Optional OpenTelemetry integration with graceful degradation. Tracing is installed via `--extra tracing` and enabled via environment variable.

Implementation:

```
# In core/src/crackz/tracing/setup.py
from crackz.tracing import setup_tracing, is_tracing_enabled

# Automatic instrumentation on setup
if is_tracing_enabled():
    setup_tracing()  # Instruments Anthropic client automatically

# The instrumentation happens automatically via AnthropicInstrumentor
# No manual client instrumentation needed
```

Configuration:

```
# Install tracing dependencies
uv sync --extra tracing

# Enable at runtime
export CRACKZ_TRACING_ENABLED=1
```

When to Use: Production deployments needing observability, performance debugging, SLA monitoring.

Benefits: - Optional dependency (no bloat for basic usage) - Graceful degradation (works without tracing installed) - Standard OpenTelemetry format (compatible with Jaeger, Zipkin, etc.) - Minimal performance overhead

8. Result Aggregation & Export Pattern

Where: `core/src/crackz/ai/evaluation/reporter.py`, `core/src/crackz/report/generator.py`

Problem: Detection results need to be exported in multiple formats (JSON for APIs, CSV for analysis, DOCX for reports), but each format has different requirements.

Solution: Separate aggregation (collect results) from export (format results). Use adapter pattern for format-specific exporters.

Implementation:

```
from crackz.ai.evaluation.reporter import export_json, export_csv
from crackz.report.generator import ReportGenerator

# Aggregation (format-agnostic)
results = {
    "accuracy": 0.95,
    "precision": 0.92,
    "recall": 0.89,
    "f1_score": 0.90,
    "iou": 0.85
}

# Export to JSON (lightweight)
export_json(
    results,
    output_path=project_path / "evaluation_results.json"
)

# Export to CSV (analysis-friendly)
export_csv(
    results,
    output_path=project_path / "evaluation_results.csv"
)

# Export to DOCX (rich formatting)
generator = ReportGenerator(
```

```
    project_path=project_path,
    template_path=template_path
)
generator.generate_report(
    detection results=results,
    output_path=project_path / "report.docx"
)
```

When to Use: Any system exporting data in multiple formats (reports, APIs, integrations).

Benefits: - Single aggregation logic, multiple exporters - Easy to add new formats (extend reporter) - Format-specific optimization (JSON compact, DOCX rich) - Testable (mock exporters)

9. Device Abstraction Pattern

Where: `core/src/crackz/ai/device/device.py`

Problem: PyTorch supports multiple accelerators (CUDA, MPS, XLA, CPU) but detection logic differs per platform. Hard-coding `torch.device("cuda")` breaks on Mac/TPU.

Solution: Enum-based device abstraction with auto-detection and priority-based selection (CUDA > MPS > CPU).

Implementation:

```
from crackz.ai.device.device import AIDevice

# Auto-select best available device
device = AIDevice.default()  # Returns CUDA if available, else MPS, else CPU

# Check availability
available = AIDevice.available_devices()  # [AIDevice.CPU, AIDevice.CUDA]

# Test device works
success, message, details = AIDevice.test_device("cuda")
if success:
    print(f"CUDA available: {details['device_name']}")

# Use in model
model = SegmentationModel()
model.to(device.value)  # device.value is "cuda", "mps", "cpu", etc.
```

Supported Devices: - **CPU**: Always available - **CUDA**: NVIDIA GPUs (Linux, Windows) - **MPS**: Apple Silicon (M1/M2/M3 Macs) - **XLA**: TPUs (optional torch_xla dependency)

When to Use: Any PyTorch application targeting multiple platforms.

Benefits: - Platform-independent code - Auto-detection prevents manual configuration - Testable (mock torch.cuda.is_available()) - Extensible (add new accelerators via enum)

10. Severity Classification Pattern

Where: `core/src/crackz/ai/detection/severity.py`

Problem: Detection confidence alone doesn't convey risk to users. A 3% crack area might be critical for harbor walls but negligible for a test image.

Solution: Threshold-based categorization with configurable thresholds. Single source of truth function prevents inconsistent categorization.

Implementation:

```
from crackz.ai.detection.severity import categorize_crack_severity

# Default thresholds: 5% (high), 2% (medium)
severity = categorize_crack_severity(confidence=0.08)  # "Crack Detected"
severity = categorize_crack_severity(confidence=0.03)  # "Suspected Crack"
severity = categorize_crack_severity(confidence=0.01)  # "No Crack"

# Custom thresholds via config
severity = categorize_crack_severity(
    confidence=0.04,
    high_threshold=0.06,    # Raise "Crack Detected" threshold
    medium_threshold=0.03    # Raise "Suspected" threshold
)  # "Suspected Crack" (was "Crack Detected" with defaults)

# Override with has_crack flag (confidence + area thresholds met)
severity = categorize_crack_severity(confidence=0.01, has_crack=True)
# "Crack Detected" (flag indicates combined criteria met)
```

Configuration:

```
# project.yaml
ai:
  detection:
    high_confidence_threshold: 0.05   # 5% crack area
    medium_confidence_threshold: 0.02  # 2% crack area
    min_confidence: 0.005             # 0.5% minimum to save
```

When to Use: Any classification system needing human-readable risk categories.

Benefits: - Single source of truth (one function, no duplication) - Configurable thresholds (domain-specific tuning) - Clear semantics ("Crack Detected" vs "Suspected Crack") - Type-safe (returns literal strings, not magic numbers)

Code Organization Patterns

1. Centralized Options Pattern (CLI)

Problem: CLI options duplicated across commands.

Solution: Define options once, import everywhere.

Implementation:

```
# core/src/crackz/cli/options.py
PROJECT_PATH_OPTION = typer.Option(
    None,
    "--project",
    help="Path to project directory",
)

FORCE_OPTION = typer.Option(
    False,
    "--force",
    help="Force operation without confirmation",
)

# core/src/crackz/cli/some_command.py
from crackz.cli.options import PROJECT_PATH_OPTION, FORCE_OPTION

def my_command(
    project_path: Path | None = PROJECT_PATH_OPTION,
    force: bool = FORCE_OPTION,
) -> None:
    pass
```

Benefits: - DRY: Define once, use everywhere - Consistency: All commands use same option names - Maintainability: Change once, applies everywhere

2. Output Convention Pattern

Problem: Inconsistent output between `print()`, `logger`, `typer.echo()`.

Solution: Strict separation of user messages vs operational logs.

Implementation:

```
from crackz.log import logger
import typer

# User-facing success/error messages
typer.secho("✅ Detection completed", fg=typer.colors.GREEN)
typer.secho("❌ Training failed", fg=typer.colors.RED)

# Technical/operational logs
logger.info("Loading {} images from {}", len(images), path)
logger.debug("Model config: {}", config.model_dump())
logger.error("Failed to load checkpoint: {}", error, exc_info=True)
```

Rule: - `typer.secho()` with colors → End-user messages (success, errors, warnings) - `logger.*` → Technical details, debugging, operational info - NEVER `print()` or `typer.echo()` in CLI commands

3. Error Handling Pattern

Problem: Technical exceptions exposed to users, inconsistent error messages.

Solution: Centralized CLI error handler with user-friendly messages.

Implementation:

```
from crackz.cli.exceptions import cli_failure

def train_command():
    try:
        # Business logic
        result = train_model(project_path)
        typer.secho("✅ Training completed", fg=typer.colors.GREEN)
    except FileNotFoundError as e:
        cli_failure(e, "File access")
    except RuntimeError as e:
        cli_failure(e, "Training execution")
```

```
    except Exception as e:
        cli_failure(e, "Unexpected error")
```

What `cli_failure` does: - Logs technical details with `logger.error()` - Shows user-friendly message with `typer.secho()` - Exits with appropriate error code - Handles traceback formatting

4. Configuration Hierarchy Pattern

Problem: Configuration from multiple sources (YAML, env, CLI, defaults).

Solution: Pydantic Settings with clear precedence.

Precedence (highest to lowest): 1. CLI arguments (runtime overrides) 2. Environment variables 3. YAML file 4. Default values

Implementation:

```
class AIConfig(BaseModel):
    model: ModelConfig = Field(default_factory=ModelConfig)
    training: TrainingConfig = Field(default_factory=TrainingConfig)

# Usage
config = AIConfig()  # Loads from defaults only
config_from_yaml = AIConfig(**yaml_data)  # Loads from YAML + defaults

# CLI override (doesn't persist to YAML)
if cli_epochs:
    config.training.epochs = cli_epochs  # Runtime only
```

5. Hybrid Logging Pattern

Problem: Need both system-wide logs and project-specific logs.

Solution: Dual log destinations with same logger instance.

Architecture:

```
logger.info("message")
    ↓
┌─────────────────┐
│  Loguru Router  │
└─────────────────┘
         │
         ├──→ Central Log: /tmp/crackz_logs/app.log (all operations)
         └──→ Project Log: <project>/logs/operations.log (project-specific)
```

Implementation:

```
from loguru import logger

# Configure at startup
logger.add(
    central_log_path,
    format="{time} {level} {message}",
    level="INFO",
)

# Add project-specific handler when project opens
logger.add(
    project_log_path,
    format="{time} {level} {message}",
    level="DEBUG",
    filter=lambda record: record["extra"].get("project") == project_name,
)

# Usage
logger.bind(project=project.name).info("Training started")  # Goes to both logs
```

Configuration Architecture

Pydantic Schema Design

Principles: 1. **Immutability by default**: Use `frozen=False` only when needed 2. **Validation**: Add validators for complex constraints 3. **Defaults**: Provide sensible defaults for all optional fields 4. **Documentation**: Use `Field(description=...)` for clarity 5. **Versioning**: Include `schema_version` for migration support

Example:

```
class ProjectConfig(BaseModel):
    """Project configuration with validation and versioning."""

    schema_version: int = Field(
        default=2,
        ge=1,
        description="Config schema version for migration support",
    )
```

```
    name: str = Field(
        ...,  # Required
        min_length=1,
        max_length=100,
        pattern=r"^[a-zA-Z0-9_-]+$",
        description="Project name (alphanumeric, underscore, hyphen)",
    )

    root: Path = Field(
        ...,  # Required
        description="Project root directory",
    )

    ai: AIConfig = Field(
        default_factory=AIConfig,
        description="AI/ML configuration",
    )

    @field_validator("root")
    @classmethod
    def root_must_exist(cls, v: Path) -> Path:
        """Validate project root exists."""
        if not v.exists():
            raise ValueError(f"Project root does not exist: {v}")
        return v

    @model validator(mode="after")
    def name_matches_root(self) -> Self:
        """Validate name matches root directory."""
        if self.root.name != self.name:
            raise ValueError(f"Name '{self.name}' doesn't match directory '{self.root.name}'")
        return self
```

Schema Versioning & Migration

Why: Backward compatibility for customer deployments.

Strategy: 1. **Version field**: `schema_version: int` in all configs 2. **Migration scripts**: `crackz migrate <project>` command 3. **Breaking changes**: Increment version, add migration path 4. **Non-breaking**: Add with defaults, no version change needed

Example Migration:

```
def migrate_v1_to_v2(config_data: dict) -> dict:
    """Migrate project config from v1 to v2."""
    if config_data.get("schema_version", 1) == 1:
        # v2 added 'ai.detection.batch_size'
        if "ai" not in config_data:
            config_data["ai"] = {}
        if "detection" not in config_data["ai"]:
            config_data["ai"]["detection"] = {}
        if "batch_size" not in config_data["ai"]["detection"]:
            config_data["ai"]["detection"]["batch_size"] = 8

        config_data["schema_version"] = 2

    return config_data
```

Error Handling Architecture

Error Categories

1. **User Errors** (show friendly message, exit gracefully)
2. Invalid CLI arguments
3. Missing project files
4. Invalid configuration
5. **System Errors** (log details, show generic message, exit)
6. Out of memory
7. Disk full
8. GPU not available
9. **Programmer Errors** (log full traceback, should not happen in production)
10. Assertion failures
11. Type errors
12. Logic bugs

Exception Hierarchy

```
class CrackzError(Exception):
    """Base exception for all Crackz errors."""
    pass

class ConfigurationError(CrackzError):
    """Invalid configuration."""
    pass

class ProjectError(CrackzError):
    """Project-related error."""
    pass

class ModelError(CrackzError):
    """Model loading/execution error."""
    pass
```

Error Handling Strategy

```
def command():
    try:
        # Business logic
        result = operation()
        typer.secho("✅ Success", fg=typer.colors.GREEN)

    except FileNotFoundError as e:
        # User error: file doesn't exist
        logger.error("File not found: {}", e)
        typer.secho(f"❌ File not found: {e}", fg=typer.colors.RED)
        raise typer.Exit(1)

    except ConfigurationError as e:
        # User error: invalid config
        logger.error("Configuration error: {}", e)
        typer.secho(f"❌ Configuration error: {e}", fg=typer.colors.RED)
        raise typer.Exit(1)

    except MemoryError:
        # System error: out of memory
        logger.error("Out of memory during operation")
        typer.secho("❌ Out of memory. Try reducing batch size.", fg=typer.colors.RED)
        raise typer.Exit(1)

    except Exception as e:
        # Programmer error: unexpected
        logger.exception("Unexpected error: {}", e)
        typer.secho("❌ Unexpected error. Check logs for details.", fg=typer.colors.RED)
        raise typer.Exit(1)
```

Testing Architecture

Test Pyramid

```
        /\
       /  \        E2E (Integration)         ← Few, slow, expensive
      /────\       Tests: 5% (25 tests)
     /      \
    /────────\     Integration Tests        ← Some, medium speed
   /          \    Tests: 15% (100 tests)
  /────────────\
 /______________\  Unit Tests               ← Many, fast, cheap
                   Tests: 80% (570 tests)
```

Test Structure

```
core/tests/
├── unit/                    # Fast, isolated, no I/O
│   └── crackz/
│       ├── ai/
│       │   ├── test_models.py      # Model creation, logic
│       │   ├── test_training.py    # Training loop logic
│       │   └── test_detection.py   # Detection logic
│       ├── cli/
│       │   ├── test_options.py     # CLI option validation
│       │   └── test_commands.py    # Command parsing
│       └── config/
│           └── test project.py     # Config validation
└── integration/             # Slower, system-level, I/O allowed
    ├── crackz/
    │   ├── test_cli_workflows.py   # End-to-end CLI
    │   └── test_training_pipeline.py # Full training
    └── docs/
        └── test_docs_build.py      # Documentation build

gui/tests/
├── unit/                    # GUI unit tests
└── integration/             # GUI integration tests
```

Test Patterns

Unit Test Pattern

```
def test_model_creation():
    """Test model factory with different configs."""
    # Arrange
    config = ModelConfig(encoder="resnet34", num_classes=1)

    # Act
    model = create_segmentation_model(config)

    # Assert
    assert isinstance(model, nn.Module)
    assert hasattr(model, "encoder")
    assert hasattr(model, "decoder")
```

Integration Test Pattern

```
@pytest.mark.integration
def test full training workflow(tmp path):
    """Test complete training workflow."""
    # Arrange
    project = create_test_project(tmp_path, num_images=10)

    # Act
    result = train_model(project, epochs=1)

    # Assert
    assert result["final_loss"] < result["initial_loss"]
    assert (project.project_path / "artifacts" / "models" / "checkpoint.ckpt").exists()
```

Performance Test Pattern

```
@pytest.mark.benchmark
def test detection performance(benchmark, sample images):
    """Ensure detection meets performance targets."""
    model = load_model(TEST_CHECKPOINT)

    result = benchmark(detect_cracks, model, sample_images)

    # Verify performance target: <5s per image on CPU
    assert result.mean < 5.0
```

Test Fixtures

Location: `core/tests/conftest.py` for shared fixtures

Pattern:

```
@pytest.fixture
def tiny model():
    """Fast model for unit tests."""
    return create_segmentation_model(ModelConfig(
        encoder="resnet18",
        encoder_weights=None,  # No pre-trained weights
        num_classes=1,
    ))

@pytest.fixture
def sample project(tmp path):
    """Sample project with test data."""
    project_dir = tmp_path / "test_project"
    initialize_project(project_dir, name="test_project")

    # Add minimal test data
    (project_dir / "images" / "raw").mkdir(parents=True)
    create_test_image(project_dir / "images" / "raw" / "test.png")

    return project_dir

@pytest.fixture
def mock_model(mocker):
    """Mock model for testing without actual inference."""
    mock = mocker.Mock(spec=nn.Module)
    mock.predict.return_value = torch.zeros(1, 1, 512, 512)
    return mock
```

Performance Architecture

Memory Management

Principle: Process data in chunks, not all at once.

Pattern:

```
# ❌ BAD: Load everything into memory
all_images = [Image.open(p) for p in image_paths]  # OOM for large datasets
for img in all_images:
```

```
    process(img)

# ✅ GOOD: Process in batches
for batch_paths in chunk_list(image_paths, batch_size=8):
    batch = [Image.open(p) for p in batch_paths]
    process_batch(batch)
    # batch garbage collected after each iteration
```

GPU Memory Management

Pattern:

```
def train_with_cleanup():
    try:
        # Training code
        model.train()
        for batch in dataloader:
            loss = model.training_step(batch)
            loss.backward()
            optimizer.step()
    finally:
        # Always clean up GPU memory
        if torch.cuda.is_available():
            torch.cuda.empty_cache()
```

Lazy Loading Pattern

When: Large datasets, optional features.

Pattern:

```
class Project:
    def __init__(self, root: Path):
        self.root = root
        self._config: ProjectConfig | None = None  # Not loaded yet
        self._images: list[Path] | None = None     # Not loaded yet

    @property
    def config(self) -> ProjectConfig:
        """Lazy load configuration."""
        if self._config is None:
            self._config = ProjectConfig.load(
                self.root / f"{self.root.name}_project.yaml"
            )
        return self._config

    @property
    def images(self) -> list[Path]:
        """Lazy load image list."""
        if self._images is None:
            self._images = list((self.root / "images").rglob("*.png"))
        return self._images
```

Security Architecture

Data Privacy Principles

1. **No telemetry**: No data sent to external servers
2. **Local storage only**: All data stays on user's machine
3. **No cloud dependencies**: Works completely offline
4. **No secrets in code**: Use environment variables
5. **No secrets in logs**: Sanitize sensitive data

Input Validation

Pattern:

```
from pydantic import BaseModel, Field, validator

class ProjectConfig(BaseModel):
    name: str = Field(
        ...,
        min_length=1,
        max_length=100,
        pattern=r"^[a-zA-Z0-9_-]+$",  # Prevent path traversal
    )

    @validator("name")
    def no_path_traversal(cls, v: str) -> str:
        """Prevent directory traversal attacks."""
        if ".." in v or "/" in v or "\\" in v:
            raise ValueError("Invalid project name: contains path separators")
        return v
```

Path Safety

Pattern:

```
def safe_path_join(base: Path, *parts: str) -> Path:
    """Join paths safely, preventing traversal attacks."""
    result = base
    for part in parts:
        if ".." in part or part.startswith("/") or part.startswith("\\"):
            raise ValueError(f"Unsafe path component: {part}")
        result = result / part

    # Verify result is still under base
    if not result.resolve().is relative to(base.resolve()):
        raise ValueError("Path traversal detected")

    return result
```

Extending the Architecture

Adding a New CLI Command

1. **Create command module**: `core/src/crackz/cli/my_command.py`
2. **Follow the command pattern**:

```
from __future__ import annotations

import typer
from crackz.cli.options import PROJECT_PATH_OPTION
from crackz.cli.exceptions import cli_failure
from crackz.log import logger

app = typer.Typer(no_args_is_help=True)

@app.command(name="my-command")
def my_command(
    project_path: Path = PROJECT_PATH_OPTION,
) -> None:
    """Description shown in --help."""
    try:
        # Business logic
        logger.info("Starting operation")
        result = do_operation(project_path)
        typer.secho("✅ Operation completed", fg=typer.colors.GREEN)
    except Exception as e:
        cli_failure(e, "Operation name")
```

3. **Register in main CLI**: `core/src/crackz/cli/main.py`
4. **Add tests**: `core/tests/unit/crackz/cli/test_my_command.py`
5. **Update docs**: Add to CLI reference

Adding a New Configuration Section

1. **Create Pydantic model**: `core/src/crackz/config/my_feature.py`

```
from pydantic import BaseModel, Field

class MyFeatureConfig(BaseModel):
    """Configuration for my feature."""

    enabled: bool = Field(
        default=False,
        description="Enable my feature",
    )

    option: str = Field(
        default="default_value",
        description="Feature option",
    )
```

2. **Add to ProjectConfig**: `core/src/crackz/config/project.py`

```
class ProjectConfig(BaseModel):
    # ... existing fields ...
    my_feature: MyFeatureConfig = Field(default_factory=MyFeatureConfig)
```

3. **Add migration if breaking**: `core/src/crackz/project/migrations.py`
4. **Update schema version** if needed
5. **Add tests**: Config validation, serialization, migration

Adding a New AI Model Type

1. **Create model module**: `core/src/crackz/ai/models/my_model.py`
2. **Implement model interface**:

```
class MyCustomModel(nn.Module):
    def __init__(self, config: ModelConfig):
```

```
        super().__init__()
        # Model setup

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # Forward pass
        return x

    def predict(self, x: torch.Tensor) -> torch.Tensor:
        """Inference interface."""
        self.eval()
        with torch.no_grad():
            return self(x)
```

3. **Register in factory**: `core/src/crackz/ai/models/__init__.py`

```
def create_model(config: ModelConfig) -> nn.Module:
    if config.type == "segmentation":
        return create_segmentation_model(config)
    elif config.type == "my custom":
        return MyCustomModel(config)
    else:
        raise ValueError(f"Unknown model type: {config.type}")
```

4. **Add configuration**: `core/src/crackz/config/ai.py`
5. **Add tests**: Model creation, forward pass, prediction

Adding a New Detection Strategy

1. **Create strategy module**: `core/src/crackz/ai/detection/my_strategy.py`
2. **Implement protocol**:

```
class MyDetectionStrategy:
    def __init__(self, model: nn.Module, config: DetectionConfig):
        self.model = model
        self.config = config

    def detect(self, images: list[Path]) -> dict[Path, np.ndarray]:
        """Run detection with my strategy."""
        results = {}
        for image_path in images:
            mask = self. detect single(image_path)
            results[image_path] = mask
        return results
```

3. **Register strategy**: `core/src/crackz/ai/detection/__init__.py`
4. **Add configuration option**: `core/src/crackz/config/ai.py`
5. **Add tests**: Strategy execution, result format

References

Related Documentation

- architecture.md - System architecture overview
- internal/adr/02-decisions.md - Architecture Decision Records (ADRs)
- contributing.md - Contribution guidelines and development standards

External Resources

- Clean Architecture - Robert C. Martin
- SOLID Principles - Software design principles
- Python Design Patterns - Common patterns
- Pydantic Documentation - Configuration validation
- PyTorch Lightning - ML framework

Maintained By: Theresia Lundgren (theresia.lundgren@anaxiatech.se) **Last Updated**: 2025-11-13 **Status**: Active

---

Diagrams

Database & Class Diagrams

Visual reference for the Crackz data model and core class hierarchy.

Interactive Architecture Graph

The primary class diagram is the **Understand Anything knowledge graph** — an interactive, force-directed visualization of every class, module, and relationship in the codebase.

To open it, run the following command in a Claude Code session:

```
/understand-dashboard
```

The dashboard lets you: - Pan, zoom, and search across all nodes - Click any class or module to see its summary, connections, and source location - Follow "guided tours" that walk the architecture in dependency order - Run `/understand-chat` to ask questions about the code while browsing

The committed graph snapshot is at `.understand-anything/knowledge-graph.json`. Refresh it after significant refactors with `/understand` in Claude Code, or `make understand-update` for a reminder of the exact command.

Database Schema

The project-level SQLite database (`artifacts/project.db`) stores model history, detection runs, and training metrics. All tables share a `project_id` foreign key that scopes records to the owning project.

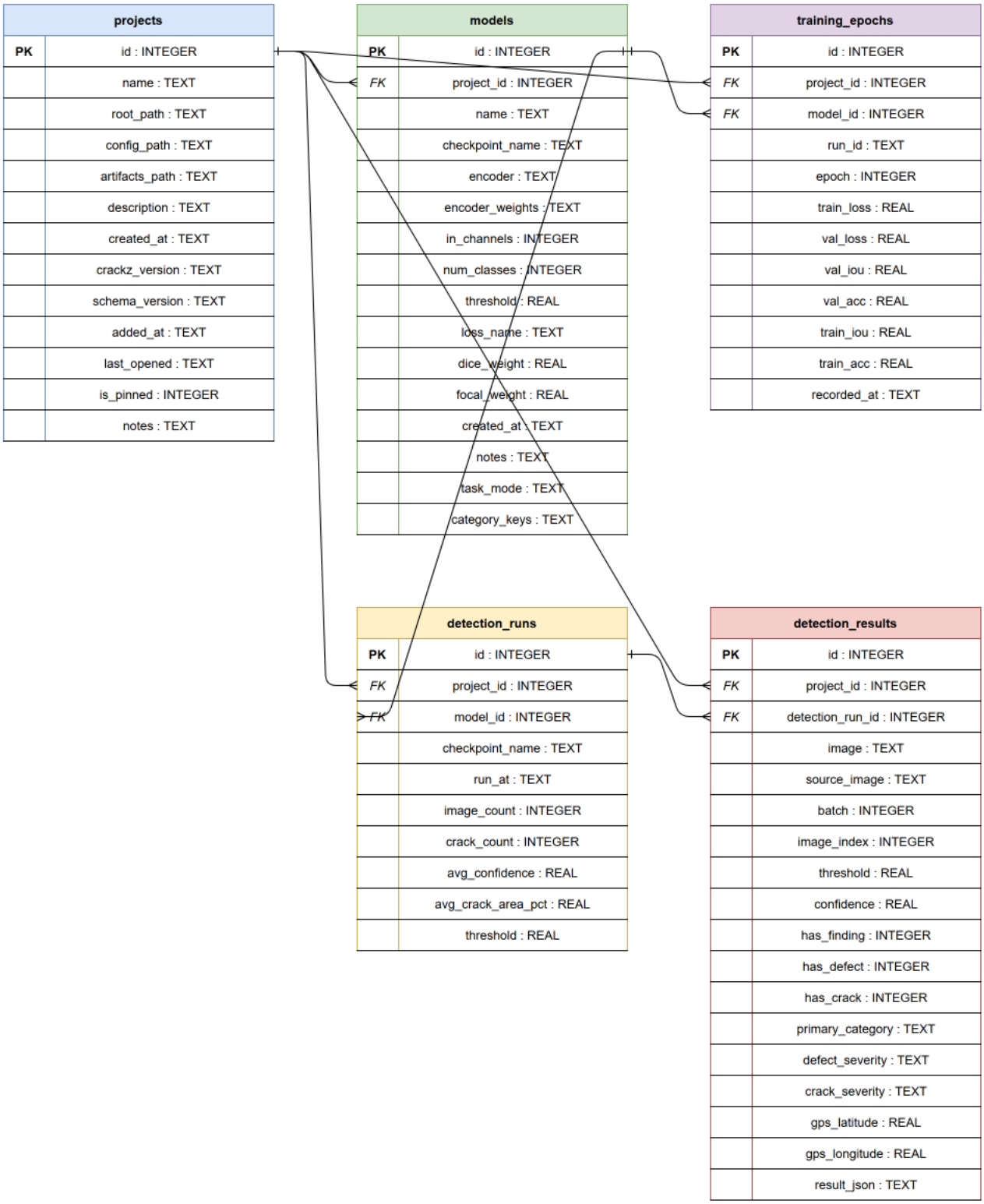


Table summary

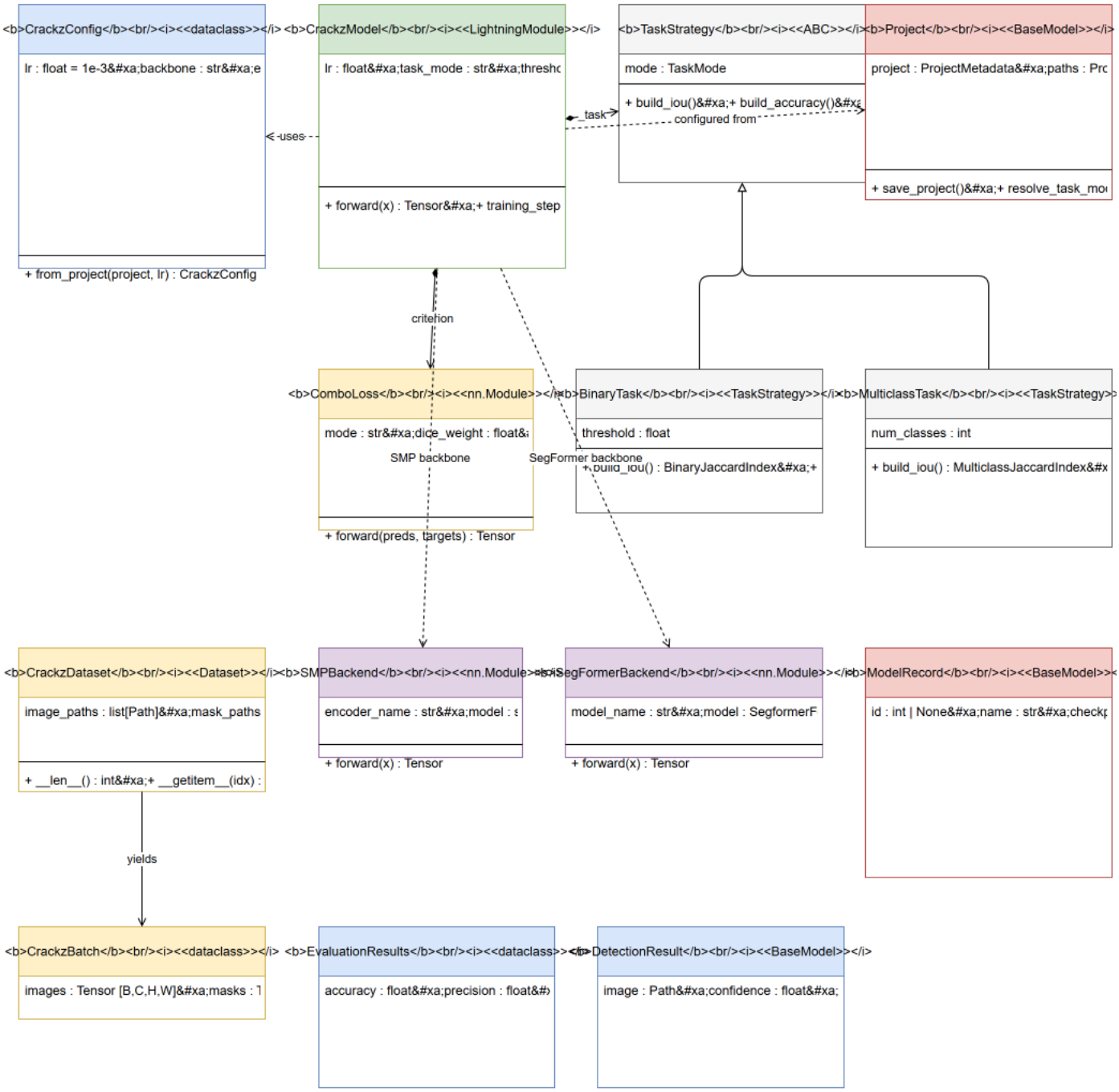
Table	Purpose
`projects`	One row per project; stores path, config, and version info
`models`	Each trained checkpoint registered via `ModelRegistry`
`detection_runs`	Aggregate stats for one batch-inference run
`detection_results`	Per-image results with GPS, confidence, and category data
`training_epochs`	Per-epoch loss / IoU / accuracy for each training run

See `crackz.ai.model.persistence` for the full CRUD layer.

Static Class Diagram (fallback)

A static export of the core AI model hierarchy — useful in environments where Claude Code is not available (e.g., plain GitHub preview, printed docs).

The interactive graph above is the authoritative source; this PNG may lag behind recent refactors.



Key relationships

Relationship	Description
`CrackzModel` uses `CrackzConfig`	Config drives backbone, loss weights, thresholds

Relationship	Description
`CrackzModel` has `TaskStrategy`	Strategy pattern isolates binary vs multiclass behaviour
`BinaryTask` / `MulticlassTask` → `TaskStrategy`	Both implement the abstract strategy
`CrackzModel` has `ComboLoss`	Dice + Focal/CE combination loss
`CrackzDataset` yields `CrackzBatch`	Torch `Dataset` → batched tensor pairs
`CrackzConfig.from_project(Project)`	Config constructed from project YAML/settings

Understand Anything

Understand Anything — Setup & Usage

Crackz uses the Understand Anything Claude Code plugin to provide an interactive, browsable knowledge graph of the codebase architecture.

Prerequisites

- Claude Code (desktop or IDE extension)
- Plugin enabled in `.claude/settings.json`:

```
"enabledPlugins": {
  "understand-anything@Lum1104": true
}
```

Day-to-Day Usage

Command	Purpose
`/understand-dashboard`	Open interactive architecture graph
`/understand-chat`	Ask questions about the code
`/understand-diff`	See which areas a current change touches
`/understand-explain <file>`	Deep-dive into a specific file or class

Keeping the Graph Fresh

The committed graph at `.understand-anything/knowledge-graph.json` is a snapshot. It's accurate at the time of last run. Update it:

- **After a large refactor:** run `/understand` (full re-analysis, ~5 min)
- **After routine commits:** run `/understand --update` (incremental, ~30 sec)

`make understand-update` prints the right command as a reminder.

What the Graph Covers

- Every Python class, function, and module in the codebase
- Import and call relationships
- Architectural layer assignments (API, Service, Data, UI, Utility)
- God nodes (highest-connectivity hubs) and surprising cross-layer connections
- Guided tours ordered by dependency for onboarding

What It Does Not Cover

- SQL schema (see the DB schema diagram in `docs/diagrams.md`)
- Runtime behaviour (use tracing — see `docs/tracing.md`)
- Test coverage (use `uv run pytest --cov`)

Agent Usage

Agents can use the graph during coding sessions via `/understand-chat`. For programmatic access, use CodeGraph's MCP server (see `docs/CODEGRAPH_FOR_AGENTS.md` — agent-facing reference, not published to the docs site) which offers symbol lookup and context queries.

API Reference

API Reference

This page documents the core Python interfaces for programmatic use of Crackz.

> Tip: All high-level tasks (init, import-images, detect, train) are available via the CLI, but you can also call them directly from Python for integration into notebooks, pipelines, or custom services.

Quick Programmatic Examples

Load a Project & Run Detection

```
from pathlib import Path
from crackz.project.project import load_project_from_path
from crackz.ai.detection import detect

project = load_project_from_path(Path("demo-project"))
# Optional: custom progress reporter
progress = lambda p, msg=None: print(f"{p:.0%}", msg or "")

detect(project, progress_reporter=progress)
```

Train a Model

```
from crackz.project.project import load_project_from_path
from crackz.ai.training import train

project = load_project_from_path("demo-project")
train(project)  # writes checkpoints & metrics under project.artifacts/checkpoints
```

Create a New Project Programmatically

```
from pathlib import Path
from crackz.project.project import (
    ProjectMetadata, ProjectPaths, create_project, save_project
)

project = create_project(
    metadata=ProjectMetadata(name="demo-project", description="Example"),
    paths=ProjectPaths(
        project_path=Path("demo-project"),
        image_path=Path("demo-project/images"),
        image_mask_path=Path("demo-project/masks"),
        artifacts_path=Path("demo-project/artifacts"),
    ),
    force=False,
)
save_project(project)
```

Import Images in Code

```
from crackz.project.project import load_project_from_path
project = load project from path("demo-project")
# Uses same logic as CLI import-images (see project.import_images signature)
project.import_images(
    source_path="./samples",  # directory with raw images
    masks_source_path="./samples/masks",  # optional
    image_type=project.ImagesType.RAW if hasattr(project, 'ImagesType') else None,  # or enum from crackz.ImagesType
    size=(512, 512),
    quality=90,
    overwrite=False,
)
```

Configuration

Settings (Environment / YAML)

`CrackzSettings` supports environment overrides with the prefix `CRACKZ_` (e.g. `CRACKZ_LOG_LEVEL=DEBUG`).

A module for managing and loading project configuration.

This module provides a configuration model and functionality to load the configuration from a YAML file.

CrackzSettings

Bases: `BaseSettings`

Configuration for a ai crackz project.

▶ Source code in `core/src/crackz/config/crackz_settings.py`

model_post_init

```
model_post_init(_context)
```

Log any CRACKZ_* env vars that are not recognised fields.

extra="ignore" silently drops them; this surfaces typos so mis-configured environments remain debuggable. Both os.environ and the .env file are checked so typos written only to .env (not yet exported) are also caught.

▶ Source code in `core/src/crackz/config/crackz_settings.py`

from_conf `classmethod`

```
from_conf(conf_file)
```

Creates and returns an instance of the class from a configuration file. This method reads a YAML configuration file from the given file path, parses its content into a dictionary, and uses it to initialize and return an instance of the class.

If the configuration file is empty, an empty dictionary will be used. If the file does not exist, FileNotFoundError is raised. :param conf_file: Path or str pointing to the YAML file. :return: A new instance of the class initialized with the configuration data from the file. :rtype: CrackzSettings

▶ Source code in `core/src/crackz/config/crackz_settings.py`

Exceptions:

This module contains exception classes related to configuration errors.

The exceptions defined here are used to handle various errors that occur during the configuration or project lifecycle process, such as missing configuration files or invalid configuration content.

ConfigError

Bases: `CrackzError`

Base class for all config-related exceptions.

▶ Source code in `core/src/crackz/config/exceptions.py`

ConfigFileNotFoundError

Bases: `ConfigError`

Raised when a required config file/path cannot be found.

▶ Source code in `core/src/crackz/config/exceptions.py`

InvalidConfigError

Bases: `ConfigError`

Raised when configuration content is invalid.

▶ Source code in `core/src/crackz/config/exceptions.py`

Unified Project Configuration (Scaffold)

A minimal serializable wrapper combining model/training/detection/logging settings.

Unified ProjectConfig scaffold.

This provides a light-weight configuration model that can be serialized independently of the full `Project` instance. It wraps the component models already defined in `project.project` and adds helper factory methods to construct a `Project` or persist the

config.

Minimal by design - extend with additional global settings later (e.g. random seeds, mixed precision flags, registry URIs, feature toggles).

ProjectConfig

Bases: `BaseModel`

Serializable project configuration wrapper (minimal).

▶ Source code in `core/src/crackz/config/project_config.py`

load `classmethod`

```
load(path)
```

Load the configuration from the given path.

▶ Source code in `core/src/crackz/config/project_config.py`

save

```
save(path)
```

Save the configuration to the given path.

▶ Source code in `core/src/crackz/config/project_config.py`

to_project

```
to_project()
```

Construct a `Project` from this `ProjectConfig`.

▶ Source code in `core/src/crackz/config/project_config.py`

from_project `classmethod`

```
from_project(project)
```

Construct a `ProjectConfig` from a `Project`.

▶ Source code in `core/src/crackz/config/project_config.py`

Project API

Project objects encapsulate paths, metadata, and convenience helpers.

A set of utility functions and classes to manage and manipulate project configurations.

This module provides functionality to create, configure, save, and manage project-related directories and metadata. It is optimized to handle common scenarios during project setup, including directory creation and YAML-based serialization for project configurations. All project configurations follow a fixed structure to ensure consistency during usage.

It includes: - A Pydantic-based `Project` model to define project metadata. - Functions to generate required project directories. - Functionality to serialize project configurations to persistent storage in YAML format.

Project

Bases: `BaseModel`

Represents a project model with configuration for storing project details.

The Project class encapsulates the essential attributes and configurations of a project. This includes details such as the project's name, description, creation date, and storage-related paths. Designed to work with a BaseModel-compatible configuration that ensures model extensibility. :

▶ Source code in `core/src/crackz/project/project.py`

name `property`

```
name
```

The name of the project.

root `property`

```
root
```

Project root directory.

config_file `property`

```
config_file
```

The expected config file name for this project.

detections_path `property`

```
detections_path
```

Gets the path to the detected images data directory.

Returns:

Name	Type	Description
`Path`	`Path`	The path to the detected images data directory derived from the provided image path.

models_path `property`

```
models_path
```

Gets the path to the models directory (new structure).

Returns:

Name	Type	Description
`Path`	`Path`	The path to the models directory.

checkpoints_path `property`

```
checkpoints_path
```

Gets the path to the checkpoints directory.

Deprecated: Use models_path instead. This property provides backward compatibility by checking for legacy 'checkpoints' directory and falling back to new 'models' directory.

Returns:

Name	Type	Description
`Path`	`Path`	The path to the models directory (or legacy checkpoints for old projects).

runtime_path `property`

```
runtime_path
```

Gets the path to the runtime directory for logs and temporary artifacts.

Returns:

Name	Type	Description
`Path`	`Path`	The path to the runtime directory.

training_path `property`

```
training_path
```

Gets the path to the training directory.

detections_visualizations_path `property`

```
detections_visualizations_path
```

Gets the path to the detection visualizations subdirectory.

For backward compatibility: if the new visualizations subdirectory doesn't exist or is empty (no PNG files), fall back to the legacy detections directory. This handles projects that were schema-migrated but not file-migrated.

Returns:

Name	Type	Description
`Path`	`Path`	The path to the detection visualizations directory (or legacy detections for old projects).

detection_results_path `property`

```
detection_results_path
```

Gets the path to the training metrics JSON file.

training_metrics_path `property`

```
training_metrics_path
```

Gets the path to the training metrics JSON file.

tensorboard_logs_path `property`

```
tensorboard_logs_path
```

Gets the path to the TensorBoard logs directory (in runtime/).

registry_db_path `property`

```
registry_db_path
```

Path to the per-project SQLite database (models, detection runs, epochs).

Canonical name is `project.db`. Legacy artifact-level `crackz.db` files are moved into this path by the registry layer.

lightning_logs_path `property`

```
lightning_logs_path
```

Gets the path to the PyTorch Lightning logs directory (in runtime/).

val_path `property`

```
val_path
```

Gets the path to the model validation data directory.

Returns:

Name	Type	Description
`Path`	`Path`	The path to the validation data directory derived from the provided image path.

test_path property

```
test_path
```

Gets the path to the model test data directory.

Returns:

Name	Type	Description
Path	Path	The path to the test data directory derived from the provided image path.

train_path property

```
train_path
```

Gets the path to the model train data directory.

Returns:

Name	Type	Description
Path	Path	The path to the train data directory derived from the provided image path.

train_mask_path property

```
train_mask_path
```

Gets the path to the model train mask data directory.

Returns:

Name	Type	Description
Path	Path	The path to the train data directory derived from the provided image mask path.

val_mask_path property

```
val_mask_path
```

Gets the path to the model validation mask data directory.

Returns:

Name	Type	Description
Path	Path	The path to the validation data directory derived from the provided image mask path.

test_mask_path property

```
test_mask_path
```

Gets the path to the model test mask data directory.

Returns:

Name	Type	Description
Path	Path	The path to the test data directory derived from the provided image mask path.

raw_path property

```
raw_path
```

Gets the path to the raw (detect) test data directory.

Returns:

Name	Type	Description
`Path`	`Path`	The path to the raw data directory derived from the provided image path.

raw_mask_path `property`

```
raw_mask_path
```

Gets the path to the raw mask (detect) test data directory.

Returns:

Name	Type	Description
`Path`	`Path`	The path to the raw data directory derived from the provided image path.

project_file_path `property`

```
project_file_path
```

Compute the path where the project YAML will be saved.

report_logo_path

```
report_logo_path(default_logo_path)
```

Return the custom report logo path when configured and present.

▶ Source code in `core/src/crackz/project/project.py`

resolve_task_mode

```
resolve_task_mode()
```

Return the active segmentation task mode.

Honors `ai.model.task_mode` when set; otherwise auto-derives from the annotation schema. `"binary"` collapses all foreground categories into a single defect-vs-none class.

▶ Source code in `core/src/crackz/project/project.py`

sync_derived_fields

```
sync_derived_fields()
```

Update compatibility fields that are derived from project-owned schema state.

▶ Source code in `core/src/crackz/project/project.py`

map_type_to_dir

```
map_type_to_dir(image_type)
```

Maps a given image type to its corresponding directory path.

This method takes an image type as input and resolves it to a corresponding directory path. It supports types such as `raw`, `train`, `test`, and `val`. If the given value does not map to a valid type, the function defaults to the `raw` directory path.

:param image_type: The type of the image to map ("raw", "train", "test", or "val"). :return: The `Path` object representing the directory mapped from the given image type, or `None` if no mapping exists.

▶ Source code in `core/src/crackz/project/project.py`

map_type_to_masks_dir

```
map_type_to_masks_dir(image_type)
```

Maps a given image type to its corresponding directory path.

This method takes an image type as input and resolves it to a corresponding directory path. It supports types such as `raw`, `train`, `test`, and `val`. If the given value does not map to a valid type, the function defaults to the `raw` directory path.

:param image_type: The type of the image mask to map ("raw", "train", "test", or "val"). :return: The `Path` object representing the directory mapped from the given image type, or `None` if no mapping exists.

▶ Source code in `core/src/crackz/project/project.py`

save_project

```
save_project()
```

Saves the given project to a file in YAML format. The function serializes the project data using the model's `model_dump` method, and writes it to a specified path. The path is determined dynamically based on the project information. Logging is performed to confirm success or log any errors encountered during the saving process.

:param project: The project object to be saved. The project must implement the `model_dump` method for serialization. :return: None

▶ Source code in `core/src/crackz/project/project.py`

import_image_files

```
import_image_files(
    files,
    dest_dir,
    size=None,
    quality=90,
    *,
    overwrite=False,
    progress_cb=None,
)
```

Import specific image files into project storage.

- Resizes and adjusts quality.
- Writes to the given destination directory.
- Raises ImageImportError on failure.

▶ Source code in `core/src/crackz/project/project.py`

import_images

```
import_images(
    source_path,
    masks_source_path=None,
    image_type=ImagesType.RAW,
    size=None,
    quality=90,
    *,
    overwrite=False,
    progress_cb=None,
    split=False,
    train_ratio=0.7,
    val_ratio=0.15,
    test_ratio=0.15,
    random_seed=None,
    group_by=GroupBy.NONE,
    group_block_size=None,
    mask_label_map=None,
)
```

Import images (and optional masks) into project storage.

- Resizes and adjusts quality.
- Writes to type-specific image/mask destinations.
- Skips masks if not provided or empty.
- Raises ImageImportError on failure.
- If split=True, automatically distributes images across train/val/test splits.

▶ Source code in `core/src/crackz/project/project.py`

create_project_dir

```
create_project_dir(*, force)
```

Creates the necessary directories for a given project. If the directories already exist, it optionally allows overwriting them based on the `force` parameter.

:param project: The project object containing the `project_path` and `artifacts_path` :param force: A boolean indicating whether to forcibly overwrite existing directories :return: None

▶ Source code in `core/src/crackz/project/project.py`

clean

```
clean(*, logs_only=False, keep_best_model=True)
```

Clean temporary files and logs from the project.

By default, cleans logs, training artifacts, models, and detections. Images and masks are always preserved. The best model checkpoint is preserved by default.

:param logs_only: If True, only clean log files and runtime artifacts, preserve models/results :param keep_best_model: If True, preserve best.ckpt when cleaning models :return: Number of items removed

▶ Source code in `core/src/crackz/project/project.py`

create_project_and_save

```
create_project_and_save(
    metadata,
    paths,
    ai=default_ai_config,
    logging=default_logging_config,
    *,
    force=False,
)
```

Creates and initializes a new project by defining its attributes, setting up its directory structure, and saving the corresponding Project object. It ensures.

▶ Source code in `core/src/crackz/project/project.py`

create_project

```
create_project(
    metadata, paths, ai=None, logging=None, *, force=False
)
```

Creates and initializes a new project by defining its attributes, setting up its directory structure, and returning the corresponding Project object. It ensures proper storage of project-related information including paths and metadata.

:param paths: necessary paths for the project. :param metadata: project metadata. :param ai: artificial intelligence configuration. :param logging: logging configuration. :param force: Boolean flag to force creation, potentially overriding existing data. :type force: bool :return: The created Project instance containing all initialized attributes. :rtype: Project

▶ Source code in `core/src/crackz/project/project.py`

save_project

```
save_project(project)
```

See `Project.save_project`.

▶ Source code in `core/src/crackz/project/project.py`

load_project

```
load_project(project_file_path)
```

Loads a project configuration from the specified file path. The configuration file is expected to be in YAML format, and its content is deserialized into a `Project` object. If the file cannot be opened, read, or parsed, the function logs an error.

Parameters:

Name	Type	Description	Default
`project_file_path`	`Path`	The path to the project configuration file.	*required*

Returns:

Type	Description
`Project`	Project | None: The deserialized `Project` object if loading is

Type	Description

`Project` successful. Returns `None` if there is an error during the process.

▶ Source code in `core/src/crackz/project/project.py`

load_project_from_path

```
load_project_from_path(project_path)
```

Loads a project configuration from a specified path and returns a Project instance.

This function reads a YAML file from the given project path, parses its content, and initializes a Project instance based on the data. If the file cannot be read or parsed, an appropriate exception is raised.

:param project_path: The path to the project directory. :type project_path: Path :return: The initialized Project instance containing the configuration data. :rtype: Project :raises TypeError: Raised when there is an OSError or YAML parsing error while loading the project configuration.

▶ Source code in `core/src/crackz/project/project.py`

load_from_project_path_or_cfg

```
load_from_project_path_or_cfg(cfg, project_path)
```

Loads a project either from the provided project path or configuration file.

This function attempts to load a project based on the provided parameters. If a project path is provided, the project is loaded from the given path. If a configuration file is provided instead, the project is loaded using the configuration file. The function prefers the project path if both are provided.

:param cfg: Path to the configuration file for loading the project. Can be None if a project path is provided. :type cfg: Path | None :param project_path: Path to the project directory from which to load the project. Can be None if a configuration file is provided. :type project_path: Path | None :return: Returns the loaded project instance of type Project. :rtype: Project

▶ Source code in `core/src/crackz/project/project.py`

Supporting types:

Project-level constants.

ImagesType

Bases: `StrEnum`

Represents different types of image datasets.

This Enum class is used to categorize and manage different types of image datasets for various stages in a machine learning workflow. It provides string values for each dataset type to ensure consistency and prevent errors due to arbitrary strings.

Attributes:

Name	Type	Description
`RAW`		Specifies raw image data that has not been processed.
`TRAIN`		Indicates the dataset used for training machine learning models.
`TEST`		Specifies the dataset used for testing or evaluation purposes.
`VAL`		Represents the validation dataset used to tune model parameters.

▶ Source code in `core/src/crackz/project/types.py`

Exceptions:

This module defines custom exception classes related to project operations.

These exception classes inherit from CrackzError and represent specific errors encountered in the context of projects, such as configuration issues or file import errors.

ProjectError

Bases: `CrackzError`

Base class for all project-related exceptions.

▶ Source code in `core/src/crackz/project/exceptions.py`

ProjectExistsError

Bases: `ProjectError`

Raised when a project path/name exists.

▶ Source code in `core/src/crackz/project/exceptions.py`

ProjectNotFoundError

Bases: `ProjectError`

Raised when a project path/name is missing or invalid.

▶ Source code in `core/src/crackz/project/exceptions.py`

ProjectConfigError

Bases: `ProjectError`

Raised for invalid or inconsistent project config.

▶ Source code in `core/src/crackz/project/exceptions.py`

ImageImportError

Bases: `ProjectError`

Raised when an image cannot be imported.

▶ Source code in `core/src/crackz/project/exceptions.py`

ImageFileError

Bases: `ProjectError`

Raised when an image cannot be opened.

▶ Source code in `core/src/crackz/project/exceptions.py`

AI / Detection API

High-level detection & training entry points:

AI detection compatibility exports with lazy loading.

This package intentionally avoids importing the full training/runtime stack at module import time so that lightweight paths such as report generation and CLI startup do not eagerly pull in PyTorch Lightning and segmentation models.

DetectionResult

Bases: `BaseModel`

Typed representation of one detection result entry.

This is a Pydantic BaseModel to provide runtime validation, type coercion, and consistent data handling throughout the application.

▶ Source code in `core/src/crackz/ai/detection/results.py`

categorize_crack_severity

```
categorize_crack_severity(
    confidence,
    *,
    high_threshold=DEFAULT_HIGH_CONFIDENCE,
    medium_threshold=DEFAULT_MEDIUM_CONFIDENCE,
    has_crack=False,
)
```

Categorize crack detection severity based on crack area percentage.

This is the single source of truth for crack categorization across the application. Categories are simplified for user clarity: detected vs suspected cracks.

Parameters:

Name	Type	Description	Default
`confidence`	`float`	Crack area percentage as decimal (0.0 to 1.0) Represents mean probability over whole image. Typical values: 0.005-0.08 (0.5%-8% crack coverage)	*required*
`high_threshold`	`float`	Threshold for "Crack Detected" category (default 0.05 = 5%)	`DEFAULT_HIGH_CONFIDENCE`
`medium_threshold`	`float`	Threshold for "Suspected Crack" category (default 0.02 = 2%)	`DEFAULT_MEDIUM_CONFIDENCE`
`has_crack`	`bool`	Boolean flag indicating if crack was detected (confidence + area thresholds met)	`False`

Returns:

Type	Description
`str`	Severity category (one of three simplified categories): - "Crack Detected" (crack_area >= high_threshold OR has_crack=True) - "Suspected Crack" (medium_threshold <= crack_area < high_threshold) - "No Crack" (crack_area < medium_threshold)

Examples:

```
>>> categorize_crack_severity(0.08)  # 8% crack area
'Crack Detected'
>>> categorize_crack_severity(0.03)  # 3% crack area
'Suspected Crack'
>>> categorize_crack_severity(0.01)  # 1% crack area
'No Crack'
>>> categorize_crack_severity(0.01, has_crack=True)
'Crack Detected'  # has_crack flag indicates both confidence and area thresholds met
```

▶ Source code in `core/src/crackz/ai/detection/severity.py`

create_tensorboard_logger

```
create_tensorboard_logger(project)
```

Create a TensorBoard logger if enabled in the project configuration.

This is a convenience function that maintains backward compatibility with the existing API while leveraging the new CrackzTensorBoardLogger class.

Parameters:

Name	Type	Description	Default
`project`	`Project`	The project configuration.	*required*

Returns:

Type	Description
`TensorBoardLogger | None`	TensorBoardLogger if tensorboard is enabled, None otherwise.

▶ Source code in `core/src/crackz/ai/training/tensorboard.py`

train

```
train(
    project,
    progress_reporter=None,
    cancellation_token=None,
    metrics_callback=None,
    ckpt path=None,
    backbone_override=None,
)
```

Trains a model for crack detection using the provided project data.

Parameters:

Name	Type	Description	Default
`project`	`Project`	Loaded project containing configuration and dataset paths.	*required*
`progress_reporter`	`ProgressReporter | None`	Optional callable for progress updates.	`None`
`cancellation_token`	`CancellationToken | None`	Optional token to check for cancellation requests.	`None`
`metrics_callback`	`MetricsReporter | None`	Optional callable to receive epoch metrics immediately for real-time GUI updates.	`None`
`ckpt_path`	`Path | None`	Optional path to a checkpoint to resume training from. Restores model weights, optimizer state, and epoch counter.	`None`
`backbone_override`	`str | None`	Optional backbone name to use for this run instead of `project.ai.model.backbone`. Ignored when `ckpt_path` is given (a resumed checkpoint's backbone always wins).	`None`

Raises:

Type	Description
`CancellationError`	If cancellation is requested during training.

▶ Source code in `core/src/crackz/ai/training/training_runner.py`

detect

```
detect(
    project,
    progress_reporter=None,
    cancellation_token=None,
    checkpoint_name=None,
    result_callback=None,
    *,
    on_incompatible="original",
)
```

Detect cracks in images using a pre-trained model and save annotated results.

Parameters:

Name	Type	Description	Default
`project`	`Project`	Loaded project containing configuration, image split paths and model settings.	*required*
`progress_reporter`	`ProgressReporter | None`	Optional callable receiving (progress: float 0.0-1.0, message: str | None) used to report incremental status (CLI/GUI). If None, a no-op reporter is used.	`None`
`cancellation_token`	`CancellationToken | None`	Optional token to check for cancellation requests.	`None`
`checkpoint_name`	`str | None`	Optional checkpoint name to use (e.g., 'best', 'last', 'final', 'epoch-0.9234'). If None, uses automatic selection (best > last > final).	`None`
`result_callback`	`Callable[[dict[str, Any]], None] | None`	Optional callable receiving detection results immediately as they're processed. Allows GUI to update in real-time without file polling.	`None`
`on_incompatible`	`IncompatibleMode`	How to handle checkpoint/project architecture mismatches. `"raise"` (default), `"untrained"`, or `"original"`.	`'original'`

Returns:

Type	Description
`None`	None. Side-effects: writes annotated detection PNGs and detection metrics to
`None`	the project's registry DB.

Raises:

Type	Description
`CancellationError`	If cancellation is requested during detection.
`CheckpointIncompatibleError`	When *on_incompatible* is `"raise"` and the checkpoint backbone differs from the project backbone.

▶ Source code in `core/src/crackz/ai/detection/detection_runner.py`

__getattr__

```
__getattr__(name)
```

Load detection exports only when they are first accessed.

▶ Source code in `core/src/crackz/ai/detection/__init__.py`

Datasets:

Dataset classes for crack images.

ToRGB

Convert image to RGB.

▶ Source code in `core/src/crackz/ai/data/dataset.py`

apply

```
apply(im)
```

Apply the transformation.

▶ Source code in `core/src/crackz/ai/data/dataset.py`

CrackzDataset

Bases: `Dataset[Sample]`

Dataset for crack images with optional masks.

▶ Source code in `core/src/crackz/ai/data/dataset.py`

load_image_with_crackz_image

```
load_image_with_crackz_image(image_path)
```

Load an image using CrackzImage for consistent error handling and file management.

This function provides a standardized way to load images across the AI pipeline, ensuring consistent error handling and proper file handle management.

Parameters:

Name	Type	Description	Default
`image_path`	`Path`	Path to the image file	*required*

Returns:

Type	Description
`Image`	PIL Image object

Raises:

Type	Description
`ImageLoadingError`	If the image cannot be loaded

▶ Source code in `core/src/crackz/ai/data/dataset.py`

CrackzDetectionDataset

```
CrackzDetectionDataset(
    image_dir, transform=None, normalization_config=None
)
```

Create a detection dataset - no masks required.

This is a factory function that returns a CrackzDataset configured for detection (require_masks=False), creating dummy masks for all images to maintain a consistent interface.

Parameters:

Name	Type	Description	Default
`image_dir`	`Path | str`	Directory containing images to process	*required*
`transform`	`ImageTransform | None`	Optional image transformations	`None`
`normalization_config`	`NormalizationConfig | None`	Optional normalization configuration	`None`

Returns:

Type	Description
`CrackzDataset`	CrackzDataset configured for detection mode

▶ Source code in `core/src/crackz/ai/data/dataset.py`

create_training_datasets

```
create_training_datasets(
    train_path,
    train_mask_path,
    val_path,
    val_mask_path,
    test_path,
    test_mask_path,
)
```

Create training, validation, and test datasets.

Parameters:

Name	Type	Description	Default
`train_path`	`Path`	Path to training images directory	*required*
`train_mask_path`	`Path`	Path to training masks directory	*required*
`val_path`	`Path`	Path to validation images directory	*required*
`val_mask_path`	`Path`	Path to validation masks directory	*required*
`test_path`	`Path`	Path to test images directory	*required*
`test_mask_path`	`Path`	Path to test masks directory	*required*

Returns:

Type	Description
`tuple[CrackzDataset, CrackzDataset, CrackzDataset]`	Tuple of (train_ds, val_ds, test_ds)

▼ Note

Images without corresponding masks will be skipped with a warning logged. Empty datasets are allowed - PyTorch will raise an error later if needed.

▶ Source code in `core/src/crackz/ai/data/dataset.py`

create_dataset

```
create_dataset(
    project,
    transforms,
    mask_transforms,
    image_dir=None,
    image_masks_dir=None,
)
```

Create detection dataset (no masks needed).

Parameters:

Name	Type	Description	Default
`project`	`Project`	Project configuration	*required*
`transforms`	`ImageTransform | None`	Optional image transformations	*required*

Name	Type	Description	Default
`mask_transforms`	`MaskTransform | None`	Optional mask transformations (ignored for detection)	*required*
`image_dir`	`Path | None`	Optional image directory (defaults to project.raw_path)	`None`
`image_masks_dir`	`Path | None`	Optional mask directory (ignored for detection)	`None`

Returns:

Type	Description
`CrackzDataset`	CrackzDataset configured for detection mode

▶ Source code in `core/src/crackz/ai/data/dataset.py`

Dataloaders:

DataLoader utilities for crack images.

collate

```
collate(batch)
```

Collate samples into a batch.

Handles both training samples (CrackzSample with real masks) and detection samples (CrackzSample with dummy masks). Detection samples have dummy zero masks that are ignored.

▶ Source code in `core/src/crackz/ai/data/dataloader.py`

make_crackz_loader

```
make_crackz_loader(
    dataset, batch_size=32, num_workers=8, *, shuffle=True
)
```

Make a dataloader for crackz (handles both training and detection datasets).

▶ Source code in `core/src/crackz/ai/data/dataloader.py`

create_training_dataloaders

```
create_training_dataloaders(
    train_ds,
    val_ds,
    test_ds,
    batch_size,
    num_workers,
    *,
    shuffle_train=True,
)
```

Create dataloaders for training, validation, and testing.

Parameters:

Name	Type	Description	Default
`train_ds`	`CrackzDataset`	Training dataset	*required*
`val_ds`	`CrackzDataset`	Validation dataset	*required*
`test_ds`	`CrackzDataset`	Test dataset	*required*
`batch_size`	`int`	Batch size for all dataloaders	*required*

Name	Type	Description	Default
`num_workers`	`int`	Number of worker processes for data loading	*required*
`shuffle_train`	`bool`	Whether to shuffle the training data	`True`

Returns:

Type	Description
`tuple[DataLoader, DataLoader, DataLoader]`	Tuple of (train_loader, val_loader, test_loader)

▶ Source code in `core/src/crackz/ai/data/dataloader.py`

Model configuration & LightningModule wrapper:

The core `CrackzModel` class utilizes a segmentation backbone for binary or multiclass segmentation tasks. It computes segmentation logits, applies a task-aware combination loss (Dice + Focal for binary, Dice + CrossEntropy for multiclass) during training, and evaluates performance with IoU and accuracy metrics.

Per-task-mode behaviour (metric construction, loss selection, prediction shaping) lives in :mod:`crackz.ai.model.segmentation.tasks` — `CrackzModel` delegates to a :class:`TaskStrategy` rather than branching on `task_mode` at every call site.

ComboLoss

Bases: `Module`

Task-aware combination loss for segmentation.

Combines a Dice term with a task-specific auxiliary term:

- binary mode: Dice + Focal, weighted by `aux_weight`.
- multiclass mode: Dice + CrossEntropy, weighted by `aux_weight`.

`focal_weight` from :class:`~crackz.ai.model.segmentation.model.CrackzConfig` is forwarded as `aux_weight` so existing project YAML keeps working.

▶ Source code in `core/src/crackz/ai/model/segmentation/loss.py`

forward

```
forward(y_pred, y_true)
```

Compute `dice_weight * Dice + aux_weight * (focal | cross_entropy)`.

▶ Source code in `core/src/crackz/ai/model/segmentation/loss.py`

CrackzConfig `dataclass`

Configuration container for `CrackzModel`.

Attributes:

Name	Type	Description
`lr`	`float`	Learning rate for the AdamW optimizer.
`backbone`	`str`	Model backbone identifier. "segformer-b0" through "segformer-b5" use HuggingFace SegFormer; anything else is treated as an SMP encoder name.
`in_channels`	`int`	Number of channels in the input images.
`num_classes`	`int`	Number of output segmentation classes, derived from the project annotation schema (1 for crack-only/binary, otherwise 1 + foreground categories).

Name	Type	Description
`threshold`	`float`	Probability threshold for converting logits to binary masks and IoU metric.
`dice_weight`	`float`	Weight applied to the Dice term in ComboLoss.
`focal_weight`	`float`	Weight applied to the auxiliary term in ComboLoss. In binary mode this is the focal loss; in multiclass mode it is the cross-entropy loss.
`task_mode`	`TaskMode`	"binary" for a single foreground category, "multiclass" otherwise.
`category_keys`	`tuple[str, ...]`	Foreground category keys in stable schema order. Used to label per-class training metrics.

▶ Source code in `core/src/crackz/ai/model/segmentation/model.py`

from_project `staticmethod`

```
from_project(project, lr)
```

Create a `CrackzConfig` from a `Project` instance.

▶ Source code in `core/src/crackz/ai/model/segmentation/model.py`

CrackzModel

Bases: `LightningModule`

PyTorch Lightning module for crack segmentation.

▶ Source code in `core/src/crackz/ai/model/segmentation/model.py`

training_step

```
training_step(batch, _batch_idx)
```

Perform a training step.

▶ Source code in `core/src/crackz/ai/model/segmentation/model.py`

validation_step

```
validation_step(batch, _batch_idx)
```

Validate a step.

▶ Source code in `core/src/crackz/ai/model/segmentation/model.py`

test_step

```
test_step(batch, _batch_idx)
```

Perform a test step.

▶ Source code in `core/src/crackz/ai/model/segmentation/model.py`

on_train_epoch_end

```
on_train_epoch_end()
```

Reset training metrics at epoch end.

▶ Source code in `core/src/crackz/ai/model/segmentation/model.py`

on_validation_epoch_end

```
on_validation_epoch_end()
```

Reset validation metrics at epoch end.

▶ Source code in `core/src/crackz/ai/model/segmentation/model.py`

on_test_epoch_end

```
on_test_epoch_end()
```

Reset test metrics at epoch end.

▶ Source code in `core/src/crackz/ai/model/segmentation/model.py`

configure_optimizers

```
configure_optimizers()
```

Configure the AdamW optimizer.

▶ Source code in `core/src/crackz/ai/model/segmentation/model.py`

get_loss_function

```
get_loss_function(config)
```

Get the loss function based on the configuration.

`loss_name == "combo"` (the default) returns the task-appropriate :class:`ComboLoss` from the active strategy. Any other value falls back to a plain Dice loss with a warning.

▶ Source code in `core/src/crackz/ai/model/segmentation/model.py`

Callbacks & utilities:

Custom callbacks for PyTorch Lightning.

This module provides callbacks for training and testing progress reporting, metrics collection, and image logging.

The ProgressCallback now includes batch-level progress tracking to provide more accurate time remaining estimates during training. Progress updates are reported every 10 batches (configurable) and at the end of each epoch, ensuring that the GUI and CLI show accurate remaining time even for long epochs with many batches.

MetricsLoggerCallback

Bases: `Callback`

Collects and persists scalar metrics; optionally forwards them to an external reporter.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_train_start

```
on_train_start(trainer, pl_module)
```

Store total epochs and resolve model_id when training starts.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_train_batch_end

```
on_train_batch_end(
    trainer, pl_module, outputs, batch, batch_idx
)
```

Update running metrics for live GUI updates during long epochs.

Writes current metrics to training_metrics.json every 10 batches. The main metrics history is preserved and only updated at epoch end.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_train_epoch_end

```
on_train_epoch_end(trainer, pl_module)
```

Capture training metrics at the end of each training epoch.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_validation_epoch_end

```
on_validation_epoch_end(trainer, pl_module)
```

Capture validation metrics at the end of validation.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_test_epoch_end

```
on_test_epoch_end(trainer, pl_module)
```

Capture test metrics at the end of testing.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_fit_end

```
on_fit_end(trainer, pl_module)
```

Final save at the end of training.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

ProgressCallback

Bases: `Callback`

Reports training/testing progress via an optional external reporter.

This callback tracks progress at both the epoch and batch level to provide accurate time remaining estimates. Progress updates are reported: - At the start and end of each training epoch - Every 10 batches during training (configurable via update_interval) - At the end of each epoch (always) - At the end of training and testing

The batch-level progress tracking ensures that long epochs with many batches show continuously updating progress and accurate time estimates, which is especially important for GUI displays.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

__call__

```
__call__(progress, message=None)
```

Report progress via the reporter or log it.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_train_start

```
on_train_start(trainer, pl_module)
```

Calculate total batches for progress tracking.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_train_epoch_start

```
on_train_epoch_start(trainer, pl_module)
```

Report progress at the start of each training epoch.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_train_batch_end

```
on_train_batch_end(
    trainer, pl_module, outputs, batch, batch_idx
)
```

Report progress after each training batch for more accurate time estimates.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_train_epoch_end

```
on_train_epoch_end(trainer, pl_module)
```

Report progress at the end of each training epoch.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_fit_end

```
on_fit_end(trainer, pl_module)
```

Report progress at the end of training.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_test_end

```
on_test_end(trainer, pl_module)
```

Report progress at the end of testing.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

ImageSampleLogger

Bases: `Callback`

Logs sample input / prediction / mask triplets to TensorBoard after validation epochs.

This callback is added only when a TensorBoard logger is active. It keeps a small, fixed set of samples (to avoid touching the dataloaders again) and performs a forward pass in eval/no_grad context to obtain predictions. Images are denormalized using the standard ImageNet statistics (same as dataset defaults) before logging.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

on_validation_epoch_end

```
on_validation_epoch_end(trainer, pl_module)
```

Log a grid of sample input / ground-truth / prediction images to TensorBoard.

Executed after each validation epoch when a TensorBoard logger is present. Skips silently if no samples or logger are available.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

make_reporter

```
make_reporter(ui_callback)
```

Wrap a UI callback to conform to the reporter interface.

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

create_training_callbacks

```
create_training_callbacks(
    project,
    val_ds,
    tb_logger,
    progress_reporter,
    runtime_tracker,
    cancellation_token,
)
```

Create callbacks for training including checkpoint, metrics, progress, and image logging.

Parameters:

Name	Type	Description	Default
`project`	`Project`	Project configuration	*required*
`val_ds`	`CrackzDataset`	Validation dataset for sampling images	*required*
`tb_logger`	`TensorBoardLogger | None`	TensorBoard logger instance or None if disabled	*required*
`progress_reporter`	`ProgressReporter | None`	Optional progress reporter callback	*required*
`runtime_tracker`	`RuntimeTracker`	Runtime tracker for time estimation	*required*
`cancellation_token`	`CancellationToken | None`	Optional cancellation token	*required*

Returns:

Type	Description
`list[Any]`	List of PyTorch Lightning callbacks

▶ Source code in `core/src/crackz/ai/training/callbacks.py`

Checkpoints:

Model checkpoint management utilities.

get_available_checkpoints

```
get_available_checkpoints(checkpoints_dir)
```

Get all available checkpoint files in the directory.

Parameters:

Name	Type	Description	Default
`checkpoints_dir`	`Path`	Directory containing checkpoint files	*required*

Returns:

Type	Description
`list[Path]`	List of checkpoint paths sorted by priority (best, final, last, then others by name)

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

has_trained_model

```
has_trained_model(project)
```

Check if the project has any trained model checkpoints.

Parameters:

Name	Type	Description	Default
`project`	`Project`	Project configuration	*required*

Returns:

Type	Description
`bool`	True if at least one checkpoint file exists, False otherwise

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

select_checkpoint

```
select_checkpoint(checkpoints_dir)
```

Select the best available checkpoint file in order of preference: 1) best.ckpt 2) last.ckpt 3) final.ckpt.

Parameters:

Name	Type	Description	Default
`checkpoints_dir`	`Path`	Directory containing checkpoint files	*required*

Returns:

Type	Description
`Path | None`	Path to selected checkpoint or None if none found

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

peek_checkpoint_backbone

```
peek_checkpoint_backbone(checkpoint_path)
```

Read the backbone name stored in a checkpoint without loading the full model.

Returns None if the backbone cannot be determined (legacy/corrupt checkpoint).

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

get_checkpoint_compatibility

```
get_checkpoint_compatibility(project)
```

Return (checkpoint_backbone, project_backbone) if they mismatch, else None.

Returns None when there is no trained model or when backbones match.

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

load_model_from_checkpoint

```
load_model_from_checkpoint(
    project,
    lr,
    checkpoint_name=None,
    *,
    on_incompatible="original",
)
```

Load a CrackzModel from the project's checkpoint directory.

Parameters:

Name	Type	Description	Default
`project`	`Project`	Project configuration	*required*
`lr`	`float`	Learning rate for the model	*required*
`checkpoint_name`	`str | None`	Optional checkpoint name (e.g., 'best', 'last', 'epoch-0.9234'). If None, uses automatic selection (best > last > final). Can be specified with or without the '.ckpt' extension.	`None`

Name	Type	Description	Default
`on_incompatible`	`IncompatibleMode`	What to do when the checkpoint architecture mismatches: `"original"` (default) - load the checkpoint using its original backbone `"untrained"` - return a fresh untrained model with the project backbone `"raise"` - raise :class: `CheckpointIncompatibleError`	`'original'`

Returns:

Type	Description
`CrackzModel`	Loaded model or new untrained model if no checkpoint found

Raises:

Type	Description
`CheckpointIncompatibleError`	When checkpoint backbone differs from project backbone and *on_incompatible* is `"raise"`.

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

create_checkpoint_callback

```
create_checkpoint_callback(project)
```

Create a ModelCheckpoint callback configured to save the best model (by val_iou) and the last checkpoint into the project's checkpoints directory.

Uses project.ai.training.save_top_k and save_last configuration.

Parameters:

Name	Type	Description	Default
`project`	`Project`	Project configuration	*required*

Returns:

Type	Description
`ModelCheckpoint`	Configured ModelCheckpoint callback

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

prune_checkpoints

```
prune_checkpoints(project, keep)
```

Prune older checkpoints, keeping only the top N based on a scoring heuristic.

Scoring heuristic: - best.ckpt: score 2.0 (highest priority) - final.ckpt: score 1.5 - last.ckpt: score 0.5 - Other checkpoints: score 0.0 (lowest priority)

Parameters:

Name	Type	Description	Default
`project`	`Project`	Project containing checkpoints directory	*required*
`keep`	`int`	Number of checkpoints to keep	*required*

Returns:

Type	Description
`list[Path]`	List of removed checkpoint paths

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

Visualization:

Visualization utilities for Crackz AI.

This package contains utilities for plotting, visualization, and graphical output of detection results and metrics.

plot_image

```
plot_image(img_path, out_file, pred_mask)
```

Plot original image, predicted mask overlay, and predicted mask side by side.

▶ Source code in `core/src/crackz/ai/visualization/visualization.py`

save_detection_figure

```
save_detection_figure(fig, out_file)
```

Save a Matplotlib figure to disk and close it safely.

▶ Source code in `core/src/crackz/ai/visualization/visualization.py`

save_metrics_json

```
save_metrics_json(results, output_dir)
```

Save metrics to a JSON file in the project's artifacts directory.

▶ Source code in `core/src/crackz/ai/visualization/visualization.py`

save_training_metrics_json

```
save_training_metrics_json(results, output_dir)
```

Save metrics to a JSON file in the project's artifacts directory.

▶ Source code in `core/src/crackz/ai/visualization/visualization.py`

show_image

```
show_image(ax, index, title, img, alpha=None, cmap='gray')
```

Show an image on a matplotlib axis.

▶ Source code in `core/src/crackz/ai/visualization/visualization.py`

to_numpy_img

```
to_numpy_img(tensor)
```

Convert a CHW tensor [0,1] to HWC numpy image for plotting.

▶ Source code in `core/src/crackz/ai/visualization/visualization.py`

Device management:

Device management and hardware utilities for Crackz AI.

This package contains utilities for device detection, hardware acceleration, and device-specific operations.

AIDevice

Bases: `Enum`

Available AI devices.

▶ Source code in `core/src/crackz/ai/device/device.py`

available_devices `staticmethod`

```
available_devices()
```

Return a list of available devices on this system.

▶ Source code in `core/src/crackz/ai/device/device.py`

default `staticmethod`

```
default()
```

Return best available device, preferring GPU > MPS > CPU.

▶ Source code in `core/src/crackz/ai/device/device.py`

test_device `staticmethod`

```
test_device(device_name)
```

Test if a device is available and working.

Parameters:

Name	Type	Description	Default
`device_name`	`str`	Device name to test ('cpu', 'cuda', 'mps', etc.)	*required*

Returns:

Type	Description
`bool`	Tuple of (success, message, details_dict) where:
`str`	• success: True if device works
`dict[str, str | None]`	• message: Human-readable result message
`tuple[bool, str, dict[str, str | None]]`	• details_dict: Additional device information

▶ Source code in `core/src/crackz/ai/device/device.py`

get_accelerator

```
get_accelerator()
```

Determine the best accelerator available for PyTorch Lightning.

Returns a string suitable for PyTorch Lightning's accelerator argument.

▶ Source code in `core/src/crackz/ai/device/utils.py`

resolve_device_and_accelerator

```
resolve_device_and_accelerator(project)
```

Helper to resolve device from project config or auto-detect.

Parameters:

Name	Type	Description	Default
`project`	`Project`	Project instance containing device configuration	*required*

Returns:

Type	Description
`str`	Device string (e.g., 'cpu', 'cuda', 'mps')

▶ Source code in `core/src/crackz/ai/device/utils.py`

General utilities:

Utility package for Crackz AI - DEPRECATED.

This package has been broken up into more logical top-level packages: - crackz.ai.device: Device management and hardware utilities - crackz.ai.performance: Performance optimization and profiling utilities - crackz.ai.visualization: Plotting and visualization utilities - crackz.ai.pipeline: Workflow orchestration and cancellation support - crackz.ai.compat: Compatibility utilities for different environments - crackz.ai.core: Core types and protocols - crackz.ai.exceptions: AI-related exceptions

Please import directly from these packages instead of using crackz.ai.util.

Examples:

from crackz.ai.device import AIDevice, resolve_device_and_accelerator from crackz.ai.performance import RuntimeTracker, set_global_seed from crackz.ai.core import CrackzBatch, ProgressReporter

Exceptions:

Exceptions related to AI processing and pipeline errors.

This module defines custom exceptions for handling errors within the AI processing pipeline. It includes a hierarchy of exceptions that allow for more nuanced error handling in applications leveraging AI-related functionalities.

Classes:

Name	Description
`AIPipelineError`	Base class for AI-related exceptions.
`ModelNotFoundError`	Exception raised when a specified AI model cannot be found.
`DataLoadingError`	Exception raised when an error occurs during the data loading process.

DetectionError

Bases: `CrackzError`

Base class for Detection-related exceptions.

▶ Source code in `core/src/crackz/ai/exceptions.py`

AIPipelineError

Bases: `CrackzError`

Base class for AI-related exceptions.

▶ Source code in `core/src/crackz/ai/exceptions.py`

ModelNotFoundError

Bases: `AIPipelineError`

Raised when a specified AI model cannot be found.

▶ Source code in `core/src/crackz/ai/exceptions.py`

DataLoadingError

Bases: `AIPipelineError`

Raised when there is an error loading data for AI processing.

▶ Source code in `core/src/crackz/ai/exceptions.py`

CheckpointIncompatibleError

Bases: `AIPipelineError`

Raised when a checkpoint was trained with a different model architecture.

▶ Source code in `core/src/crackz/ai/exceptions.py`

CheckpointTaxonomyMismatchError

Bases: `AIPipelineError`

Raised when a checkpoint taxonomy does not match the project annotation schema.

▶ Source code in `core/src/crackz/ai/exceptions.py`

ImageLoadingError

Bases: `AIPipelineError`

Raised when there is an error loading images.

▶ Source code in `core/src/crackz/ai/exceptions.py`

Detection Results (Scaffold)

Typed schema + loader for `detection_results.json`.

Detection results schema and loader utilities.

Minimal scaffold providing a typed representation of detection results and helper functions to load them from the project DB, with legacy JSON fallback.

▼ Intended usage

from crackz.ai.results import load_detection_results results = load_detection_results(project)

▼ The legacy JSON structure is a list of dicts like

{ "image": "batch0_img0_detect.png", "batch": 0, "index": 0, "threshold": 0.50 }

Note: IoU is optional and only present when ground truth masks are available.

Future extensions (not implemented here but easy to add): - Add timestamp, precision/recall, per-pixel stats. - Provide aggregation helpers or integrate with `crackz.ai.eval`.

DetectionResult

Bases: `BaseModel`

Typed representation of one detection result entry.

This is a Pydantic BaseModel to provide runtime validation, type coercion, and consistent data handling throughout the application.

▶ Source code in `core/src/crackz/ai/detection/results.py`

load_detection_results

```
load_detection_results(project_or_path, *, strict=False)
```

Load detection results from project DB, falling back to legacy JSON.

Parameters

project_or_path: Project | Path | str Either a project instance or path to directory / file. strict: bool If True, raise ValidationError on the first invalid row. If False, skip invalid rows (logging a warning).

▶ Source code in `core/src/crackz/ai/detection/results.py`

result_has_finding

```
result_has_finding(result)
```

Return whether a result contains any finding.

Prefer the generic field when available, but keep crack-only results working.

▶ Source code in `core/src/crackz/ai/detection/results.py`

result_present_categories

```
result_present_categories(result)
```

Return all present categories for a result with crack-only fallback.

▶ Source code in `core/src/crackz/ai/detection/results.py`

result_primary_category

```
result_primary_category(result)
```

Return the primary category for a result.

▶ Source code in `core/src/crackz/ai/detection/results.py`

result_area_pct

```
result_area_pct(result)
```

Return the most relevant area percentage for a result.

▶ Source code in `core/src/crackz/ai/detection/results.py`

result_category_label

```
result_category_label(category)
```

Format a category key for UI/report display.

▶ Source code in `core/src/crackz/ai/detection/results.py`

iter_iou

```
iter_iou(results)
```

Yield IoU values from a sequence of DetectionResult objects.

Only yields IoU values that are not None (i.e., when ground truth was available).

▶ Source code in `core/src/crackz/ai/detection/results.py`

Evaluation Helpers (Scaffold)

Basic aggregation utilities (mean IoU, counts) – extend for precision/recall.

Evaluation helpers (minimal scaffold).

Provides utility functions to compute aggregated statistics from `DetectionResult` entries. Designed to be lightweight and easily extensible later (precision, recall, F1, confusion metrics, etc.).

aggregate_iou

```
aggregate_iou(results)
```

Return mean IoU over all results (0 if empty).

▶ Source code in `core/src/crackz/ai/detection/eval.py`

basic_summary

```
basic_summary(results)
```

Return a minimal summary dict.

Currently: count + mean_iou. Extend with median, std, percentiles later.

▶ Source code in `core/src/crackz/ai/detection/eval.py`

Checkpoint Utilities (Scaffold)

List, select, and prune checkpoints.

Model checkpoint management utilities.

get_available_checkpoints

```
get_available_checkpoints(checkpoints_dir)
```

Get all available checkpoint files in the directory.

Parameters:

Name	Type	Description	Default
`checkpoints_dir`	`Path`	Directory containing checkpoint files	*required*

Returns:

Type	Description
`list[Path]`	List of checkpoint paths sorted by priority (best, final, last, then others by name)

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

has_trained_model

```
has_trained_model(project)
```

Check if the project has any trained model checkpoints.

Parameters:

Name	Type	Description	Default
`project`	`Project`	Project configuration	*required*

Returns:

Type	Description
`bool`	True if at least one checkpoint file exists, False otherwise

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

select_checkpoint

```
select_checkpoint(checkpoints_dir)
```

Select the best available checkpoint file in order of preference: 1) best.ckpt 2) last.ckpt 3) final.ckpt.

Parameters:

Name	Type	Description	Default
`checkpoints_dir`	`Path`	Directory containing checkpoint files	*required*

Returns:

Type	Description
`Path | None`	Path to selected checkpoint or None if none found

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

peek_checkpoint_backbone

```
peek_checkpoint_backbone(checkpoint_path)
```

Read the backbone name stored in a checkpoint without loading the full model.

Returns None if the backbone cannot be determined (legacy/corrupt checkpoint).

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

get_checkpoint_compatibility

```
get_checkpoint_compatibility(project)
```

Return (checkpoint_backbone, project_backbone) if they mismatch, else None.

Returns None when there is no trained model or when backbones match.

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

load_model_from_checkpoint

```
load_model_from_checkpoint(
    project,
    lr,
    checkpoint_name=None,
    *,
    on_incompatible="original",
)
```

Load a CrackzModel from the project's checkpoint directory.

Parameters:

Name	Type	Description	Default
`project`	`Project`	Project configuration	*required*
`lr`	`float`	Learning rate for the model	*required*

Name	Type	Description	Default
`checkpoint_name`	`str | None`	Optional checkpoint name (e.g., 'best', 'last', 'epoch-0.9234'). If None, uses automatic selection (best > last > final). Can be specified with or without the '.ckpt' extension.	`None`
`on_incompatible`	`IncompatibleMode`	What to do when the checkpoint architecture mismatches: `"original"` (default) - load the checkpoint using its original backbone `"untrained"` - return a fresh untrained model with the project backbone `"raise"` - raise :class:`CheckpointIncompatibleError`	`'original'`

Returns:

Type	Description
`CrackzModel`	Loaded model or new untrained model if no checkpoint found

Raises:

Type	Description
`CheckpointIncompatibleError`	When checkpoint backbone differs from project backbone and *on_incompatible* is `"raise"`.

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

create_checkpoint_callback

```
create_checkpoint_callback(project)
```

Create a ModelCheckpoint callback configured to save the best model (by val_iou) and the last checkpoint into the project's checkpoints directory.

Uses project.ai.training.save_top_k and save_last configuration.

Parameters:

Name	Type	Description	Default
`project`	`Project`	Project configuration	*required*

Returns:

Type	Description
`ModelCheckpoint`	Configured ModelCheckpoint callback

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

prune_checkpoints

```
prune_checkpoints(project, keep)
```

Prune older checkpoints, keeping only the top N based on a scoring heuristic.

Scoring heuristic: - best.ckpt: score 2.0 (highest priority) - final.ckpt: score 1.5 - last.ckpt: score 0.5 - Other checkpoints: score 0.0 (lowest priority)

Parameters:

Name	Type	Description	Default
`project`	`Project`	Project containing checkpoints directory	*required*
`keep`	`int`	Number of checkpoints to keep	*required*

Returns:

Type	Description
`list[Path]`	List of removed checkpoint paths

▶ Source code in `core/src/crackz/ai/model/checkpoints.py`

Pipeline Orchestrator (Scaffold)

Minimal helper to run train/detect sequences programmatically.

Pipeline orchestration and workflow utilities for Crackz AI.

This package contains utilities for workflow orchestration, cancellation support, and pipeline management.

CancellationToken

Thread-safe cancellation token for AI workflows.

This class allows callers to request cancellation of long-running operations and workers to check if cancellation has been requested.

▶ Source code in `core/src/crackz/ai/pipeline/cancellation.py`

__init__

```
__init__()
```

Initialize a new cancellation token.

▶ Source code in `core/src/crackz/ai/pipeline/cancellation.py`

cancel

```
cancel()
```

Request cancellation of the operation.

▶ Source code in `core/src/crackz/ai/pipeline/cancellation.py`

is_cancelled

```
is_cancelled()
```

Check if cancellation has been requested.

Returns:

Type	Description
`bool`	True if cancellation has been requested, False otherwise.

▶ Source code in `core/src/crackz/ai/pipeline/cancellation.py`

check_cancelled

```
check_cancelled()
```

Check if cancelled and raise exception if so.

Raises:

Type	Description
`CancellationError`	If cancellation has been requested.

▶ Source code in `core/src/crackz/ai/pipeline/cancellation.py`

Logging

Centralized logging helper & convenience setup.

Logging utilities for the Crackz project.

Integrates loguru with Python's stdlib logging, so 3rd-party libs work. Uses Rich for beautiful console output with colorized logs and tracebacks, while keeping plain text file logs for production parsing.

InterceptHandler

Bases: `Handler`

Redirect standard logging records to loguru.

▶ Source code in `core/src/crackz/log/crackz_logger.py`

emit

```
emit(record)
```

Emit a record.

▶ Source code in `core/src/crackz/log/crackz_logger.py`

use_rich_console

```
use_rich_console()
```

Determine if Rich console logging should be used.

Returns False in CI environments or when explicitly disabled via CRACKZ_RICH_LOGGING=0 environment variable.

▶ Source code in `core/src/crackz/log/crackz_logger.py`

setup_logging

```
setup_logging(level=None, log_dir=None, *, use_rich=None)
```

Configure loguru + redirect stdlib logging. Safe for tests & PyInstaller.

Console logging uses Rich for beautiful, colorized output with rich tracebacks. File logging uses plain structured format for production parsing.

Parameters:

Name	Type	Description	Default
`level`	`str | None`	Log level (e.g., "DEBUG", "INFO"). Defaults to settings.log_level.	`None`
`log_dir`	`Path | None`	Directory for log files. Defaults to settings.log_dir.	`None`
`use_rich`	`bool | None`	If True, use Rich handler for console. If None, auto-detect based on environment (disabled in CI). Defaults to None.	`None`

Returns:

Type	Description
`Logger`	Configured loguru Logger instance.

▶ Source code in `core/src/crackz/log/crackz_logger.py`

CLI Entrypoints

(Generated by Typer; prefer CLI docs for usage examples.)

Root CLI entrypoint with lazy command loading.

LazyTyperGroup

Bases: `TyperGroup`

Typer-compatible group that keeps subcommand imports lazy.

▶ Source code in `core/src/crackz/cli/crackz_cli.py`

main

```
main(
    args=None,
    prog_name=None,
    complete_var=None,
    standalone_mode=True,
    windows_expand_args=True,
    **extra,
)
```

Normalize test arguments before Click starts parsing them.

▶ Source code in `core/src/crackz/cli/crackz_cli.py`

list_commands

```
list_commands(ctx)
```

Return static command names without importing subcommand modules.

▶ Source code in `core/src/crackz/cli/crackz_cli.py`

get_command

```
get_command(ctx, cmd_name)
```

Return an eager command or a lightweight lazy stub.

▶ Source code in `core/src/crackz/cli/crackz_cli.py`

format_commands

```
format_commands(ctx, formatter)
```

Render command help without importing lazy subcommands.

▶ Source code in `core/src/crackz/cli/crackz_cli.py`

root_callback

```
root_callback(
    ctx,
    version=False,
    debug=False,
    quiet=False,
    log_dir=None,
    telemetry=False,
)
```

Main entrypoint for Crackz CLI.

▶ Source code in `core/src/crackz/cli/crackz_cli.py`

main

```
main()
```

Execute the main application entry point.

▶ Source code in `core/src/crackz/cli/crackz_cli.py`

A module for initializing and managing project configurations.

This module provides Typer CLI commands for initializing and exporting project configurations. It saves configs as YAML and ensures paths (artifacts, images, logs, etc.) are resolved consistently.

Log directory behavior: - If user provides a log dir inside the project root stored as relative in YAML. - If external stored absolute.

init_project

```
init_project(
    name=NAME_OPTION,
    image_path=IMAGE_PATH_OPTION,
    image_mask_path=IMAGE_MASK_PATH_OPTION,
    artifacts_path=ARTIFACTS_PATH_OPTION,
    description=DESCRIPTION_OPTION,
    category_template=CATEGORY_TEMPLATE_OPTION,
    task_mode=TASK_MODE_OPTION,
    log_dir=LOG_DIR_OPTION,
    *,
    force=FORCE_OPTION,
)
```

Initialize a new Crackz project directory and save its config.

▶ Source code in `core/src/crackz/cli/project_cli.py`

export_project_config

```
export_project_config(
    cfg=CFG_OPTION,
    project_path=PROJECT_PATH_OPTION,
    out=EXPORT_OUTPUT_OPTION,
)
```

Export a unified ProjectConfig YAML derived from an existing project or config file.

▶ Source code in `core/src/crackz/cli/project_cli.py`

migrate_project_config

```
migrate_project_config(
    cfg=CFG_OPTION,
    project_path=PROJECT_PATH_OPTION,
    *,
    dry_run=DRY_RUN_OPTION,
    no_backup=NO_BACKUP_OPTION,
    migrate_files=typer.Option(
        default=False,
        help="Also migrate artifacts directory structure (checkpoints to models, etc.)",
    ),
)
```

Migrate a project configuration to the current schema version.

Automatically upgrades legacy YAML (no schema_version) by adding missing fields with safe defaults (e.g. ai.training.tensorboard) and stamping schema_version.

Optionally migrates the artifacts directory structure from v2 to v3: - checkpoints/ to models/ - training/tensorboard/ to runtime/tensorboard/ - training/lightning_logs/ to runtime/lightning_logs/ - detections/*.png to detections/visualizations/

▶ Source code in `core/src/crackz/cli/project_cli.py`

clean_project

```
clean_project(
    project_path=PROJECT_PATH_OPTION,
    *,
    dry_run=DRY_RUN_OPTION,
    logs_only=LOGS_ONLY_OPTION,
    keep_best_model=KEEP_BEST_MODEL_OPTION,
)
```

Clean temporary files and logs from a project.

By default, cleans logs, training artifacts, checkpoints, and detections. Images and masks are always preserved. The best model checkpoint is preserved by default.

▶ Source code in `core/src/crackz/cli/project_cli.py`

Management module for importing datasets and images into specified projects.

This module provides functionality to import datasets or images into project directories or YAML configuration-based setups. It offers options for specifying image type, size, quality, and other parameters to streamline the importing process. Errors during the process are logged and handled properly.

validate_group_options

```
validate_group_options(*, split, group_by, group_size)
```

Reject inconsistent --group-by / --group-size / --split combinations.

Raises `typer.BadParameter` (usage error, exit code 2) so a bad combination is distinguishable from an import failure (exit code 1) without parsing any output.

▶ Source code in `core/src/crackz/cli/image_import_cli.py`

import_images

```
import_images(
    cfg=CFG_OPTION,
    project_path=PROJECT_PATH_OPTION,
    source_path=SOURCE_PATH_OPTION,
    masks_source_path=MASKS_SOURCE_PATH_OPTION,
    image type=TYPE OPTION,
    image_size=SIZE_OPTION,
    quality=QUALITY_OPTION,
    target_size=TARGET_SIZE_OPTION,
    *,
    overwrite=OVERWRITE_OPTION,
    split=SPLIT_OPTION,
    train ratio=TRAIN RATIO OPTION,
    val_ratio=VAL_RATIO_OPTION,
    test_ratio=TEST_RATIO_OPTION,
    random_seed=RANDOM_SEED_OPTION,
    group_by=GROUP_BY_OPTION,
    group_size=GROUP_SIZE_OPTION,
    mask_label_map=MASK_LABEL_MAP_OPTION,
)
```

Import images into the specified project with optional configurations.

This function imports images from a given source path to a project or a configuration file, allowing additional parameters like image type, size, and quality. It supports progress display for tracking the import process. Errors during import are logged and appropriately handled.

Parameters:

Name	Type	Description	Default
`cfg`	`Path | None`	Path | None The configuration file path (optional).	`CFG_OPTION`
`project_path`	`Path | None`	Path | None The path to the project directory (optional).	`PROJECT_PATH_OPTION`
`source_path`	`Path`	Path The directory path containing the source images to be imported.	`SOURCE_PATH_OPTION`
`masks_source_path`	`Path | None`	Path The directory path containing the source for image masks to be imported. Expected to be in the same format as `source_path` with same filenames but with suffix .png	`MASKS_SOURCE_PATH_OPTION`
`image_type`	`ImagesType`	ImagesType The type/category of the images to import (ignored if split=True).	`TYPE_OPTION`
`image_size`	`SizeType`	tuple[int, int] Tuple specifying the width and height to resize the images.	`SIZE_OPTION`
`quality`	`int`	int Compression quality level for the images.	`QUALITY_OPTION`
`target_size`	`int | None`	int | None Square resolution (NxN) to resize images to during import (64-4096). When explicitly provided, it overrides the legacy image_size option	`TARGET_SIZE_OPTION`

Name	Type	Description	Default
		and is persisted in project config so training and detection use the same resolution.	
`overwrite`	`bool`	bool Flag indicating whether to overwrite existing images if present.	`OVERWRITE_OPTION`
`split`	`bool`	bool Flag to enable automatic splitting of images into train/val/test sets.	`SPLIT_OPTION`
`train_ratio`	`float`	float Ratio of images for training set when using split (default: 0.7).	`TRAIN_RATIO_OPTION`
`val_ratio`	`float`	float Ratio of images for validation set when using split (default: 0.15).	`VAL_RATIO_OPTION`
`test_ratio`	`float`	float Ratio of images for test set when using split (default: 0.15).	`TEST_RATIO_OPTION`
`random_seed`	`int | None`	int | None Random seed for reproducible splits (default: None for random).	`RANDOM_SEED_OPTION`
`group_by`	`GroupBy`	GroupBy Keep images of one group in the same split (requires split).	`GROUP_BY_OPTION`
`group_size`	`int | None`	int | None Files per block for group_by=sequence-block.	`GROUP_SIZE_OPTION`
`mask_label_map`	`str | None`	str | None Optional source-id to project-id remap for imported mask labels.	`MASK_LABEL_MAP_OPTION`

Raises:

Type	Description
`BadParameter`	Raised if neither configuration nor project path is provided.
`Exit`	Raised to handle any unexpected exceptions during the process.

Returns:

Type	Description
`None`	None

▶ Source code in `core/src/crackz/cli/image_import_cli.py`

This module provides functionality to run detection or training pipelines based on a specified YAML configuration file.

It includes commands to execute both detection and training processes, leveraging the typer CLI framework for seamless command-line interaction.

run_detection

```
run_detection(
    cfg=CFG_OPTION,
    project_path=PROJECT_PATH_OPTION,
    model=CHECKPOINT_NAME_OPTION,
    embed=typer.Option(
        False,
        "--embed/--no-embed",
        help="After detection completes, backfill defect embeddings for the new results. Requires a reachable Postgres DB (CRACKZ_DATABASE_URL). Embedding failures are warned
    ),
)
```

Run the detection process based on the given configuration or project path.

▶ Source code in `core/src/crackz/cli/detection_cli.py`

detection_train

```
detection_train(
    cfg=CFG_OPTION,
    project_path=PROJECT_PATH_OPTION,
    epochs=EPOCHS_OPTION,
    batch_size=BATCH_SIZE_OPTION,
    resume=RESUME_FROM_OPTION,
)
```

Executes the training phase of a detection model based on the provided configuration or project path.

This function initiates the training of a detection model by loading the appropriate configuration file or project path. At least one of the two options must be specified. It logs the training process and properly handles exceptions encountered during execution.

Parameters:

Name	Type	Description	Default
`cfg`	`Path | None`	Optional path to the configuration file defining the training parameters. Can be None if `project_path` is provided.	`CFG_OPTION`
`project_path`	`Path | None`	Optional path to an existing project directory. Can be None if `cfg` is provided.	`PROJECT_PATH_OPTION`
`epochs`	`int | None`	Optional override for training epochs.	`EPOCHS_OPTION`
`batch_size`	`int | None`	Optional override for training batch size.	`BATCH_SIZE_OPTION`
`resume`	`str | None`	Optional checkpoint name to resume training from.	`RESUME_FROM_OPTION`

Raises:

Type	Description
`BadParameter`	If neither `cfg` nor `project_path` is specified.
`Exit`	If an error occurs during the execution of the training process.

▶ Source code in `core/src/crackz/cli/detection_cli.py`

prune_checkpoints_cmd

```
prune_checkpoints_cmd(
    keep=KEEP_CHECKPOINTS_OPTION,
    cfg=CFG_OPTION,
    project_path=PROJECT_PATH_OPTION,
)
```

Prune older checkpoints keeping only the newest/best N (heuristic).

▶ Source code in `core/src/crackz/cli/detection_cli.py`

Extensibility Points

Area	How to Extend	Notes
Models	Create a new model class similar to `CrackzModel`	Register via factory (future hook)
Datasets	Wrap or subclass `CrackzDataset`	Maintain (image, mask, path) structure
Callbacks	Add PyTorch Lightning callbacks	Compose with existing metrics/progress callbacks
Metrics	Extend `MetricsLoggerCallback` or post-process JSON metrics	Metrics saved as JSON in artifacts
Device selection	Modify `resolve_device_and_accelerator`	Could add multi-GPU strategies
Results schema	Extend `DetectionResult` model	Add new fields for downstream analytics
Pipeline	Subclass `Pipeline` or add steps	Build higher-level automation workflows

---

Suggested Future Additions (You Can Add to This Page)

Below is a curated list of improvements you might add to make this API section richer and more maintainable:

1. Result Data Schemas
2. Document JSON layout produced by detection (fields: image, batch, index, iou, threshold).
3. Provide a pydantic model for typed loading (e.g. `DetectionResult`).
4. Metrics & Evaluation
5. Add an evaluation module (`evaluation.py`) to compute aggregated IoU / precision / recall.
6. Publish its API here with examples.
7. Custom Threshold Strategies
8. Show how to compute per-image thresholds or dynamic threshold tuning.
9. Checkpoint Management
10. Add functions to list, promote, or prune checkpoints; surface in docs.
11. Batch Export Helpers
12. Utility to export masks/predictions to GeoJSON or COCO-style formats.
13. Callback Cookbook
14. Examples: early stopping, LR scheduling, rich progress console.
15. Device & Performance Tuning
16. Section on enabling mixed precision, gradient accumulation, multi-GPU (DDP) strategy.
17. Configuration Layer
18. Provide a unified `ProjectConfig` pydantic model referencing AI + paths; load/save YAML round-trip.
19. Error Handling Patterns
20. Table mapping common exceptions to remediation steps.
21. Programmatic Pipeline Object
22. A `Pipeline` class orchestrating import → train → detect → report for scripting.

Minimal Detection Flow Diagram (Textual)

```
Project -> load_model_from_checkpoint -> CrackzModel.eval() -> DataLoader -> forward -> sigmoid -> threshold -> mask -> metrics(json)
```

Versioning Notes

The public API may evolve (alpha status). Pin a version for reproducibility (e.g. in `requirements.txt` or lockfile). Breaking changes will be noted in the CHANGELOG.

Feedback

Found gaps or want new hooks? Open an issue or PR with a short example of the extension you need.

Async Callbacks

Async Callback Architecture

Crackz now includes an asynchronous callback infrastructure that provides Go channel-like patterns for non-blocking communication between AI operations and UI updates.

Overview

The async callback architecture solves a key performance issue: synchronous callbacks can slow down training/detection when UI updates take time to process. By using asyncio queues, we decouple AI operations from UI updates, achieving:

- **Non-blocking updates**: Training/detection doesn't wait for UI rendering
- **Backpressure handling**: Queues automatically manage burst updates
- **Thread-safe communication**: Queue-based design works seamlessly with threading
- **Backward compatibility**: Existing synchronous callbacks continue to work

Key Components

1. CallbackDispatcher (Using Janus Queues)

The core component that bridges synchronous callbacks to async consumers using the janus library:

```
from crackz.ai.core.async_callbacks import CallbackDispatcher

dispatcher = CallbackDispatcher(maxsize=100)

# Create sync reporter for PyTorch Lightning (training thread)
progress_reporter = dispatcher.create_sync_progress_reporter()

# Use in training (synchronous context)
train(project, progress_reporter=progress_reporter)

# Consume updates asynchronously (main thread)
async for progress, message in dispatcher.iter_progress():
    update_progress_bar(progress, message)
```

Features: - **Janus queues**: Provides both sync and async interfaces to same queue - **Thread-safe**: Safe sync puts from training thread, async gets from main thread - **Backpressure handling**: Drops oldest items when queue is full - **Statistics tracking**: Monitor queue performance with built-in stats - **Graceful shutdown**: Proper cleanup with sentinel values

Why Janus? - Proper synchronization between sync/async worlds - Eliminates race conditions from manual bridging - Industry-standard solution for sync/async queue bridging

2. Async Consumers

Helper functions to create async consumers that process queue updates:

```
from crackz.ai.core.async_callbacks import (
    create_async_progress_consumer,
    create_async_metrics_consumer,
)

# Define callback
def update_ui(progress: float, message: str | None = None) -> None:
    progress_bar.set_value(progress)

# Create async consumer
consumer = create_async_progress_consumer(dispatcher, update_ui)

# Run in async context
await consumer()
```

3. GUI Async Helpers

High-level utilities for GUI integration:

```
from crackz_gui.qt.async_bridge import (
    start_gui_callback_consumers,
    stop_gui_callback_consumers,
)

# Start consumers (manages event loop)
loop, tasks = start_gui_callback_consumers(
    dispatcher,
    progress_callback=update_progress,
    metrics_callback=update_metrics,
)

# ... run training ...
```

```
# Cleanup
stop_gui_callback_consumers(dispatcher, loop)
```

ℹ️ GUI Thread Safety with PySide6

PySide6's signal/slot mechanism is inherently thread-safe — signals emitted from any thread are automatically queued and delivered on the GUI thread. Use `TaskSignals` from `async_bridge.py` for background work:

✅ CORRECT - Qt Signals (Thread-Safe)

```
from crackz_gui.qt.async_bridge import TaskSignals, run_in_background

def _do_work(signals: TaskSignals) -> None:
    signals.progress.emit(0.5)   # Safe from any thread
    signals.status.emit("Processing...")
    result = expensive_operation()
    signals.data.emit(result)
    signals.finished.emit()

signals = run in background( do work)
signals.progress.connect(self._on_progress)
signals.finished.connect(self._on_complete)
signals.error.connect(self._on_error)
```

No manual `threading.Lock` or `after()` wrappers are needed — Qt handles cross-thread dispatch automatically.

> For the PySide6 GUI, prefer the Qt-native helpers in `crackz_gui.qt.async_bridge`.

Usage Patterns

Pattern 1: Synchronous (Backward Compatible)

Existing code continues to work without changes:

```
def progress_callback(progress: float, message: str | None = None) -> None:
    print(f"Progress: {progress:.2%}")

train(project, progress_reporter=progress_callback)
```

Pattern 2: Queue-Based (Intermediate)

Use dispatcher with manual async consumption:

```
dispatcher = CallbackDispatcher(maxsize=100)

# Create sync reporters
progress_reporter = dispatcher.create_sync_progress_reporter()
metrics_reporter = dispatcher.create_sync_metrics_reporter()

# Start training in background
training_thread = threading.Thread(
    target=train,
    args=(project,),
    kwargs={
        "progress_reporter": progress_reporter,
        "metrics_callback": metrics_reporter,
    }
)
training_thread.start()

# Consume updates asynchronously
async def consume_updates():
    async for progress, message in dispatcher.iter_progress():
        print(f"Progress: {progress:.2%} - {message}")

asyncio.run(consume_updates())
```

Pattern 3: Full Async with Thread Safety (GUI - Recommended)

Complete async pattern for GUI applications with automatic thread safety:

```
from crackz.ai.core.async_callbacks import CallbackDispatcher
from crackz_gui.qt.async_bridge import (
    start_gui_callback_consumers,
    stop_gui_callback_consumers,
)

dispatcher = CallbackDispatcher(maxsize=100)

# Start async consumers with Qt-thread-safe GUI updates
loop, tasks, emitters = start_gui_callback_consumers(
    dispatcher,
    progress_callback=self.update_progress_bar,
```

```
    metrics_callback=self.update_metrics_display,
)
self._qt_emitters = emitters  # Keep signal emitters alive

# Create reporters
progress_reporter = dispatcher.create_sync_progress_reporter()
metrics_reporter = dispatcher.create_sync_metrics_reporter()

try:
    # Run training in background thread
    training_thread = threading.Thread(
        target=train,
        args=(project,),
        kwargs={
            "progress_reporter": progress_reporter,
            "metrics_callback": metrics_reporter,
        }
    )
    training_thread.start()
    training_thread.join()
finally:
    stop_gui_callback_consumers(dispatcher, loop)
```

Performance Benefits

Without Async Callbacks (Synchronous)

```
Training Loop:
  Process batch → Update metrics → **Wait for UI** → Next batch
                                    ↑ 1-5ms overhead
```

For 1000 callbacks/epoch: ~4 seconds wasted waiting for UI

With Async Callbacks (Asynchronous)

```
Training Loop:
  Process batch → Put in queue → Next batch (no wait!)
                    ↓ 0.1ms
  Async Consumer: Takes from queue → Updates UI
```

For 1000 callbacks/epoch: ~0.1 seconds overhead (40x faster)

Queue Statistics & Monitoring

The dispatcher provides built-in statistics for monitoring queue health:

```
dispatcher = CallbackDispatcher(maxsize=100)

# ... use reporters ...

# Check progress queue stats
stats = dispatcher.progress_stats
print(f"Total items put: {stats.total_put}")
print(f"Total items retrieved: {stats.total_get}")
print(f"Total items dropped: {stats.total_dropped}")
print(f"Current queue size: {stats.current_size}")
print(f"Max queue size: {stats.max_size}")

# Check metrics queue stats
metrics_stats = dispatcher.metrics_stats
```

Use statistics to: - Monitor queue backpressure (check `total_dropped`) - Tune queue size based on actual usage - Identify performance bottlenecks - Debug callback issues

Thread Safety Architecture

The async callback infrastructure is thread-safe by design:

- **Janus queues**: Provides thread-safe `sync_q` for puts, async-safe `async_q` for gets
- **Event loop**: Runs in a dedicated thread with proper synchronization via `threading.Event`
- **GUI callbacks**: Routed through Qt signals so delivery is queued onto the GUI thread automatically
- **No busy waits**: Proper event-based synchronization eliminates race conditions

Thread Flow:

```
Training Thread          Async Event Loop Thread     GUI Main Thread
    |                           |                        |
    ├─ sync_q.put() ──────────>|                        |
    |  (thread-safe)            |                        |
    |                           ├─ async_q.get()         |
    |                           |                        |
```

```
    |                          ├─ Qt signal emit ──────>|
    |                          |                        |─ Update UI
```

This architecture ensures: - Training never blocks on UI updates - UI updates always happen on the correct thread - No race conditions or deadlocks - Proper cleanup and shutdown

Migration Guide

For Existing Code

No changes required! Existing synchronous callbacks continue to work:

```
# Before (still works)
train(project, progress_reporter=my_callback)

# After (optional upgrade for better performance)
dispatcher = CallbackDispatcher()
train(project, progress_reporter=dispatcher.create_sync_progress_reporter())
```

For New Code

Use the async pattern from the start:

```
from crackz.ai.core.async_callbacks import CallbackDispatcher
from crackz_gui.qt.async_bridge import start_gui_callback_consumers

dispatcher = CallbackDispatcher(maxsize=100)
loop, tasks = start_gui_callback_consumers(
    dispatcher,
    progress_callback=update_progress,
    metrics_callback=update_metrics,
)

# Use dispatcher's sync reporters in training
train(
    project,
    progress_reporter=dispatcher.create_sync_progress_reporter(),
    metrics_callback=dispatcher.create_sync_metrics_reporter(),
)

stop_gui_callback_consumers(dispatcher, loop)
```

API Reference

See the module docstrings for detailed API documentation:

Core Components (`crackz.ai.core.async_callbacks`)

- `CallbackDispatcher` - Main sync-to-async bridge using janus queues
- `QueueStats` - Statistics dataclass for monitoring
- `create_async_progress_consumer()` - Create progress consumer function
- `create_async_metrics_consumer()` - Create metrics consumer function

GUI Helpers (`crackz_gui.qt.async_bridge`)

- `AsyncEventLoop` - Event loop manager with proper synchronization
- `create_threadsafe_gui_callback()` - Wrap callback for thread-safe GUI updates
- `create_threadsafe_metrics_callback()` - Wrap metrics callback for thread safety
- `start_gui_callback_consumers()` - High-level helper to start consumers
- `stop_gui_callback_consumers()` - Clean shutdown of consumers

Use `help()` in Python to view full API documentation:

```
from crackz.ai.core.async_callbacks import CallbackDispatcher
help(CallbackDispatcher)
```

Examples

Complete examples are available in docs/examples/async_callbacks_example.py.

See Also

- Python asyncio documentation
- Go channels
- Threading in Python

MCP Integration

MCP (Model Context Protocol) Integration

Crackz includes built-in support for the Model Context Protocol (MCP), which enables AI assistants to interact with development tools and services in a standardized way using **code execution** for maximum efficiency.

Quick Facts

- **MCP Servers Configured**: 4 (filesystem, github, python, uv)
- **Who Uses MCP**: Developers using the configured Claude Code CLI setup
- **Who Doesn't Use MCP**: End users (GUI chat)
- **Purpose**: Enable Claude Code to efficiently interact with files, GitHub, Python, and package management

  Note: This documentation is for developers setting up MCP tools. End users of the Crackz GUI do not need MCP configuration.

Overview

MCP provides a unified interface for AI assistants to interact with development tools. Crackz is configured with **4 MCP servers**:

1. **filesystem** - File operations with progressive disclosure
2. **github** - GitHub API operations (issues, PRs, releases, workflows)
3. **python** - Code execution environment for composing MCP operations
4. **uv** - Package manager operations

The configuration follows Anthropic's best practices for code execution with MCP, using code to compose MCP operations efficiently and minimize token consumption.

Who Uses MCP?

Claude Code CLI (✅ Uses MCP): - Uses the repo-local MCP setup when configured - Typically run from terminal sessions such as tmux - Shares baseline instructions with Codex through `AGENTS.md`

Codex (tool-specific access): - Uses `AGENTS.md` and `.agents/skills/` - Uses the tools exposed by the Codex runtime rather than this MCP setup

End-User GUI Chat (❌ No MCP): - Built into Crackz GUI for harbor inspectors - Intentionally kept simple without developer tools - No filesystem, GitHub, or code execution access

AI Assistant Comparison

Claude Code CLI	Codex	End-User GUI
✅ 4 MCP Servers • filesystem • github • python • uv	Runtime tools AGENTS.md .agents/skills	❌ No MCP Simple API Project info only
Tool: claude in tmux	Tool: codex	Tool: Crackz GUI

Key Takeaway: MCP is part of the Claude Code CLI development setup. Codex uses the shared repo instructions and its own runtime tools. End-user chat does not use MCP.

Key Concepts

Code Execution with MCP

Instead of making direct tool calls that pass all data through the model's context window, AI assistants write Python code to:

- **Filter data before returning results**: Process large datasets in code, return only relevant information
- **Compose operations efficiently**: Loop over operations without alternating tool calls through the model
- **Load tools on-demand**: Use progressive disclosure to read tool definitions only when needed
- **Maintain state**: Keep intermediate results in the execution environment, not in context

Example - Traditional approach (inefficient):

```
TOOL CALL: filesystem.readDirectory("src/")
→ returns 50 files in context

TOOL CALL: filesystem.readFile("src/file1.py")
→ returns full file in context

TOOL CALL: filesystem.readFile("src/file2.py")
→ returns full file in context
```

Example - Code execution approach (efficient):

```
# Discover and filter in code
files = await filesystem.readDirectory("src/")
python_files = [f for f in files if f.endswith(".py")]

# Load only what's needed
for filename in python_files[:5]:  # Top 5 only
    content = await filesystem.readFile(f"src/{filename}")
    # Process in code, log summary only
    print(f"{filename}: {len(content)} lines")
```

Progressive Disclosure

Load tool definitions on-demand rather than upfront:

1. List directories to discover available modules
2. Read specific files only when needed
3. Search for relevant tools before loading full definitions
4. Keep context lean by loading incrementally

AI Assistant Development Standards

AI assistants working with Crackz MUST follow these conventions:

1. Branch Naming Convention

Format: `<type>/<scope>-<short-slug>` (lowercase, hyphenated)

Examples:

```
feat/gui-live-preview        # New GUI feature
fix/pdf-rendering-crash      # Bug fix in PDF generation
perf/dataloader-cache        # Performance improvement
docs/api-reference-update    # Documentation update
refactor/config-cleanup      # Code refactoring
```

2. Commit Message Format

Format: `<type>(<scope>): <description>` (lowercase type with colon)

Examples:

```
feat(gui): add real-time detection preview
fix(cli): correct path handling in Windows
perf(dataloader): implement batch caching
docs(api): update training configuration reference
```

3. Pull Request Title Format (CRITICAL)

The PR title MUST follow conventional commit format to trigger releases!

Format: `<type>(<scope>): <description>`

✅ **CORRECT** (triggers release):

```
feat(gui): add export panel for detection results
fix(pdf): handle missing hero image gracefully
perf(training): optimize data loading pipeline
```

❌ **INCORRECT** (no release triggered):

```
Fix GUI bugs and add new feature
Update documentation
Add export functionality
```

Release Triggers: - `feat:` or `feat(<scope>):` → MINOR version bump (1.2.0 → 1.3.0) - `fix:` or `fix(<scope>):` → PATCH version bump (1.2.0 → 1.2.1) - `perf:` or `perf(<scope>):` → PATCH version bump (1.2.0 → 1.2.1)

No Release (maintenance): - `docs:` - Documentation changes only - `refactor:` - Code restructuring without behavior changes - `test:` - Test-only changes - `chore:` - Maintenance, dependencies, tooling - `ci:` - CI/CD pipeline changes - `build:` - Build system or

Docker changes

4. Code Quality Standards

CLI Commands: - Use `typer.secho()` with colors for user messages - Use `logger` from `crackz.log` for operational details - Import centralized options from `crackz.cli.options` - Never use `print()` statements

Testing: - Add/update unit tests for all behavior changes - Maintain minimum 75% test coverage - Run tests before committing: `uv run task tests`

Type Safety: - Add type hints to all public interfaces - Run mypy: `uv run task mypy`

See Contributing Guide for complete details.

Quick Start

Initialize MCP Configuration

Create a default MCP configuration file:

```
crackz mcp init
```

This creates `.mcp/config.json` with sensible defaults for the Crackz project.

View Configuration

Display the current MCP configuration:

```
crackz mcp show
```

Validate Configuration

Check that the configuration is valid:

```
crackz mcp validate
```

Configuration Structure

The MCP configuration is a JSON file (`.mcp/config.json`) with servers, metadata, and patterns sections:

MCP Servers (4 Total)

Crackz is configured with **4 MCP servers** available to AI development assistants:

```
{
  "mcpServers": {
    "filesystem": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "."],
      "description": "Filesystem operations with progressive disclosure"
    },
    "github": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-github"],
      "env": {
        "GITHUB_PERSONAL_ACCESS_TOKEN": "${GITHUB_TOKEN}"
      },
      "description": "GitHub operations (issues, PRs, releases)"
    },
    "python": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-python"],
      "description": "Python code execution environment"
    },
    "uv": {
      "type": "stdio",
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-uv"],
      "description": "UV package manager operations"
    }
  }
}
```

Metadata

Provides context about the project:

```
{
  "metadata": {
    "project": "Crackz",
    "language": "python",
    "version": "3.13",
    "package_manager": "uv",
    "framework": "pytorch-lightning",
    "cli": "typer",
    "gui": "pyside6",
    "testing": "pytest",
    "ci": "github-actions",
    "release": "semantic-release",
    "code_execution": true,
    "progressive_disclosure": true
  }
}
```

Patterns

Defines recommended usage patterns:

```
{
  "patterns": {
    "code_execution": {
      "enabled": true,
      "description": "Use code execution to interact with MCP servers efficiently",
      "examples": [
        "Filter large datasets in code before returning results",
        "Loop over operations without alternating tool calls",
        "Compose multiple MCP operations in a single execution"
      ]
    },
    "progressive_disclosure": {
      "enabled": true,
      "description": "Load tool definitions on-demand",
      "workflow": [
        "List directories to discover modules",
        "Read specific files as needed",
        "Load documentation when relevant"
      ]
    }
  }
}
```

Available Commands

crackz mcp init

Initialize a new MCP configuration file.

Options: - `--output` / `-o`: Output path for config file (default: `.mcp/config.json`) - `--force` / `-f`: Overwrite existing config file

Examples:

```
# Create with defaults
crackz mcp init

# Create in custom location
crackz mcp init --output custom/mcp.json

# Overwrite existing config
crackz mcp init --force
```

crackz mcp show

Display the current MCP configuration with syntax highlighting.

Options: - `--config` / `-c`: Path to config file (default: `.mcp/config.json`)

Examples:

```
# Show default config
crackz mcp show

# Show custom config
crackz mcp show --config custom/mcp.json
```

crackz mcp validate

Validate that the MCP configuration is well-formed and follows the expected schema.

Options: - `--config` / `-c`: Path to config file (default: `.mcp/config.json`)

Examples:

```
# Validate default config
crackz mcp validate

# Validate custom config
crackz mcp validate --config custom/mcp.json
```

Customization

Adding Custom Servers

Add your own MCP-compatible servers to `.mcp/config.json`:

```
{
  "mcpServers": {
    "filesystem": { ... },
    "custom-api": {
      "type": "stdio",
      "command": "node",
      "args": ["./custom-mcp-server.js"],
      "description": "Custom MCP server for specialized operations"
    }
  }
}
```

Extending Metadata

Add custom project metadata:

```
{
  "metadata": {
    "project": "Crackz",
    "language": "python",
    "team": "ML Ops",
    "repository": "https://github.com/Anaxiatech-AB/crackz",
    "custom_field": "your_value"
  }
}
```

Integration with AI Assistants

AI assistants that support MCP can use this configuration to:

1. **Execute Python code**: Write code to compose MCP operations efficiently
2. **Use progressive disclosure**: Load tool definitions on-demand to minimize tokens
3. **Filter data in code**: Process large results before passing to context
4. **Understand project context**: Access metadata about language, frameworks, and tools
5. **Maintain state**: Keep intermediate results in execution environment

Code Execution Best Practices

DO: - Write Python code to filter and transform data before returning results - Use loops and conditionals in code rather than chaining tool calls - Load only the files and tools you need for the current task - Keep intermediate data in the execution environment

DON'T: - Pass large datasets through the model context unnecessarily - Load all tool definitions upfront - Make sequential tool calls when a loop would be more efficient - Return raw data when a summary would suffice

Example Workflows

Efficient file discovery:

```
# List Python files in src/
files = await filesystem.readDirectory("src/crackz")
py_files = [f for f in files if f.endswith(".py")]

# Read only the first 3 for initial analysis
for filename in py_files[:3]:
    content = await filesystem.readFile(f"core/src/crackz/{filename}")
    lines = len(content.split('\n'))
    print(f"{filename}: {lines} lines")
```

Efficient GitHub operations:

```
# Get recent PRs and filter in code
prs = await github.listPullRequests(repo="Anaxiatech-AB/crackz", state="open")
priority_prs = [pr for pr in prs if "feat:" in pr.title or "fix:" in pr.title]

print(f"Found {len(priority_prs)} priority PRs out of {len(prs)} total")
for pr in priority_prs[:5]:  # Show top 5 only
    print(f"#{pr.number}: {pr.title}")
```

Efficient package operations:

```
# Install multiple packages in one operation
packages = ["pytest", "ruff", "mypy"]
for pkg in packages:
    await uv.install(package=pkg)
    print(f"✓ Installed {pkg}")
```

Default Configuration

The default MCP configuration for Crackz includes:

- **Filesystem**: Progressive discovery and file operations
- **Python**: Code execution environment for composing operations
- **UV**: Package management with uv tooling
- **GitHub**: Version control operations (issues, PRs, releases)

This configuration follows Anthropic's best practices for code execution with MCP to minimize token usage and maximize agent efficiency.

Logging Practices

Crackz uses **Loguru** for logging with a hybrid approach: central logs + per-project logs.

When to Use Logging

Use `logger` for: - Operational/detailed information (progress updates, technical details) - Internal state changes and debugging information - Warnings about internal conditions - Error details for troubleshooting - Any information useful for debugging or auditing

DO NOT use `print()` statements - Always use the logger instead.

Logging Locations

1. **Central Log**: System-wide log for all Crackz operations
2. Location priority: `CRACKZ_LOG_DIR` env var > `--log-dir` CLI flag > system temp directory
3. Contains all logging output from all projects and operations

4. Useful for debugging across multiple projects

5. **Project Log**: Per-project log at `<project>/logs/`

6. Always stored relative to the project directory
7. Contains only logs specific to that project's operations
8. Automatically created when project operations run

Logging Examples

```python from crackz.log import logger
```

Info level - operational progress

logger.info("Starting image import for {} files", len(files))

Warning level - non-critical issues

logger.warning("Mask not found for image {}, skipping", image_name)

Error level - failures with details

logger.error("Failed to load checkpoint: {}", error, exc_info=True)

Debug level - detailed troubleshooting info

logger.debug("Model config: encoder={}, weights={}", encoder, weights)

See Also

- Configuration Reference
- CLI Reference
- Contributing Guide
- API Reference

Python 3.15 Readiness

Python 3.15 Readiness for Crackz

Tracking notes for issue #609. Python 3.15 final lands October 2026. This doc captures what we plan to adopt, what we explicitly will not, and the import-cost audit that motivates the lazy-import work.

Repository floor stays at `requires-python = ">=3.11,<4"` until 3.15 is released and our supported runtimes catch up. No runtime code in this PR uses 3.15-only syntax.

PEP 810: Explicit Lazy Imports

New `lazy` soft keyword that defers module loading until first attribute access.

```
lazy import torch
lazy from segmentation_models_pytorch import Unet
```

Constraints that matter for crackz

- **Module level only.** Forbidden inside functions, class bodies, `try` blocks, and `import *`. The current convention in `docs/agent-playbook.md` is to push heavy imports into the function that needs them — PEP 810 cannot replace that pattern, it complements it.
- **No `try:` guards.** Optional-dependency probes that today look like `try: import torch except ImportError: ...` cannot be rewritten as `lazy`.
- **`from X import name` is supported**, but the import still resolves the module on first access — there is no per-symbol granularity beyond what Python already gives.
- **Star imports stay eager.** Not a problem for crackz (we do not use star imports in shipped code).

Where PEP 810 would help

Candidates are modules that:

1. Import heavy deps at module top-level today, AND
2. Are imported by light entry points (CLI, GUI bootstrap, dashboard) even when the heavy code path is not taken.

Audit results from current `main`:

Module	Heavy top-level import	Used eagerly by lightweight entry?
`core/src/crackz/ai/model/segmentation/model.py`	torch, segmentation_models_pytorch	Yes, via `crackz.ai` re-exports
`core/src/crackz/ai/model/segmentation/backends/smp.py`	torch, smp	Indirect — only via model registry
`core/src/crackz/ai/training/callbacks.py`	torch	Only during training
`core/src/crackz/ai/evaluation/metrics.py`	torch	Only during evaluation
`core/src/crackz/ai/data/dataset.py`	torch	Only during data loading
`core/src/crackz/dashboard/app.py`	streamlit	Only when dashboard is launched

CLI and GUI entry points already keep torch/numpy/Qt off the top level (see `agent-playbook.md` § Project Standards). The remaining win is at the `crackz.ai` package boundary — module-level `lazy import` in `ai/__init__.py` and the segmentation `model.py` would let `crackz --help` and the GUI splash skip torch initialization. Defer that change until 3.15 is the floor.

Compatibility plan

- Phase 0 (now): document, do not change runtime code.
- Phase 1 (when 3.15 floor): convert audited modules to `lazy import` at module level. Keep function-local imports where the import is conditional (e.g., optional backends, ImportError-guarded probes).
- Phase 2: enable `-X lazy_imports` globally for the dashboard and GUI entry points if measurement shows further wins.

What NOT to do

- Do not add `sys.version_info` branches that contain `lazy import` — the statement is a syntax error on <3.15, so the file fails to parse before any runtime check runs.
- Do not migrate the existing function-local lazy pattern to PEP 810 in GUI entry modules. The function-local pattern is still required there (PEP 810 forbids `lazy` inside functions).

Other 3.15 Features Worth Tracking

Free-threading (PEP 703 / PEP 779 / PEP 803)

- Free-threaded builds get a stable ABI (`abi3t`). Once torch, numpy, and our Qt binding ship `abi3t` wheels, training and inference on free-threaded CPython become viable. Not a 3.15-release-day option for us — wait for upstream wheel coverage.
- No code action needed in crackz yet.

JIT (PEP 744, ongoing)

The copy-and-patch JIT shipped experimental in 3.13 and is still tuned, not re-architected, in 3.15. Crackz spends almost all its CPU time inside numpy, torch, and Pillow C extensions, so JIT wins on our pure-Python code are expected to be small. Worth a one-off benchmark after 3.15 final on the training-config CLI and image-tile pipeline, but no design changes required.

`profiling` module + Tachyon (PEP 799)

Statistical sampling profiler in the stdlib, attachable by PID, flamegraph output. Replaces the ad-hoc `py-spy` recipe in `docs/advanced-performance.md` once we are on 3.15. Track for a follow-up doc update; do not migrate now.

PEP 798: unpacking in comprehensions

```
[*tile for tile in tiles]
{**meta for meta in metas}
```

Removes a few `itertools.chain.from_iterable` and `dict(ChainMap(...))` patterns in `core/src/crackz/ai/data/` and the dashboard merge code. Cosmetic; defer to the cleanup pass after the floor bump.

PEP 686: UTF-8 default I/O

We already pass `encoding="utf-8"` everywhere the linter catches; verify no regressions when the floor moves.

Stdlib niceties we can adopt later

- `frozendict` (PEP 814): replace a handful of `MappingProxyType(dict(...))` usages in config loaders.
- `TaskGroup.cancel()` (PEP 127): cleaner cancellation in `gui/` async callbacks.
- Tomllib 1.1: only relevant if config files start using inline-table newlines.

Out of Scope for #609

- No `pyproject.toml` floor bump.
- No `uv.lock` changes.
- No conversion of existing function-local lazy imports.
- No JIT or free-threading benchmarks (separate issue once 3.15 is GA).

References

- PEP 810 — https://peps.python.org/pep-0810/
- Python 3.15 whatsnew — https://docs.python.org/3.15/whatsnew/3.15.html
- PEP 703 free-threading — https://peps.python.org/pep-0703/
- PEP 744 JIT — https://peps.python.org/pep-0744/
- PEP 799 profiling module — https://peps.python.org/pep-0799/
- Issue #609 — https://github.com/Anaxiatech-AB/crackz/issues/609

Tracing & Observability

Tracing with OpenTelemetry

Crackz supports distributed tracing for AI operations using OpenTelemetry. This enables monitoring, debugging, and performance analysis of AI model calls and workflows.

Quick Start

1. Install Tracing Dependencies

```
uv sync --extra tracing
```

This installs: - `opentelemetry-instrumentation-anthropic` - Instruments Anthropic SDK - `opentelemetry-sdk` - OpenTelemetry core SDK - `opentelemetry-exporter-otlp-proto-http` - OTLP HTTP exporter

2. Start AI Toolkit Trace Collector

In VS Code, run command palette (`Ctrl+Shift+P` / `Cmd+Shift+P`):

```
AI Toolkit: Open Tracing
```

Or use command ID: `ai-mlstudio.tracing.open`

This starts the local trace collector and opens the trace viewer.

3. Enable Tracing

Option A: Environment Variable (Recommended)

```
export CRACKZ_TRACING_ENABLED=1
crackz-gui  # Tracing auto-enabled
```

Option B: Programmatic

```
from crackz.tracing import setup_tracing

# Initialize with default AI Toolkit endpoint
setup_tracing()

# Or custom endpoint
setup_tracing(
    endpoint="http://custom-collector:4318/v1/traces",
    service_name="crackz-production"
)
```

4. View Traces

Traces appear automatically in the AI Toolkit trace viewer. Each AI operation creates spans showing: - Request/response payloads - Latency and timing - Token usage - Errors and exceptions

What Gets Traced

When tracing is enabled, the following operations are automatically instrumented:

Anthropic API Calls (AI Chat)

- Message creation requests
- Streaming responses
- Token counts
- Model selection
- Error conditions

Example trace:

```
crackz
└── anthropic.messages.create
    ├── model: claude-sonnet-4-5-20250929
    ├── input_tokens: 1245
    ├── output_tokens: 382
    └── duration: 2.3s
```

Future: PyTorch Lightning Training

- Training epochs
- Validation loops
- Checkpoint saves
- Data loading

Configuration

Environment Variables

Variable	Default	Description
`CRACKZ_TRACING_ENABLED`	`0`	Enable/disable tracing (`1` / `0`)

Endpoints

AI Toolkit (Default): - HTTP: `http://localhost:4318/v1/traces` - gRPC: `http://localhost:4317` (not used by default)

Custom Collector:

```
setup_tracing(endpoint="http://your-collector:4318/v1/traces")
```

Usage Examples

CLI with Tracing

```
# Enable tracing for all CLI operations
export CRACKZ_TRACING_ENABLED=1
crackz detect run --project my_project

# Disable for specific run
export CRACKZ_TRACING_ENABLED=0
crackz detect run --project my_project
```

GUI with Tracing

```
# In main.py or startup script
from crackz.tracing import setup_tracing

# Enable before starting GUI
if os.getenv("CRACKZ_TRACING_ENABLED") == "1":
    setup_tracing()

# Run GUI
from crackz_gui.qt.app import main
main()
```

Programmatic API

```
from crackz.tracing import setup_tracing, shutdown_tracing

# Initialize tracing
setup_tracing(service_name="crackz-batch-processor")

# Your AI operations (automatically traced)
from crackz gui.services.ai_chat import AIChatService
chat = AIChatService(...)
chat.send_message("Analyze this crack detection result")

# Clean shutdown (flushes pending spans)
shutdown_tracing()
```

Troubleshooting

"Tracing dependencies not installed"

Solution:

```
uv sync --extra tracing
```

"Connection refused to localhost:4318"

Cause: AI Toolkit trace collector not running

Solution: Open trace viewer in VS Code:

```
Ctrl+Shift+P → AI Toolkit: Open Tracing
```

No traces appearing

Checklist: 1. ✅ Tracing dependencies installed (`uv sync --extra tracing`) 2. ✅ AI Toolkit trace collector running 3. ✅ `CRACKZ_TRACING_ENABLED=1` set 4. ✅ Application restarted after enabling 5. ✅ Check logs for "OpenTelemetry tracing initialized" message

Spans missing or incomplete

Cause: Application crashed or didn't shut down cleanly

Solution: Call `shutdown_tracing()` before exit:

```
import atexit
from crackz.tracing import shutdown_tracing

atexit.register(shutdown_tracing)
```

Performance Impact

Tracing has minimal overhead: - **Span creation**: ~0.1ms per operation - **Memory**: ~2MB for span buffering - **Network**: Async batch export (default 5s intervals)

Recommendation: Enable in development/staging, optional in production.

Integration with AI Toolkit

AI Toolkit provides: - **Local trace collector** - No external dependencies - **Visual trace viewer** - Interactive span timeline - **Filtering and search** - Find specific operations - **Export capabilities** - Save traces for analysis

Workflow: 1. Write code with AI operations 2. Enable tracing (`CRACKZ_TRACING_ENABLED=1`) 3. Run application 4. View traces in AI Toolkit panel 5. Analyze performance and behavior

Advanced Configuration

Custom Span Attributes

```
from opentelemetry import trace

tracer = trace.get_tracer(__name__)

with tracer.start_as_current_span("custom-operation") as span:
    span.set_attribute("project.name", "harbor_alpha")
    span.set_attribute("image.count", 1024)
    # Your code here
```

Sampling

By default, all spans are exported. For high-volume scenarios:

```
from opentelemetry.sdk.trace.sampling import TraceIdRatioBased

# Sample 10% of traces
sampler = TraceIdRatioBased(0.1)

# Pass to setup_tracing (requires custom implementation)
```

See Also

- OpenTelemetry Python Docs
- Anthropic Instrumentation
- AI Toolkit Documentation
- Crackz AI Chat Guide

Support

- **Tracing issues**: Check AI Toolkit trace viewer logs
- **Crackz issues**: GitHub Issues
- **OpenTelemetry issues**: OpenTelemetry GitHub

Contributing

Contributing Guide

Thanks for considering a contribution to Crackz! This page summarizes the essentials. For full context you can also view the repository root files: - README - CHANGELOG - Code of Conduct

---

AI Assistant Integration & Standards

Crackz supports AI-assisted development through shared repo instructions and tool-specific adapters:

- **Codex**: Shared instructions start in the repository root `AGENTS.md`; portable skills live under `.agents/skills/`
- **Claude Code CLI**: Claude-specific agents and skills live under `.claude/`; MCP details are in MCP Integration

AI assistants can help with development tasks when they follow our project conventions.

Key Standards for AI Assistants: 1. **Branch Naming**: Must follow `<type>/<scope>-<short-slug>` format (lowercase, hyphenated) 2. **Commit Messages**: Must use conventional commit format with lowercase type prefix and colon 3. **PR Titles**: MUST match conventional commit format to trigger releases 4. **Code Quality**: Follow CLI output patterns, use centralized options, maintain test coverage

Example AI-Assisted Workflow:

```
# AI creates feature branch
git checkout -b feat/gui-export-panel

# AI makes changes with proper commit messages
git commit -m "feat(gui): add export panel component"
git commit -m "feat(gui): integrate export panel with main view"

# When creating PR, AI suggests proper title
# ✅ PR Title: "feat(gui): add export panel for detection results"
# This triggers a MINOR version release (e.g., 1.2.0 → 1.3.0)
```

See MCP Integration for configuration details.

Ways to Help

- Bug reports (include OS, Python version, crackz version, steps to reproduce, expected vs actual)
- Feature requests (describe the workflow / problem first, not only the solution idea)
- Pull requests (small & focused; draft early to get feedback)
- **Pull request reviews** (see PR Review Workflow for detailed guide)
- Documentation improvements (typos, clarifications, examples)
- Performance / profiling suggestions

Environment Setup (uv)

You need Python 3.11+ and uv.

```
# create & activate venv
uv venv
source .venv/bin/activate  # Windows: .venv\Scripts\activate

# install with dev dependencies
uv sync --dev

# run CLI help
uv run crackz --help

# run GUI (optional)
uv run crackz-gui
```

Useful tasks (defined in `pyproject.toml` under `[tool.taskipy.tasks]`):

```
uv run task lint        # ruff lint (auto-fix)
uv run task format      # ruff format
uv run task mypy        # static type check
uv run task tests       # unit tests only
uv run task integration # integration tests
uv run task all_tests   # full test matrix
uv run task docs        # live docs (mkdocs serve)
uv run task build_docs  # strict build
```

Note: Static analysis is handled by the existing ruff/mypy tooling. Previous Qodana integration was removed to optimize CI performance and reduce workflow complexity.

Tests

- Unit tests: `tests/unit` (shared/global), `core/tests/unit`, `gui/tests/unit`
- Integration tests: `tests/integration` (marked with `@pytest.mark.integration`)
- Benchmark tests: `backend/backend/tests/unit/crackz/cli/test_cli_benchmark.py` (marked with `@pytest.mark.benchmark`)

Test Data Convention

- Keep shared canonical test data in `tests/test_data`.
- Keep module-local test data in module test roots only when it is private to one module and not reused.
- Avoid duplicating the same data across `core/tests` and `gui/tests`.

Run fast unit suite (CI default):

```
uv run task tests
```

Run everything:

```
uv run task all_tests
```

Run benchmark tests with automatic regression detection:

```
uv run task benchmark
```

Coverage threshold (see `pyproject.toml`) is currently `fail_under = 75`.

GUI Testing

GUI tests are located in `tests/integration/crackz/gui/` and include smoke tests to verify the GUI can launch without errors.

Smoke Tests verify basic GUI initialization: - Test that GUI entry point exists - Test that GUI can start and exit cleanly in headless mode - Test that critical components (BrowserTree, CrackzApp) can be initialized - Regression tests for known GUI component issues

Running GUI Smoke Tests:

GUI tests are automatically skipped in CI environments or when Tkinter is not available. To run GUI smoke tests locally:

```
# Run all integration tests (includes GUI smoke tests)
uv run task integration

# Run only GUI smoke tests
uv run pytest tests/integration/crackz/gui/test_gui_smoke.py

# Run GUI in smoke mode (auto-close after initialization)
set CRACKZ_GUI_SMOKE=1  # Windows CMD
export CRACKZ_GUI_SMOKE=1  # Linux/macOS
uv run crackz-gui
```

Environment Variables: - `CRACKZ_GUI_SMOKE=1` – Runs the GUI in smoke test mode where it initializes all components and exits immediately without showing the window or entering the main loop. Useful for CI pipelines and automated testing.

Platform Considerations: - GUI tests require a display server (X11, Wayland, or Windows display) - Tests automatically skip in CI environments (`CI` or `GITHUB_ACTIONS` env vars) - Tests check for Tkinter availability before running - Some tests are Windows-only due to platform-specific behavior

Test Structure:

```
tests/integration/crackz/gui/
├── test_gui_smoke.py        # Smoke tests for GUI initialization
└── ...                       # Future GUI integration tests
```

Common GUI Testing Patterns:

1. **Smoke Mode Testing** – Quick verification that GUI starts without errors:

   ```
   env = os.environ.copy()
   env["CRACKZ_GUI_SMOKE"] = "1"
   subprocess.run([sys.executable, "gui/src/crackz_gui/crackz_gui.py"], env=env)
   ```

2. **Component Initialization Testing** – Verify components can be created:

   ```
   @pytest.mark.skipif(not  has display() or os.environ.get("CI"),
                       reason="Requires display")
   def test_component(qapp):
       component = MyComponent()
       assert component is not None
   ```

3. **Regression Testing** – Prevent known issues from reoccurring:

```
def test_no_unsupported_parameters():
    # Ensure widget initialization works without errors
    browser.toggle_toolbar_buttons(visible=True)  # Should not raise
```

Performance Regression Detection

Benchmark tests automatically track performance and **fail if execution time degrades significantly**: - **Mean time threshold**: Fails if >20% slower than baseline - **Max time threshold**: Fails if >30% slower than baseline

Use the `benchmark` task to run benchmarks with automatic comparison and failure on regression:

```
# Run benchmarks - compares to last saved baseline and fails on regression
uv run task benchmark
```

The first run will save a baseline. Subsequent runs compare against this baseline automatically. If you make intentional changes that affect performance, update the baseline:

```
# Update the baseline after intentional performance-impacting changes
uv run task benchmark_save
```

Alternatively, you can run benchmarks manually with pytest:

```
# Manual benchmark with comparison
uv run pytest -m benchmark --benchmark-only --benchmark-autosave --benchmark-compare --benchmark-compare-fail=mean:20%
```

Baselines are stored in `.benchmarks/` directory (gitignored).

Coding Standards

- Ruff handles formatting & most style (`uv run task format` / `lint`).
- Keep functions focused; prefer explicit names over comments.
- Add/adjust type hints for public interfaces.
- Log with `loguru` (already configured) rather than `print` .
- Avoid large monolithic pull requests; split by feature slice.

Commit Style & Release Triggers

CRITICAL: Pull Request titles MUST follow conventional commit format to trigger releases.

Conventional Commit Format

```
<type>(<optional-scope>): <description>
```

Required for releases: - `feat:` or `feat(<scope>):` - Triggers MINOR version bump (e.g., 1.2.0 → 1.3.0) - `fix:` or `fix(<scope>):` - Triggers PATCH version bump (e.g., 1.2.0 → 1.2.1) - `perf:` or `perf(<scope>):` - Triggers PATCH version bump

No release triggered (documentation/maintenance): - `docs:` - Documentation only changes - `refactor:` - Internal code restructuring - `test:` - Test-only changes - `chore:` - Maintenance tasks, dependency updates - `ci:` - CI/CD workflow changes - `build:` - Build system or packaging changes - `style:` - Code formatting (no logic changes)

PR Title Format Rules

When merging Pull Requests, the **merge commit title** determines if a release is triggered:

✅ **CORRECT** (triggers release):

```
feat(gui): add return-to-run-view functionality
fix(pdf): correct missing hero image
perf(dataloader): optimize batch processing
```

❌ **INCORRECT** (no release):

```
Fix GUI image display and add return-to-run-view functionality
Add new feature to handle user authentication
Update documentation for new feature
```

Key Requirements: 1. Type prefix MUST be lowercase (`feat:` , `fix:` , not `Feat:` , `Fix:`) 2. Type prefix MUST end with colon (`:`) 3. Scope is optional but recommended for clarity: `feat(gui):` , `fix(cli):` 4. Description should be concise and imperative mood

Semantic-release analyzes the merge commit title to determine version bumps and generate changelogs.

Automated Validation (Pre-Commit Hook)

A pre-commit hook automatically validates commit messages to ensure they follow the conventional commit format:

```
# Install pre-commit hooks (one-time setup)
pip install pre-commit  # or: uv tool install pre-commit
pre-commit install
pre-commit install --hook-type commit-msg
```

Once installed, the hook will: - **Validate commit message format** before each commit - **Reject invalid formats** with helpful error messages showing correct examples - **Allow valid formats** (feat:, fix:, docs:, etc.) to proceed

Testing the validator manually:

```
# Test with a valid message
echo "feat(gui): add export panel" > temp_msg.txt
cz check --commit-msg-file temp_msg.txt

# Test with an invalid message
echo "Fix bug" > temp_msg.txt
cz check --commit-msg-file temp_msg.txt
```

Bypassing the hook (not recommended):

```
git commit --no-verify -m "your message"
```

Pull Request Checklist

- [ ] Linked issue (or clearly explained rationale)
- [ ] Tests updated / added (unit + integration if needed)
- [ ] Docs updated (API / usage / changelog entry if user-facing)
- [ ] Lint + mypy + tests pass locally
- [ ] Avoided unrelated formatting churn

Documentation

Live preview:

```
uv run task docs  # serves at http://127.0.0.1:8000
```

Strict build (CI equivalent):

```
uv run task build_docs
```

PDF export (uses `scripts/export_pdf.py`):

```
uv run task pdf
```

Release Process (Overview)

- Commits merged to `main` trigger commitizen (`cz bump`), which determines the next version from conventional commits, updates `pyproject.toml`, generates/updates `CHANGELOG.md`, creates a git tag, and pushes everything back to `main`.
- The release workflow then creates a draft GitHub release, uploads built artifacts, and publishes that draft in the same pipeline.
- Docker images are built for full / slim / gpu variants in a separate workflow after a successful release.
- PyPI upload is currently disabled.
- Full current behavior: see Release Flow.

Reporting Bugs

Provide: 1. Environment: OS, Python version, CPU/GPU (CUDA?), crackz version. 2. Command(s) run (exact CLI) or GUI action. 3. Stack trace and the log snippet (project `logs/crackz.log` + central log if set). 4. Minimal reproducible steps (sample images if possible).

Security Issues

Do NOT open a public issue for sensitive security problems; instead open a private discussion / email the maintainer.

Code of Conduct

We follow the repository Code of Conduct. Be respectful and inclusive.

Questions?

Open an issue or start a discussion describing your use case. Contributions welcome!

Branching & Workflow

A lightweight trunk-based model is used: all releasable work lands in `main`; feature branches are short-lived.

Quick Reference

Branch Format: `<type>/<scope>-<short-slug>` (lowercase, hyphenated)

Common Patterns: - New feature: `feat/gui-live-preview` - Bug fix: `fix/pdf-rendering-crash` - Performance: `perf/dataloader-optimization` - Documentation: `docs/api-reference-update` - Refactoring: `refactor/config-module-cleanup`

IMPORTANT: When merging, ensure PR title matches conventional commit format: - Branch: `feat/gui-live-preview` → PR Title: `feat(gui): add live preview panel` - Branch: `fix/pdf-crash` → PR Title: `fix(pdf): handle null hero image`

Core Branch

- `main`: always green & releasable. Protected (recommend: squash merges, required checks).

Ephemeral Branch Naming

Format: `<type>/<scope>-<short-slug>` (lowercase, hyphenated). Examples:

```
feat/gui-live-preview
fix/pdf-hero-image
refactor/logging-bootstrap
perf/dataloader-prefetch
chore/ci-release-step
hotfix/0.17.1-pdf-null-crash
exp/vector-db-poc
```

Allowed type prefixes: - `feat` – new user-visible functionality (triggers MINOR bump) - `fix` – bug fix (triggers PATCH bump) - `perf` – performance improvements - `refactor` – internal restructuring (no behavior change) - `docs` – documentation only - `test` – tests only - `chore` – maintenance / deps / infra - `ci` – workflow pipelines - `build` – packaging / build scripts / Docker - `style` – formatting / lint (no logic) - `hotfix` – urgent production issue (fast patch release) - `exp` – experimental spike (must be renamed before merge if kept) - `wip` – draft (never merge directly; rename when ready)

`hotfix/*` branches are cut from `main` and merged back quickly, producing a patch via a `fix:` commit.

Commit Messages (Conventional Commits)

```
&lt;type&gt;(&lt;optional-scope&gt;): &lt;imperative summary&gt;

&lt;body — optional&gt;

&lt;footer — BREAKING CHANGE / issue refs&gt;
```

Examples:

```
feat(gui): add real-time detection preview panel
fix(pdf): correct missing hero when logo == cover image
refactor(logging): unify central + project logger bootstrap
chore(ci): refine release workflow version verification
```

Breaking changes:

```
feat!: remove legacy --batch flag

BREAKING CHANGE: The --batch flag was replaced by --batch-size.
```

Semantic release interpretation (default): - `feat` → MINOR bump - `fix` → PATCH bump - `!` or `BREAKING CHANGE:` footer → MAJOR bump

Pull Request Guidelines

CRITICAL: PR title format determines if a release is triggered!

- **Title Format**: MUST follow conventional commit format (see "Commit Style & Release Triggers" section above)
- ✅ `feat(gui): add live preview` (triggers MINOR release)
- ✅ `fix(cli): correct path handling` (triggers PATCH release)
- ✅ `docs: update installation guide` (no release)
- ❌ `Fix GUI bugs` (no release - wrong format)
- ❌ `Add new feature` (no release - wrong format)
- **Merge Strategy**: Squash merge → one clean conventional commit
- **Single Focus**: One feature/fix per PR for clear changelog entries
- **Documentation**: Update docs for user-facing changes
- **Tests**: Add/update tests for behavior changes

Before Merging: 1. Verify PR title follows `<type>(<scope>): <description>` format 2. Ensure type is lowercase with colon: `feat:`, `fix:`, `docs:` 3. Confirm tests pass and code is reviewed 4. Check if release should be triggered (feat/fix/perf) or not (docs/chore/refactor)

Dev / Pre-Release Builds

Manual workflow (`Dev Build`) creates ephemeral pre-release artifacts: - Version pattern: `<base>.devYYYYMMDD+<shortsha>` - Tag pattern: `dev-<computed-dev-version>` - Artifacts: wheel + sdist, docs site, PDF, optional Docker images (`dev-` tags) Use for early validation without bumping stable semantic version.

Release Flow

1. Merge feature/fix branches into `main` (squash).
2. CI runs tests + commitizen (`cz bump`):
3. Determines next version from conventional commit history.
4. Updates `CHANGELOG.md`, commits, tags `vX.Y.Z`, and pushes the release commit back to `main`.
5. Artifact jobs build wheels, sdist, checksums, and optionally the documentation PDF.
6. The workflow creates a draft GitHub release, uploads artifacts, and publishes it.
7. Docker images (full + slim + gpu) built & pushed with tags: `:X.Y.Z` and `:latest`.
8. Docs site + PDF published to GitHub Pages via docs workflow (separate pipeline).
9. **Optional**: PyInstaller executables can be built manually and added to published releases:
10. Go to Actions → PyInstaller Build workflow
11. Click "Run workflow"
12. Enter version tag (e.g., `v0.35.0` or `latest`)
13. Select platforms (Windows only by default, or all platforms)
14. Binaries are added to the existing GitHub release

For the exact current-state sequence and triggers, see Release Flow.

Hotfix Flow

1. Branch: `hotfix/<next-patch>-<short-desc>` from `main`.
2. Implement fix; PR with `fix:` commit message.
3. Merge → `cz bump` issues patch version.

Schema Stability & Pre-Commit Hooks

Crackz persists project configuration (YAML) using Pydantic models. To avoid accidental breaking changes, we enforce a lightweight **schema snapshot** for `TrainingConfig`.

Current safeguards: - `tests/unit/crackz/project/test_training_config_schema.py` – snapshot of ordered field names + defaults. CI fails if the shape drifts. - `.pre-commit-config.yaml` – runs the schema test (and a fast project unit test) before each commit.

Workflow to enable locally:

```
pip install pre-commit  # or uv tool install pre-commit
pre-commit install
```

On commit, hooks will: 1. Run Ruff lint/format (auto-fix where possible). 2. Run the TrainingConfig schema snapshot test. 3. Run a fast project core test (`test_project.py`).

Intentional schema change checklist: 1. Update the Pydantic model (e.g. add optional field with safe default). 2. Update the snapshot list in the test file. 3. Update `docs/configuration.md` under *TrainingConfig Contract & Compatibility*. 4. Add / adapt a unit test for new logic (e.g. default behavior). 5. Mention in `CHANGELOG.md` (feat / fix) if user-visible.

If you accidentally break the snapshot, the failure message will show: - Added fields - Removed fields - Default mismatches

Future: we may extend snapshots to `ModelConfig` & `DetectionConfig` once they stabilize further.

Quick Reference Table

Type	Purpose	Example Branch
feat	New feature	`feat/cli-batch-export`
fix	Bug fix	`fix/train-metrics-path`
perf	Performance	`perf/dataloader-cache`
refactor	Internal change	`refactor/model-config`
docs	Docs only	`docs/pdf-section`
test	Tests only	`test/cli-train-e2e`
chore	Maintenance	`chore/update-ruff`
ci	Pipeline	`ci/release-tag-fix`
build	Packaging / Docker	`build/multiarch-images`
style	Formatting	`style/ruff-cleanup`
hotfix	Urgent patch	`hotfix/0.17.2-threshold-null`
exp	Experiment	`exp/alt-model-head`
wip	Draft (rename later)	`wip/gui-layout-pass1`

DO / AVOID

DO	AVOID
Squash merge to keep linear history	Merging large unrelated bundles
Write imperative summaries	Past tense or vague messages
Include scope when clear (`feat(gui):`)	Overloading scope with multiple modules
Add tests for each behavior change	Relying only on manual testing
Use `BREAKING CHANGE:` explicitly	Implicit breaking changes

Optional Enhancements (Future)

- Commit linting (GitHub Action + commitlint).
- Enforce branch name regex via a CI check.
- Automatic labeling based on branch prefix (e.g., `feat/*` → label: feature).

CLI Development Standards

CRITICAL: All CLI commands must follow these standardized output patterns for consistent user experience.

Output Conventions

For End-User Messages (use `typer.secho`)

```
# Success messages
typer.secho("✅ Operation completed successfully", fg=typer.colors.GREEN)

# Error messages
typer.secho("❌ Operation failed", fg=typer.colors.RED)

# Warning messages
typer.secho("⚠️ Warning message", fg=typer.colors.YELLOW)

# Info messages
typer.secho("ℹ️ Information", fg=typer.colors.CYAN)
```

For Operational/Detailed Information (use `logger`)

```
from crackz.log import logger

# Detailed progress, technical info, debugging
logger.info("Detailed operational information")
logger.warning("Warning about internal state")
logger.error("Error details for troubleshooting")
```

❌ NEVER USE:

- `print()` statements - Use logger instead
- `typer.echo()` - Use `typer.secho()` with colors
- Rich `console.print()` in CLI commands - Use logger for detailed output, typer.secho for user messages

- Mixed output methods within the same function

CLI Option Management

Follow the centralized option pattern:

1. Use centralized options from `core/src/crackz/cli/options.py`

```
from crackz.cli.options import PROJECT_PATH_OPTION, FORCE_OPTION

def command_function(
    project_path: Path | None = PROJECT_PATH_OPTION,
    force: bool = FORCE_OPTION,
) -> None:
```

2. Only define module-specific options locally if truly unique

```
# Only for options not reused elsewhere
LOCAL_OPTION = typer.Option(
    None,
    "--local-flag",
    help="Module-specific option not reused elsewhere",
)
```

❌ NEVER:

- Define duplicate options across modules (e.g., multiple `FORCE_OPTION` definitions)
- Use inline `typer.Option()` in function signatures for reusable options
- Mix centralized and inline option definitions

Required CLI Command Pattern

```
"""Module docstring explaining CLI command purpose."""

from __future__ import annotations

from pathlib import Path
import typer

from crackz.cli.exceptions import cli_failure
from crackz.cli.options import PROJECT_PATH_OPTION, FORCE_OPTION
from crackz.log import logger  # Always import logger

app = typer.Typer(no_args_is_help=True)

@app.command(name="command")
def command_function(
    project_path: Path | None = PROJECT_PATH_OPTION,
    force: bool = FORCE_OPTION,
) -> None:
    """Command description."""
    try:
        # Business logic here
        logger.info("Processing started")
        typer.secho("✅ Operation completed", fg=typer.colors.GREEN)
    except Exception as e:
        cli_failure(e, "Operation name")
```

Code Quality Standards

General Principles

- **Minimal changes**: Make the smallest possible modifications to achieve the goal
- **Type hints**: Add type hints for all public interfaces
- **Explicit naming**: Prefer clear function/variable names over comments
- **Logging**: Use `loguru` logger, not `print()` statements
- **Focus**: Keep functions focused on a single responsibility

Ruff Configuration

- Line length: 100 characters
- Indent: 4 spaces
- Target: Python 3.13
- Convention: Google-style docstrings

Testing Requirements

- Minimum coverage threshold: **75%**
- Unit tests in `tests/unit/`
- Integration tests in `tests/integration/`
- Mark tests with `@pytest.mark.integration` or `@pytest.mark.benchmark`

When Making Changes

Code Changes

1. Check current output patterns follow conventions
2. Use centralized CLI options from `options.py`
3. Add/update tests for new behavior
4. Run linting: `uv run task lint && uv run task format`
5. Run type checking: `uv run task mypy`
6. Verify tests pass: `uv run task tests`

CLI Command Changes

1. **Always** use `typer.secho()` for end-user messages with appropriate colors
2. **Always** use `logger` for operational/detailed information
3. **Always** import and use centralized options from `options.py`
4. **Never** mix output methods within the same function
5. Test the command manually to verify consistent user experience

Common Pitfalls to Avoid

1. ❌ **Don't use `print()`** - Use loguru logger instead
2. ❌ **Don't use `typer.echo()`** - Use `typer.secho()` with colors
3. ❌ **Don't define duplicate options** - Use centralized options from `options.py`
4. ❌ **Don't mix output methods** - Stick to the standardized patterns
5. ❌ **Don't skip tests** - Add/update tests for behavior changes
6. ❌ **Don't ignore type hints** - Add them to public interfaces
7. ❌ **Don't modify project YAML directly** - Use CLI commands or proper config methods

---

Release Flow

Release Flow

Current-state documentation for Crackz release automation.

This page describes what the repository does today via `.github/workflows/release.yml`.

Overview

Crackz uses a Commitizen-based release workflow that runs after integration tests complete successfully on `main`.

In the happy path, the workflow:

1. Determines whether a release is needed from conventional commits.
2. Bumps the version with Commitizen.
3. Updates `CHANGELOG.md`, creates a git commit, and tags `vX.Y.Z`.
4. Pushes the release commit and tag back to `main`.
5. Builds wheel artifacts, source distribution, checksums, and optionally a docs PDF.
6. Creates a draft GitHub release, uploads artifacts, then publishes it in the same workflow.

Triggering

The workflow supports two entry paths:

- Automatic: `workflow_run` after `Integration Tests` completes successfully on `main`
- Manual: `workflow_dispatch` with `confirm=YES`

The automatic path is the normal release path. The manual path exists for controlled reruns or maintainer-triggered releases.

Release Decision Rules

The workflow first runs:

```
uv run cz bump --dry-run
```

Outcome:

- Exit code `0`: a release is needed
- Exit code `21`: no release is needed (`NoneIncrementExit`)
- Any other exit code: fail the workflow

By default, the meaningful release-triggering conventional commit types are:

- `feat:` → MINOR bump
- `fix:` → PATCH bump
- `perf:` → PATCH bump

Docs-only and maintenance titles such as `docs:`, `chore:`, `refactor:`, `test:`, `ci:`, `build:`, and `style:` do not produce a release by themselves.

Step-by-Step Workflow

1. Version Bump

The `version-bump` job runs on the self-hosted runner:

- Checks out the repository with push credentials
- Installs `commitizen`
- Configures Git bot identity
- Explicitly disables commit and tag signing for CI
- Runs `cz bump --yes --changelog`

Commitizen is responsible for:

- selecting the next semantic version
- updating version metadata
- updating `CHANGELOG.md`

- creating the release commit
- creating the git tag

2. Lockfile Synchronization

After the bump, the workflow runs:

```
uv lock
```

If `uv.lock` changed, the workflow amends the Commitizen-created commit and moves the tag to the amended commit so the release commit remains internally consistent.

3. Push Back to `main`

If a release was created, the workflow pushes:

- `HEAD` to `main`
- the new git tag

This means the release commit is created by CI and written back to the main branch.

4. Draft Release Creation

Still inside the `version-bump` job, the workflow:

- extracts the matching changelog section from `CHANGELOG.md`
- creates a draft GitHub release for `vX.Y.Z`

At this point the release exists as a draft, but artifact attachment and publication still happen later in the workflow.

5. Artifact Builds

Separate jobs build release artifacts:

- `build-wheels`
- CPU wheel
- CUDA 11.8 wheel
- CUDA 12.8 wheel
- `build-sdist`
- source distribution (`.tar.gz`)

The wheel jobs also perform a smoke check by importing `crackz` and showing CLI help.

6. Release Assembly and Publication

The `release` job:

- downloads all build artifacts
- combines them into `dist/`
- optionally generates `site/crackz-docs.pdf` for minor/major releases ending in `.0`
- copies `CHANGELOG.md`
- generates `SHA256SUMS.txt`
- verifies artifact filenames match the bumped version
- uploads everything to the draft GitHub release
- publishes the release by setting `--draft=false`

This is why the workflow is more than “create draft release”: it creates a draft first, then turns that draft into the public release once artifact assembly succeeds.

Outputs and Artifacts

Published GitHub release artifacts may include:

- CPU wheel
- CUDA wheels (`cu118`, `cu128`)
- source distribution
- `CHANGELOG.md`
- `SHA256SUMS.txt`
- documentation PDF for `.0` releases

The workflow also stores a short-lived combined workflow artifact (`dist-combined`) for CI debugging and traceability.

Related Workflows

The main release workflow does not do everything:

- `ai-release-notes.yml`
- runs after a release is published
- adds customer-facing AI-generated release notes
- `pyinstaller-build.yml`
- manual
- builds standalone executables and can attach them to an existing release
- `docker-build.yml`
- manual
- builds container images after a release

Operational Notes

- The release flow depends on conventional commit semantics, which in practice means the PR title matters when using squash merge.
- The version bump job currently uses a self-hosted runner.
- CI explicitly disables Git signing in the release job so missing runner GPG configuration does not break automated releases.
- There is no human approval gate in the current workflow.

Troubleshooting

No release was created

Likely causes:

- no `feat:`, `fix:`, or `perf:` change reached `main`
- PR title / squash commit title was not in conventional commit format

Release created but later jobs failed

Check:

- wheel build logs
- version consistency check output
- artifact upload step
- PDF generation step for `.0` releases

Need to inspect recent runs

```
gh run list --limit 10
gh run view <run-id>
gh run watch
```

PR Review Workflow

Pull Request Review Workflow with Claude Code

This guide shows how to use Claude Code with GitHub CLI to review pull requests efficiently.

Prerequisites

- GitHub CLI (`gh`) installed and authenticated
- Claude Code CLI tool running
- Repository cloned locally

Quick Reference Commands

```
# List open PRs
gh pr list

# View PR details
gh pr view <number>

# View PR diff
gh pr diff <number>

# Check CI status
gh pr checks <number>

# Review PR
gh pr review <number> --approve
gh pr review <number> --comment -b "your comment"
gh pr review <number> --request-changes -b "needs changes"
```

Step-by-Step PR Review Workflow

1. List Available PRs

```
# List all open PRs
gh pr list

# List PRs with specific labels
gh pr list --label "ready-for-review"

# List your own PRs
gh pr list --author @me
```

2. View PR Details

```
# View PR summary
gh pr view 254

# View PR with comments
gh pr view 254 --comments

# Open PR in browser
gh pr view 254 --web
```

Example Output:

```
fix(docs): resolve build failures and clarify release standards #254
Open • theresiasnow wants to merge 4 commits into main from fix/docs-build-and-standards

  This PR fixes critical documentation build failures...

  View this pull request on GitHub: https://github.com/owner/repo/pull/254
```

3. Review PR Changes

```
# View full diff
gh pr diff 254

# View only changed filenames
gh pr diff 254 --name-only

# View diff in browser
gh pr diff 254 --web
```

4. Check CI Status

```
# View all checks
gh pr checks 254
```

```
# View only required checks
gh pr checks 254 --required

# Watch checks until completion
gh pr checks 254 --watch

# Open checks in browser
gh pr checks 254 --web
```

Example Output:

```
✓ Build and Test (ubuntu-latest) — pass
✓ Build and Test (windows-latest) — pass
✓ Integration Tests — pass
✓ Lint and Format — pass
```

5. Checkout PR Locally for Detailed Review

```
# Checkout PR to review code locally
gh pr checkout 254

# This creates/switches to the PR branch
# Now you can:
# - Run tests: uv run task tests
# - Run linting: uv run task lint
# - Build docs: uv run task build_docs
# - Run the code locally
```

6. Review PR Files with Claude Code

Once you've checked out the PR, you can ask Claude Code to review specific aspects:

Review code quality:

```
Review the changes in scripts/validate_commit_msg.py for:
- Code quality and readability
- Error handling
- Edge cases
- Documentation completeness
```

Review documentation:

```
Review the changes in docs/contributing.md for:
- Clarity and completeness
- Correct formatting
- Broken links
- Consistency with existing docs
```

Review tests:

```
Check if the changes need additional tests and identify any edge cases not covered.
```

7. Submit Your Review

Approve PR:

```
gh pr review 254 --approve -b "LGTM! Code quality looks good, tests pass."
```

Comment on PR:

```
gh pr review 254 --comment -b "Good work! Minor suggestion: consider adding error handling for X."
```

Request Changes:

```
gh pr review 254 --request-changes -b "Please address the linting issues in validate_commit_msg.py before merging."
```

Review with multi-line comment:

```
gh pr review 254 --approve -b "$(cat <<'EOF'
Great PR! Here's my review:

✅ Code quality: Excellent
✅ Tests: All passing
✅ Documentation: Clear and complete
✅ Linting: All checks pass

Minor suggestions:
- Consider adding a note about pre-commit hook in README
- The error messages are very helpful

Approved for merge.
EOF
)"
```

8. Return to Main Branch

```
# After review, switch back to main
git checkout main
```

Advanced Workflows

Review Multiple PRs Efficiently

```
# List all PRs and store numbers
gh pr list

# Review each PR in sequence
for pr in 254 255 256; do
    echo "Reviewing PR #$pr"
    gh pr view $pr
    gh pr diff $pr --name-only
    gh pr checks $pr
    # Ask Claude Code to review changes
done
```

Review with JSON Output for Automation

```
# Get PR data as JSON
gh pr view 254 --json title,body,author,createdAt,additions,deletions,files

# Get check status as JSON
gh pr checks 254 --json name,state,completedAt
```

Watch PR Checks in Real-Time

```
# Continuously monitor checks until completion
gh pr checks 254 --watch --interval 30

# This will update every 30 seconds until all checks complete
```

Best Practices

1. Always Check CI Before Reviewing

```
# Wait for checks to complete before reviewing
gh pr checks 254 --watch
```

2. Review Locally for Complex Changes

```
# For complex PRs, checkout and test locally
gh pr checkout 254
uv run task lint
uv run task tests
uv run task build_docs
```

3. Use Claude Code for Deep Review

After checking out the PR, ask Claude Code to: - Analyze code quality and patterns - Identify potential bugs or edge cases - Suggest improvements - Check test coverage - Review documentation completeness

4. Provide Constructive Feedback

```
# Instead of just "needs work"
gh pr review 254 --request-changes -b "$(cat <<'EOF'
Thanks for the PR! A few items to address:

1. Linting: Please run `uv run task lint` to fix PLR2004 error
2. Tests: Add test case for empty commit message
3. Docs: Update contributing.md with hook installation steps

Otherwise looks great!
EOF
)"
```

5. Verify Release Triggers

For Crackz specifically, always check:

```
# Check PR title format
gh pr view 254 --json title

# Should match: <type>(<scope>): <description>
# Examples:
```

```
# ✅ feat(gui): add export panel
# ✅ fix(docs): resolve build failures
# ❌ Fix bugs
```

Common Review Scenarios

Reviewing a Bug Fix

```
# 1. View the PR
gh pr view <number>

# 2. Check what files changed
gh pr diff <number> --name-only

# 3. View the actual changes
gh pr diff <number>

# 4. Checkout and test the fix
gh pr checkout <number>
# Test the fix...

# 5. Run all pre-commit checks (lint, format, mypy, tests)
uv run task pre_commit

# 6. Approve if all looks good
gh pr review <number> --approve -b "Bug fix verified, all pre-commit checks pass."
```

Reviewing New Documentation

```
# 1. View the PR
gh pr view <number>

# 2. Checkout the branch
gh pr checkout <number>

# 3. Build docs to check for errors
uv run task build_docs

# 4. Review locally with live server
uv run task docs
# Opens at http://127.0.0.1:8000

# 5. Ask Claude Code to review
"Review the documentation changes for clarity, completeness, and broken links"

# 6. Approve or request changes
gh pr review <number> --approve
```

Reviewing New Features

```
# 1. View PR and check size
gh pr view <number> --json additions,deletions,files

# 2. Check CI status
gh pr checks <number>

# 3. Checkout locally
gh pr checkout <number>

# 4. Run full test suite
uv run task all_tests

# 5. Test the feature manually
# ... manual testing ...

# 6. Ask Claude Code for code review
"Review this feature for:
- Code quality and maintainability
- Test coverage
- Documentation completeness
- Potential edge cases"

# 7. Provide detailed review
gh pr review <number> --approve -b "Feature works well, tests comprehensive."
```

Integration with CI/CD

The PR review workflow integrates with Crackz's CI/CD:

1. **Build Checks**: Ensure code builds on all platforms
2. **Test Checks**: Verify unit and integration tests pass
3. **Lint Checks**: Confirm code style compliance
4. **Release Trigger**: PR title determines if release is triggered

Release Workflow Behavior

When a PR is merged to main, the release workflow runs and provides clear feedback:

✅ **Release Created** (feat/fix/perf commits):

```
✅ Release workflow completed successfully
🎉 New release published: v0.52.1
📦 Artifacts built and uploaded
📝 CHANGELOG updated
🏷️ Git tag created
```

ℹ️ **No Release Created** (docs/chore/refactor commits):

```
ℹ️ Release workflow completed - No release created
🔍 Reason: No conventional commits found that trigger releases
✅ This is expected behavior for documentation and maintenance commits
```

The workflow always completes successfully - it never fails just because no release was triggered.

Always verify: - ✅ All checks pass before approving - ✅ PR title follows conventional commit format - ✅ Documentation updated for user-facing changes - ✅ Tests added for new functionality

Troubleshooting

GitHub CLI Not Authenticated

```
gh auth login
```

Can't Find PR

```
# Make sure you're in the right repository
gh repo view

# Or specify repository explicitly
gh pr view 254 -R Anaxiatech-AB/crackz
```

PR Checkout Conflicts

```
# Stash local changes first
git stash

# Then checkout PR
gh pr checkout <number>

# Restore changes later
git stash pop
```

See Also

- Contributing Guide - Full contribution guidelines
- MCP Integration - AI assistant standards
- GitHub CLI Manual - Complete gh reference

Distribution

Distribution & Commercial Delivery

Docker images are now built manually only via `.github/workflows/docker-build.yml` to reduce CI costs. Trigger manually when needed. This document outlines recommended patterns for building, packaging, and delivering the commercial Crackz application to customers. Adapt the placeholders (ORG / COMPANY / INDEX URL) to your environment.

> Goal: Consistent, auditable, secure delivery of wheels/binaries/containers with licensing terms managed via legal agreements. Automated via `.github/workflows/docker-build.yml` (manual trigger only, no longer automatic):

---

1. Artifact Types

| Artifact | Purpose | Build Command | Notes | - Trigger manually: Actions > Docker Build > Run workflow |----------|---------|---------------|-------| | Wheel (.whl) | Primary Python runtime install | `uv build --wheel` | Fast install, preferred default | | CUDA Wheel (.whl) | GPU-accelerated install | `python scripts/build_cuda_wheel.py` | See CUDA Wheels section below | | Source dist (sdist) | (Optional) source escrow / reproducibility | `uv build --sdist` | Provide only if contract requires | | PyInstaller Binary | GUI or "no Python" user install | `uv run task build_exe` | Larger (181 MB), platform-specific; includes full PyTorch | | Docker Image (CPU) | Reproducible CLI runtime | `docker build -f Dockerfile -t crackz-cli:latest .` | Multi-stage; contains wheel | | Docker Image (Slim) | Minimal runtime distribution | `docker build -f Dockerfile.slim -t crackz-cli:slim .` | Wheel + runtime libs only | | Docker Image (GPU) | CUDA acceleration | `docker build -f Dockerfile.gpu -t crackz-cli:gpu .` | Ensure host driver compatibility |

CUDA Wheels

Build wheels with CUDA support for GPU acceleration:

```
# Build with CUDA 11.8 (default, most compatible for Windows)
python scripts/build_cuda_wheel.py

# Build with CUDA 12.1
python scripts/build_cuda_wheel.py --cuda-version cu121

# Build with CUDA 12.8 (default for Linux)
python scripts/build_cuda_wheel.py --cuda-version cu128

# Build without cleaning artifacts first
python scripts/build_cuda_wheel.py --no-clean
```

Installation methods:

After building, customers can install the wheel with CUDA support:

```
# Method 1: Install with specific CUDA index
pip install dist/crackz-*.whl --extra-index-url https://download.pytorch.org/whl/cu118

# Method 2: Install with CUDA extra (uses pyproject.toml config)
pip install dist/crackz-*.whl[cu128]  # For CUDA 12.8 on Linux

# Method 3: Install CPU-only version
pip install dist/crackz-*.whl[cpu]
```

CUDA Version Guide: - `cu118` - CUDA 11.8 (most compatible, recommended for Windows) - `cu121` - CUDA 12.1 (newer GPUs) - `cu128` - CUDA 12.8 (latest, requires newest drivers, default for Linux)

Notes: - The wheel itself is platform-independent - CUDA dependencies are resolved at installation time based on the `--extra-index-url` flag or optional extras - Platform detection is configured in `pyproject.toml` (Windows defaults to cu118, Linux to cu128) - Verify CUDA availability: `python -c "import torch; print(f'CUDA available: {torch.cuda.is_available()}')"`

PyInstaller Executables

Build standalone executables for users without Python installed:

```
# Build both CLI and GUI executables (CPU-only)
uv run task build_exe

# Build only CLI
uv run task build_exe_cli

# Build only GUI
uv run task build_exe_gui

# Clean and rebuild
uv run task build_exe_clean
```

CUDA-enabled executables:

For GPU acceleration, use the CUDA build script:

```
# Build with CUDA 11.8 (default, recommended for Windows)
python scripts/build_cuda_exe.py
```

```
# Build with CUDA 12.8 (for newer GPUs/Linux)
python scripts/build_cuda_exe.py --cuda-version cu128

# Build only GUI with CUDA
python scripts/build_cuda_exe.py --gui-only

# Skip CUDA check if already installed
python scripts/build_cuda_exe.py --skip-cuda-check
```

Build outputs: - `dist/crackz` - CLI executable (~181 MB CPU-only, ~250 MB with CUDA) - `dist/crackz-gui` - GUI executable (~181 MB CPU-only, ~250 MB with CUDA)

Notes: - Executables include the entire PyTorch runtime (hence the size) - CUDA builds are larger and require NVIDIA GPU + drivers - Platform-specific (build on macOS for macOS, Windows for Windows, etc.) - Build time: 4-5 minutes on modern hardware - UPX compression is enabled by default - Assets and data files are bundled automatically

Advanced build script:

```
# Use the build script directly for more control
python scripts/build_exe.py --help
python scripts/build_exe.py --clean  # Clean then build both
python scripts/build_exe.py --no-build  # Only clean, don't build

# CUDA build script
python scripts/build_cuda_exe.py --help
python scripts/build_cuda_exe.py --clean --cuda-version cu128
```

---

2. Versioning & Changelog

- Semantic versioning (MAJOR.MINOR.PATCH) driven by commit messages (python-semantic-release).
- Each release produces: wheel in `dist/`, git tag (e.g. `v0.14.0`), changelog entry.
- Keep `upload_to_pypi = false` while using a private index; toggle later if public release.

---

3. Private Package Index Options

Option	When to Use	Pros	Cons
GitHub Releases (manual download)	Few customers / PoC	Simple, no extra cost	Manual install flow
~~GitHub Packages (pip)~~	❌ Not supported for Python	N/A	GitHub Packages doesn't support PyPI
AWS CodeArtifact / Azure Artifacts	Enterprise cloud alignment	IAM / central control	Cloud vendor lock-in
Cloudsmith / Artifactory / Gemfury	Scalable commercial delivery	Metrics, entitlement	Subscription cost
Self-hosted devpi / pypiserver	Air-gapped / on-prem customers	Full control	Ops overhead

Note: GitHub Packages does NOT support Python/PyPI repositories. Use GitHub Releases for distribution or configure a separate private package index.

Pick one primary + one fallback (offline wheel bundle).

---

4. Recommended Initial Flow (Lean)

1. Keep repository private.
2. Automate wheel build + attach to GitHub Release (action).
3. Distribute wheel via Release asset OR mirrored private index.
4. Provide offline bundle for regulated / air-gapped users.

Installing from GitHub Releases

Since GitHub Packages doesn't support Python/PyPI, use these methods:

Method 1: Direct pip install from release URL

```
# Install specific version from GitHub release
pip install https://github.com/OWNER/REPO/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl

# Or download then install
wget https://github.com/OWNER/REPO/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl
pip install crackz-0.30.2-py3-none-any.whl
```

Method 2: Using GitHub API with token (for private repos)

```
# Set GitHub token
export GITHUB_TOKEN="ghp_xxxxxxxxxxx" # gitleaks:allow
```

```
# Download using gh CLI
gh release download v0.30.2 --pattern '*.whl' --repo OWNER/REPO
pip install crackz-*.whl

# Or use curl with authentication
curl -L \
  -H "Authorization: token $GITHUB_TOKEN" \
  -o crackz.whl \
  https://github.com/OWNER/REPO/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl
pip install crackz.whl
```

Method 3: Private package index (recommended for production)

Configure a private PyPI index (see options above) and set GitHub secrets: - `PRIVATE_INDEX_URL` - Your package index URL - `PRIVATE_INDEX_USER` - Username (or `__token__`) - `PRIVATE_INDEX_PASSWORD` - Password or API token

The publish-release workflow will automatically upload to this index.

Then customers install normally:

```
pip install --index-url https://pypi.company.com/simple/ crackz
```

---

5. CI/CD Workflow (Wheel + Release)

Skeleton GitHub Actions workflow (`.github/workflows/release.yml`):

```
name: release
on:
  push:
    tags:
      - 'v*.*.*'
jobs:
  build-release:
    runs-on: ubuntu-latest
    permissions:
      contents: write   # allow releasing
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.13'
      - name: Install uv
        run: pip install uv
      - name: Sync deps (locked)
        run: uv sync --frozen
      - name: Build wheel + sdist
        run: uv build --wheel --sdist
      - name: Build documentation PDF
        run: |
          uv run mkdocs build --clean --strict
          uv run playwright install chromium
          uv run python scripts/export_pdf.py --outline \
            --cover-image 'assets/crackz-logo.png' \
            --logo 'assets/crackz-logo.png' \
            --out site/crackz-docs.pdf
          cp site/crackz-docs.pdf dist/
      - name: Upload to private index (twine)
        if: env.PRIVATE_INDEX_URL != ''
        env:
          TWINE_USERNAME: ${{ secrets.PRIVATE_INDEX_USER }}
          TWINE_PASSWORD: ${{ secrets.PRIVATE_INDEX_PASSWORD }}
        run: |
          pip install twine
          twine upload --repository-url ${{ secrets.PRIVATE_INDEX_URL }} dist/*.whl dist/*.tar.gz
      - name: Attach artifacts to Release
        uses: softprops/action-gh-release@v2
        with:
          files: dist/*
```

Note: The actual implementation publishes packages in the `publish-release.yml` workflow after the release is made public. The Release workflow creates a draft release, which is then automatically published.

PyInstaller Executables: PyInstaller builds are now optional and manual to reduce CI costs. The CI workflow builds only the supported Qt GUI executable (`crackz-gui`), not the CLI. With immutable GitHub releases enabled, published releases cannot be modified later, so this workflow behaves as follows: 1. If the target release is still mutable, the workflow uploads the binaries to that release. 2. If the target release is already immutable/published, the workflow skips release upload and keeps the binaries as workflow artifacts only. 3. Workflow artifacts are retained for 30 days, so use a longer-lived distribution channel if customers need binaries beyond that window.

To build executables for a release: 1. Go to the GitHub Actions tab 2. Select "PyInstaller Build" workflow 3. Click "Run workflow" and enter the version tag (e.g., `v0.35.0` or `latest`) 4. Select whether to build all platforms or Windows only 5. If the release is immutable, download the binaries from the workflow run artifacts within 30 days or publish them through a separate distribution channel

To publish to a private package index, configure these repository secrets: - `PRIVATE_INDEX_URL` - Your private PyPI index URL (e.g., `https://pypi.company.com/simple/`) - `PRIVATE_INDEX_USER` - Username for authentication (optional, defaults to `__token__`) -

`PRIVATE_INDEX_PASSWORD` - Password or API token for authentication

GitHub Packages does not support Python/PyPI repositories, so wheels are distributed via: 1. GitHub Releases (always, built-in) 2. Custom private package index (optional, if secrets configured)

Docker images are built automatically in a separate workflow (`.github/workflows/docker-build.yml`) after successful release, or can be triggered manually.

---

6. Docker Image Publishing

Automated via `.github/workflows/docker-build.yml` (triggered after release or manually): - Builds full, slim, and GPU variants - Pushes to GitHub Container Registry (ghcr.io) - Supports multi-platform (amd64 + arm64)

Manual build example:

```
docker login ghcr.io -u USER -p TOKEN
# Build variants
docker build -f Dockerfile       -t ghcr.io/anaxiatech-ab/crackz:latest .
docker build -f Dockerfile.slim  -t ghcr.io/anaxiatech-ab/crackz:slim   .
docker build -f Dockerfile.gpu   -t ghcr.io/anaxiatech-ab/crackz:gpu    .
# Push
for tag in latest slim gpu; do docker push ghcr.io/anaxiatech-ab/crackz:$tag; done
```

Customers pull:

```
docker pull ghcr.io/anaxiatech-ab/crackz:slim
```

---

7. License Enforcement

Strategy tiers: 1. Legal-only (recommended) – rely on contract + LICENSE file. 2. Passive reminder – log banner (can be added to application startup if needed). 3. Key validation – check signed key (JWT / HMAC) at startup (implement if needed). 4. Online entitlement – remote API validation + cached token (implement if needed).

Keep it simple until abuse risk justifies complexity. The application currently uses a legal-only approach, trusting the license file and contractual agreements.

---

8. Customer Installation Patterns

Scenario	Command Pattern
Standard CPU (from private index)	`pip install --index-url https://pypi.company.com/simple/ crackz`
Standard CPU (from GitHub Release)	`pip install https://github.com/OWNER/REPO/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl`
GPU (explicit)	`pip install torch torchvision --index-url <TORCH_URL> && pip install <crackz-wheel-url>`
No internet / offline	`pip install --no-index --find-links wheelhouse crackz-VERSION-py3-none-any.whl`
Locked reproducible	`pip install --require-hashes -r requirements.txt`

Provide a customer PDF with these flows (you can export docs using existing mkdocs PDF tooling).

---

9. Offline / Air-Gapped Delivery

1. Build wheels (app + deps):

```
 # Download Crackz wheel from GitHub Release first
wget https://github.com/OWNER/REPO/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl

# Create wheelhouse directory and download all dependencies
mkdir wheelhouse
pip download crackz-VERSION-py3-none-any.whl -d wheelhouse
```

2. Add SHA256 manifest:

```
(cd wheelhouse && shasum -a 256 * > SHA256SUMS.txt)
```

3. Zip + deliver via secure channel.
4. Customer installs:

```
pip install --no-index --find-links wheelhouse crackz-VERSION-py3-none-any.whl
```

---

10. Security & Integrity

- Optionally sign wheels (GPG) and publish `.asc` files.
- Provide SBOM (e.g., `pip install pip-audit` or `cyclonedx-bom`).
- Maintain CVE scanning (safety / pip-audit in CI).

---

11. Configuration of pip (Token Hiding)

Example `~/.config/pip/pip.conf`:

```
[global]
extra-index-url = https://TOKEN@pkg.company.com/simple
```

Windows `%APPDATA%/pip/pip.ini` similar. Avoid placing tokens in shell history.

---

12. Support & SLA Metadata

Include in customer welcome pack: | Item | Value (example) | |------|-----------------| | Support email | support@company.com | | Severity 1 target | < 4h acknowledgment | | Update cadence | Monthly minor / quarterly patch | | EOL policy | Last 2 minor versions supported |

---

13. Escrow / Source Access (If Required)

- Provide sdist to escrow agent OR controlled customer group.
- Tag escrow deliveries with `escrow-*` naming convention.

---

14. Future Enhancements

- Add license telemetry (opt-in) via anonymized usage pings.
- Integrate SBOM + provenance attestations (SLSA / sigstore).
- Introduce plugin architecture: open core + commercial extensions.

---

15. Quick Reference Command Block

```
# Build wheel
uv build --wheel
# Build + attach release (manual example)
git tag v0.14.0 && git push origin v0.14.0
# Install from private index
pip install --index-url https://pypi.company.com/simple/ crackz
# Install from GitHub Release
pip install https://github.com/OWNER/REPO/releases/download/vVERSION/crackz-VERSION-py3-none-any.whl
# Offline install
pip install --no-index --find-links wheelhouse crackz-VERSION-py3-none-any.whl
```

---

16. Glossary

Term	Meaning
Wheel	Built distribution artifact for Python packages
Private Index	Authenticated package repository (internal)
Offline Bundle	Pre-downloaded wheel set + hashes
Entitlement	Licensed right to use version/features

Keep this document updated when delivery channels change.

Customer Guide

Customer Guide: Crackz for Infrastructure Management

This guide is designed for **decision-makers, infrastructure managers, and asset owners** evaluating Crackz for their organization. For technical getting started instructions, see the Getting Started Guide.

Use Cases & Success Stories

Harbor & Port Infrastructure

Challenge: Large port facilities need to monitor concrete deterioration across thousands of structural elements (piers, docks, seawalls) with limited inspection budgets.

Solution: Crackz processes drone and handheld camera imagery to automatically identify and measure crack patterns.

Results: - **5x faster** inspection cycles - **90%+ detection accuracy** on validation datasets - **Prioritized maintenance** based on severity scores - **Complete documentation** for regulatory compliance

Bridge & Overpass Monitoring

Challenge: Transportation agencies must monitor bridge integrity across distributed assets while maintaining safety standards and budget constraints.

Solution: Integration of Crackz into regular inspection workflows with automated reporting and trend analysis.

Benefits: - **Early warning system** detecting micro-cracks before expansion - **Historical tracking** of crack progression over time - **Resource optimization** by focusing manual inspection on high-risk areas - **Compliance documentation** with exportable metrics and annotated imagery

Industrial Facilities & Plants

Challenge: Chemical plants, refineries, and manufacturing facilities require continuous structural monitoring but face access restrictions and safety concerns.

Solution: Crackz enables remote analysis of structural images captured by maintenance teams or robotic inspection systems.

Value Delivered: - **Reduced downtime** through predictive maintenance - **Safety improvement** by minimizing confined space entry - **Cost savings** on emergency repairs through early intervention - **Integration** with existing CMMS and asset management systems

---

Creating Training Masks for Custom Models

To train Crackz on your specific infrastructure, you need labeled training data. **Masks** are binary images that identify which pixels contain cracks (white/255) versus background (black/0).

What is a Mask?

A mask is a grayscale or binary image with the same dimensions as your source image: - **White pixels (255)** = crack present - **Black pixels (0)** = no crack / background

Example:

```
Original Image          Mask Image           Result
[photo of concrete] → [white lines on    = Trained model learns
                       black background]    to detect cracks
```

When Do You Need Masks?

- **For detection only**: Use the pre-trained model (no masks needed)
- **For custom training**: Create masks for 100-500 representative images
- Minimum viable: 100 images (quick pilot)
- Recommended: 200-300 images (good accuracy)
- Optimal: 500+ images (production-grade)

Recommended Tools for Creating Masks

1. CVAT (Computer Vision Annotation Tool) - Best for Teams

Free, open-source, web-based - **Website**: https://www.cvat.ai/ - **Best for**: Teams, collaborative labeling, large datasets - **Features**: - Browser-based (no installation) - Multiple annotators can work simultaneously - Polygon and brush tools for precise crack tracing - Export directly as PNG masks - Built-in quality control and review workflows

Setup:

```
# Self-hosted option (Docker)
docker run -d -p 8080:8080 cvat/server
# Or use cloud-hosted version at app.cvat.ai
```

Workflow: 1. Create a project and upload your images 2. Use the "Brush" or "Polygon" tool to trace cracks 3. Set annotation class to "crack" (class ID: 1) 4. Export as "Segmentation mask 1.1" format 5. Masks are generated as PNG files matching your image names

2. Labelme - Best for Individual Users

Free, open-source, desktop application - **Website**: https://github.com/wkentaro/labelme - **Best for**: Individual annotators, quick labeling, offline work - **Features**: - Simple desktop GUI (Windows/Mac/Linux) - Polygon drawing for crack outlines - JSON format with mask export capability - Lightweight and fast

Installation:

```
pip install labelme
labelme
```

Workflow: 1. Open image in Labelme 2. Use polygon tool to trace crack boundaries 3. Label as "crack" 4. Save (creates JSON file) 5. Convert to mask: `labelme_json_to_dataset <json_file>` 6. Extract `label.png` from output folder

3. Photoshop / GIMP - Best for Fine Control

Commercial (Photoshop) or Free (GIMP) - **Best for**: High-precision manual work, complex crack patterns - **Features**: - Full pixel-level control - Brush tools for painting cracks - Layers for organization - Batch processing capabilities

Workflow (GIMP example): 1. Open your crack image 2. Create new layer (white background) 3. Use black brush to paint over ALL cracks 4. Invert colors (black background, white cracks) 5. Export as PNG (same filename as original) 6. Save to `masks/train/`, `masks/val/`, or `masks/test/`

4. Python Scripting - Best for Automation

For developers or semi-automated workflows

```
# Example: Convert polygon annotations to masks
from PIL import Image, ImageDraw
import json

def polygon_to_mask(image_path, polygon_coords, output_path):
    # Load original image to get dimensions
    img = Image.open(image_path)
    width, height = img.size

    # Create black background
    mask = Image.new('L', (width, height), 0)
    draw = ImageDraw.Draw(mask)

    # Draw white polygon for crack
    draw.polygon(polygon_coords, fill=255)

    # Save mask
    mask.save(output_path)

# Usage
polygon_coords = [(100, 200), (150, 250), (200, 230), ...]
polygon_to_mask('image_01.jpg', polygon_coords, 'masks/train/image_01.png')
```

Best Practices for Mask Creation

Quality Guidelines

- **Accuracy over speed**: Take time to trace cracks precisely
- **Consistent width**: Paint the full width of visible cracks
- **Include hairline cracks**: Don't skip small cracks (they're valuable training signal)
- **Clean edges**: Avoid fuzzy or semi-transparent pixels (use pure white/black)
- **Match filenames exactly**: `image_01.jpg` → `image_01.png` (same stem, extension doesn't matter)

File Naming Convention

Masks must have the **exact same filename** as their corresponding image (excluding extension):

```
✓ CORRECT:
images/train/harbor_crack_001.jpg
masks/train/harbor_crack_001.png

✗ INCORRECT:
```

```
images/train/harbor_crack_001.jpg
masks/train/harbor_crack_001_mask.png  ← Extra suffix
masks/train/HarborCrack001.png         ← Different name
```

Organizational Tips

1. **Start with obvious examples**: Label clear, visible cracks first
2. **Diverse scenarios**: Include different crack types (horizontal, vertical, branching)
3. **Various conditions**: Mix lighting, angles, distances
4. **Quality check**: Review 10-20 masks visually before labeling entire dataset
5. **Version control**: Keep backups of masks (they represent significant effort!)

Train/Val/Test Split

Distribute your labeled images across directories:

```
masks/
  train/    # 70% of labeled data (200-350 images)
  val/      # 15% of labeled data (30-50 images)
  test/     # 15% of labeled data (30-50 images)
```

Same split structure should exist in `images/`:

```
images/
  train/    # Same filenames as masks/train/
  val/      # Same filenames as masks/val/
  test/     # Same filenames as masks/test/
```

Using Crackz with Your Masks

Once you've created masks, import them with your images:

GUI Method: 1. Right-click on `images/train/` folder 2. Select "Import Images to train" 3. Choose your source images (Crackz auto-detects matching masks)

CLI Method:

```
# Automatic mask detection (if masks are in source_folder/masks/)
crackz import-images \
  --project my-project \
  --src ./labeled-data \
  --type train \
  --size 512 512

# Explicit mask source
crackz import-images \
  --project my-project \
  --src ./images \
  --masks_src ./masks \
  --type train \
  --size 512 512
```

With automatic train/val/test split:

```
crackz import-images \
  --project my-project \
  --src ./labeled-data \
  --masks_src ./labeled-data/masks \
  --split \
  --train-ratio 0.7 \
  --val-ratio 0.15 \
  --test-ratio 0.15 \
  --size 512 512 \
  --random-seed 42
```

For overlapping photogrammetry or video frames, add `--group-by folder` (or `sequence-block --group-size N`) so near-duplicate frames stay in the same split; see the CLI cookbook.

Troubleshooting Mask Issues

Problem	Cause	Solution
"Missing masks" warning	Filename mismatch	Verify stems match exactly (case-sensitive)
Poor training accuracy	Masks inverted (black cracks)	Invert colors: white = crack, black = background
Model learns nothing	All-black or all-white masks	Check mask files aren't corrupted
Slow convergence	Inconsistent labeling	Review and fix annotation quality
File not found	Wrong directory structure	Ensure masks in `masks/train/`, not `masks/` root

Advanced: Semi-Automated Mask Generation

For large datasets, consider a hybrid approach: 1. Manually label 50-100 images (high quality) 2. Train initial model on this small set 3. Use model to predict masks for remaining images 4. **Manually review and correct** predicted masks 5. Retrain with expanded dataset 6. Iterate until quality is acceptable

This can reduce labeling time by 50-70% while maintaining accuracy.

Time Estimates

Based on customer experience: - **Simple cracks** (single line): 2-3 minutes per image - **Complex cracks** (branching, multiple): 5-10 minutes per image - **Very complex** (spiderweb patterns): 10-20 minutes per image

Example project timeline: - 200 images at 5 min/image = **16-17 hours of labeling** - Split across 2-3 annotators = **1 week part-time** - Professional labeling service: **3-5 days** (at additional cost)

Professional Labeling Services

If your team lacks time or resources for manual labeling, consider: - **Amazon SageMaker Ground Truth**: Pay-per-image labeling service - **Labelbox**: Managed annotation platform with QA - **Scale AI**: High-quality human annotation at scale - **V7 Darwin**: Automated QA and consistency checks

Typical costs: $0.50-$5.00 per image depending on complexity and turnaround time.

Reproducible Project Structure

Every analysis is self-contained with complete provenance:

```
inspection-campaign-2025/
  images/           # Organized by train/val/test/raw splits
  masks/            # Ground truth for training (if available)
  artifacts/        # Detection results, metrics, checkpoints
  logs/             # Complete audit trail
  project.yaml      # Configuration snapshot
```

Hybrid Logging System

- **Central logging** for multi-project oversight
- **Per-project logs** for detailed troubleshooting
- **Audit compliance** with timestamped entries
- **Portable logs** that move with project data

Configurable Detection Parameters

Fine-tune the system to your specific requirements: - **Threshold adjustment** to balance sensitivity vs. false positives - **Batch size optimization** for GPU utilization - **Model selection** (choose encoder architecture for accuracy/speed trade-off) - **Custom training** on your labeled dataset for maximum accuracy

---

Return on Investment (ROI)

Typical Cost-Benefit Analysis

Traditional Manual Inspection (baseline): - Inspector rate: $75–150/hour - Time per 100 images: 8–12 hours - Cost per 100 images: $600–1,800 - Annual inspection (5,000 images): $30,000–90,000

With Crackz (automated): - Processing time per 100 images: 15–30 minutes (GPU) - Review time: 1–2 hours (inspector validates results) - Cost per 100 images: $100–300 (primarily reviewer time) - Annual inspection (5,000 images): $5,000–15,000 - **Annual savings**: $25,000–75,000 (per inspection program)

Additional Value Drivers

- **Avoided emergency repairs** through early detection
- **Extended asset lifespan** via proactive maintenance
- **Reduced liability** from documented inspection protocols
- **Faster regulatory approval** with comprehensive documentation

---

Implementation & Support

Quick Start Timeline

Week	Milestone	Activities
1	Environment Setup	Install Crackz, configure infrastructure, initial training
2	Pilot Project	Process first inspection dataset, validate results

Week	Milestone	Activities
3	Training & Refinement	Custom model training on customer data (if required)
4	Production Rollout	Integration with existing workflows, team training

Training & Onboarding

We provide comprehensive support to ensure successful adoption: - **Technical documentation** covering all features and workflows - **Example projects** demonstrating best practices - **Configuration templates** for common use cases - **Video tutorials** (available upon request)

Licensing & Pricing

Crackz operates under a commercial license model designed for enterprise customers: - **Perpetual license** for internal use - **Source-available** code for transparency and customization - **Volume discounts** for multi-site deployments - **SLA options** for mission-critical operations

Contact: theresia.sofia.snow@gmail.com for: - Custom pricing for your organization - Proof-of-concept projects - Extended commercial rights (SaaS, redistribution) - On-premises or air-gapped deployments

---

Security & Compliance

Data Privacy

- **On-premises deployment** – your data never leaves your infrastructure
- **No telemetry** or external API calls by default
- **Air-gapped operation** for sensitive facilities
- **Containerized isolation** for multi-tenant environments

Audit & Compliance

- **Complete reproducibility** with versioned configurations
- **Cryptographic hashing** for artifact integrity verification
- **Timestamped logs** for regulatory requirements
- **Exportable reports** in industry-standard formats

Enterprise Security

- **No cloud dependencies** – fully self-contained runtime
- **License enforcement** options (from legal-only to key validation)
- **Role-based access** (via external authentication if integrated)
- **SBOM generation** for supply chain security

Next Steps

For Technical Users

Ready to get started? See the Getting Started Guide for: - Quick 5-minute installation - Your first project setup - Running detection and training

For Decision Makers

Contact us to discuss your requirements:

📧 **Email**: theresia.sofia.snow@gmail.com

Include in your inquiry: - Organization name and use case - Estimated inspection volume - Current inspection workflow - Timeline and budget considerations

Request a Proof-of-Concept

We offer pilot projects to demonstrate Crackz on your actual infrastructure data: 1. Provide representative sample images (100-500 images) 2. We'll run detection analysis and provide results 3. Review accuracy, performance, and integration requirements 4. Discuss licensing and deployment options

Schedule a Demo

See Crackz in action with a personalized walkthrough: - Live demonstration of GUI and CLI workflows - Q&A with technical team - Discussion of your specific use case and requirements - Custom configuration recommendations

---

Comparison: Crackz vs. Alternatives

Feature	Crackz	Generic ML Platforms	Manual Inspection
Infrastructure-specific	✓ Pre-trained on cracks	✗ Requires custom training	✓ Domain expertise
Deployment	Self-hosted, containerized	Cloud-only (often)	N/A
Cost	One-time license	Ongoing cloud fees	High labor cost
Speed	100s of images/hour	Varies	10-15 images/hour
Consistency	Perfect repeatability	Depends on model	Subjective variance
Offline capability	✓ Air-gapped ready	✗ Requires internet	✓ Always offline
Customization	Source-available	Black box	N/A
GPU acceleration	✓ Native support	✓ (if cloud)	N/A

Frequently Asked Questions

Q: Can Crackz handle different types of infrastructure?

A: Yes. While initially developed for harbor concrete, Crackz's deep learning architecture adapts to various materials and crack patterns through custom training. Customers have successfully used it on bridges, tunnels, building facades, and industrial equipment.

Q: What accuracy can we expect?

A: On validation datasets similar to training data, detection accuracy typically exceeds 90%. For new asset types, we recommend a pilot phase with ground truth labeling to fine-tune the model for optimal performance.

Q: Do we need labeled training data?

A: For initial testing, use the pre-trained model. For production deployment on your specific infrastructure, we recommend labeling a representative sample (100-500 images) to train a custom model for maximum accuracy.

Q: What are the hardware requirements?

A: - **CPU**: Any modern x64 processor (sufficient for small-scale processing) - **GPU**: NVIDIA GPU with 4+ GB VRAM recommended for batch processing - **RAM**: 8 GB minimum, 16+ GB recommended for large projects - **Storage**: 10-50 GB depending on image volume and training data

Q: Can Crackz integrate with our existing systems?

A: Crackz provides CLI and Docker interfaces designed for integration. Output is JSON + annotated images, easily consumed by CMMS, GIS, or custom reporting tools. We can provide integration consulting for complex workflows.

Q: What about regulatory compliance (ISO, ASTM)?

A: Crackz provides the technical capabilities (documentation, reproducibility, audit trails) required for compliance. Specific certification depends on your industry and jurisdiction. We recommend involving your compliance team during pilot phases.

Q: Is training required for our team?

A: Basic usage (GUI workflow) requires minimal training—most users are productive within 1-2 hours. Advanced features (custom model training, CLI automation) benefit from our documentation and example projects. Custom training workshops are available upon request.

Next Steps

Crackz: AI-powered infrastructure inspection that's accurate, affordable, and audit-ready.

Copyright © 2025 Theresia Sofia Snow. All rights reserved. Crackz is commercial software. See LICENSE for terms.

Customer Access

When scaling becomes critical: 1. Build customer portal (web app for license management) 2. Automate onboarding via API 3. Integrate with payment processor (Stripe, etc.) 4. Analytics dashboard

Cost: Variable (development + hosting)
Effort: Mostly automated

Next Steps

Immediate Actions (This Week)

1. **Create customer tracking spreadsheet** using template above
2. **Document onboarding email template** for consistency
3. **Test GitHub team workflow** with a test user
4. **Write customer installation guide** specific to your setup

Short Term (This Month)

1. **Create customer-access folder** with templates:
2. `templates/welcome-email.md`
3. `templates/access-revocation-email.md`
4. `templates/renewal-reminder-email.md`
5. **Set up access audit calendar**:
6. Quarterly: Review all customer teams
7. Monthly: Check for expiring licenses
8. Weekly: Process new customer onboarding

Long Term (This Quarter)

1. **Evaluate Azure Artifacts** if customer count grows
2. **Build automation scripts** for common tasks
3. **Create customer success metrics** dashboard

FAQ

Q: Can customers share their access with subcontractors?
A: Yes, if it's within their license terms. Add subcontractors to the customer's GitHub team. Consider charging for additional users if license is per-user.

Q: How do we handle license transfers?
A: Remove old organization's team, create new team for new organization. Transfer is administrative only (same package access).

Q: Can we offer trial licenses?
A: Yes! Create time-limited teams (30-day trial). Set calendar reminder to revoke access after trial period.

Q: How do we handle enterprise customers with 100+ users?
A: Use GitHub Enterprise features (SAML SSO, team sync with Azure AD). Or migrate to Azure Artifacts with AD integration.

Support

For questions about customer access management: - **Email:** theresia.sofia.snow@gmail.com - **Documentation:** This file (`docs/customer-access.md`)

Customer Access Management Guide v1.0 · Copyright © 2025 Theresia Sofia Snow

Customer Access Management

This guide explains how to manage customer access to Crackz packages and Docker images.

Overview

Crackz uses a **tiered access control** model:

1. **GitHub Releases** - Public releases (for open distribution) or private releases (for authorized customers only)
2. **GitHub Container Registry (ghcr.io)** - Docker images with token-based access
3. **Optional Private PyPI Index** - For simplified `pip install crackz` workflow

---

Access Control Models

Model 1: Private GitHub Repository (Current Setup)

How it works: - Keep repository private - Grant customers repository access (read-only) - Customers use Personal Access Tokens (PATs) for authentication

Pros: - ✅ Free (no additional cost) - ✅ Simple to manage via GitHub Teams - ✅ Works for both Docker and wheel files - ✅ Audit trail via GitHub access logs

Cons: - ❌ Customers can see source code (if that's a concern) - ❌ Requires GitHub account per customer

Best for: Small customer base (<20 organizations), source-available licensing model

Model 2: Public Repository + Private Packages

How it works: - Repository is public (source-available) - Docker images in ghcr.io with token authentication - Wheels distributed via private links or separate package index

Pros: - ✅ Public code (transparency, marketing) - ✅ Controlled access to binaries - ✅ No GitHub accounts needed for customers

Cons: - ❌ More complex access management - ❌ Need separate authentication for Docker/wheels

Best for: Source-available with commercial binaries model

Model 3: Private Package Index (Enterprise)

How it works: - Use Azure Artifacts, AWS CodeArtifact, or Cloudsmith - Customers get unique credentials per organization - Simple `pip install crackz` with configured index

Pros: - ✅ Professional customer experience - ✅ Fine-grained access control - ✅ Usage analytics - ✅ Revocation without GitHub access changes

Cons: - ❌ Monthly cost ($0-150/month depending on service) - ❌ Additional infrastructure to manage

Best for: Enterprise customers, SaaS model, many customers (20+)

Implementation Guide

Option A: GitHub Repository Access (Recommended for Now)

Setup Organization Teams

1. **Create customer teams** in your GitHub organization:

```
# Via GitHub web UI:
# Settings > Teams > New Team
# Team name: "Customer-Acme-Corp"
# Privacy: Secret
# Repository access: crackz (Read)
```

2. **Invite customer contacts** to the team:

3. Each customer organization gets one team
4. Add their technical contacts as members
5. They get read-only access to releases and packages

Customer Onboarding Process

Step 1: Send welcome email

```
Subject: Crackz Access - Welcome to [Customer Name]

Hi [Contact Name],

Thank you for your Crackz license purchase!

To access Crackz packages and Docker images, please:

1. Create a GitHub account (if you don't have one): https://github.com/signup
2. Reply with your GitHub username
3. We'll add you to the private repository

Once added, you can access:
- Releases: https://github.com/Anaxiatech-AB/crackz/releases
- Docker images: https://github.com/anaxiatech?tab=packages&repo_name=crackz
- Documentation: https://github.com/Anaxiatech-AB/crackz/tree/main/docs

License: [LICENSE-KEY or CONTRACT-REF]
Now the `release.yml` workflow will automatically publish to Azure Artifacts when configured!

Best regards,
Theresia
```

Step 2: Add customer to team 1. Go to GitHub Settings > Teams 2. Find "Customer-[CompanyName]" team 3. Add member by GitHub username 4. Set role to "Member" (not "Maintainer")

Step 3: Send access confirmation

```
Subject: Crackz Access Confirmed

Hi [Contact Name],

You now have access to the Crackz repository!

Installation instructions:
https://github.com/Anaxiatech-AB/crackz/blob/main/docs/installation.md

Getting started:
https://github.com/Anaxiatech-AB/crackz/blob/main/docs/getting-started.md

Docker registry:
ghcr.io/anaxiatech-ab/crackz:latest

Need help? Email theresia.sofia.snow@gmail.com

Best regards,
Theresia
```

Customer Installation (GitHub Releases)

Option 1: Direct pip install from release

```
# Find latest version at: https://github.com/Anaxiatech-AB/crackz/releases
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl
```

Option 2: Download then install

```
# Download from GitHub Releases page
wget https://github.com/Anaxiatech-AB/crackz/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl

# Install locally
pip install crackz-0.30.2-py3-none-any.whl
```

Option 3: Using GitHub CLI (with authentication)

```
# Install GitHub CLI: https://cli.github.com/
gh auth login

# Download latest release
gh release download --repo anaxiatech/crackz --pattern '*.whl'

# Install
pip install crackz-*.whl
```

Customer Docker Access

Step 1: Generate Personal Access Token (PAT)

Send customers these instructions:

```
# Generate GitHub Personal Access Token

1. Go to: https://github.com/settings/tokens/new
```

```
2. Note: "Crackz Docker Access"
3. Expiration: 1 year (or custom)
4. Scopes: Select ONLY:
   - ✅ read:packages
5. Click "Generate token"
6. Copy the token (starts with ghp_)
```

Step 2: Authenticate Docker

```
# Customer runs this once
export CR_PAT=ghp_xxxxxxxxxxxxxxxxxxxx  # Their PAT # gitleaks:allow
echo $CR_PAT | docker login ghcr.io -u GITHUB_USERNAME --password-stdin

# Now they can pull
docker pull ghcr.io/anaxiatech-ab/crackz:latest
docker pull ghcr.io/anaxiatech-ab/crackz:slim
docker pull ghcr.io/anaxiatech-ab/crackz:gpu
```

Step 3: Run containers

```
# Use as normal
docker run --rm ghcr.io/anaxiatech-ab/crackz:latest --version
```

Option B: Azure Artifacts (Enterprise Scale)

If you need more control and analytics, set up Azure Artifacts:

Setup Azure Artifacts Feed

1. **Create Azure DevOps organization** (free tier available)
2. Go to: https://dev.azure.com/
3. Create organization: "crackz" or your company name
4. **Create Artifacts feed**

   ```
   # Via Azure DevOps UI:
   # Artifacts > Create Feed
   # Name: crackz-packages
   # Visibility: Private (organization)
   # Upstream sources: Enable PyPI (optional)
   ```

5. **Configure feed permissions**
6. Feed Settings > Permissions
7. Create groups per customer:
   - "Customer-Acme-Corp" (Reader role)
   - "Customer-Beta-Inc" (Reader role)
8. **Generate feed URL**

   ```
   https://pkgs.dev.azure.com/YOUR-ORG/_packaging/crackz-packages/pypi/simple/
   ```

Configure GitHub Workflow

Set these repository secrets:

```
# Settings > Secrets and variables > Actions > New repository secret

PRIVATE_INDEX_URL: https://pkgs.dev.azure.com/YOUR-ORG/_packaging/crackz-packages/pypi/upload
PRIVATE_INDEX_USER: azure
PRIVATE_INDEX_PASSWORD: <Azure-PAT-with-Packaging-Write-scope> # gitleaks:allow
```

Now the `publish-release.yml` workflow will automatically publish to Azure Artifacts!

Customer Setup (Azure Artifacts)

Step 1: Generate customer credentials 1. Azure DevOps > User Settings > Personal Access Tokens 2. Create PAT for customer: "Acme-Corp-Crackz-Access" 3. Scopes: Packaging (Read) 4. Expiration: 1 year 5. Send PAT securely to customer

Step 2: Customer pip configuration

Send customers these instructions:

```
# Option 1: Install with explicit index
pip install crackz \
  --index-url https://pkgs.dev.azure.com/YOUR-ORG/_packaging/crackz-packages/pypi/simple/

# Option 2: Configure pip globally
# Linux/macOS: ~/.config/pip/pip.conf
# Windows: %APPDATA%\pip\pip.ini
```

```
[global]
index-url = https://YOUR-PAT@pkgs.dev.azure.com/YOUR-ORG/_packaging/crackz-packages/pypi/simple/
```

Step 3: Verify access

```
pip install crackz
crackz --version
```

Cost Estimate (Azure Artifacts)

- **Storage**: $0 for first 2 GB (covers ~200 releases)
- **Users**: First 5 free, then $6/user/month
- **Bandwidth**: Free within Azure, free for external downloads

Total monthly cost: $0-12 for typical usage

Option C: Self-Hosted Private PyPI (Advanced)

For customers who need on-premise or air-gapped installations:

Setup devpi Server

```
# Install devpi
pip install devpi-server devpi-web

# Initialize server
devpi-init

# Start server
devpi-server --start --host=0.0.0.0 --port=3141

# Create user for yourself
devpi use http://localhost:3141
devpi user -c admin password=SECRET # gitleaks:allow

# Create index
devpi index -c crackz bases=root/pypi
devpi use crackz
```

Upload Packages

```
# Install devpi client
pip install devpi-client

# Login
devpi use http://YOUR-SERVER:3141
devpi login admin --password=SECRET # gitleaks:allow
devpi use admin/crackz

# Upload wheel
devpi upload dist/crackz-*.whl
```

Customer Access

```
# Customer configures pip
pip install --index-url http://YOUR-SERVER:3141/admin/crackz/+simple/ crackz
```

Hosting options: - **Cloud VM**: $10-30/month (AWS, Azure, DigitalOcean) - **On-premise**: Use existing infrastructure - **Docker**: `docker run -p 3141:3141 muccg/devpi`

Access Management Workflows

Onboard New Customer

Checklist: - [ ] Contract signed and license key generated - [ ] Create GitHub team: "Customer-[CompanyName]" - [ ] Add customer contacts to team (GitHub usernames) - [ ] Send welcome email with access instructions - [ ] Provide installation guide and support contact - [ ] Add to customer tracking spreadsheet/CRM

Template tracking spreadsheet: | Customer | License Key | GitHub Team | Contacts | Start Date | Renewal Date | Status | |----------|-------------|-------------|----------|------------|--------------|--------| | Acme Corp | LIC-001 | Customer-Acme-Corp | john@acme.com | 2025-01-15 | 2026-01-15 | Active | | Beta Inc | LIC-002 | Customer-Beta-Inc | jane@beta.com | 2025-02-01 | 2026-02-01 | Active |

Revoke Access

When license expires or is terminated:

1. **Remove from GitHub team**

2. Settings > Teams > Customer-[Name] > Members
3. Remove all members
4. **Revoke Docker access** (if using PATs)
5. Customers' PATs automatically stop working when team access is removed
6. **Revoke PyPI access** (if using Azure Artifacts)
7. Disable customer's PAT in Azure DevOps
8. Remove from feed permissions group
9. **Send termination notice**

```
Subject: Crackz License Expired - Access Revoked

Hi [Contact],

Your Crackz license expired on [DATE].
Repository and package access has been revoked.

To renew, please contact theresia.sofia.snow@gmail.com

Thank you for using Crackz.
```

Upgrade/Downgrade Customer

Scenario: Customer upgrades from single-user to team license

1. Add additional GitHub users to their team
2. No technical changes needed (packages remain the same)
3. Update customer tracking spreadsheet

Transfer Access Between Team Members

Scenario: Customer's team member leaves, new person joins

1. Remove old member from GitHub team
2. Add new member to GitHub team
3. New member generates their own PAT for Docker
4. No disruption to existing installations

Customer Support Scenarios

Customer: "I can't access the releases page"

Diagnosis: - Check if they're logged into GitHub - Verify they're in the correct team - Check team has "Read" access to repository

Solution:

```
# Ask customer to verify:
1. Go to: https://github.com/Anaxiatech-AB/crackz
2. If you see "404 Not Found", you don't have access yet
3. Reply with your GitHub username for access
```

Customer: "Docker pull fails with authentication error"

Diagnosis: - PAT expired or incorrect - PAT doesn't have `read:packages` scope - Not logged into ghcr.io

Solution:

```
# Ask customer to:
1. Generate new PAT: https://github.com/settings/tokens/new
2. Select ONLY: read:packages
3. Run: echo PAT | docker login ghcr.io -u USERNAME --password-stdin
4. Try again: docker pull ghcr.io/anaxiatech-ab/crackz:latest
```

Customer: "Which version should I install?"

Diagnosis: - Customer needs guidance on release versions

Solution: - **Latest stable**: Highest version number (e.g., v0.30.2) - **Specific version**: Check CHANGELOG.md for features - **Pre-release/beta**: Only if customer is testing new features

```
# Recommend:
pip install https://github.com/Anaxiatech-AB/crackz/releases/download/v0.30.2/crackz-0.30.2-py3-none-any.whl
```

Customer: "Can we host Crackz internally?"

Diagnosis: - Customer wants air-gapped or on-premise deployment

Solution: 1. **Offline wheel bundle**: Provide wheelhouse with all dependencies

```
# You prepare:
pip download crackz -d wheelhouse
tar -czf crackz-offline-bundle.tar.gz wheelhouse/

# Customer installs:
tar -xzf crackz-offline-bundle.tar.gz
pip install --no-index --find-links wheelhouse crackz
```

1. **Private Docker registry**: Customer mirrors your images

   ```
   # Customer pulls and retags
   docker pull ghcr.io/anaxiatech-ab/crackz:latest
   docker tag ghcr.io/anaxiatech-ab/crackz:latest registry.company.com/crackz:latest
   docker push registry.company.com/crackz:latest
   ```

2. **Self-hosted PyPI**: Set up devpi for customer (consulting service)

Analytics & Monitoring

Track Downloads (GitHub)

Via GitHub API:

```
# Get release download counts
gh api repos/anaxiatech/crackz/releases \
  --jq '.[] | {name: .name, downloads: [.assets[].download_count] | add}'
```

Output example:

```
{"name":"v0.30.2","downloads":47}
{"name":"v0.30.1","downloads":23}
```

Track Docker Pulls (GitHub)

Via GitHub web UI: 1. Go to: https://github.com/anaxiatech?tab=packages&repo_name=crackz 2. Click on package name 3. View "Package Insights" tab for pull statistics

Via API:

```
# Requires GitHub GraphQL API (complex)
# Easier to use web UI for now
```

Track PyPI Downloads (Azure Artifacts)

Azure DevOps UI: 1. Artifacts > crackz-packages 2. Analytics tab shows: - Downloads per day/week/month - Top consumers - Package versions

Customer Usage Report Template

Generate monthly reports:

```
# Crackz Usage Report - October 2025

## Active Customers: 5

| Customer | License | Status | Last Access |
|----------|---------|--------|-------------|
| Acme Corp | LIC-001 | Active | 2025-10-08 |
| Beta Inc | LIC-002 | Active | 2025-10-05 |
| Gamma LLC | LIC-003 | Expired | 2025-09-30 |

## Download Statistics

- Total releases: 12
- Total downloads (wheels): 156
- Docker pulls (latest): 89
- Docker pulls (gpu): 34

## Top Versions
1. v0.30.2: 47 downloads
2. v0.30.1: 23 downloads
```

```
3. v0.30.0: 18 downloads

## Action Items
- [ ] Contact Gamma LLC for renewal
- [ ] Send v0.31.0 release announcement
```

Security Best Practices

For You (Package Publisher)

1. **Never commit secrets to repository**
2. Use GitHub Secrets for `PRIVATE_INDEX_*`
3. Rotate Azure/AWS credentials quarterly
4. **Use least-privilege access**
5. Customers get "Read" role only
6. Separate teams per customer
7. **Audit access regularly**
8. Review team memberships quarterly
9. Remove inactive users
10. **Version control access changes**
11. Document when/why customers were added/removed
12. Keep changelog of access modifications

For Customers

1. **Use Personal Access Tokens (PATs), not passwords**
2. Set expiration (max 1 year)
3. Scope to minimum required (`read:packages` only)
4. **Store credentials securely**
5. Use environment variables, not hardcoded
6. Consider secrets management (Vault, Azure Key Vault)
7. **Rotate credentials**
8. Regenerate PATs before expiration
9. Update CI/CD pipelines
10. **Limit credential sharing**
11. One PAT per user/system
12. Don't share PATs between team members

Pricing & Revenue Tracking

License Models

Option 1: Per-Organization (Recommended) - Flat rate per customer organization - Unlimited users within organization - Example: $5,000/year per customer

Option 2: Per-User - Charge per GitHub user added to team - Better for large organizations - Example: $500/user/year

Option 3: Per-Download - Track downloads via API - Charge based on usage - Example: $10/1000 downloads

Option 4: Tiered - Starter: 1-5 users, $2,000/year - Professional: 6-20 users, $8,000/year - Enterprise: Unlimited, $20,000/year + custom features

Revenue Tracking Spreadsheet

Customer	Plan	License Fee	Payment Date	Renewal Date	Status
Acme Corp	Professional	$8,000	2025-01-15	2026-01-15	Paid
Beta Inc	Starter	$2,000	2025-02-01	2026-02-01	Paid
Gamma LLC	Professional	$8,000	2024-10-01	2025-10-01	**Expired**

Annual Recurring Revenue (ARR): $18,000
Monthly Recurring Revenue (MRR): $1,500

Automation Scripts

Customer Onboarding Script

```
#!/bin/bash
# scripts/customer-access/onboard-customer.sh

set -euo pipefail

CUSTOMER_NAME="$1"
GITHUB_USERNAME="$2"
LICENSE_KEY="$3"

echo "🎉 Onboarding customer: $CUSTOMER_NAME"
echo "GitHub username: $GITHUB_USERNAME"
echo "License: $LICENSE_KEY"

# Create GitHub team (requires gh CLI with admin access)
TEAM_SLUG="customer-$(echo $CUSTOMER_NAME | tr '[:upper:]' '[:lower:]' | tr ' ' '-')"
gh api orgs/YOUR-ORG/teams -f name="Customer-$CUSTOMER_NAME" -f privacy=secret

# Add member to team
gh api orgs/YOUR-ORG/teams/$TEAM_SLUG/memberships/$GITHUB_USERNAME -X PUT

# Add team to repository
gh api orgs/YOUR-ORG/teams/$TEAM_SLUG/repos/YOUR-ORG/crackz -X PUT -f permission=pull

echo "✅ Customer onboarded successfully!"
echo ""
echo "Next steps:"
echo "1. Send welcome email to customer"
echo "2. Add to tracking spreadsheet"
echo "3. Schedule renewal reminder for 1 year from now"
```

Access Revocation Script

```
#!/bin/bash
# scripts/customer-access/revoke-customer.sh

set -euo pipefail

CUSTOMER_NAME="$1"
REASON="${2:-License expired}"

echo "🔒 Revoking access for: $CUSTOMER_NAME"
echo "Reason: $REASON"

TEAM_SLUG="customer-$(echo $CUSTOMER_NAME | tr '[:upper:]' '[:lower:]' | tr ' ' '-')"

# Remove team from repository
gh api orgs/YOUR-ORG/teams/$TEAM_SLUG/repos/YOUR-ORG/crackz -X DELETE

# Optionally delete team entirely
read -p "Delete team entirely? (y/n) " -n 1 -r
echo
if [[ $REPLY =~ ^[Yy]$ ]]; then
    gh api orgs/YOUR-ORG/teams/$TEAM_SLUG -X DELETE
    echo "✅ Team deleted"
else
    echo "ℹ️ Team kept (access removed from repository only)"
fi

echo "✅ Access revoked successfully!"
echo ""
echo "Next steps:"
echo "1. Send termination notice to customer"
echo "2. Update tracking spreadsheet"
echo "3. Archive customer records"
```

Recommended Setup for Your Current Scale

Based on your current state (early commercial phase, likely <10 customers):

Phase 1: GitHub Repository Access (Now - 20 customers)

What to do now: 1. ✅ Keep using GitHub Releases for wheels (already working) 2. ✅ Keep using ghcr.io for Docker (already working) 3. ✅ Create customer tracking spreadsheet (see template above) 4. ✅ Document customer onboarding process (send welcome emails manually)

Cost: $0/month
Effort: ~30 min per customer onboarding

Phase 2: Add Azure Artifacts (20-100 customers)

When you hit 20+ customers: 1. Set up Azure Artifacts feed 2. Configure `PRIVATE_INDEX_*` secrets in GitHub 3. Customers get simple `pip install crackz` experience

Cost: $0-12/month
Effort: ~10 min per customer onboarding (semi-automated)

Phase 3: Full Automation (100+ customers)

License

License

GNU GENERAL PUBLIC LICENSE Version 3, 29 June 2007

```
Copyright (C) 2025  Theresia Sofia Snow

This program is free software: you can redistribute it and/or modify
it under the terms of the GNU General Public License as published by
the Free Software Foundation, either version 3 of the License, or
(at your option) any later version.

This program is distributed in the hope that it will be useful,
but WITHOUT ANY WARRANTY; without even the implied warranty of
MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  See the
GNU General Public License for more details.

You should have received a copy of the GNU General Public License
along with this program.  If not, see <http://www.gnu.org/licenses/>.
```

Also add information on how to contact you by electronic and paper mail.

You should also get your employer (if you work as a programmer) or school, if any, to sign a "copyright disclaimer" for the program, if necessary. For more information on this, and how to apply and follow the GNU GPL, see http://www.gnu.org/licenses/.

The GNU General Public License does not permit incorporating your program into proprietary programs. If your program is a subroutine library, you may consider it more useful to permit linking proprietary applications with the library. If this is what you want to do, use the GNU Lesser General Public License instead of this License. But first, please read http://www.gnu.org/philosophy/why-not-lgpl.html.